1年目エンジニア研修 資料

目次

1年目エンジニア研修 全体概要

1年目エンジニア研修カリキュラム（Linux・Git・設計・Web・TDD）

第1回 知識体系の紹介＋Linux の全体像

第2回 シェル・環境変数・通信の切り分け

環境設定手順書（WSL・Ubuntu）

第3回 Web 通信の基礎（HTTP・Cookie・Web API）

第4回 実行環境の整備

環境構築手順

第5回 Git と GitHub

研修の Git の規約（コミット・ブランチ・PR・セルフレビュー）

第6回 Python の基本

第7回 読みやすく壊れにくい Python

第8回 テストと TDD

第9回 要件と図

第10回 設計書作成

第11回 TDD による Web バックエンド

第12回 画面の部分更新と成果発表

成果発表: TODO アプリ

タグ一覧

db　env　env-mise　env-python　git　git-github　git-基礎　http　http-ブラウザ　http-リアルタイム　http-基本　http-構成　linux　linux-アプリ　linux-シェル　linux-ネットワーク　linux-ファイル　pm　pm-見積　pm-進捗　python　python-oop　python-可読性　python-構文　sec　test　test-CI　test-レビュー　test-自動　test-静的解析　web　web-サーバ　web-ブラウザ　標準　設計　設計-構成　設計-要件　設計-詳細

テンプレート

API設計書　システム構成図　不具合票　DB設計書　ユーザーストーリー　タスク　PR の説明　要件メモ <アプリ名>

未確定事項

１年目エンジニア研修 全体概要

週１回×1時間×全１２回の研修です。カリキュラムの正本は curriculum.md です。この全体概要の表と図は curriculum.md から生成するので、直すときは curriculum.md のほうを直します。

目的

Linux の基礎から始めて、配属後に知らない領域に触れたときの判断基準となるベースを作っておくことが目的です。1つの題材「TODO・タスク管理アプリ」を通して、「作る」よりも「仕事を進める型」（記録する・小さく変更する・設計で合意する・テストで確かめる）を身につけます。評価の中心は Git 運用・設計書の質・テストのある実装で、Web 開発はそのための手段です。

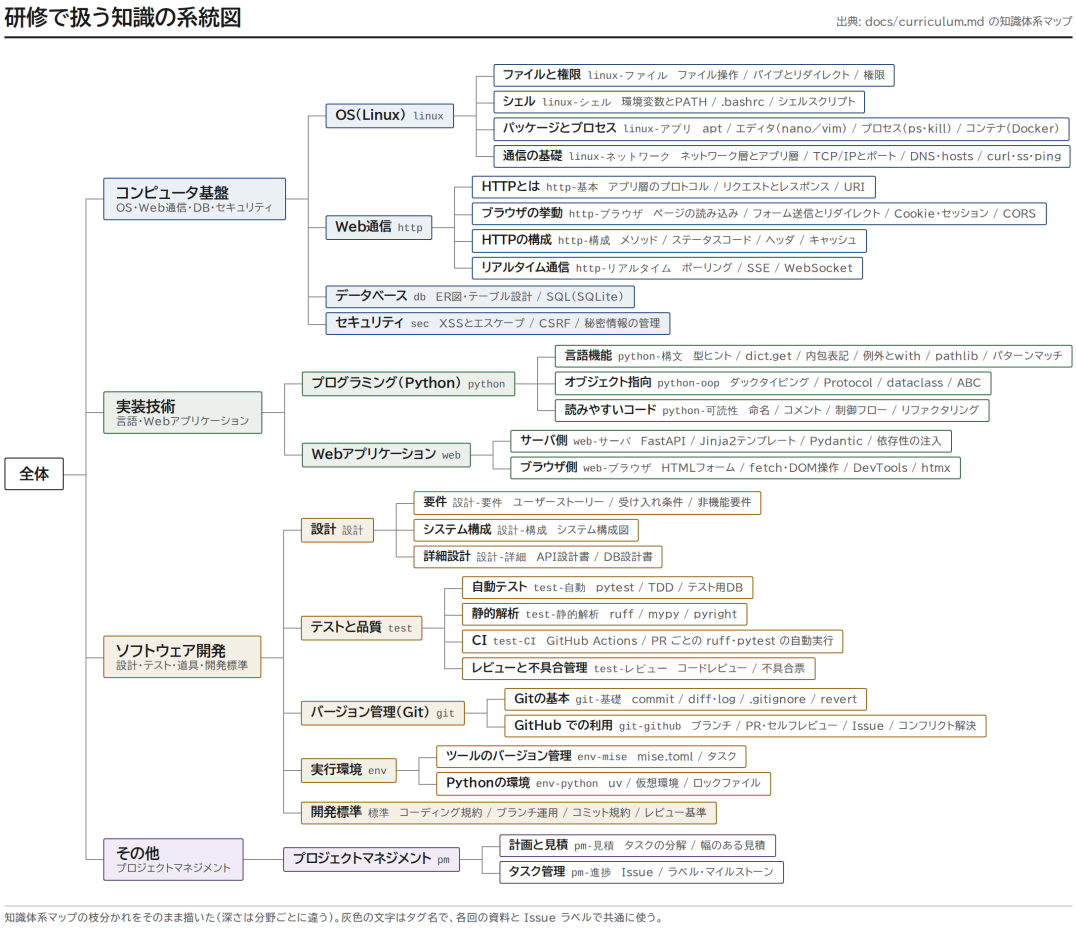
- 全回を個人で進めます。共通のリポジトリは使わず、各自のリポジトリで作ります。レビューは PR のセルフレビューと、講師や隣の人に見てもらう形で行います
- 第１〜２回: Linux の全体像、シェル・環境変数・通信の切り分け
- 第３回: Web 通信の基礎（HTTP のヘッダ・Cookie とセッション・Web API・CORS・XSS をデモのサーバとブラウザで観察する）
- 第４〜５回: 実行環境（mise と uv）と構成管理（Git と GitHub）
- 第６〜８回: Python で TODO の CLI を作り、読みやすく整理して、テストと CI を付ける
- 第９〜10回: Web 版の要件と設計書を書く
- 第11〜12回: FastAPI＋Jinja2＋SQLite＋素の HTML/JavaScript で Web 版をテスト付きで作る（第12回は fetch による画面の部分更新）
- 第12回の後半: １人ずつ成果を発表する
- １回の配分: 座学 → ハンズオン（合計６０分）。冒頭の座学で要点をまとめて話し、ハンズオンの合間にも短い座学を挟みます。配分は回ごとに違い、各回の資料の「所要」の行と下の一覧に書いています。振り返り・宿題・発展課題はありません
- プロジェクトマネジメントは軽く扱います。第９回で作る機能をユーザーストーリーに分け、幅を持たせて見積もり、GitHub の Issue とラベルで管理してみます（研修そのものの見積と実績の記録はしません）

全回を通して、次の４つを横断テーマとして扱います。

1. Git: 第５回以降、コードと設計書は自分のリポジトリにコミットし、まとまった変更は PR にしてセルフレビューしてからマージする
2. ドキュメント: 設計書・README・手順書をリポジトリ内に Markdown で置く。第９回で図と要件、第10回で設計書の書き方を学び、以降も要所で Mermaid の図を使う
3. 環境の再現性: 第４回以降、mise.toml と手順書だけで他人が再現できることを完了条件にする
4. TDD: 第８回以降の実装はテストを先に書き、テストと実装のコミットを分ける

研修で扱う知識の系統図

研修で扱う知識を、知識体系マップの枝分かれのとおりに描いた系統図です（深さは分野ごとに違います）。第１回の冒頭で全体を見回すのに使い、どの回で扱うかは下の全体像の図で説明します。

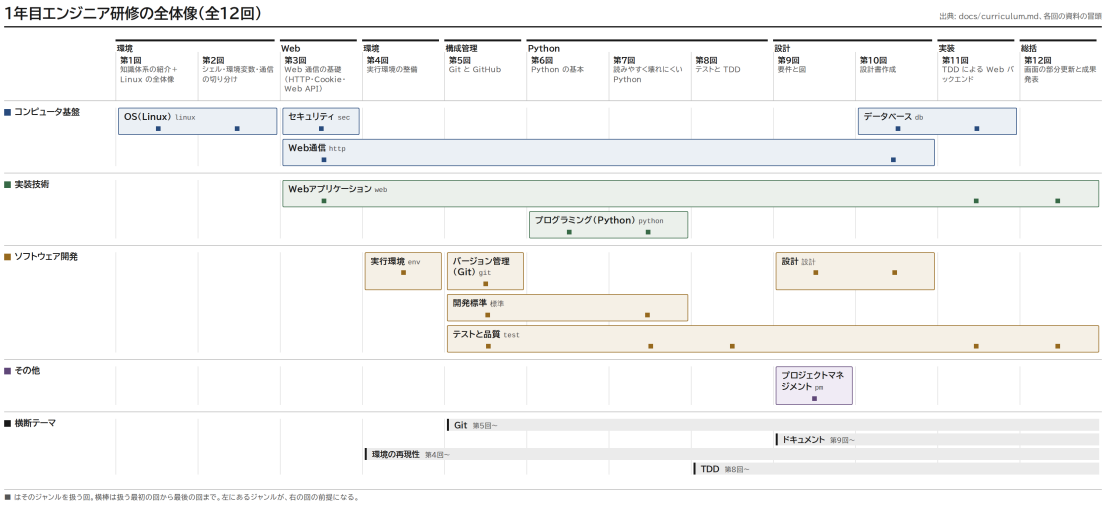


研修で扱う知識の系統図

PDF 版は knowledge-map.pdf です。

研修の全体像

列はフェーズと回、行は大分野です。ジャンル（分野）は、扱う最初の回から最後の回までの横棒で表します。■ が扱う回です。左にあるジャンルが、右の回の前提になります。系統図とこの図は、カリキュラムと各回の資料の冒頭から生成しているので、手で編集しません（`tools/` で `mise run diagrams`）。



研修の全体像

PDF 版は overview.pdf、同じ配置の Mermaid 版（Markdown に貼れる）は overview.mmd です。

各回の内容

日程は研修ごとに決めます。回のテーマ・タグ・成果物は curriculum.md の「全Ｎ回の一覧」から生成しています。

回	テーマ	所要	tag	前提タグ	成果物	資料	見本
第１回	知識体系の紹介＋Linux の全体像	座学15分 / ハンズオン45分	linux	なし	—	session01	session01
第２回	シェル・環境変数・通信の切り分け	座学15分 / ハンズオン45分	linux-シェル, linux-アプリ, linux-ネットワーク	linux	環境設定手順書	session02	session02
第３回	Web 通信の基礎（HTTP・Cookie・Web API）	座学15分 / ハンズオン45分	http, web-ブラウザ, sec	linux-シェル, linux-ネットワーク	—	session03	session03
第４回	実行環境の整備	座学15分 / ハンズオン45分	env-mise, env-python	linux-シェル, linux-アプリ, linux-ネットワーク	mise.toml＋環境構築手順書（README）	session04	session04

回	テーマ	所要	tag	前提タグ	成果物	資料	見本
第5回	Git と GitHub	座学15分 / ハンズオン45分	git-基礎, git-github, 標準, test-レビュー	linux-ファイル, linux-アプリ, env-mise, env-python	手順書のリポジトリ＋PR 2本(コンフリクト解決つき)	session05	session05
第6回	Python の基本	座学15分 / ハンズオン45分	python-構文	env-python, git-基礎, git-github	TODO の CLI＋アプリのリポジトリ(期限の追加を PR でマージ)	session06	session06
第7回	読みやすく壊れにくい Python	座学15分 / ハンズオン45分	python-oop, python-可読性, test-静的解析, 標準	python-構文, env-python, git-github	`src/todo/` パッケージ＋ruff・mypy の設定と README(PR でマージ)	session07	session07
第8回	テストと TDD	座学15分 / ハンズオン45分	test-自動, test-CI	python-oop, test-静的解析, git-github	テスト＋CI＋期限順の一覧(TDD の履歴が残る PR でマージ)	session08	session08
第9回	要件と図	座学15分 / ハンズオン45分	設計-要件, pm-見積, pm-進捗	python-構文, test-自動, git-github	要件メモ(業務フロー図・状態遷移図)＋ストーリー S-01〜S-04(Issue とその控え、PR でマージ)	session09	session09
第10回	設計書作成	座学15分 / ハンズオン45分	設計-詳細, 設計-構成, http-構成, db	設計-要件, git-github	API・DB設計書＋システム構成図(隣の人の質問を反映し、PR でマージ)	session10	session10
第11回	TDD による Web バックエンド	座学15分 / ハンズオン45分	web-サーバ, db, test-自動	設計-詳細, python-oop, test-自動, test-CI	Web 版(FastAPI＋Jinja2＋SQLite)＋受け入れ条件 (1)〜(15) のテスト(TDD の履歴が残る PR でマージし、ストーリーの Issue を閉じる)	session11	session11

資料の読み方

各回の資料の冒頭は、次の形にそろえています。

```
# 第2回 シェル・環境変数・通信の基礎
tag: linux-シェル, linux-アプリ, linux-ネットワーク
前提タグ: linux-ファイル
所要: 座学20分 / ハンズオン40分
```

- `tag`：その回で扱う知識。タグ一覧 から、タグごとの索引（扱う回・読む本）に飛べます
- `前提タグ`：その回の前に身につけておきたい知識
- `所要`：座学・ハンズオンの配分（回ごとに違う）
- 資料はチートシートです。「まとめ」はその回の要点（コマンド・用語・判断の基準）を表と箇条書きにしたもので、冒頭の座学で使います。詳しい説明は各行の「参照」と「読む章」にある参考書と無料資料で読みます（副読本は持っている人向けの紹介）
- 見出しは「ゴール → まとめ → ハンズオン → 読む章」の順です
- ハンズオンは１ステップずつ「🖋 やること → 📚 参考書の節 → 📒 説明 → コマンド」の形です
- `instructor.md` は講師用です（受講者に配るときは除きます）

ディレクトリ

ディレクトリ	内容
`docs/sessionNN/`	各回の資料（`README.md` 受講者用、`instructor.md` 講師用、`assets/` 演習素材）
`docs/templates/`	設計書・ユーザーストーリー（Issue）・不具合票・PR のひな形
`docs/tags.md`、`docs/tags/`	タグ一覧とタグ索引（`tools/` で生成）
`docs/open-questions.md`	未確定事項（社内プロキシ、Git ホスティングなど）
`docs/_template/`	資料を書くときのひな形（講師・作成者向け）
`docs/diagrams/`	知識の系統図（`knowledge-map.png`・`.pdf`・`.html`）と研修の全体像の図（`overview.png`・`.pdf`・`.html`、Mermaid 版 `overview.mmd`・`overview-mermaid.png`。`tools/` で生成）
`docs/_build/`	HTML 版（`html/`）と１冊の PDF 版（`handsoff.pdf`）。`tools/` で生成
`examples/sessionNN/`	各回の成果物の見本と、その回の見本への案内
`examples/todo-app/`	見本の TODO アプリの最新（第１２回終了時点）
`examples/todo-app-snapshots/`	見本の TODO アプリの各回終了時点の状態（`session-06/` ～ `session-12/`）。一覧と追いつき方は examples/README.md

事前準備

- WSL（Ubuntu）が使えること（open-questions.md の Q2）
- git（Ubuntu に入っているもの）
- mise（第４回でインストールする）

回の構成やタグを変えるときは、curriculum.md を直してから `tools/` で `mise run tag-index`（一覧と索引の再生成）と `mise run check`（資料との食い違いの検査）を実行してください。

１年目エンジニア研修カリキュラム（Linux・Git・設計・Web・TDD）

最終更新: 2026-10-13

変更履歴:

- 2026-10-13 第３回に「Web 通信の基礎（HTTP ヘッダ・Cookie とセッション・CSRF・Web API と JSON・CORS・DOM と fetch・XSS）」を新設し、旧第３〜10回を第４〜11回に繰り下げた。旧第11回のアプリに依存しない Web の基礎は第３回へ移し、残り（画面の部分更新・JSON 版・BUG-001）と成果発表を第12回にまとめた。KPT とスクラムは扱わないことにした
- 2026-10-13 振り返り・宿題・発展課題をやめ、各回を座学とハンズオンだけの60分にした。ハンズオンは第1回と同じく「🖋 やること → 📚 参考書の節 → 📒 説明（Mermaid の図を多めに）→ コマンド」のステップ形式にし、yomiyasu で推敲する。宿題で作っていたものはハンズオンに移した
- 2026-10-12 第1回を『入門 モダンLinux』の流れ（全体→プロセス→ファイルシステム→ネットワーク→押さえるコマンド→コンテナ→モダンなコマンド）で Linux 全体を通して見る回にした（テーマ「知識体系の紹介＋Linux の全体像」、タグ `linux`、座学15分 / ハンズオン35分）
- 2026-10-12 チーム開発と題材「社内備品貸出システム」をやめ、個人で TODO・タスク管理アプリを作る流れにした。Git は1回（第４回、基本からブランチ・PR・コンフリクト解決まで）にまとめ、ユーザーストーリーと見積は軽くして設計を２回（要件と図／設計書）に組み直した。空いた分は Python を２回（基本／読みやすく壊れにくい Python）とテストの回に配った。共通リポジトリは使わず、Git で管理するのは必要な回だけにした
- 2026-10-12 第５回の後に「図と設計書の書き方」の回を追加し、全13回にした（以降の回番号を１つずつ繰り下げ、以降の回で図を使う箇所を明記）
- 2026-10-12 データ分析の回を外して全12回にした。プロジェクトマネジメントは考え方の講義と GitHub での管理の体験だけにし、見積と実績の記録はやめた。座学の時間を回ごとに取り、ハンズオン資料はチートシートの形にした
- 2026-10-12 知識体系マップを整理した。開発標準（旧 `pm-標準`）をソフトウェア開発の下へ移してタグを `標準` に変え、タグのない中間の枝（開発ツールと環境）をなくして Git と実行環境をソフトウェア開発の直下に置いた。プロジェクトマネジメントは浅いまま（計画と見積・タスク管理・振り返り・ドキュメント）
- 2026-10-12 大分野を４つに組み替えた（コンピュータ基盤／実装技術／ソフトウェア開発／その他）。プロジェクトマネジメントは「その他」の下の `pm` にし、`pm-振り返り` と `pm-文書` はなくした（第12回のタグは `pm`）。大分野はタグ名に入らないので、ほかのタグ名は変わらない
- 2026-10-12 コンピュータ基盤を整理した。社内ネットワーク（`net-社内`）は分野から外し、プロキシは必要な場面で案内する。通信の基礎は Linux の下（`linux-ネットワーク`）、Web通信は OS と同じ層（`http`。HTTPとは→ブラウザの挙動→HTTPの構成→リアルタイム）にした。DB は深く扱わないので細分せず `db` だけにした（セキュリティも細分しない）
- 2026-10-12 設計は、アジャイルで設計書を少なめにする範囲にした。要件はユーザーストーリーと受け入れ条件（第７回のテーマを「ユーザーストーリーと見積」に変更）、設計書は要件メモ・API・DB・システム構成図だけにし、画面設計書とテスト仕様書はなくした。アーキテクチャのモジュール性と結合・DRYと直交性は扱わない

- 2026-10-12 全体を見直した。自己評価シートと週２回の拡張案をなくし、CI（`test-CI`、第10回）を足した。第２回は「シェル・環境変数・通信の基礎」にし、プロキシは社内で必要な設定の１項目に抑え、第11回のリアルタイム通信は発展課題だけにした。`test-設計` はマップから外した
- 2026-10-12 参考書を３冊（計9,460円）に組み替えた。『ソフトウェアエンジニアリングの基礎』を副読本から参考書へ移して研修の軸にし、初心者向けの『新しいLinuxの教科書 第２版』と『Web技術の基本 第２版』を引く本として足した。『Python実践レシピ』は追加候補へ移した
- 2026-10-12 副読本は紹介だけで全員が持つ前提ではないことにし、資料と宿題を参考書と無料資料だけで完結させる方針にした。『入門 モダンLinux』を副読本から参考書へ移し、『新しいLinuxの教科書 第２版』は候補へ移した（参考書は計9,240円）

方針と全体像

週１回×1時間×12回（約３か月）を基本形とします。目的は、Linux の基礎から始めて、配属後に知らない領域に触れたときの判断基準となるベースを作っておくことです。そのために、１つの題材を通して「作る」より「仕事を進める型」（記録する・小さく変更する・設計で合意する・テストで確かめる）を身につけ、知識体系マップで全体のどこに何があるかを押さえます。Web開発は手段であり、評価の中心はGit運用・設計書の質・テストのある実装に置きます。

- **１回の進め方**：冒頭の座学で要点をまとめて話し、ハンズオンの合間にも短い座学を挟みます。座学で使うのはハンズオン資料の「まとめ」か、参考書の該当章です。ハンズオンでは講師と一緒にその場で動かし、後半の回では各自が実際にプログラムを作ります。配分は回ごとに変え、「全Ｎ回の一覧」の所要の列に書きます。振り返り・宿題・発展課題は設けず、その回でやることは60分の中で終えます。
- **ハンズオン資料はチートシート＋ステップ**：まとめはエッセンス（コマンド・用語・判断の基準を表や箇条書きで）と、参考書と無料の公式資料への参照だけを載せます。ハンズオンは１ステップずつ「🖋 やること → 📚 参考書の節 → 📒 短い説明（Mermaid の図を多めに）→ コマンド」の形で書きます。副読本は全員が持っているとは限らないので、資料は副読本なしで完結させます。短く保ちつつ、その回に必要な要素は漏らしません。詳しい説明は本に任せます。
- **言語はPythonで統一**：言語の学習コストを抑え、設計・テストの学習に回します。Python は２回（第６～７回）かけて基本から Python らしい書き方（ダックタイピングなど）まで扱い、第８回でテストを書けるようにします。Web は移り変わりが激しいため、HTTP・HTML・Cookie・WebSocket など変わりにくい基礎を厚く学びます。バックエンドは FastAPI、画面はサーバが HTML を返す軽い構成とし、React などのフレームワークやビルドツールは使いません。
- **受講者のレベル差**：情報系・非情報系の混在を想定し、ハンズオンを短いステップに分けて、全員がその場で同じところまで進めるようにします。
- **進め方**：全回を個人で進めます。共通のリポジトリは使わず、各自のリポジトリで作ります。Git で管理するのはコード・設計書など履歴を残す意味があるものだけにし、毎回の演習を必ず Git に載せることはしません。レビューは PR のセルフレビューと、講師や隣の人に見てもらう形で行います。
- **プロジェクトマネジメントは軽く**：作る機能をユーザーストーリーに分け、幅を持たせて見積もり、Issue で管理する考え方を体験する程度にします（第９回）。研修そのもの（講義やハンズオン）について見積と実績を記録する進捗管理は行いません。

画面側はライブラリを使わず、素の HTML と JavaScript（fetch）で作ります。ブラウザ標準の機能は流行に左右されず、通信を自分で書くので初学者でも何が起きているかを追えるからです。基本形は「サーバが HTML 断片を返

し、JavaScript は受け取って差し替えるだけ」とし、画面の中身をサーバ側に残して pytest による TDD の範囲に収めます。第12回では1機能だけ JSON を返す版も書き、次の観点で比べます。htmx は最後に「この作りをライブラリにしたもの」として紹介します。

観点	HTML 断片を返す（基本形）	JSON を返す（比較用）
サーバが返すもの	Jinja2 で描いた HTML 断片	JSON
画面を組み立てる場所	サーバ	ブラウザ
JavaScript の役割	fetch して受け取った HTML を差し替えるだけ	JSON から DOM を組み立てる
状態の持ち主	サーバに一元化しやすい	ブラウザの変数とサーバで二重に持ちやすい
テスト	返す HTML を pytest で検証でき、TDD の範囲に収まる	API は pytest で検証できるが、画面の組み立てはブラウザ側のテストが別に要る
安全性	テンプレートが自動でエスケープする	innerHTML に値を直接入れると XSS になる。textContent を使う
向いている場面	CRUD 中心の画面、管理画面	複雑なクライアント側の状態、オフライン対応

オンラインへのリリースは前提として設計に含めますが、実際には公開しません。第9〜10回の非機能要件とシステム構成図は本番公開を想定して書き、設定は環境変数で切り替えられるようにします。CI で PR ごとにテストを自動実行するところまでを扱い、デプロイや実際のリリースは行いません。

通しの題材と横断テーマ

題材は「TODO・タスク管理アプリ」です。第6〜8回で Python の CLI（追加・一覧・完了・期限）として作ってテストを付け、第9〜10回で Web 版の要件と設計を書き、第11〜12回で画面（FastAPI＋Jinja2＋SQLite＋素の HTML/JavaScript）をテスト付きで作ります。全員が個人で一通り作ります。

全回を貫く横断テーマは次の4つです。どの回でも必ず触れるので、個別の回で教え切る必要はありません。

1. **Git**：第5回以降、コードと設計書は自分のリポジトリにコミットし、まとまった変更は PR にしてセルフレビューしてからマージする
2. **ドキュメント**：設計書・README・手順書をリポジトリ内にMarkdownで置く。第9回で図と要件、第10回で設計書の書き方を学び、以降の回でも要所で Mermaid の図を書いて考え・伝える
3. **環境の再現性**：第4回以降、「自分のPCでは動く」を禁止。mise.tomlと手順書だけで他人が再現できることを完了条件にする
4. **TDD**：第8回以降の実装はテストを先に書く。テストのコミットと実装のコミットを分け、PR の履歴でレッド→グリーン→リファクタリングの順序が追えることをレビュー基準にする

知識体系とタグ

研修で扱う知識を４つの大分野に整理し、第１回の冒頭で全体地図として紹介します。大分野は知識の性質で分けており、コンピュータ基盤（動かす土台）、実装技術（作るための技術）、ソフトウェア開発（作り方と道具）、その他（プロジェクトマネジメント）の４つです。その下は書籍の守備範囲（章立て）に合わせて切っています。このため、枝ごとに「この本を読めばよい」と副読本を案内できます。知識体系は研修の回とは独立させ、回との対応は各回のアジェンダ側からタグで付けます。

枝は分野ごとに深さを変えてよく、必要に応じて何段でも伸ばせます。タグは大分野を除いた経路を「-」でつないだ名前（例：`http-リアルタイム`）とし、資料の粒度に合わせてどの深さのタグを付けても構いません（`http` のように途中の枝で止めてもよい）。同じタグ名をハンズオン資料と Issue ラベルで共通に使います。

図にしたもの：研修で扱う知識の系統図（`tools/` の `mise run diagrams` でこのマップから生成）

知識体系マップ（木構造）

- **コンピュータ基盤**（OS・Web通信・DB・セキュリティ）
  - `linux` OS（Linux）
    - `linux-ファイル` ファイルと権限：ファイル操作／パイプとリダイレクト／権限
    - `linux-シェル` シェル：環境変数とPATH／.bashrc／シェルスクリプト
    - `linux-アプリ` パッケージとプロセス：apt／エディタ（nano／vim）／プロセス（ps・kill）／コンテナ（Docker）
    - `linux-ネットワーク` 通信の基礎：ネットワーク層とアプリ層／TCP/IPとポート／DNS・hosts／curl・ss・ping
  - `http` Web通信
    - `http-基本` HTTPとは：アプリ層のプロトコル／リクエストとレスポンス／URI
    - `http-ブラウザ` ブラウザの挙動：ページの読み込み／フォーム送信とリダイレクト／Cookie・セッション／CORS
    - `http-構成` HTTPの構成：メソッド／ステータスコード／ヘッダ／キャッシュ
    - `http-リアルタイム` リアルタイム通信：ポーリング／SSE／WebSocket
  - `db` データベース：ER図・テーブル設計／SQL（SQLite）
  - `sec` セキュリティ：XSSとエスケープ／CSRF／秘密情報の管理
- **実装技術**（言語・Webアプリケーション）
  - `python` プログラミング（Python）
    - `python-構文` 言語機能：型ヒント／dict.get／内包表記／例外とwith／pathlib／パターンマッチ
    - `python-oop` オブジェクト指向：ダックタイピング／Protocol／dataclass／ABC
    - `python-可読性` 読みやすいコード：命名／コメント／制御フロー／リファクタリング
  - `web` Webアプリケーション
    - `web-サーバ` サーバ側：FastAPI／Jinja2テンプレート／Pydantic／依存性の注入
    - `web-ブラウザ` ブラウザ側：HTMLフォーム／fetch・DOM操作／DevTools／htmx

- **ソフトウェア開発**（設計・テスト・道具・開発標準）
  - `設計` 設計
    - `設計-要件` 要件：ユーザーストーリー／受け入れ条件／非機能要件
    - `設計-構成` システム構成：システム構成図
    - `設計-詳細` 詳細設計：API設計書／DB設計書
  - `test` テストと品質
    - `test-自動` 自動テスト：pytest／TDD／テスト用DB
    - `test-静的解析` 静的解析：ruff／mypy／pyright
    - `test-CI` CI：GitHub Actions／PR ごとの ruff・pytest の自動実行
    - `test-レビュー` レビューと不具合管理：コードレビュー／不具合票
  - `git` バージョン管理（Git）
    - `git-基礎` Gitの基本：commit／diff・log／.gitignore／revert
    - `git-github` GitHub での利用：ブランチ／PR・セルフレビュー／Issue／コンフリクト解決
  - `env` 実行環境
    - `env-mise` ツールのバージョン管理：mise.toml／タスク
    - `env-python` Pythonの環境：uv／仮想環境／ロックファイル
  - `標準` 開発標準：コーディング規約／ブランチ運用／コミット規約／レビュー基準
- **その他**（プロジェクトマネジメント）
  - `pm` プロジェクトマネジメント
    - `pm-見積` 計画と見積：タスクの分解／幅のある見積
    - `pm-進捗` タスク管理：Issue／ラベル・マイルストーン

設計書で押さえること（設計-要件・設計-構成・設計-詳細）

アジャイルで設計書を少なくする前提で、書くのは次の５つだけにします。何を作るかはユーザーストーリーと受け入れ条件で合意し、細かい振る舞いは TDD のテストで示します（画面設計書とテスト仕様書は作りません）。図は Mermaid で Markdown に書き、Git で差分を追える形にします。合格基準は「実装者が質問せずに作れるか」です。

設計書	主な図	必須記載事項
ユーザーストーリー（Issue）	—	「～として、～したい。なぜなら～」、受け入れ条件（チェックリスト）、幅のある見積
要件メモ	業務フロー図	目的・背景、非機能要件（性能・セキュリティ・運用）、制約、対象外、ストーリー一覧へのリンク
API設計書	シーケンス図	エンドポイント、メソッド、リクエスト・レスポンス例、ステータスコード、入力チェックとエラー表示の文言
DB設計書	ER図	テーブル定義（カラム・型・NULL可否・キー）、制約
システム構成図	構成図	構成要素、通信経路とポート、ミドルウェアとバージョン

ハンズオン資料へのタグ付け

資料の冒頭にタグと前提タグを書きます。同じタグを GitHub の Issue ラベルにも使い、タグごとの索引（扱う回と読む本）を `docs/tags/` に生成します。

```
# 第2回 シェル・環境変数・通信の切り分け
tag: linux-シェル, linux-アプリ, linux-ネットワーク
前提タグ: linux
所要: 座学15分 / ハンズオン45分

## ゴール
- `command not found` のとき、PATH とコマンドを探す順番から原因を見つけて直せる
- つながらないとき、`curl` の表示と終了コードから、名前解決・接続・TLS・HTTP のどこで止まったかを言える
```

副読本と参考書

参考書は３冊（計9,240円）で、全員に配る前提です。研修の軸として読み進める『ソフトウェアエンジニアリングの基礎』、Linux の仕組みを理解する『入門 モダンLinux』、困ったときに引く『Web技術の基本 第２版』を組み合わせ、ハンズオン資料に書く説明を本の章に任せます。Git・Python・FastAPI などは無料の公式資料を使います。副読本は紹介するだけで、全員が利用できるわけではない前提です。資料とハンズオンは参考書と無料資料だけで進められるようにし、副読本は「さらに読むなら」として案内します。どの本もタグに紐付け、各回の資料に「読む章」として書きます。

副読本（紹介のみ。全員が持つ前提ではない）

書名	価格（税込）	読み方	対応タグ	研修で使う章と回
Effective Python 第２版	3,960円	該当章	python-構文, python-oop, test-静的解析, env-python	1〜2章（項目16 dict.get など）、8章 項目65〜66（例外と with）（第６回）／5章 クラスと継承（第７回）／10章 項目83 仮想環境（第４回）、項目90 静的解析（第７回）

書名	価格(税込)	読み方	対応タグ	研修で使う章と回
リーダブルコード	2,640円	通読	python-可読性, test-レビュー, 標準	2〜6章 名前とコメント(第7回の規約づくり)／7〜9章 制御フローと変数(第7回のリファクタリング演習)／14章 テストと読みやすさ(第8回)／巻末解説の添削コミット(第5回のセルフレビュー)
達人プログラマー 第2版(オーム社)	3,520円	通読	linux-シェル, git-基礎, pm-見積	基本ツール(シェル、プレーンテキスト、バージョン管理)(第1〜5回)／見積もり(第9回)
Webを支える技術(技術評論社)	電子版2,827円	該当章	http, 設計-詳細	第2部 URI、第3部 HTTP(第3・10回)／第5部 Web サービスの設計(第10回の API 設計)
達人に学ぶDB設計徹底指南書 第2版(翔泳社)	3,080円	該当章	db, 設計-詳細	3章 正規化、4章 ER図(第10回)／6章 インデックス設計、7章 論理設計のアンチパターン(第11回)
ソフトウェアアーキテクチャの基礎 第2版(オライリー・ジャパン)	未確認	発展	設計-構成	アーキテクチャ特性(非機能要件)の考え方(第9回)／アーキテクチャスタイルの比較は配属後の発展課題

Effective Python 第2版は Python 3.8 対応です。新しい型ヒントの書き方、パターンマッチ、uv・Ruff は、Python と uv の公式ドキュメントで補います(まとまった本が欲しい場合は、下の候補の『Python実践レシピ』)。

参考書（リファレンス、1万円以内）

書名	価格（税込）	発行	対応タグ	引きどころ
ソフトウェアエンジニアリングの基礎（オライリー・ジャパン）	4,180円	2026年9月	設計, test, 標準, pm	研修の軸として読み進める1冊。2章 コードを読む・3章 コードを書く（第5・7回）／4章 モデル＝どの図が必要か（第9〜10回）／5章 自動テスト（第8・11回）／6章 初めて触れるシステムの調査と変更（第1・12回）／8章 データの取り扱い（第10〜11回）／9章 ソフトウェアアーキテクチャ（第9〜10回）／10章 本番環境へ（第8回のCI）／12章 学び方を学ぶ（第1回）／13章 ソフトスキル（第12回）

書名	価格（税込）	発行	対応タグ	引きどころ
入門 モダンLinux(オライリー・ジャパン)	3,080円	2023年4月	linux, linux-ネットワーク	Linux の仕組みを理解する本。第1回はこの本の流れで全体を通して見る。1章 Linuxの入門・2章 Linuxカーネル(第1回)／3章 シェルとスクリプト、4章 アクセス制御(第1〜2回)／5章 ファイルシステム(第1回)／6章 アプリケーション、パッケージ管理、コンテナ(6.6 コンテナは第1回、パッケージは第2・4回)／7章 ネットワーク(第1〜2回)／9.1 プロセス間通信(シグナル、第1回)／付録A 便利なコマンド集(第1回、以降は引く)・付録B モダンLinuxツール(第1回)

書名	価格（税込）	発行	対応タグ	引きどころ
イラスト図解式 この一冊で全部わかるWeb技術の基本 第2版(SBクリエイティブ)	1,980円	2024年6月	linux-ネットワーク, http, 設計-構成, sec	見開き1テーマの図解で引くWebの入門書。2章 Webとネットワーク技術(第2回)／3章 HTTPでやりとりする仕組み(第3・10回)／4章 データ形式(HTML・DOM・JSON、第3・12回)／5章 Webアプリケーションの基本(Web API、第3回／3層構造、第10〜11回／Ajax、第12回)／6章 セキュリティと認証(プロキシは第2回、Cookie・セッション・CSRF・XSSは第3回)／7章 構築と運用(構成図、第10回)
合計	**9,240円**			予算の残りは 760円

無料の公式資料

資料	対応タグ	用途
Pro Git(日本語版)	git	Git 公式の解説書。仕組みからブランチ運用まで網羅しており、Git のリファレンスはこれで足りる
Python 公式ドキュメント	python	言語仕様と標準ライブラリの原典
PEP 8・Ruff・mypy 公式	python-可読性, test-静的解析	コーディング規約と、規約・型を自動で確かめる道具の一次資料(第７回)
MDN Web Docs	http, web-ブラウザ, sec	HTTP、HTML、Cookie、WebSocket、CORS のリファレンス。Web の基礎はここを一次資料にする(第３回のヘッダ・キャッシュ・Cookie・CORS・フェッチ API・DOM、第10・12回でも引く)
htmx 公式・Hypermedia Systems	web-ブラウザ	htmx の属性リファレンスと例集。Hypermedia Systems は htmx 作者らによる無料の書籍で、サーバが HTML を返す設計の考え方がわかる(第12回の講師の補足で紹介)
mise・uv 公式ドキュメント	env	まとまった書籍がないため、公式が主な資料
FastAPI 公式ドキュメント	web-サーバ	チュートリアルがそのまま第11回の教材になる
pytest 公式・GitHub Actions 公式	test-自動, test-CI	テストの書き方(fixture・parametrize)と CI の設定の一次資料(第８回)
Mermaid 公式・GitHub Docs(Issues)	設計-要件, pm-進捗	図の書き方(Syntax Reference)と、Issue・ラベル・マイルストーンの使い方の一次資料(第９回)

そのほかの候補(紹介用)

分野	書名	価格(税込)	向いている使い方
pm	ずっと受けたかったソフトウェアエンジニアリングの新人研修 第３版	2,200円	工程・見積・プロジェクト管理を体系的に学べる。プロジェクトマネジメントの副読本を足す場合に
http	Real World HTTP 第３版	未確認	HTTP の仕様と歴史を深く扱う中上級向け。『Web を支える技術』の先を知りたい人や講師用(第３回の先)
linux	新しい Linux の教科書 第２版(SB クリエイティブ、2024年４月)	3,300円	コマンドの使い方を１章１テーマで丁寧に追える入門書。非情報系でモダン Linux が難しい人や、シェルスクリプト・Git の基礎を手順で学びたい人に
python	改訂新版 Python 実践レシピ(技術評論社、2026年３月)	3,520円	Python 3.14 対応。uv、Ruff、型ヒント、pathlib などをライブラリ単位で引ける。第４・６〜７回を本で補いたい場合に

見積の本格的なリファレンス(マコネル『ソフトウェア見積り』など)は、受講者に配るより講師用として部署に１冊置く運用を勧めます。

全12回の一覧

所要は「座学・ハンズオン」などの配分で、合計60分にします。座学は冒頭にまとめて取り、ハンズオンの合間の短い座学はハンズオンの時間に含めます。

回	フェーズ	テーマ	所要	タグ	成果物
1	環境	知識体系の紹介＋Linux の全体像	座学15分 / ハンズオン45分	linux	—
2	環境	シェル・環境変数・通信の切り分け	座学15分 / ハンズオン45分	linux-シェル, linux-アプリ, linux-ネットワーク	環境設定手順書
3	Web	Web 通信の基礎（HTTP・Cookie・Web API）	座学15分 / ハンズオン45分	http, web-ブラウザ, sec	—
4	環境	実行環境の整備	座学15分 / ハンズオン45分	env-mise, env-python	mise.toml＋環境構築手順書（README）
5	構成管理	Git と GitHub	座学15分 / ハンズオン45分	git-基礎, git-github, 標準, test-レビュー	手順書のリポジトリ＋PR 2本（コンフリクト解決つき）
6	Python	Python の基本	座学15分 / ハンズオン45分	python-構文	TODO の CLI＋アプリのリポジトリ（期限の追加を PR でマージ）
7	Python	読みやすく壊れにくい Python	座学15分 / ハンズオン45分	python-oop, python-可読性, test-静的解析, 標準	`src/todo/` パッケージ＋ruff・mypy の設定と README（PR でマージ）
8	Python	テストと TDD	座学15分 / ハンズオン45分	test-自動, test-CI	テスト＋CI＋期限順の一覧（TDD の履歴が残る PR でマージ）
9	設計	要件と図	座学15分 / ハンズオン45分	設計-要件, pm-見積, pm-進捗	要件メモ（業務フロー図・状態遷移図）＋ストーリー S-01〜S-04（Issue とその控え、PR でマージ）
10	設計	設計書作成	座学15分 / ハンズオン45分	設計-詳細, 設計-構成, http-構成, db	API・DB設計書＋システム構成図（隣の人の質問を反映し、PR でマージ）
11	実装	TDD による Web バックエンド	座学15分 / ハンズオン45分	web-サーバ, db, test-自動	Web 版（FastAPI＋Jinja2＋SQLite）＋受け入れ条件 (1)〜(15) のテスト（TDD の履歴が残る PR でマージし、ストーリーの Issue を閉じる）
12	総括	画面の部分更新と成果発表	座学5分 / ハンズオン25分 / 発表準備5分 / 発表25分	web-ブラウザ, test-レビュー	fetch による部分更新（HTML 断片＋比較用の JSON 版）＋不具合票 BUG-001 とその修正＋発表資料（PR 3本でマージ）

各回の詳細

第１回　Linux の全体像

tag: `linux`　冒頭の座学で系統図と全体像の図を使って知識体系を紹介する。Linux は『入門 モダンLinux』の流れに沿って、全体を一度通して見る（各分野の深掘りは第２回以降）

- **狙い**：Linux がどんな部品でできているかの地図を持ち、黒い画面への抵抗をなくし、「何をしたか」を記録する習慣をつける
- **内容**：次の順に、参考書の節・短い説明・理解のためのコマンドの流れで進める
    1. 全体：カーネルとユーザー空間、WSL と Windows の関係（uname・/mnt/c）
    2. プロセス：プロセスと PID、シェルもプロセス、PID 1（init／systemd）、親子関係、ジョブとシグナル（ps・pstree・kill）
    3. ファイルシステム：ディレクトリの構成、すべてはファイル（/proc・/dev）、権限、標準入出力とリダイレクト・パイプ（ls・chmod・>・|・/proc/$$/fd）
    4. ネットワーク：TCP/IP の各層（リンク・インターネット・トランスポート・アプリケーション）と、各層を見るコマンド（ip・ping・ss・curl）
    5. ここまでで出なかったが押さえておくコマンド：grep・wc・sort・uniq・find・xargs・man
    6. コンテナ：namespace と cgroup、Docker の基本
    7. モダンなコマンド：ripgrep・fd・bat・fzf・jq など
- **演習**：ログファイル（講師配布）から grep と wc で特定のエラー件数を数える。振り返り・宿題・発展課題は設けない

第２回　シェル・環境変数・通信の切り分け

tag: `linux-シェル` `linux-アプリ` `linux-ネットワーク`　第１回で一度通したプロセス・ネットワークの層・DNS・curl -v は繰り返さず、シェル・パッケージ・通信の切り分けを深掘りする

- **狙い**：「コマンドが見つからない」「ネットにつながらない」を自力で切り分けられるようにする
- **内容**：次の順に、参考書の節・短い説明・理解のためのコマンドの流れで進める
    1. 変数：シェル変数と環境変数、export が効く範囲（子にはコピーが渡り、親は変わらない）
    2. PATH：コマンドを探す順番（エイリアス → 関数 → 組み込み → PATH）、type -a、自分のコマンドを PATH に足す、子のシェルで PATH を壊して戻す
    3. .bashrc：ログインシェルと対話シェルで読まれるファイル、コピーを取ってから追記し、わざと壊して直す
    4. apt と sudo：リポジトリ・apt update・install・dpkg -L、sudo で環境変数が消えること
    5. 通信の切り分け：curl の表示と終了コードで、名前解決（6）・接続（7・28）・TLS（60）・HTTP（404、-f で 22）のどこで止まったかを見分ける
    6. 社内で必要な設定としてのプロキシ：http_proxy／https_proxy／no_proxy（小文字と大文字）、http と https（CONNECT）の頼み方、apt の設定ファイル、社内CA証明書、git で確かめる
    7. 環境設定手順書を仕上げる：新しいターミナルで確かめ、元に戻す方法を書く

- **演習**：子のシェルと `.bashrc` で PATH を壊して直す／curl で各段階の失敗をわざと起こし、表示と終了コードを記録する。手順３から、打ったコマンドと確認方法をその場で環境設定手順書（第４回で隣の人が使う）に書く。振り返り・宿題・発展課題は設けない

第３回　Web 通信の基礎（HTTP・Cookie・Web API）

tag: `http` `web-ブラウザ` `sec`　第１回の `curl -v`（リクエスト行・ヘッダ・空行・本文）と第２回の切り分け・プロキシを前提に、届いた後のやり取りを深掘りする。mise・uv はまだないので、素材は Ubuntu の `python3` の標準ライブラリだけで動くデモのサーバ（`docs/session03/assets/web_demo.py`。このサイト 8000 と別のサイト 8001）にする

- **狙い**：ブラウザとサーバの間で何が起きているかを、ヘッダ・Cookie・Web API の実物を読んで説明できるようにする
- **内容**：次の順に、参考書・公式資料の節・短い説明・理解のためのコマンドの流れで進める
    1. デモのサーバとヘッダの中身：`curl -v` で Host・User-Agent・Accept・Cache-Control・Set-Cookie・Content-Type・Content-Length を読む、`curl -I`（HEAD）、`/headers` で送ったヘッダを見る
    2. Network タブ：document と stylesheet、再読み込みで `ETag` ／ `If-None-Match` の 304、ブラウザが自動で付けるヘッダ（Accept・Sec-Fetch-*・Cookie）
    3. Cookie とセッション：curl の `-c`・`-b`、Cookie の値を書き換えられること、ログイン（303 と Set-Cookie の属性 HttpOnly・SameSite・Max-Age）、Cookie の有無で変わる `/api/me`（401）、ログアウトでサーバのセッションを消す、ブラウザの「アプリケーション」と `document.cookie`
    4. CSRF：トークン付きのフォーム、`127.0.0.1:8001` の罠（同じサイト、403）と `localhost:8001` の罠（別のサイト、SameSite で Cookie が付かず 401）、オリジンとサイトの違い
    5. Web API と JSON：REST（URL はもの、メソッドはすること）、一覧・クエリ文字列・jq、POST の 201 と Location、PATCH・DELETE（204）、わざと失敗させる（415・400・422・405・404）、ETag の 304、公開 API（JSONPlaceholder）をプロキシ越しに呼ぶ
    6. CORS とプロキシ：別のオリジンからの fetch がブラウザで止まり、サーバには届いている（ログの Origin）、`Access-Control-Allow-Origin` で許可した API、curl では読める、`no_proxy` で 127.0.0.1 に直接つながる
    7. DOM・fetch・XSS をコンソールで：DOM の木、`textContent` の書き換え、fetch で追加して `<li>` を足す、`innerHTML` の XSS、エスケープしないページ（`/search-unsafe`）で `document.cookie` に HttpOnly の Cookie が出ないこと
- **演習**：デモのサーバと公開 API を相手に、上の流れでヘッダ・Cookie・JSON を観察する。成果物はない。振り返り・宿題・発展課題は設けない（プリフライト・GitHub の API の回数の制限は講師が見せる補足）

第４回　実行環境の整備（mise・uv）

tag: `env-mise` `env-python`　第２回の PATH・`.bashrc`・プロキシを前提に、Python とパッケージを OS から切り離してディレクトリごとに固定する。第５回でリポジトリにするファイル（mise.toml・pyproject.toml・uv.lock・main.py・環境構築手順書の README.md）は、すべてハンズオンの中で作る

- **狙い**：バージョン管理ツールで「誰でも同じ環境を再現できる」状態を作り、手順書だけで他人が再現できるかを確かめる
- **内容**：次の順に、参考書・公式資料の節・短い説明・理解のためのコマンドの流れで進める
    1. システムの Python：OS が使っていること、版が PC ごとに違うこと、`EXTERNALLY-MANAGED`（OS・ツール・パッケージの３つの層）
    2. mise を入れる：`.bashrc` のコピーを取って activate を書く。`cd` とプロンプトのたびに PATH が変わる仕組み。環境構築手順書のテンプレートをコピーし、ここから書きながら進める
    3. mise で固定する：`mise use --path .` で Python 3.12 と uv を `mise.toml` に書く、親の `mise.toml` との関係、`cd` で切り替わることを確かめる
    4. uv：`uv init`・`uv add`・`uv tree`、pyproject.toml と uv.lock と `.venv` の関係、`.venv` の中（mise の Python へのリンク）
    5. mise.toml の `[env]`（`UV_PYTHON_DOWNLOADS`・`.venv` の自動有効化）と `[tasks]`（setup・hello）。`requires-python` に上限を付け、`uv sync --locked` が止まるのを見て `uv lock` で直す
    6. 渡すものを決める：`.venv` を渡さない理由。別の場所にコピーし、`mise trust` → `mise install` → `mise run setup` → `mise run hello` で再現する
    7. 隣の人の手順書で再現する：第２回の環境設定手順書の「確認」で前提を見てから、README だけで再現し、止まった所を指摘し合って直す
- **演習**：手順２から、打ったコマンドと確認をその場で環境構築手順書（README.md）に書く。隣の人がその手順書だけで再現し、止まった所・推測した所を指摘し合う。振り返り・宿題・発展課題は設けない

第５回　Git と GitHub

tag: `git-基礎` `git-github` 標準（コミット規約・ブランチ運用） `test-レビュー`（セルフレビュー） 第４回のハンズオンで作った環境構築手順書の一式をリポジトリにし、Issue からマージまでを１人で２回通す。第６回以降は、ここで通したブランチ・PR・セルフレビュー・マージの流れを前提にする

- **狙い**：変更履歴で「何を・なぜ変えたか」を説明でき、main を壊さずに変更を入れる流れを一通り体験する
- **内容**：次の順に、参考書・公式資料の節・短い説明・理解のためのコマンドの流れで進める
    1. Git の設定：user.name・user.email・init.defaultBranch・core.autocrlf・core.quotepath・core.editor（`--global` と `.git/config` の違い）
    2. リポジトリを作る：第４回の一式をコピーし、`mise trust` と `mise run setup` で `.venv` を作ってから `git init`
    3. .gitignore を先に書く：uv の `.venv` が自分を無視していること、`.env` が消えるのを見て、`.gitignore` だけを最初にコミットする
    4. 目的ごとにコミットする：作業ツリー・ステージ・リポジトリ、add・diff --staged・commit・log、「１行目＝何を・本文＝なぜ」、git restore で変更を捨てる
    5. リモートに push する：空のリポジトリ、remote add・push -u、origin/main
    6. Issue を２つ立て、main から分けた２本のブランチで README の同じ行を変えて push する
    7. PR（目的 `Closes #番号`・変更点・確認方法）を２つ出し、セルフレビューして１本目をマージする。Issue が閉じ、２本目がコンフリクトになるのを見る
    8. ２本目のブランチに main を取り込んでコンフリクトを両方の目的が成り立つ形で解決し、再レビューしてマージする。pull・branch -d・log --graph で全体を確かめる

- **演習**：上の流れを各自のリポジトリで１人で通す。PR の本文とセルフレビューのコメントは研修の規約（docs/session05/assets/commit-convention.md）に沿って書く。振り返り・宿題・発展課題は設けない（revert・reset・rebase・ブランチ保護は講師が見せる補足）

第６回　Python の基本

tag: `python-構文`　雛形の関数を順に書いて１ファイルの TODO の CLI を作り、アプリのリポジトリ（`workspace/todo-app/`）を作って push する。期限の機能は第５回と同じ Issue → ブランチ → PR → マージの流れで入れる。第７回以降はこのリポジトリを前提にする

- **狙い**：小さなプログラムを自分で書き、型ヒント付きで読める Python に慣れる。アプリの変更を第５回の流れで入れ始める
- **内容**：次の順に、参考書・公式資料の節・短い説明・理解のためのコマンドやコードの流れで進める
  1. REPL：list と dict、`d["k"]` と `d.get("k")`（KeyError）、内包表記、`max(..., default=0)`、f-string
  2. アプリのリポジトリを作る：見本の `.gitignore`・`mise.toml`・`pyproject.toml`（`[tool.uv] package = false` の意味）・`uv.lock` をコピーし、`mise run setup` → `git init` → 設定をコミット
  3. 雛形を読む：型ヒントと型の別名、argparse のサブコマンド、`match` による振り分け、`if __name__ == "__main__"`、トレースバックの読み方
  4. 読み込みと保存：pathlib、`with`、`FileNotFoundError` だけを捕まえる、JSON と Python の値の対応（`ensure_ascii=False`）
  5. add・list・done：データの操作（値を返す）と表示（`main` の print）を分ける、for の変数はリストの中身そのもの
  6. エラーを確かめて push する：`raise` と `except`、標準エラー出力と終了コード（1 と argparse の 2）、空のリポジトリに push
  7. 期限を足す（Issue #1 → ブランチ）：`date` と文字列の変換（JSON に日付の型はない）、`argparse` の `type=`、期限切れの判定（`today` を引数で渡す）、`--overdue` を内包表記で、古いデータを `get` で読む
  8. PR（`Closes #1`）を出し、セルフレビューしてマージする
- **演習**：上の流れで CLI（add・list・done・`--due`・`list --overdue`）を作り、tasks.json に保存する。期限の追加は PR にしてマージする。振り返り・宿題・発展課題は設けない（EAFP と LBYL・ジェネレータは講師が見せる補足）

第７回　読みやすく壊れにくい Python

tag: `python-oop` `python-可読性` `test-静的解析` `標準`（コーディング規約）　第６回の `todo.py` を動きを変えずに `src/todo/` パッケージに分け、ruff と mypy で確かめる。整理は Issue #3 → ブランチ → 目的ごとの４つのコミット → PR → セルフレビュー → マージで入れる

- **狙い**:「動く」から「読みやすく壊れにくい」Python へ引き上げる。機械が確かめる規約と人が見る規約を分けてリポジトリに置く
- **内容**:次の順に、参考書・公式資料の節・短い説明・理解のためのコマンドやコードの流れで進める
    1. Issue とブランチ、パッケージに移す:`git mv todo.py src/todo/cli.py`、`pyproject.toml` の `package = false` を消して `[project.scripts]`・`[build-system]`・`module-name`、`uv sync`、`uv run todo`。移しただけのコミット
    2. models.py:`Enum`(Status)、`dataclass(frozen=True)`(Task、`status(today)`・`replace`)、`FrozenInstanceError`、入力チェック(`validate_title`・`parse_due`・`InvalidTaskError`、ガード節)
    3. store.py:`JsonStore` を読む(`_from_json` で古いデータも読める)、`typing.Protocol`(TaskStore)、`MemoryStore`、ダックタイピング
    4. cli.py を置き換える:モジュールの依存の向き、`main(argv, store, today)` で保存先と「今日」を差し替えられる形、一覧が第６回と同じことと、エラーの文言・終了コードの変化を確かめてコミット
    5. ruff と mypy:`uv add --dev`、キャッシュを `.gitignore` に、`[tool.ruff]`(ルールごとの理由のコメント)・`[tool.mypy]`(strict)、mise の `lint`・`fmt`・`typecheck`・`check`。属性の打ち間違い・Protocol に合わない名前の変更・未使用の import をわざと入れて指摘を見る
    6. README:開発環境・使い方・コードの構成・人が見るコーディング規約
    7. PR(`Closes #3`)を出し、変えたことと変えていないことを書いてセルフレビューし、マージする
- **演習**:上の流れで CLI をパッケージに分け、`mise run check` を通して PR でマージする。振り返り・宿題・発展課題は設けない(ABC と Protocol の比較、`__post_init__` で確かめる案、ルールの選び直しは講師が見せる補足)

第８回　テストと TDD

tag: `test-自動` `test-CI`　第７回の `src/todo/` と `main(argv, store, today)` を前提に、今の動作をテストで固定し、CI を置いてから「期限が近い順に並べる」をテスト先行で足す。Issue #5 → ブランチ → テスト・CI・レッドとグリーンのコミット → PR → CI が通ってからマージ、の流れで入れる

- **狙い**:テストで「動く」を確かめられる形にし、テストを先に書いてから実装する手順と、CI が通ってからマージする習慣を身につける
- **内容**:次の順に、参考書・公式資料の節・短い説明・理解のためのコマンドやコードの流れで進める
    1. Issue とブランチ、pytest を入れる:テストの種類(単体・結合・E2E)、`uv add --dev pytest`、`.pytest_cache/`、`testpaths`・テスト名の N802 を外す・mypy の `files` に `tests`、mise の `test` と `check`、テストが０件だと失敗すること
    2. assert と parametrize:状態のテストを組ごとに、`pytest.raises`、境目(期限の当日)をわざと壊して失敗の表示を読む
    3. fixture と tmp_path:`params` で同じテストを `MemoryStore` と `JsonStore` で動かす、`tmp_path` で手元の `tasks.json` に触れない
    4. CLI の動作を固定する:`main` に `MemoryStore` と固定の日付を渡す、`capsys`、`テスト:` のコミット
    5. CI を置いて push する:`.github/workflows/check.yml`(mise-action → `mise run setup` → `mise run check`)。GitLab の場合は GitLab CI に読み替える

6. TDD で `sort_by_due`：レッド（ImportError）を push して CI の赤も見る → 最小のグリーン（mypy だけが気づく）→ 期限なし → 同じ期限（三角測量）→ テストのリファクタ。テストと実装のコミットを分ける
7. TDD で一覧を期限が近い順に表示する（`test_cli.py` にレッド → `run_list` でグリーン）
8. README を更新し、PR（`Closes #5`）を出して Commits でレッド → グリーンの順を確かめ、CI が緑になってからマージする

- **演習**：上の流れを各自のリポジトリで通す。振り返り・宿題・発展課題は設けない（もう１機能を TDD で足す例・カバレッジ・レッドのコミットの取り出しは講師が見せる補足）

第９回　要件と図

tag: `設計-要件` `pm-見積` `pm-進捗`　Mermaid を第８回までの CLI の図で練習してから、講師の曖昧な依頼（「CLI の TODO を Web でも使いたい」）を要件メモとストーリーにする。Issue #7 → ブランチ → 自分の版と見本に合わせた版のコミット → ストーリーを Issue #8〜#11 に登録 → PR → マージの流れで入れる

- **狙い**：作る前に「何を作るか」を決め、図で考え・伝える習慣をつける
- **内容**：次の順に、参考書・公式資料の節・短い説明・理解のための作業の流れで進める
  1. Issue とブランチ：設計書の役割と合格基準（実装者が質問せずに作れるか）、第９回 → 第10回 → 第11回のつながり
  2. Mermaid の基本：`add` の流れを flowchart で、同じ `add` を sequenceDiagram で書いてプレビューする。図の種類と使い分け（業務フロー図・状態遷移図・シーケンス図・ER図・構成図・クラス図）
  3. 依頼を聞き、質問する：困りごと → 誰が → 何を → どのくらい（数字）→ やらないこと。答えを要件メモ（目的・非機能要件・制約・対象外・未定）に振り分ける
  4. 業務フロー図（`subgraph` のレーン）と状態遷移図（期限切れの条件を不等号まで）を書く
  5. ユーザーストーリーと受け入れ条件：したいことを S-01〜S-04 に分け、S-01 の１文と受け入れ条件（数字・文言・境目）を書く
  6. 見積：S-01 をタスクに分けて幅で見積もり、根拠を一言書く
  7. 自分の版をコミットし、見本（依頼人と合わせた版、受け入れ条件 (1)〜(15)）に置き換えて `git diff` で比べる
  8. Issue・ラベル・マイルストーン：ラベル `ストーリー`・`タスク`、マイルストーン `Web 版`、ストーリーを Issue に登録して絞り込む
  9. PR（`Closes #7`）を出し、GitHub の画面で図を確かめてマージする
- **演習**：上の流れを各自のリポジトリで通す。振り返り・宿題・発展課題は設けない（アーキテクチャ特性での見直し・Issue テンプレートの置き場所は講師が見せる補足）
- **PM観点**：見積の根拠を言語化させる。研修の進捗管理として見積と実績を記録・比較することはしない

第10回　設計書作成

tag: `設計-詳細`（API・DB） `設計-構成` `http-構成` `db`　第９回の受け入れ条件 (1)〜(15) と要件メモから、API 設計書・DB 設計書・システム構成図を書く。Issue #13 → ブランチ → 自分の版のコミット → 隣の人と交換して質問を反映 → 見本と合わせた版のコミット → PR → マージの流れで入れる

- **狙い**：実装前に合意するための「伝わる設計書」を書く
- **内容**：次の順に、参考書・公式資料の節・短い説明・理解のための作業の流れで進める
    1. Issue とブランチ、テンプレートを置く：第９回の文書 → ３つの設計書 → 第11回のテストのつながり
    2. API設計書の URL とステータスコード：URL は名詞・操作はメソッド、POST → 303 → GET（再読み込みで二重に追加しない）、リクエストとレスポンスの例、200・303・400・404 の選び方
    3. API設計書の入力チェックとシーケンス図：受け入れ条件のうまくいかない場合を文言そのままで表にし、追加のシーケンス図を `alt` で書く。画面設計書は作らず、返す HTML の要点とエラー表示欄を書く
    4. DB設計書：`Task` を `tasks` テーブルに写す（SQLite の日付と真偽値）、ER図とテーブル定義（NULL の意味）、状態は持たずに計算で求める条件（不等号まで）、並び順
    5. システム構成図：３層（ブラウザ・アプリ・DB）、uvicorn の待ち受けとポート、バージョンを固定する場所、環境変数 `TODO_DB_PATH`。自分の版をコミットする
    6. 交換して確かめる：隣の人の設計書を、書き手に聞かずに読み、実装者の問いに設計書だけで答える。答えられない点を相手の API設計書の「設計レビューで出た質問」に書き、書き手は本文に反映する（書かないなら理由）
    7. 見本と合わせる：見本（第11回が前提にする URL・文言・テーブル）に置き換えて `git diff` で比べ、README とストーリーの控えと一緒にコミットする
    8. PR（`Closes #13`）を出し、GitHub の画面で図を確かめてマージする
- **演習**：上の流れを各自のリポジトリで通す。振り返り・宿題・発展課題は設けない（HTML 断片の URL を足す例・タグの多対多の ER図・完了済みをもう一度完了にしたときの扱いは講師が見せる補足）
- **図**：URL の木、POST → 303 → GET とフォーム送信のシーケンス図、ステータスコードの選び方、ER図、状態を計算で求める状態遷移図、３層の構成図、環境変数の流れ
- **SE観点**：「実装者が質問せずに作れるか」を合格基準にする。出た質問には口で答えず、設計書に書き足す

第11回　TDD による Web バックエンド

tag: `web-サーバ` `db` `test-自動`　第10回の設計書と第９回の受け入れ条件（1）〜（15）から、テストを先に書いて Web 版を作る。第７回の `TaskStore`（Protocol）に SQLite 版を足して差し替える。Issue #15 → ブランチ → レッドとグリーンのコミット → PR（`Closes #15`・`#8` 〜 `#11`）→ CI が通ってからマージ、の流れで入れる

- **狙い**：設計書の仕様を「通るべきテスト」に書き換えてから、Web の画面を実装する
- **内容**：次の順に、参考書・公式資料の節・短い説明・理解のためのコマンドやコードの流れで進める
    1. Issue とブランチ、依存パッケージ：３層に FastAPI・uvicorn・Jinja2・python-multipart を当てはめる、テスト用の httpx2、`mise.toml` の `TODO_DB_PATH` と `dev`・`db-reset`、`.gitignore` に `var/`
    2. SqliteStore：保存先のテストの `params` に `"sqlite"` を足して失敗を見る → 見本の `SqliteStore`（`open`・`close`、`with conn:` のコミット、`?` のプレースホルダ、日付と真偽値の変換）で通す
    3. レッド：追加のテスト。fixture で `get_store`・`get_today` を `dependency_overrides` で `tmp_path` の DB と固定日に差し替える（依存性の注入）

4. グリーン：`web.py`（`Depends` と `yield` の依存関係、`render_index`）とテンプレート（Jinja2 の継承と自動エスケープ）、最小の `add_task` と 303
5. 入力エラー：空のタイトルのテストで 422（FastAPI の検証）と例外を見る → 第７回の `validate_title`・`parse_due` の例外を 400 の一覧ページにする
6. 完了と 404：303 と、ないタスクの 404 のテスト（文言まで確かめる）→ `mark_done`
7. 残りの受け入れ条件のテストを見本からコピーして読み、わざと壊して（並び順・`| safe`）落ちるのを見る
8. `mise run dev` でブラウザから操作し、curl で 303・400・404 を見る
9. 設計書と見比べ、README と受け入れ条件の控えをそろえて PR を出し、CI が緑になってからマージして Issue #8〜#11・#15 を閉じる

- **演習**：上の流れを各自のリポジトリで通す。手で書くテストは追加・入力エラー・完了・404 で、残りは見本からコピーして読む。振り返り・宿題・発展課題は設けない（Pydantic のフォームモデル・`render_index` へのまとめ方・境目のテストの足し方は講師が見せる補足）
- **図**：層の構成、保存先のクラス図、依存性の注入の差し替え、fixture とリクエストの流れのシーケンス図、422 と 400 の分かれ道、完了の 404 のシーケンス図、TDD の履歴の gitGraph。実装した機能の流れを API 設計書のシーケンス図と突き合わせる
- **PM観点**：PR に `Closes #番号` を１行ずつ書き、マージでストーリーの Issue を閉じる（マイルストーン `Web 版` が進む）

第12回　画面の部分更新と成果発表

tag: `web-ブラウザ` `test-レビュー`（不具合票） 第３回で触った fetch と DOM で、第11回の Web 版の一覧を部分更新にする。Web の基礎（Network タブ・Cookie・CORS・XSS）は第３回で済んでいるので繰り返さず、テストとコードは素材から足して読み、ブラウザで動きを確かめることに時間を使う。後半は１人ずつ発表する。Issue #17 → PR #18（部分更新）、Issue #19 → PR #20（不具合）、Issue #21 → PR #22（発表資料）の順に入れる

- **狙い**：サーバが HTML 断片を返し、JavaScript は差し替えるだけという作りで画面を部分更新にする。見つけた不具合を票とテストで直す。作ったものを根拠を添えて発表する
- **内容**：次の順に、参考書・公式資料の節・短い説明・理解のためのコマンドやコードの流れで進める
    1. 断片の URL と app.js：素材のテスト（断片・`data-fragment`・app.js の読み込み）を足してレッド → 素材のテンプレート（一覧を `_task_list.html` に切り出す）・ルート・素朴な `app.js` を足してグリーン
    2. イベントの委譲：素朴な版では、差し替えた一覧の「完了」がふつうの送信（POST 303）になるのを Network で見る → 見本の `app.js`（`document` に１つだけイベントを付ける）に替える
    3. JSON 版との比較：素材のテストとルートで完了の JSON 版を足し、見本の `done-json.js` に差し替えて「期限切れだけ」の表示で比べる（返すもの・組み立てる場所・テスト）
    4. 設計書（API設計書・システム構成図 1.1）を見本にそろえ、PR #18 でマージ
    5. 受け入れ条件（15）を部分更新で試し直して不具合を見つけ、Issue #19（ラベル `不具合`）→ 不具合票 BUG-001 → 再現テスト（レッド）→ 修正（グリーン）→ PR #20 でマージ
    6. 発表準備：素材の発表資料の型（何を作ったか → 構成 → デモ手順 → 工夫１つ）を埋め、PR の数・レッドのコミット数・テストの件数を根拠に添える。デモを１回通し、PR #22 でマージ

7. 発表：1人ずつ発表とデモ、聞き手は良かった点と質問を1つずつ。持ち時間は人数から講師が決める

- **演習**：上の流れを各自のリポジトリで通す。手で書くのは不具合票・再現テスト・修正と、発表資料の工夫の節。振り返り・宿題・発展課題は設けない（スクラムと KPT は扱わない。ポーリング・SSE・WebSocket・htmx は講師が見せる補足）
- **図**：ページと断片のテンプレートの関係、fetch で差し替えるシーケンス図、イベントの委譲、JSON 版のシーケンス図、不具合が出る画面の状態遷移図、不具合を直す流れ、TDD の履歴の gitGraph
- **SE観点**：不具合票の書き方（再現手順・期待結果と根拠・実際の結果）。発表では主張に根拠（テスト・PR・設計書・不具合票）を付ける

事前準備・運営のコツ・評価

最大のリスクは第1・2・4回の環境構築で詰まることです。社内ネットワークと権限まわりは研修開始前に講師が一度通しで確認してください。

事前準備

- [ ] 受講者PCでWSLを有効化できるか（管理者権限・情報システム部門の許可）
- [ ] 社内プロキシのアドレス、認証の有無、社内CA証明書の入手方法
- [ ] GitHub／GitLab など利用するGitホスティングとアカウント発行
- [ ] mise・uv・PyPI・npm へのアクセスがプロキシ越しに通るかの確認
- [ ] 講師用の模範の TODO アプリ（各回終了時点の状態を用意しておくと、遅れた人が追いつける）

運営のコツ

- 詰まった人は「30分調べて解決しなければ質問する」ルールにし、質問の書き方（何をして・何を期待し・何が起きたか）も教える
- 実装が遅れても設計書は必ず全員がその回のうちに書く。研修の本題はそちら側

評価の観点

観点	見るもの
構成管理	コミットの粒度とメッセージ、PRの説明とセルフレビュー
PM	ストーリーを Issue に分けて登録し、見積の根拠を説明できるか
SE	設計書が第三者に伝わるか、不具合票の質
技術	動くこと、テストがあること、ruff を通っていること
再現性	手順書だけで他人が環境を再現できるか

第1回 知識体系の紹介＋Linux の全体像

tag: linux
前提タグ: なし
所要: 座学15分 / ハンズオン45分

ゴール

- 研修の知識の全体地図（4つの大分野とタグ）を知り、どの回で何を扱うかを言える
- Linux の部品（カーネル・プロセス・ファイルシステム・ネットワーク）を、それぞれ見るコマンドと一緒に言える
- `grep`・`wc` で、ログから特定のエラーの件数を数えられる

事前準備

- WSL と Ubuntu が使えること（講師から案内。open-questions.md の Q2）
- 研修ディレクトリ一式が `~/handsoff` にあること

まとめ

冒頭の座学で使います。ハンズオンは下の流れの順に、1ステップずつ参考書の節を開きながら進めます。

研修の地図

項目	内容
系統図	研修で扱う知識の木（knowledge-map）→ 回ごとの流れ（全体像）
大分野	コンピュータ基盤／実装技術／ソフトウェア開発／その他（タグ一覧）
`tag:`	その回で扱う知識。リンク先のタグ索引に扱う回と読む本
`前提タグ:`	その回の前に身につけておきたい知識
評価の中心	Git 運用・設計書・テストのある実装。Web 開発は練習の題材

Linux の全体像（この回の流れ）

#	分野	ひとことで	参照（モダンLinux）	深掘りする回
1	全体	ハードウェアの上にカーネル、その上にユーザー空間（シェル・アプリ）	1.6、2.1、2.3	—
2	プロセス	動いているプログラム。番号（PID）と親子関係を持つ。PID 1 が全部の先祖	2.3.1、3.1.2、6.2、9.1.1	第２回
3	ファイルシステム	ディレクトリの木。デバイスやプロセスの情報もファイルとして見える	5.1〜5.3、4.3.1	第２回
4	ネットワーク	層の積み重ね（リンク → インターネット → トランスポート → アプリケーション）	7.1〜7.4	第２回、第３回
5	押さえておくコマンド	探す・数える・並べる・調べる	付録A	毎回
6	コンテナ	namespace と cgroup でほかから隔離したプロセス。Docker はコンテナを扱う道具	6.6	—
7	モダンなコマンド	昔からのコマンドの使いやすい代替	3.1.3、付録B	—

WSL とファイルの置き場所

場所	見え方	使い方
Linux のホーム	`~` = `/home/<ユーザー名>`	研修のファイルはここ
Windows の C ドライブ	`/mnt/c`	遅く、権限の扱いも違うので作業しない
Linux 側を Windows から	`\\wsl.localhost\Ubuntu\...`	`explorer.exe .` で開く

ハンズオン

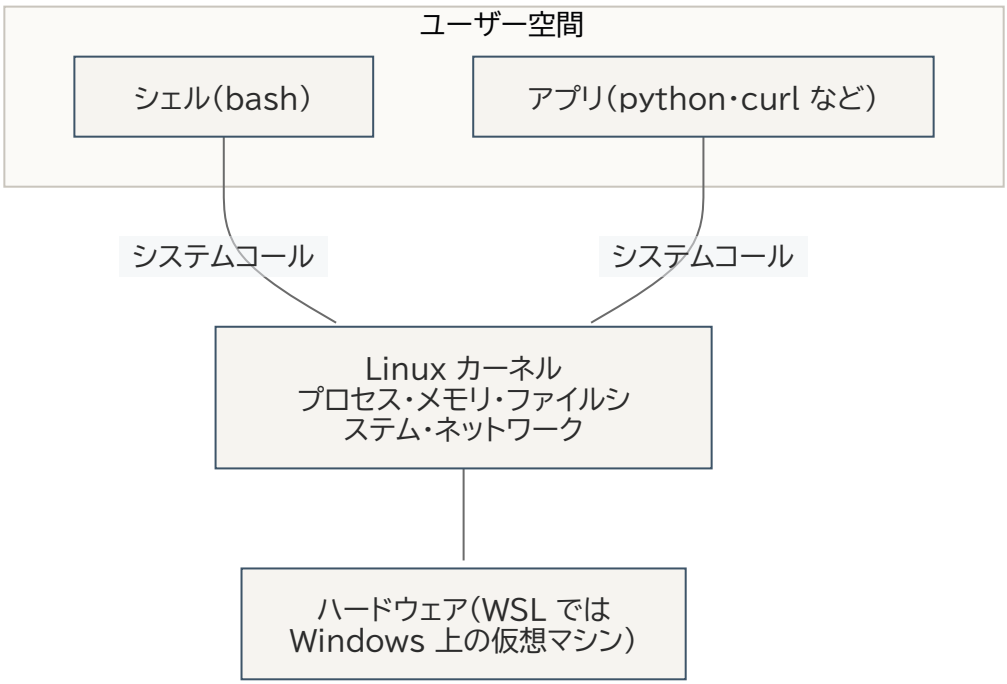
1ステップ1コマンド。各ステップは「🖋 やること → 📚 参考書の節 → 📒 説明 → コマンド」。

1. 全体: カーネルとユーザー空間（4分）

> **🖋 カーネルのバージョンを見る**

📚 モダンLinux 1.6 Linuxの全体像、2.1 Linuxアーキテクチャ

📒 カーネルはハードウェアを管理する。シェルやアプリ（ユーザー空間）は、システムコールでカーネルの機能を使う。WSL は Windows の上の仮想マシンで、本物の Linux カーネルを動かしている




```
uname -a        # microsoft-standard-WSL2 と出る
```

🖊 ディストリビューションを見る

📚 モダンLinux 1.4 Linuxディストリビューション

📒 カーネルに道具一式（シェル・パッケージ管理など）をまとめたものがディストリビューション。研修では Ubuntu を使う

```
cat /etc/os-release
```

🖊 カーネルが管理している資源を見る

📚 モダンLinux 2.3 カーネルコンポーネント、1.5 リソースの可視性

```
nproc; free -h; df -h /     # CPU の数・メモリ・ディスク
```

🖊 Windows のファイルを Linux から見る

📒 Windows の C ドライブは `/mnt/c` に見える。`/mnt/c` は遅く、権限の扱いも違う。そのため研修の作業は Linux 側の `~` で行う

```
ls /mnt/c/Users
explorer.exe .  # Linux のホームを Windows のエクスプローラーで開く
```

- 確認: アドレス欄に `\\wsl.localhost\Ubuntu\home\<ユーザー名>`

2. プロセス: シェルもプロセス、PID 1 が先祖（6 分）

🖊 PID を確認する

📚 モダンLinux 2.3.1 プロセス管理、3.1.2 シェル

📒 動いているプログラムがプロセス。コマンドを打っているシェル（bash）もプロセスで、PID という番号を持つ

```
echo $$
```

🖊 PID と PPID を見比べる

📚 モダンLinux 2.3.1 プロセス管理

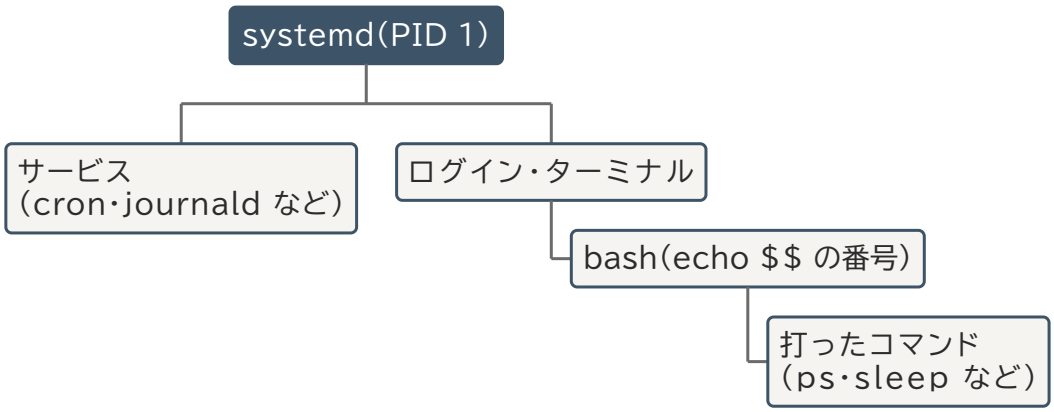
📒 コマンドはシェルの子プロセスとして動く。PPID は親の PID

```
ps -f           # ps の PPID が、echo $$ の番号と同じ
```

🖊 PID 1 を確認する

📚 モダンLinux 6.2 Linuxの起動プロセス、6.3 systemd

📒 PID 1 は起動時に最初に動くプロセス（systemd または init）。ほかの全プロセスはこの子孫




```
ps -p 1 -o pid,comm
```

🖊 親子関係をたどる

📚 モダンLinux 6.2 Linuxの起動プロセス

```
pstree -p | head -20    # PID 1 から自分の bash までたどる
```

🖊 裏でプロセスを動かす

📒 コマンドの後に `&` を付けると、シェルはコマンドの終了を待たずに次の入力を受け付ける(ジョブ)

```
sleep 300 &
jobs; ps -f     # sleep の PPID が自分の bash の PID
```

🖊 シグナルで止める

📚 モダンLinux 9.1.1 シグナル

📒 `kill` は「殺す」ではなく「シグナルを送る」コマンド。既定で送るのは SIGTERM(終了を求めるシグナル)

```
kill %1; jobs   # Terminated と出る
```

🖊 全体を眺める

📚 モダンLinux 付録A.2 ユーザとプロセス

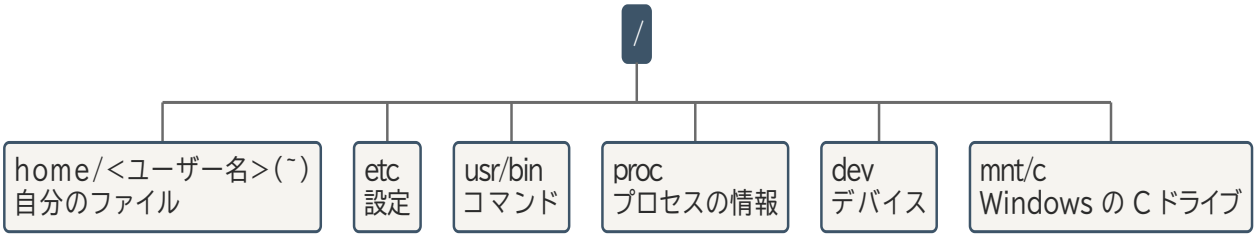
```
top             # q で終了
```

3. ファイルシステム: すべてはファイル(7分)

🖊 ルートの下を見る

📚 モダンLinux 5.1 基本、5.2.3 一般的なファイルシステムレイアウト

📒 ディレクトリは `/` を根とする1つの木構造。`/proc` と `/dev` の下は、ディスク上にない擬似ファイル




```
ls /
```

🖊 マウントを見る

📚 モダンLinux 5.2.2 ファイルシステムの操作

📒 ディスクやデバイスは、木構造の途中のディレクトリにマウント(接続)される

```
findmnt | head  # / と /mnt/c がどこにつながっているか
```

🖊 プロセスの情報をファイルで見る

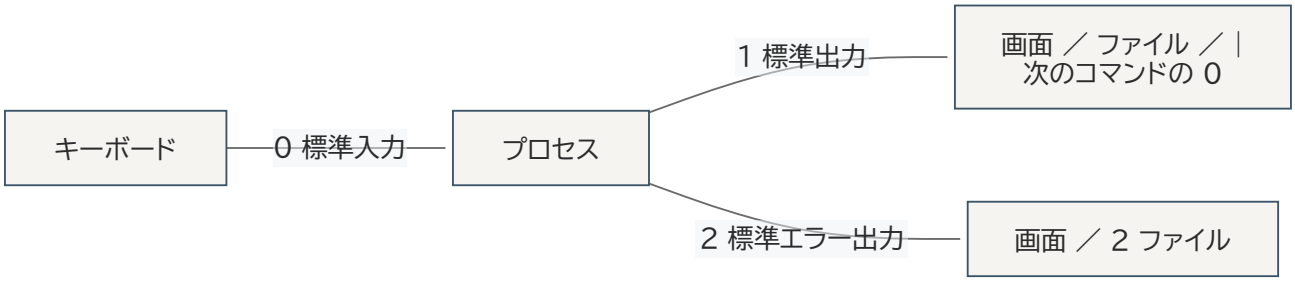
📚 モダンLinux 5.3.1 procfs

```
ls /proc/$$     # 自分の bash の情報がファイルとして並ぶ
```

🖊 標準入出力を見る

📚 モダンLinux 3.1.2 シェル、付録A.5 リダイレクトとパイプ

📒 プロセスは最初から 0(標準入力)・1(標準出力)・2(標準エラー出力)を開いている。リダイレクトとパイプは、この 0・1・2 のつなぎ先を変える




```
ls -l /proc/$$/fd   # 0 1 2 が端末（/dev/pts/...）につながっている
```

🖊 作業場所を作り、ログをコピーする

📚 モダンLinux 付録A.4 ファイルとディレクトリ

```
cd ~/handsoff/workspace/session01
mkdir logs
cp ../../docs/session01/assets/logs/*.log logs/
ls -l logs      # todo-app-2026-04-01.log ～ 04-07.log の7ファイル
```

🖊 出力をファイルにする

📚 モダンLinux 付録A.5 リダイレクトとパイプ

📒 `>` は上書き、`>>` は追記

```
echo hello > memo.md; echo world >> memo.md; cat memo.md
```

🖊 エラーだけをファイルにする

📒 エラーは 2 番(標準エラー出力)から出る。`>` は 1 番だけをファイルにつなぐので、エラーは捕まらない

```
ls /nothing 2> err.txt; cat err.txt
```

🖊 権限を読む

📚 モダンLinux 4.3.1 ファイルのパーミッション

📒 `-rw-r--r--` = 種類(`-` ファイル、`d` ディレクトリ)+ 所有者・グループ・その他 ×(r 読む・w 書く・x 実行)

```
ls -l memo.md
```

🖊 権限を変える

📒 `644` は r=4, w=2, x=1 の和(所有者 6=読み書き、ほか 4=読むだけ)

```
chmod u-r memo.md; cat memo.md   # Permission denied
chmod 644 memo.md; ls -l memo.md
```

4. ネットワーク: 層ごとに見る(10 分)

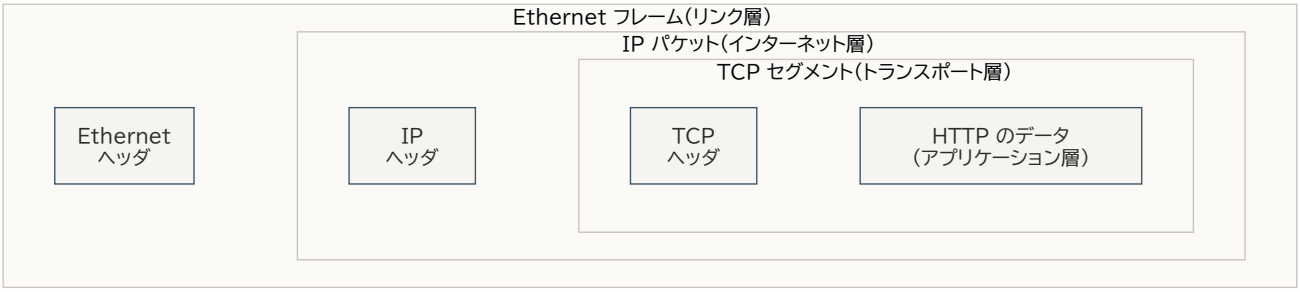
🖊 インタフェースを見る(リンク層: Ethernet)

📚 モダンLinux 7.1 基本、7.2 TCP/IPスタック、7.2.1 リンク層

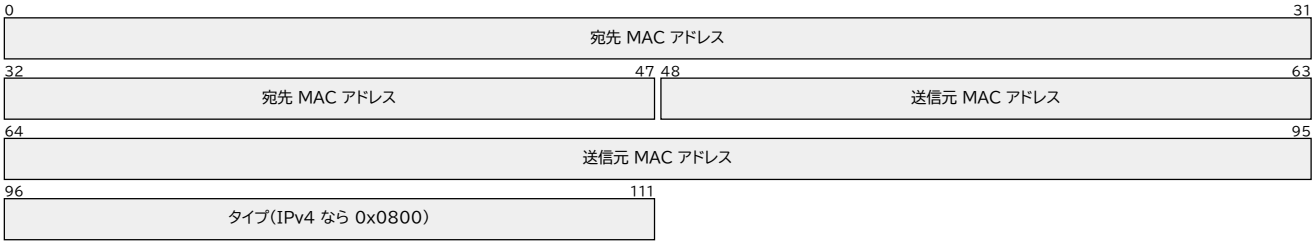
📒 通信は4つの層に分かれ、層ごとに役目が決まっている。つながらないときは下の層から確かめる

層	やること	ヘッダに入る主なもの	見るコマンド
アプリケーション(HTTP など)	アプリどうしが決まった手順(プロトコル)でやり取りする	(HTTP なら)メソッド・URL・ヘッダ	`curl -v`、`getent hosts`
トランスポート(TCP・UDP)	どのアプリに渡すかを決め、TCP なら届いたかを確かめて順番をそろえる	送信元・宛先のポート、順番の番号	`ss -tln`
インターネット(IP)	宛先の IP アドレスまで、ルーターを経由して運ぶ	送信元・宛先の IP アドレス、TTL	`ip addr`、`ip route`、`ping`
リンク(Ethernet・Wi-Fi)	同じネットワークの隣の機器まで運ぶ	送信元・宛先の MAC アドレス	`ip link`

📒 送るときは上の層から順にヘッダを付ける。ヘッダは入れ子になり、受け取った側は外側(下の層)から順に外す



📒 いちばん外側の Ethernet ヘッダ（14 バイト）。同じネットワークの中で、どの機器に渡すかを MAC アドレスで示す



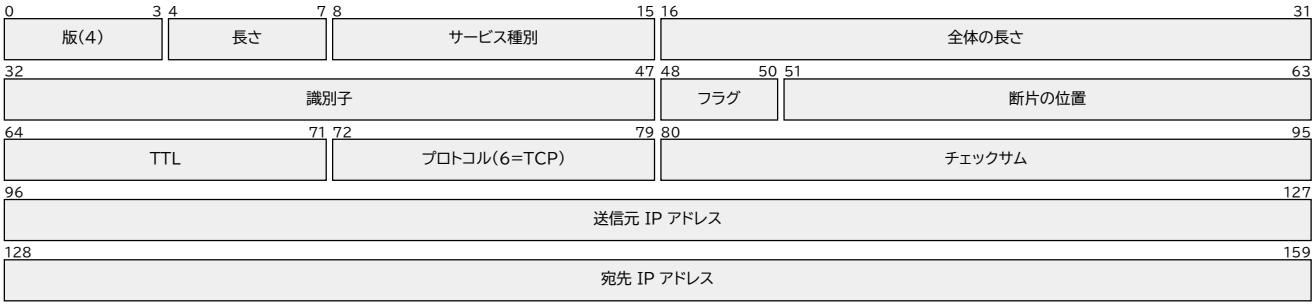
Ethernet ヘッダ（14バイト）

```
ip link          # link/ether の後ろが自分の MAC アドレス
```

🔗 IP アドレスと経路を見る（インターネット層: IP）

📚 モダンLinux 7.2.2 インターネット層

📒 IP ヘッダ（20 バイト＋オプション）。IP パケットは、ルーターが宛先の IP アドレスまで運ぶ。TTL はルーターを 1 つ通るたびに 1 減り、0 になると捨てられる。プロトコルは中身が TCP（6）か UDP（17）かを示す



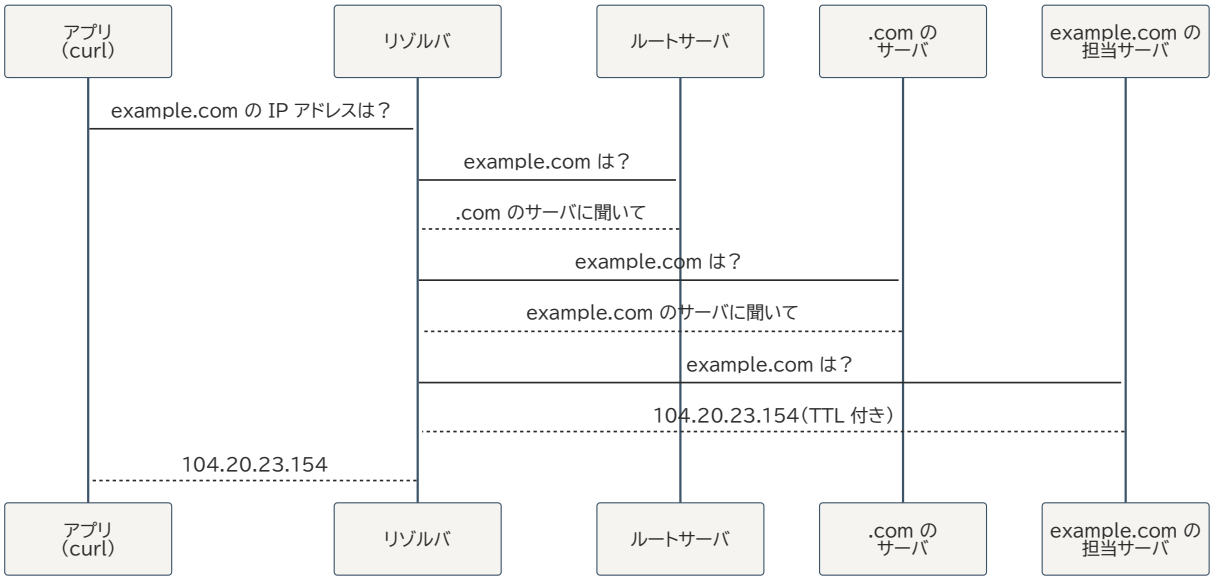
IPv4 ヘッダ（主な部分）

```
ip -br addr; ip route    # 自分の IP アドレスと、外への出口（default）
```

🔗 DNS の仕組みを知る

📚 モダンLinux 7.3 DNS、7.3.2 DNS ルックアップ

📒 DNS は名前（example.com）を IP アドレスに変える仕組み。アプリは近くのリゾルバに問い合わせる。リゾルバは答えを知らなければ、ルート → .com → example.com の担当サーバの順にたどって調べる。リゾルバは調べた答えを TTL の秒数だけ保存し（キャッシュ）、次からはすぐ返す




```
cat /etc/resolv.conf    # nameserver の行が、図のリゾルバ
```

🔗 hosts ファイルを見る

📒 `/etc/hosts` に書いた名前は DNS より先に使われる。どの順に調べるかは `/etc/nsswitch.conf` の hosts の行で決まる

```
cat /etc/hosts
grep hosts /etc/nsswitch.conf   # files (/etc/hosts) → dns の順
```

🔗 OS の仕組みで名前を引く

📒 `getent hosts` は、アプリと同じ順(hosts → DNS)で名前を引く

```
getent hosts example.com
```

🔗 dig を入れる

📒 `dig` は DNS サーバに直接問い合わせて、答えをそのまま見せる道具。`/etc/hosts` は見ない。入れられないときは、最初から入っている `resolvectl` を代わりに使う(下の各ステップに併記)

```
sudo apt install -y bind9-dnsutils   # 社内ではプロキシの設定が要る（第2回）
```

🔗 dig で IP アドレスを引く

📚 モダンLinux 7.3.1 DNSレコード

📒 DNS の答えはレコードという単位で返る。`dig` の出力では、QUESTION(何を聞いたか)、ANSWER(答え)、最後の SERVER(どのリゾルバが答えたか)を見る

レコード	中身
A	IPv4 アドレス
AAAA	IPv6 アドレス
CNAME	別名(本当の名前を指す)
NS	そのドメインを担当する DNS サーバ
MX	メールの受け取り先
TXT	文字列(ドメインの持ち主の確認などに使う)

```
dig example.com
# ;; ANSWER SECTION:
# example.com.   300   IN   A   104.20.23.154
#   名前           TTL(秒)    種類  答え
# ;; SERVER: 10.255.255.254#53   ← 答えたリゾルバ
dig +short example.com          # 答えだけ表示する
resolvectl query example.com    # dig が使えないとき（A と AAAA をまとめて表示）
getent ahosts example.com       # resolvectl も使えないとき（WSL で systemd が無効）
```

🔗 ほかのレコードを引く

```
dig +short AAAA example.com
dig +short NS example.com
dig +short MX gmail.com
resolvectl query -t NS example.com   # dig が使えないとき（-t でレコードの種類を指定）
resolvectl query -t MX gmail.com
```

🔗 ルートからたどる様子を見る

📒 `+trace` を付けると、リゾルバに任せずに、ルート → .com → 担当サーバの順に自分でたどる。社内では外の DNS サーバに直接問い合わせられず、失敗することがある。`resolvectl` に同じ機能はないので、dig が使えないときは講師の画面で見る

```
dig +trace example.com
```

🔗 届くか確かめる

📚 モダンLinux 7.2.2 インターネット層

```
ping -c 3 example.com   # 社内では止められていることがある
```

🔗 サーバを立てる

📚 モダンLinux 7.4.1 ウェブ

```
python3 -m http.server 8000 &
```

🔗 待ち受けているポートを見る(トランスポート層: TCP)

📚 モダンLinux 7.2.3 トランスポート層、7.2.4 ソケット

📒 TCP ヘッダ(20バイト+オプション)。宛先ポートで、どのアプリ(ここでは 8000 番の http.server)に渡すかが決まる。シーケンス番号と確認応答番号で、届いたことを確かめて順番をそろえる



TCP ヘッダ

```
ss -tln | grep 8000
```

🔗 HTTP のやり取りを見る(アプリケーション層: HTTP)

📚 モダンLinux 7.4.1 ウェブ

📒 いちばん内側の HTTP。HTTP はビット単位ではなく、文字の行で書く。リクエストは「リクエスト行 → ヘッダ（名前: 値）→ 空行 → 本文」の順に並び、TCP セグメントのデータ部分に入る。レスポンスは 1 行目がステータス行になり、あとは同じ形

```
GET / HTTP/1.1                          ← リクエスト行（メソッド・パス・版）
Host: 127.0.0.1:8000                    ← ヘッダ（名前: 値）
User-Agent: curl/8.5.0
Accept: */*
                                        ← 空行でヘッダが終わる（GET には本文がない）

HTTP/1.0 200 OK                         ← ステータス行（版・ステータスコード）
Server: SimpleHTTP/0.6 Python/3.12      ← ヘッダ
Content-type: text/html; charset=utf-8
Content-Length: 389
                                        ← 空行
<!DOCTYPE HTML>                         ← 本文（ファイル一覧の HTML）
<html lang="en">
...
```

📒 `curl -v` の出力では、`>` の行が送ったリクエスト、`<` の行が受け取ったレスポンス。ヘッダの意味は第 3 回で読む

```
curl -v http://127.0.0.1:8000/ | head
kill %1          # サーバを止める
```

5. 押さえておくコマンド: ログを数える（8 分）

🖋 ログの先頭を見る

📚 モダンLinux 付録A.3 ファイル情報

📒 ログの 1 行は `日付 時刻 レベル [部品] メッセージ`。レベルは `INFO` 通常／`WARN` 注意／`ERROR` 失敗

```
head logs/todo-app-2026-04-01.log
```

🖋 検索しながら読む

```
less logs/todo-app-2026-04-01.log    # /ERROR で検索 → n で次 → q で終了
```

🖊 語を含む行を数える

📚 モダンLinux 付録A.5 リダイレクトとパイプ

📒 `grep` で行を選び、パイプ(`|`)で `wc -l` に渡して数える

```
grep ERROR logs/todo-app-2026-04-01.log | wc -l
```

🖊 grep だけで数える

```
grep -c ERROR logs/todo-app-2026-04-01.log
```

🖊 使い方を調べる

📒 迷ったら `--help` か `man`。`grep` の `-i`(大小文字を区別しない)・`-h`(ファイル名を付けない)もここで確かめる

```
grep --help | head -30
```

🖊 演習: 数える

答えと使ったコマンドを両方記録する。

1. 2026-04-03 の `ERROR` の行数
2. 7ファイル合計の `ERROR` の行数(ヒント: `cat logs/*.log | grep ...`)
3. 2 を `grep -i error` にすると件数はどう変わるか。理由を実際の行で説明
4. 1週間の `ERROR` のうち `database is locked` の行数。`grep 'database is locked'` だけの件数とも比べる

> 座学: 数え方の条件で答えが変わる。仕事でログを数えるときに気をつけること

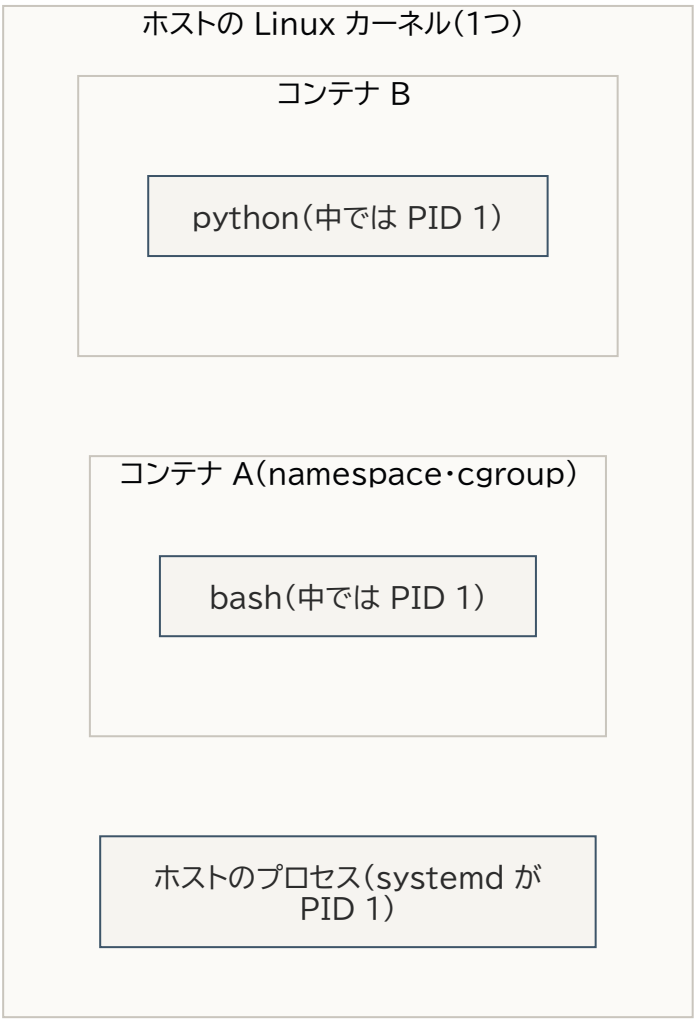
6. コンテナ: 隔離されたプロセス(2分・講師の画面)

受講者 PC で Docker が使えるかは未確定(open-questions.md の Q12)。使えない場合は講師の画面で見る。

🔗 コンテナを動かす

📚 モダンLinux 6.6 コンテナ、6.6.4 Docker

📒 コンテナは仮想マシンではなく、namespace（見える範囲）と cgroup（使える資源）でほかから隔離したプロセス。カーネルはホストと共有する




```
docker run --rm hello-world
```

🔗 コンテナの中の PID 1 を見る

📚 モダンLinux 6.6.1 namespace

```
docker run --rm -it ubuntu:24.04 bash
ps -ef          # 中では bash が PID 1（手順2の PID 1 と比べる）。exit で出る
```

🔗 コンテナとイメージを一覧する

```
docker ps -a; docker images
```

7. モダンなコマンド（2分・講師の画面）

🔗 モダンなコマンドを入れる

📚 モダンLinux 3.1.3 モダンなコマンド、付録B モダンLinuxツール

昔からのコマンド	モダンな代替	良いところ
`grep -r`	`rg`（ripgrep）	速い、.gitignore を見て余計なファイルを飛ばす
`find`	`fd`（Ubuntu では `fdfind`）	書き方が短い
`cat`	`bat`（Ubuntu では `batcat`）	色付き・行番号つき
`history | grep`	`fzf`	絞り込みながら選べる
（なし）	`jq`	JSON を整形・抽出
`top`	`htop`	見やすく操作できる

```
sudo apt install -y ripgrep fd-find bat fzf jq htop    # 社内ではプロキシの設定が要る（第2回）
```

🔗 ripgrep で数える

```
rg -c ERROR logs/
```

🔗 JSON を整形する

```
echo '{"task":"牛乳を買う","done":false}' | jq .
```

読む章

区分	資料	範囲	読み方
参考書	入門 モダンLinux(モダンLinux)	1章、2章、5章、6.6、7章、付録B	該当章(ハンズオンの「参照」の節を中心に)

区分	資料	範囲	読み方
参考書	同上	3章、4章、付録A	該当章（第2回と合わせて。付録Aは引く）
参考書	ソフトウェアエンジニアリングの基礎（SE基礎）	12章 学び方を学ぶ	通読（研修の軸。第12回まで回ごとに読み進める）
参考書	同上	6章 初めて触れるシステムの調査と変更	該当章（第12回でも使う）
紹介（副読本）	達人プログラマー 第2版	基本ツール（シェル、プレーンテキスト）	持っている人は通読（第1〜5回で基本ツールの範囲）

第２回 シェル・環境変数・通信の切り分け

tag: linux-シェル, linux-アプリ, linux-ネットワーク
前提タグ: linux
所要: 座学15分 / ハンズオン45分

ゴール

- シェル変数と環境変数の違い、`export` が効く範囲を説明できる
- `command not found` のとき、PATH とコマンドを探す順番から原因を見つけて直せる
- `.bashrc` がいつ読まれるかを説明し、コピーを取ってから書き換えられる
- apt でパッケージを入れ、`sudo` で環境変数が変わることを説明できる
- つながらないとき、`curl` の表示と終了コードから、名前解決・接続・TLS・HTTP のどこで止まったかを言える
- 社内で必要なプロキシの設定(環境変数・`no_proxy`・apt・社内 CA 証明書)を入れ、環境設定手順書にまとめられる

事前準備

- 第１回を終えていること(`~/handsoff` があり、第１回の手順４でネットワークの層・DNS・`curl -v` を一度見ている)
- 講師から受け取る: `<社内プロキシのアドレス>`・`<プロキシ認証の有無>`(open-questions.md の Q1)、`<社内CA証明書の入手方法>`(Q3)

まとめ

冒頭の座学で使います。第１回で見た部品のうち、シェル・パッケージ・ネットワークを深掘りします。ハンズオンで打ったコマンドは、その場で環境設定手順書に書いていきます。

シェルと環境変数

コマンド・ファイル	意味	参照(モダン Linux)
`NAME=値`	シェル変数。そのシェルの中だけ。`=` の前後に空白を入れない	3.1.2
`export NAME`	環境変数にする。子プロセスにコピーが渡る。子から親は変えられない	3.1.2
`NAME=値 コマンド`	そのコマンドにだけ渡す	3.1.2

コマンド・ファイル	意味	参照(モダンLinux)
`env`・`printenv NAME`	環境変数を見る	付録A
`echo "$PATH" | tr ':' '\n'`	コマンドを探すディレクトリ。前から順に探す	3.1.2
`type -a 名前`	探す順番(エイリアス → 関数 → 組み込み → PATH)で全部見せる	3.1.2
`export PATH="$HOME/mybin:$PATH"`	PATH に**足す**(元を残す)	3.1.2
`~/.bashrc`	対話シェルを開くたびに読まれる。WSL のターミナルは `~/.profile` 経由で読む	3章
`source ~/.bashrc`	今のシェルで読み直す。2回読むと PATH が重複する	3章
`cp ~/.bashrc ~/.bashrc.bak`	書き換える前に必ずコピー。追記は `>>`(`>` は全部消す)	付録A.5

apt と sudo

コマンド・ファイル	意味	参照(モダンLinux)
`/etc/apt/sources.list.d/ubuntu.sources`	パッケージの取得元(リポジトリ)	6.4、6.5
`sudo apt update`	取得元からパッケージの一覧を取り直す	6.5
`sudo apt install -y 名前`	依存するパッケージごと入れる	6.5
`apt policy 名前`・`dpkg -L 名前`	入っている版と候補／入ったファイル	6.5
`sudo コマンド`	root で実行。環境変数はほぼ消え、PATH は決まった値(`secure_path`)になる	4.2

つながらないときの切り分け

下の段階から順に確かめます。`curl` の終了コードは `echo $?` で見ます。この回の HTTP はステータス行が届くまでで、届いた後のヘッダ・Cookie は第3回の範囲です。

段階	止まったときの `curl` の表示	終了コード	次に見るもの	参照
名前解決(DNS)	`Could not resolve host`	6	`getent hosts 名前`、`/etc/resolv.conf`	モダンLinux 7.3
接続(TCP)	`Could not connect to server`(相手が待っていない)	7	相手が動いているか、`ss -tln`	モダンLinux 7.2.3

段階	止まったときの `curl` の表示	終了コード	次に見るもの	参照
接続(TCP)	`Connection timed out`(返事がない)	28	途中で止められていないか、プロキシが要るか	モダンLinux 7.2.2
TLS	`SSL certificate problem` などの証明書のエラー	60	社内 CA 証明書、日時	Web技術の基本6章
HTTP	`< HTTP/1.1 404` などのステータス行	0(`-f` を付けると 22)	URL、サーバのログ	Web技術の基本2章

社内で必要な設定: プロキシ

設定	書き方・場所	参照
環境変数	`http_proxy`・`https_proxy`・`no_proxy` を**小文字と大文字の両方** `.bashrc` に書く。curl は http の URL では小文字だけ読む	Web技術の基本 6章
`no_proxy`	`localhost,127.0.0.1` がないと、自分の PC のサーバへの通信までプロキシに送る	Web技術の基本 6章
curl・git	環境変数を読む	Web技術の基本 6章
apt	`sudo` で環境変数が消えるので `/etc/apt/apt.conf.d/95proxy` に書く	モダンLinux 6.5
社内 CA 証明書	未登録だと TLS で止まる(60)。`.crt` で `/usr/local/share/ca-certificates/` に置き `update-ca-certificates`	Web技術の基本 6章
pip・uv	環境変数＋独自の証明書の設定。通すのは第4回	—
パスワード	コマンド・`.bashrc`・手順書に書かない(履歴とファイルに残る)	—

ハンズオン

1ステップ1コマンド。各ステップは「🖋 やること → 📚 参考書の節 → 📒 説明 → コマンド」。作業場所は `~/handsoff/workspace/session02/`。

1. 変数: シェル変数と環境変数（4 分）

🖊 作業場所に移る

```
cd ~/handsoff/workspace/session02
```

🖊 シェル変数を作る

📚 モダンLinux 3.1.2 シェル

📒 `名前=値` で変数を作る。`=` の前後に空白を入れると、コマンドとして扱われてエラーになる

```
GREETING=hello; echo "$GREETING"
```

🖊 子プロセスから見る

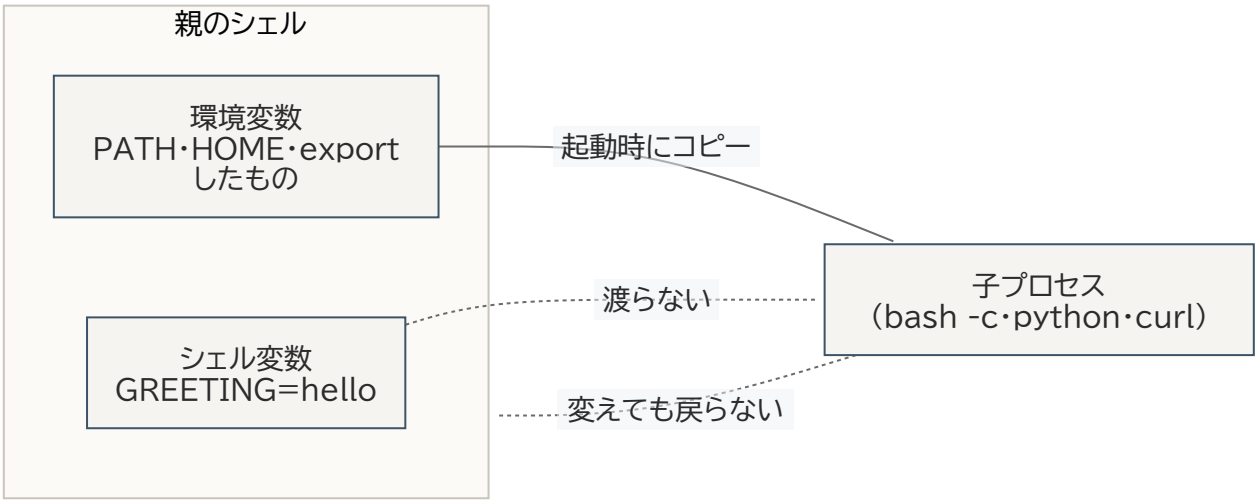
📒 `bash -c` は新しいシェルを子プロセスとして動かす。シェル変数は子に渡らないので、`子:` の後ろは空になる

```
bash -c 'echo "子: $GREETING"'
```

🖊 export して子に渡す

📚 モダンLinux 3.1.2 シェル

📒 `export` した変数（環境変数）だけが、子プロセスの起動時にコピーされる。コピーなので、子で変えても親には戻らない



```
export GREETING; bash -c 'echo "子: $GREETING"'
```

🖊 子では親を変えられないことを確かめる

```
bash -c 'GREETING=changed'; echo "$GREETING"   # hello のまま
```

🖊 1つのコマンドにだけ渡す

📒 `名前=値 コマンド` の形は、そのコマンドの環境変数にだけ入る。プロキシを1回だけ試すときに使う（手順6）

```
GREETING=bye bash -c 'echo "子: $GREETING"'; echo "$GREETING"
```

🖊 環境変数の一覧を見る

📚 モダンLinux 付録A

```
printenv HOME USER SHELL LANG; env | wc -l
```

2. PATH: コマンドを探す順番（6分）

🖊 PATH を1行ずつ見る

📚 モダンLinux 3.1.2 シェル

📒 PATH は `:` 区切りのディレクトリの並び。前から順に探し、最初に見つかったものを動かす。WSL では後ろに Windows 側（`/mnt/c/...`）が付く

```
echo "$PATH" | tr ':' '\n'
```

🖊 どれが動くかを見る

📒 シェルはエイリアス → 関数 → 組み込みコマンド → PATH の順に探し、最初に見つかったものを動かす。`type -a` は見つかったものを全部、この順に表示する

```
type -a echo; type cd ll
```

🖊 自分のコマンドを作る

📚 モダンLinux 4.3.1 ファイルのパーミッション

📒 先頭の `#!/bin/bash` は、どのプログラムで動かすかの指定。実行の権限（`x`）がないと動かない

```
mkdir -p ~/mybin && printf '#!/bin/bash\necho "hello from mybin"\n' > ~/mybin/hello &&
chmod +x ~/mybin/hello
```

🖊 名前だけでは動かないことを確かめる

📒 `~/mybin` が PATH にないので見つからない。パスを書けば動く

```
hello; ~/mybin/hello
```

🖊 PATH に足す

📒 元の `$PATH` を後ろに残して足す。前に足したので、同じ名前があれば `~/mybin` が先に使われる

```
export PATH="$HOME/mybin:$PATH"; hello; type hello
```

🖊 子のシェルで PATH を壊す

📒 子のシェルの中で壊すので、`exit` で親に戻れば元どおり（手順１の図）。壊れたシェルでもフルパスなら動く

```
bash
PATH="$HOME/mybin"; ls; /usr/bin/ls; hello   # ls だけ command not found
exit
```

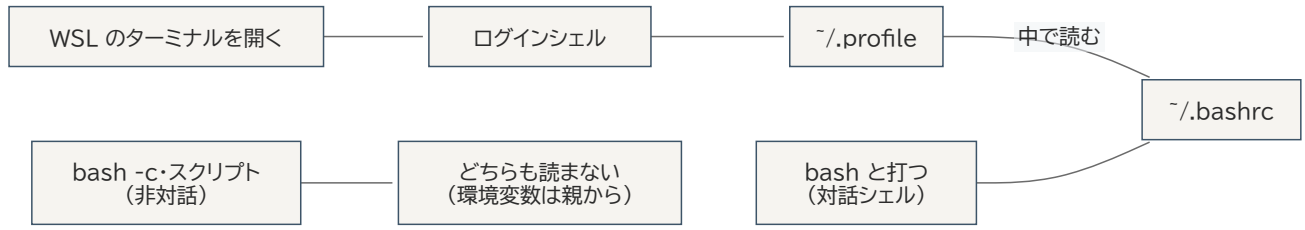
- 確認: 親のシェルで `ls` が動く

3. .bashrc: 設定を残す（８分）

🖊 ログインシェルかどうかを見る

📚 モダンLinux ３章 シェルとスクリプト

📒 `export` はそのシェルと子だけに効き、ターミナルを閉じると消える。残すには起動時に読まれるファイルに書く。どのファイルを読むかは、シェルの起動のされ方で決まる




```
shopt -q login_shell && echo ログインシェル || echo ログインシェルではない
```

🖊 .profile が .bashrc を読んでいるのを見る

```
grep -n -A2 'bashrc' ~/.profile
```

🖊 コピーを取る

📚 モダンLinux 付録A.4 ファイルとディレクトリ

```
cp ~/.bashrc ~/.bashrc.bak
```

🖊 PATH を足す行を追記する

📚 モダンLinux 付録A.5 リダイレクトとパイプ

📒 `>>` は追記、`>` は上書き（`.bashrc` が1行になる）。単一引用符で囲むと `$HOME` が展開されずにそのまま書かれる

```
echo 'export PATH="$HOME/mybin:$PATH"' >> ~/.bashrc; tail -1 ~/.bashrc
```

🖊 読み直す

📒 `source` は今のシェルでファイルを読み直す。手順2で足した分と重なり、`mybin` が2つになる。新しいターミナルなら1つ

```
source ~/.bashrc; echo "$PATH" | tr ':' '\n' | grep -c mybin
```

🖊 わざと壊す

📒 元を残さない行を追記すると、以降に開くシェルは全部壊れる。壊しても戻せるのは、コピーを取ってあるから

```
echo 'export PATH="$HOME/mybin"' >> ~/.bashrc; bash
ls              # command not found（新しいシェルは壊れている）
exit
```

🖊 差分を見て直す

📒 親のシェルは `.bashrc` を読み直していないので動く。最後の１行だけを消す。どのターミナルも壊れたときは `/usr/bin/nano ~/.bashrc` とフルパスで開くか、`cp ~/.bashrc.bak ~/.bashrc` で戻す

```
diff ~/.bashrc.bak ~/.bashrc; sed -i '$d' ~/.bashrc; diff ~/.bashrc.bak ~/.bashrc
```

- 確認: ２回目の `diff` は `export PATH="$HOME/mybin:$PATH"` の１行だけ。新しいターミナルで `ls` と `hello` が動く

🖊 環境設定手順書を用意する

📒 ここからの手順は、打ったらすぐ手順書に書く。各節に「コマンド」と「確認（成功したときの表示）」を書く。第４回の手順７で、隣の人がこの手順書の「確認」の行を打って前提がそろっているかを確かめる

```
cp ~/handsoff/docs/session02/assets/環境設定手順書.md .; code 環境設定手順書.md
```

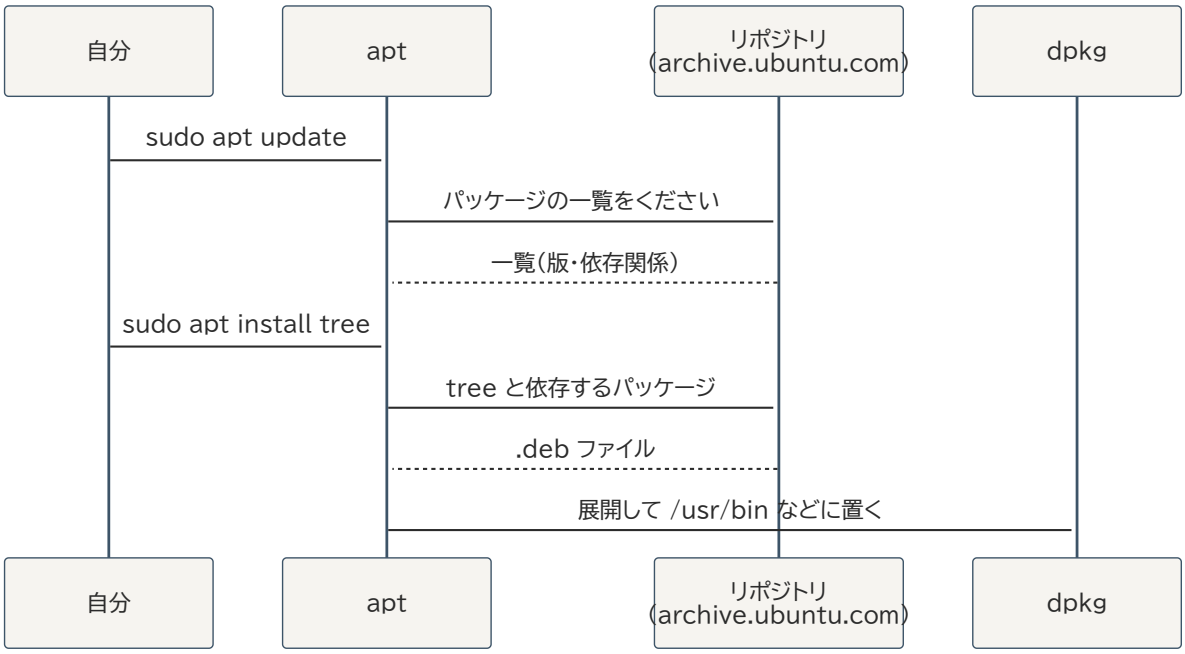
- 書く: 「0. 今の状態を確かめる」と「1. PATH に自分のコマンドの場所を足す」（コピー → 追記 → 反映 → `type hello`）
- `code` が使えないときは `nano 環境設定手順書.md`

4. apt と sudo（6分）

🖊 パッケージの取得元を見る

📚 モダンLinux 6.4 Linuxアプリケーションサプライチェーン、6.5 パッケージとパッケージマネージャ

📒 apt はリポジトリ（取得元）から一覧を取り、パッケージ（.deb）と依存するパッケージを落として dpkg で入れる




```
grep -E '^(URIs|Suites)' /etc/apt/sources.list.d/ubuntu.sources
```

🖊 一覧を更新する

📒 社内で止まったら `Ctrl+C` で止め、手順６でプロキシを設定してからやり直す。止まった理由は手順５でわかる

```
sudo apt update
```

🖊 入れる前の状態を見る

```
apt policy tree     # Installed: (none) と Candidate (入れられる版)
```

🖊 入れる

```
sudo apt install -y tree; tree -L 1 ~/handsoff
```

🖊 どこに入ったかを見る

📒 パッケージのコマンドは `/usr/bin` に入る。PATH にあるので、名前だけで動く

```
dpkg -L tree | grep bin; type tree
```

🖊 sudo で環境変数が変わるのを見る

📚 モダンLinux 4.2 ユーザ

📒 `sudo` は root としてコマンドを動かす。そのとき自分の環境変数はほぼ捨てられ、PATH は決まった値になる。プロキシの環境変数も渡らないので、apt には別の設定が要る（手順６）

```
export DEMO=1; printenv DEMO; sudo printenv PATH DEMO   # DEMO は出ない
```

🖊 手順書に書く

- 書く:「2. apt でパッケージを入れる」（`apt update` → `apt install` → 確認は `type tree`）

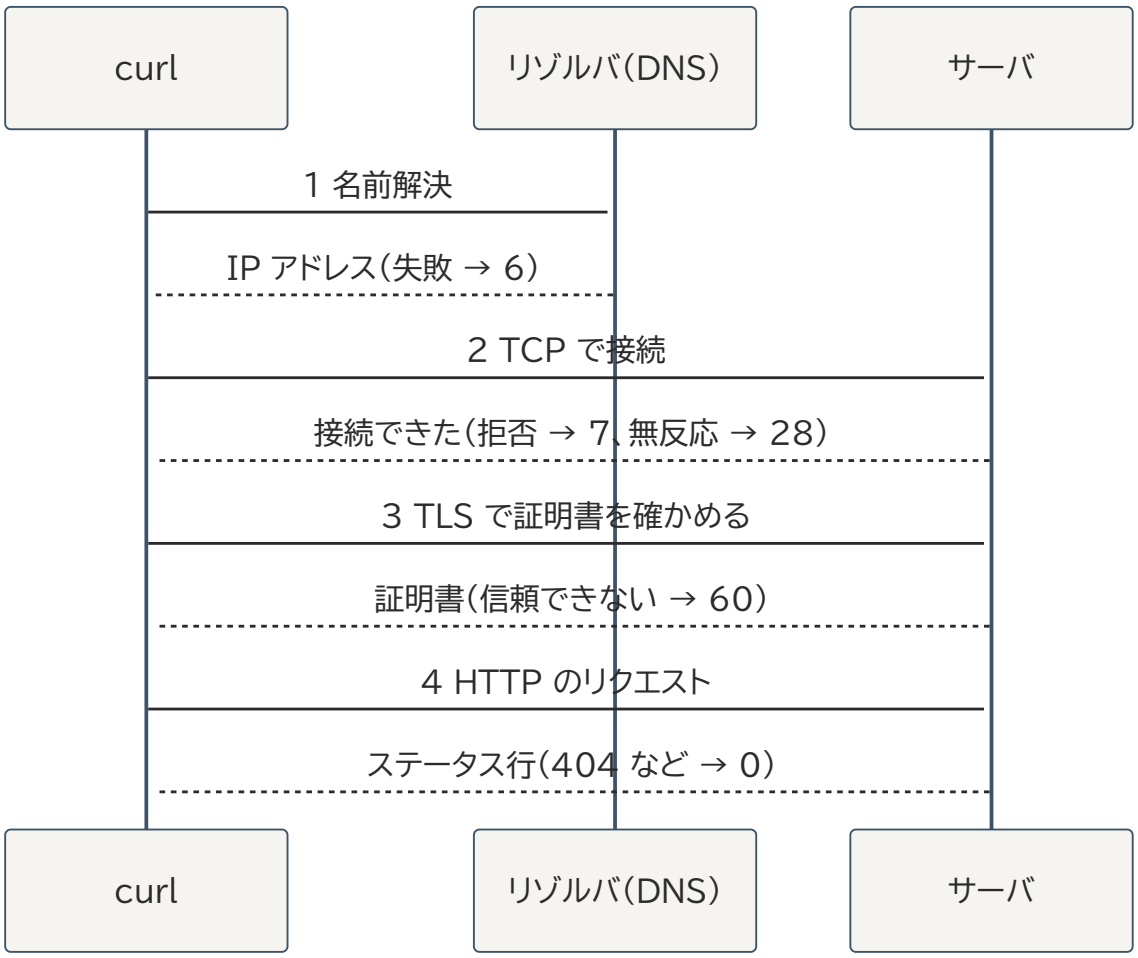
5. つながらないときの切り分け（９分）

第１回の手順４で層ごとのコマンドを見た。ここでは、わざと各段階で止めて、`curl` の表示と終了コードを覚える。

🖊 段階ごとの時間を見る

📚 モダンLinux 7.4.1 ウェブ、Web技術の基本 ２章

📒 `curl` は名前解決 → TCP 接続 → TLS（https だけ）→ HTTP の順に進む。どこで止まったかで、表示と終了コードが変わる




```
curl -sS -o /dev/null -w 'dns=%{time_namelookup} tcp=%{time_connect} tls=%
{time_appconnect} code=%{http_code}\n' https://example.com
```

🖋 名前解決で止める

📚 モダンLinux 7.3 DNS

📒 `.invalid` は、どこにも存在しないと決められた名前。`$?` は直前のコマンドの終了コード(0 が成功)

```
curl -v http://nothing.invalid/; echo "終了コード: $?"   # Could not resolve host、6
```

🖋 接続で止める: 相手が待っていない

📚 モダンLinux 7.2.3 トランスポート層

📒 相手の PC には届いても、そのポートで誰も待っていなければすぐに断られる

```
curl -v http://127.0.0.1:8000/; echo "終了コード: $?"    # Could not connect to server、7
```

🖊 接続で止める: 返事がない

📚 モダンLinux 7.2.2 インターネット層

📒 途中で捨てられると返事が来ず、待ち時間が切れる。社内で外に直接出ようとしたときによく見る形

```
curl -v --connect-timeout 3 http://10.255.255.1/; echo "終了コード: $?"   # timed out、28
```

🖊 TLS で止める

📚 Web技術の基本 6章

📒 期限切れの証明書を返すテスト用のサイト。社内 CA 証明書を登録していないときも同じ 60 になる（手順6）

```
curl -v https://expired.badssl.com/; echo "終了コード: $?"  # certificate has expired、60
```

🖊 HTTP のエラーを見る

📚 Web技術の基本 2章

📒 HTTP まで届けば、404 でも `curl` は成功（0）を返す。`-f` を付けると 4xx・5xx を失敗（22）として扱う

```
python3 -m http.server 8000 &
curl -v http://127.0.0.1:8000/nothing; echo "終了コード: $?"     # < HTTP/1.0 404、0
curl -f http://127.0.0.1:8000/nothing; echo "終了コード: $?"     # 22
kill %1
```

🖊 切り分けの順番を確かめる

📒 つながらないときは、下の段階から1つずつ確かめる。どの段階で止まったかを言えれば、調べる場所と相談する相手が決まる

```
getent hosts example.com; echo "終了コード: $?"   # 名前解決だけを確かめる（第1回の手順4）
```

🖊 手順書に書く

- 書く:「うまくいかないとき」の表(表示・終了コード → 止まった段階 → 確かめること)

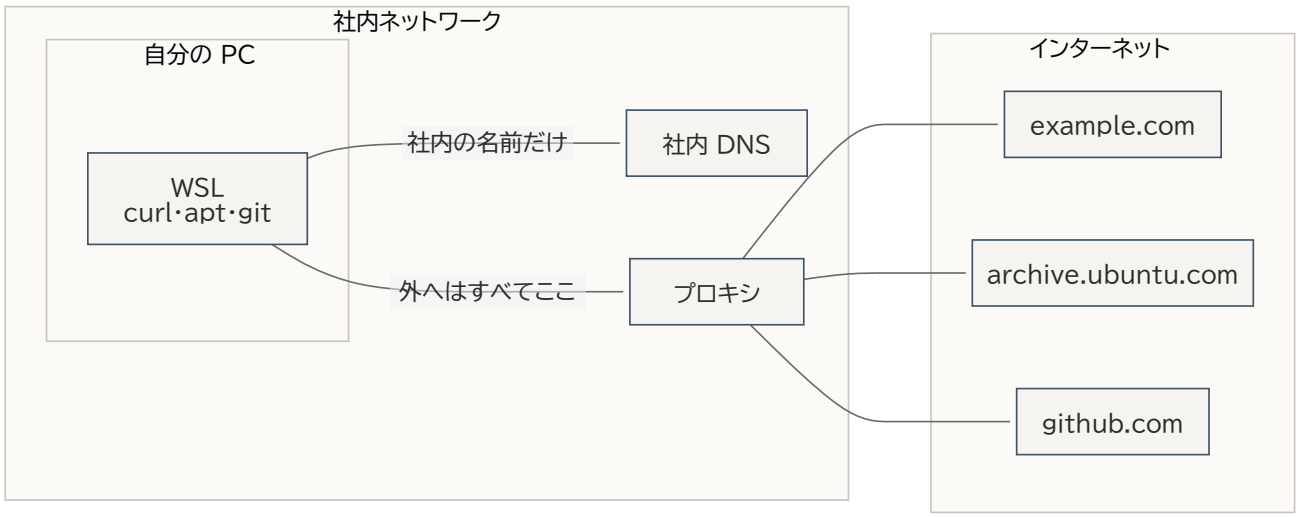
6. 社内で必要な設定: プロキシ(9 分)

プロキシのない環境では、`<社内プロキシのアドレス>` を使うステップは読むだけでよい。

🖊 今の設定を見る

📚 Web技術の基本 6章(プロキシサーバー)

📒 社内からは外に直接出られず、プロキシが代わりに外とやり取りすることが多い。外の名前もプロキシが引く




```
env | grep -i _proxy    # 何も出なければ未設定
```

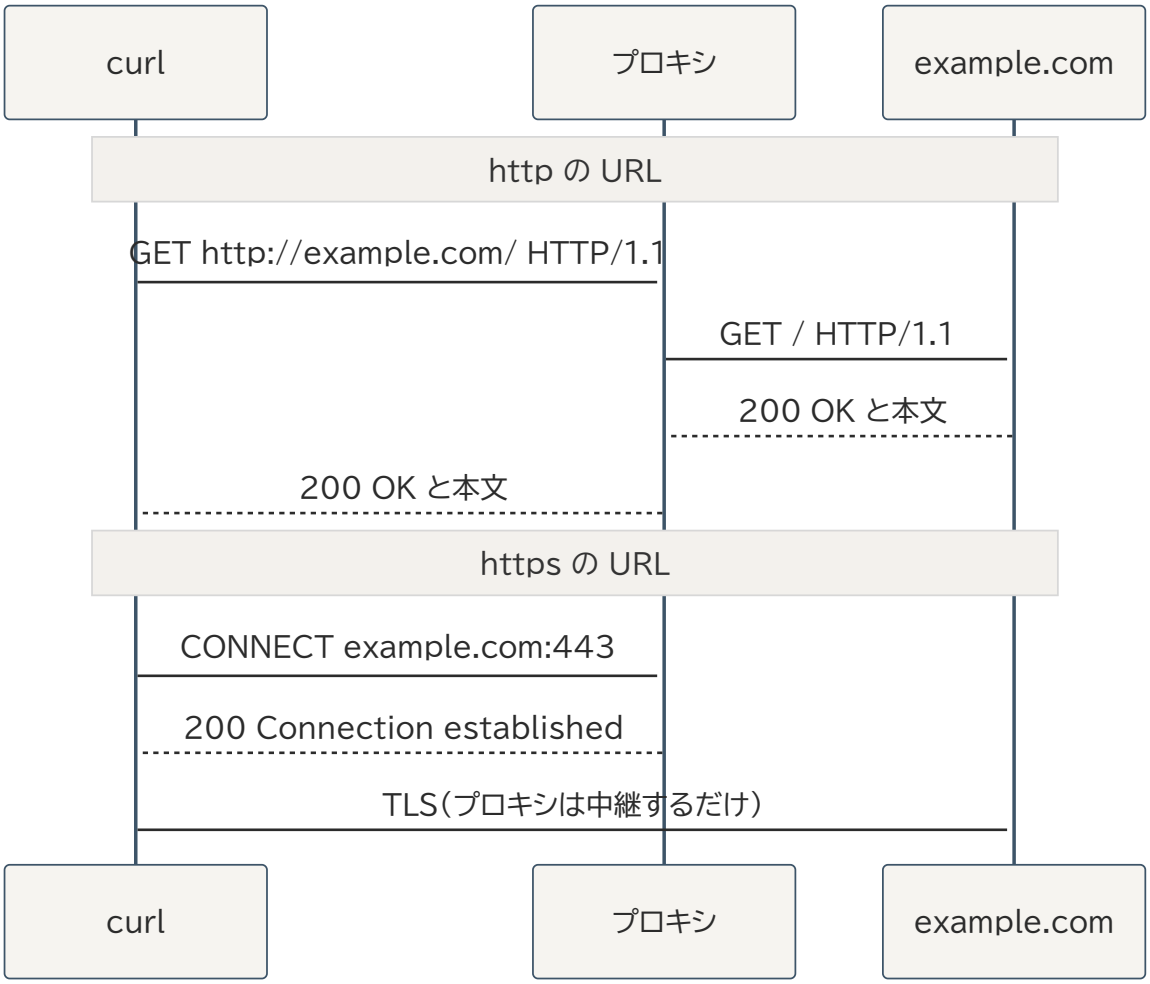
🖊 プロキシに頼む様子を見る

📒 プロキシを設定すると、`curl` は相手ではなくプロキシにつなぐ。ここでは誰も待っていないポート(9番)をプロキシにして、止まった場所を見る。表示に出るのは相手ではなくプロキシのアドレス

```
https_proxy=http://127.0.0.1:9 curl -v https://example.com; echo "終了コード: $?"
# * Uses proxy env variable https_proxy == 'http://127.0.0.1:9'
# curl: (7) Failed to connect to 127.0.0.1 port 9 ...
```

🔗 http と https の頼み方の違いを知る

📒 http では URL 全体をプロキシに渡し、プロキシが代わりに取りに行く。https では `CONNECT` で行き先だけを伝えてトンネルを張り、中身は暗号化されたまま通る



🔗 no_proxy を確かめる

📒 自分の PC のサーバまでプロキシに送ると届かない。`no_proxy` に書いた宛先は直接つなぐ

```
python3 -m http.server 8000 &
http_proxy=http://127.0.0.1:9 curl -sS -o /dev/null -w '%{http_code}\n'
http://127.0.0.1:8000/                          # 000（失敗）
http_proxy=http://127.0.0.1:9 no_proxy=localhost,127.0.0.1 curl -sS -o /dev/null -w '%
{http_code}\n' http://127.0.0.1:8000/   # 200
kill %1
```

🔗 社内プロキシを .bashrc に書く

📒 ツールによって読む名前が小文字・大文字で違うので、両方書く。`.bashrc` に書けば次のターミナルからも効く

```
cat >> ~/.bashrc <<'EOF'
export http_proxy="<社内プロキシのアドレス>" https_proxy="<社内プロキシのアドレス>"
export no_proxy="localhost,127.0.0.1"
export HTTP_PROXY="$http_proxy" HTTPS_PROXY="$https_proxy" NO_PROXY="$no_proxy"
EOF
source ~/.bashrc; curl -v -o /dev/null https://example.com 2>&1 | grep -E 'Uses
proxy|CONNECT|HTTP/'
```

- 確認: `> CONNECT example.com:443` と `< HTTP/1.1 200 Connection established`

🔗 apt にプロキシを設定する

📚 モダンLinux 6.5 パッケージとパッケージマネージャ

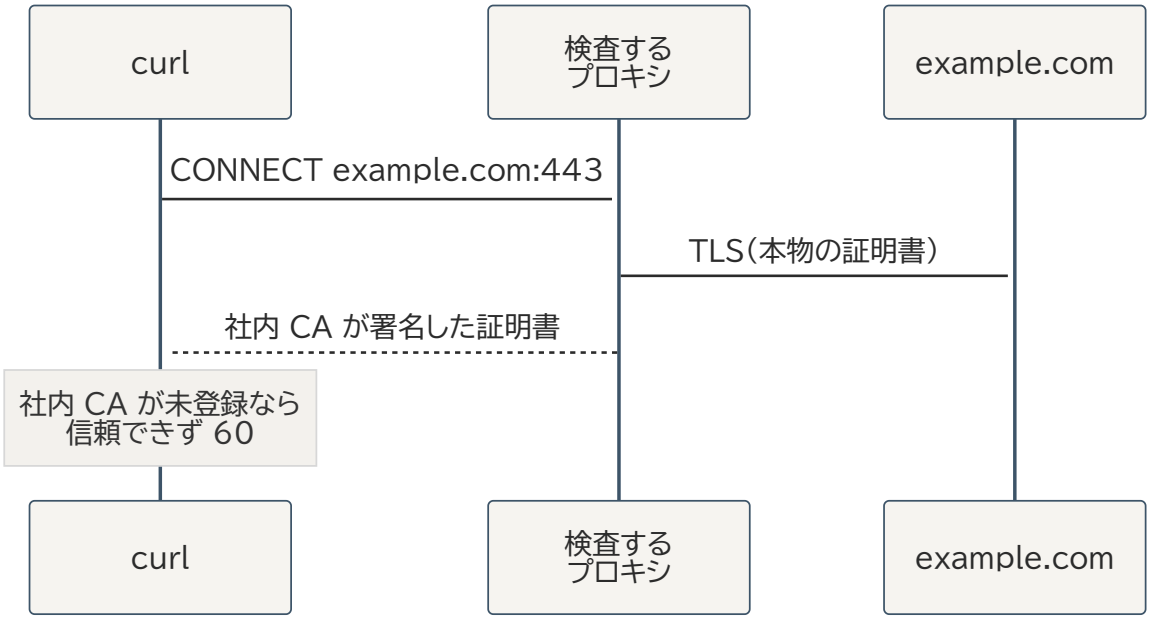
📒 `sudo apt` には環境変数が渡らない(手順4)。apt は専用の設定ファイルを読む。curl・git は環境変数を読む

```
printf 'Acquire::http::Proxy "%s";\nAcquire::https::Proxy "%s";\n' "$http_proxy"
"$http_proxy" | sudo tee /etc/apt/apt.conf.d/95proxy
apt-config dump | grep -i proxy; sudo apt update
```

🔗 社内 CA 証明書を登録する

📚 Web技術の基本 6章(暗号化・証明書)

📒 社内のプロキシが https の中身を検査する場合、プロキシは社内 CA で署名し直した証明書を返す。社内 CA を OS に登録していないと、手順5と同じ 60 で止まる




```
sudo cp <社内CA証明書のファイル名>.crt /usr/local/share/ca-certificates/ && sudo update-ca-certificates   # 1 added
```

> 🔗 **git で確かめる**

```
git ls-remote https://github.com/git/git HEAD   # 40桁の英数字と HEAD
```

- 認証があるプロキシ(Q1)は講師に相談する。パスワードは `read -rsp` で変数にだけ入れる

> 🔗 **手順書に書く**

- 書く:「3. プロキシを設定する」(3-1〜3-4 と各確認)。アドレスは `<社内プロキシのアドレス>` のまま、パスワードは書かない

7. 手順書を仕上げる(3分)

> 🔗 **新しいターミナルで確かめる**

📒 `.bashrc` に書いたものは、新しいターミナルで効いて初めて「設定した」と言える

```
type hello; env | grep -i _proxy; apt-config dump | grep -i proxy
```

> 🖋 手順書を読み直す

📒 手順書は、上から実行すれば同じ状態になり、各節の確認で成功を判断でき、元に戻せるように書く。第４回の手順７で、隣の人がこの手順書の「確認」の行で前提を確かめる

```
grep -c '（書く）' 環境設定手順書.md   # 0 になるまで埋める
```

- 書く:「元に戻す方法」（`cp ~/.bashrc.bak ~/.bashrc`、`sudo rm /etc/apt/apt.conf.d/95proxy` など）

読む章

区分	資料	範囲	読み方
参考書	入門 モダンLinux（モダンLinux）	３章 シェルとスクリプト、４章 アクセス制御	該当章（第１回からの続き）
参考書	同上	６章 パッケージ管理（6.4、6.5）、７章 ネットワーク	該当章（６章は第４回でも使う。７章は第１回の続き）
参考書	同上	付録A 便利なコマンド集	引く
参考書	イラスト図解式 この一冊で全部わかる Web技術の基本 第２版（Web技術の基本）	２章 Webとネットワーク技術、６章 のプロキシサーバー・暗号化・証明書	該当章（３章 HTTP と６章 のほかの節は第３回で読む）
紹介（副読本）	達人プログラマー 第２版	基本ツール（シェル、プレーンテキスト）	持っている人は通読（第１〜５回で基本ツールの範囲）

環境設定手順書（WSL・Ubuntu）

第２回のハンズオンで、各節の「（書く）」を自分の言葉とコマンドで埋める。第４回で隣の人がこの手順書だけで同じ設定をする。

- 対象: （書く。例: 社内ネットワークにつないだ Windows PC の WSL の Ubuntu 26.04）
- 所要時間: （書く）
- 必要なもの: `sudo` を使える WSL のユーザー、社内プロキシのアドレス `<社内プロキシのアドレス>`、社内 CA 証明書 `<社内CA証明書のファイル名>`

> パスワードはこの手順書に書かない。

0. 今の状態を確かめる

> （書く： PATH とプロキシの設定を見るコマンド）

確認: （書く: 何が表示されれば次に進んでよいか）

1. PATH に自分のコマンドの場所を足す

> （書く： .bashrc のコピーを取る → 追記する → 反映する）

確認: （書く）

2. apt でパッケージを入れる

> （書く）

確認: （書く）

3. プロキシを設定する（社内ネットワークだけ）

3-1. シェルの環境変数

> （書く： .bashrc に書く行。小文字と大文字、no_proxy）

確認:（書く）

3-2. apt

> （書く）

確認:（書く）

3-3. 社内 CA 証明書

> （書く）

確認:（書く）

3-4. git・curl で確かめる

> （書く）

確認:（書く）

うまくいかないとき

症状（表示・終了コード）	止まった段階	確かめること・対処
（書く）		

元に戻す方法

> （書く： 1〜3 を戻すコマンド）

確認:（書く）

第３回 Web 通信の基礎（HTTP・Cookie・Web API）

tag: http, web-ブラウザ, sec
前提タグ: linux-シェル, linux-ネットワーク
所要: 座学15分 / ハンズオン45分

ゴール

- リクエストとレスポンスの主なヘッダ（Host・Accept・Content-Type・Location・Cache-Control・ETag・Set-Cookie・Origin など）を、curl と開発者ツールで読んで意味を言える
- Cookie とセッションでログインを保つ仕組みを説明し、`HttpOnly`・`SameSite`・CSRF のトークンが何を防ぐかを動かして確かめられる
- Web API を curl で呼び、メソッドとステータスコード（200・201・204・304・400・404・405・415・422）の使い分けと JSON を読める
- CORS で止めるのはブラウザで、リクエストはサーバに届くことを確かめられる
- DOM と fetch をコンソールで触り、`innerHTML` に入力を入れると XSS になることを説明できる

事前準備

- 第２回を終えていること（社内ではプロキシの環境変数と `no_proxy` に `localhost,127.0.0.1`、apt のプロキシの設定が済んでいる）
- ブラウザは Windows の Chrome か Edge。開発者ツールは `F12`
- 社内では、Windows のプロキシ設定の例外に `127.0.0.1;localhost` が入っていること（open-questions.md の Q1）
- 素材: assets/web_demo.py（デモのサーバ。Ubuntu の `python3` の標準ライブラリだけで動く）

まとめ

冒頭の座学で使います。ハンズオンでは、デモのサーバを相手に curl とブラウザの開発者ツールで、ヘッダ・Cookie・JSON の実物を読みます。第１回で見た `curl -v` の「リクエスト行・ヘッダ・空行・本文」の形が前提です。

主なヘッダ（http-構成）

ヘッダ	向き	意味	見る手順	参照
`Host`	要求	どのサイト宛てか（1台のサーバが複数のサイトを持てる）	1	Web技術の基本 3章
`User-Agent`・`Accept`・`Accept-Language`	要求	送り手の種類、受け取れる形式と言語	1・2	MDN HTTP ヘッダー
`Content-Type`・`Content-Length`	両方	本文の形式（`text/html`・`application/json`・`application/x-www-form-urlencoded`）と長さ	1・5	MDN HTTP ヘッダー
`Location`	応答	次に行く URL（303）、作ったものの URL（201）	3・5	MDN HTTP ヘッダー
`Cache-Control`	応答	`no-store` 保存しない、`no-cache` 使う前に毎回確かめる、`max-age=秒` その間は確かめない	1・5	MDN HTTP キャッシュ
`ETag`・`If-None-Match`	応答・要求	中身の指紋。同じなら 304 で本文を省く	2・5	MDN HTTP キャッシュ
`Set-Cookie`・`Cookie`	応答・要求	保存させる・送り返す	3	MDN HTTP Cookie の使用
`Origin`・`Access-Control-Allow-Origin`	要求・応答	どのページの JavaScript からか、その JavaScript に読ませてよいか	6	MDN オリジン間リソース共有（CORS）

ステータスコード

コード	意味	このデモで出る場面
200 OK・201 Created・204 No Content	成功・作った（`Location` つき）・成功で本文なし	一覧・追加・削除
303 See Other・304 Not Modified	`Location` を GET し直す・手元のコピーを使ってよい	ログイン・`If-None-Match`
400・415・422	送った形が壊れている・`Content-Type` が違う・形は正しいが中身がだめ	壊れた JSON・フォームで送る・空のタイトル

コード	意味	このデモで出る場面
401・403・404・405	ログインが要る・権限がない・ない・その URL で使えないメソッド（`Allow` つき）	`/api/me`・CSRF・ない id・`DELETE /api/tasks`
5xx	サーバ側の失敗	（出ない）

Cookie・セッションとセキュリティ（sec）

用語	押さえること	参照
Cookie	サーバが `Set-Cookie` で保存させ、ブラウザが同じサイトへのリクエストに自動で付ける。値は利用者が書き換えられる	Web技術の基本 3章、MDN HTTP Cookie の使用
セッション	中身（誰がログイン中か）はサーバに置き、ブラウザには推測できない ID だけを渡す。ログアウトでサーバ側を消す	Web技術の基本 6章
属性	`HttpOnly` JavaScript から読めない／`SameSite=Lax` ほかのサイトからの POST に付けない／`Max-Age` 秒で消える／`Path`・`Secure`（https だけ）	MDN Set-Cookie
オリジンとサイト	オリジン＝スキーム＋ホスト＋ポート。サイト＝スキーム＋ドメイン（ポートを見ない）	MDN 同一オリジンポリシー
CSRF	Cookie は自動で付くので、ほかのページから送らせても通ってしまう。フォームに埋めたトークンを照合し、`SameSite` で Cookie を付けない	Web技術の基本 6章
CORS	別のオリジンへの fetch の結果を、許可（`Access-Control-Allow-Origin`）がなければブラウザが JavaScript に渡さない。リクエストは届く	MDN オリジン間リソース共有（CORS）
XSS	入力がそのまま HTML になってスクリプトが動く。エスケープと `textContent` で防ぐ。`HttpOnly` の Cookie は盗めない	Web技術の基本 6章

Web API と JSON（http-基本）

メソッドと URL	意味	成功	失敗
`GET /api/tasks`（`?done=false`）	一覧（絞り込み）	200・304	—
`POST /api/tasks`	追加	201 ＋ `Location`	400・415・422
`GET /api/tasks/{id}`	1件	200・304	404
`PATCH /api/tasks/{id}`	送った項目だけ変える	200	404・422
`DELETE /api/tasks/{id}`	消す	204	404

- REST: URL で「もの」（リソース）を、メソッドで「すること」を表す。URL に動詞（`/addTask`）を入れない（Web技術の基本 5章）

- JSON: `{}` オブジェクト（`"名前": 値`）、`[]` 配列。値は文字列・数・`true` / `false`・`null`・オブジェクト・配列（Web技術の基本 4章、MDN JSON の操作）

開発者ツール（web-ブラウザ）

パネル	見るもの
ネットワーク	1行＝1リクエスト。ステータス・タイプ（`document`・`stylesheet`・`fetch`）・ヘッダー・ペイロード・レスポンス。「ログを保持」でページを移っても消えない
アプリケーション	「ストレージ」→「Cookie」で、保存された Cookie と属性
コンソール	JavaScript を1行ずつ実行する。`await` がそのまま使える
要素	DOM の木

ハンズオン

1ステップ1コマンド。各ステップは「🖋 やること → 📚 参考書・公式資料の節 → 📒 説明 → コマンドやコード」。作業場所は `~/handsoff/workspace/session03/`。ターミナルを２つ使う（①デモのサーバ、②コマンド）。

1. デモのサーバとヘッダの中身（7分）

> 🖋 **デモのサーバを起動する（ターミナル①）**

📒 標準ライブラリだけで書いたサーバ。このサイト（8000）と別のサイト（8001）を同時に立て、1リクエストを1行のログに出す。データはメモリだけにあり、止めると消える

```
cd ~/handsoff/workspace/session03
python3 ~/handsoff/docs/session03/assets/web_demo.py
```

- `Address already in use` のときは `python3 … --port 8100 --other-port 8101` で番号を変え、以降の URL も読み替える

> 🖋 **リクエストとレスポンスのヘッダを読む（ターミナル②）**

📚 Web技術の基本 3章 HTTP でやりとりする仕組み、MDN HTTP ヘッダー

📒 本文は `-o /dev/null` で捨て、ヘッダだけを見る

```
cd ~/handsoff/workspace/session03
curl -v -o /dev/null http://127.0.0.1:8000/
```

```
> GET / HTTP/1.1
> Host: 127.0.0.1:8000                          ← 宛先のサイト（URL のホストとポート）
> User-Agent: curl/8.5.0                        ← 送り手
> Accept: */*                                   ← 何でも受け取る
>
< HTTP/1.1 200 OK
< Server: WebDemo/1.0                           ← サーバの種類
< Date: Tue, 13 Oct 2026 00:34:29 GMT           ← 時刻は世界標準時（GMT）
< Cache-Control: no-store                       ← 保存しない（毎回取り直す）
< Set-Cookie: visits=1; Path=/; Max-Age=86400; SameSite=Lax    ← Cookie を保存させる（手順
3）
< Content-Type: text/html; charset=utf-8        ← 本文は HTML、文字コードは UTF-8
< Content-Length: 1231                          ← 本文のバイト数
```

🖊 ヘッダだけを取る

📒 `-I` は HEAD メソッド。GET と同じヘッダが返り、本文は返らない

```
curl -I http://127.0.0.1:8000/
```

🖊 送ったヘッダをサーバから見る

📒 `/headers` は、受け取ったリクエストヘッダを JSON で返す。`-H` で好きなヘッダを足せる

```
curl -s http://127.0.0.1:8000/headers
curl -s -H 'Accept-Language: ja' -H 'X-Hello: 1' http://127.0.0.1:8000/headers
```

2. ブラウザの Network タブ（5 分）

🖊 Network タブを開いてページを読む（ブラウザ）

📚 Web技術の基本 3章、MDN HTTP の概要

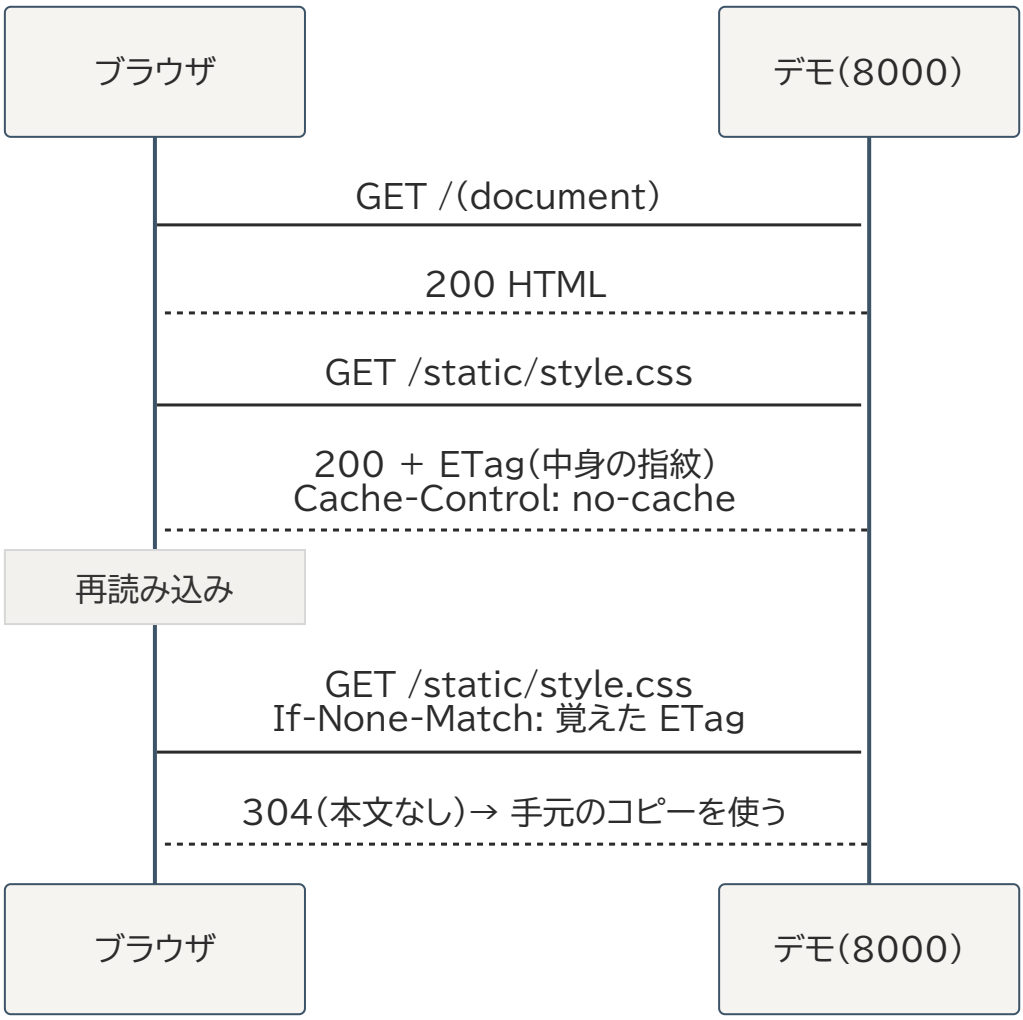
📒 `http://127.0.0.1:8000/` を開き、`F12` →「ネットワーク」→「ログを保持」に✓ → 再読み込み。HTML（`document`）を読んだ後、中の `<link>` を見て CSS（`stylesheet`）を取りに行く

- 開けない・社内のエラーページになるときは、Windows のプロキシ設定の例外を確かめる（事前準備）

🖊 再読み込みで 304 を見る

📚 MDN HTTP キャッシュ

📒 もう一度再読み込みする。`style.css` が 304 になる。行をクリックして「ヘッダー」を開くと、応答の `ETag` と要求の `If-None-Match` が同じ値になっている



🖊 ブラウザが送るヘッダを見る

📒 `http://127.0.0.1:8000/headers` を開き、手順１の curl と比べる。ブラウザは `Accept`（HTML を優先）・`Accept-Language`・`Sec-Fetch-*`（どんな操作で送ったか）・`Cookie` を自動で付ける

```
"Accept": "text/html,application/xhtml+xml,...;q=0.9,*/*;q=0.8"    ← HTML を優先（q は優先度）
"Accept-Language": "ja,en-US;q=0.9,..."                            ← ブラウザの言語設定
"Sec-Fetch-Mode": "navigate"                                       ← ページの移動で送った
"Cookie": "visits=3"                                               ← 保存した Cookie を自動で付けた
```

3. Cookie とセッション（9分）

🖊 Cookie を保存して送り返す（ターミナル②）

📚 Web技術の基本 3章、MDN HTTP Cookie の使用

📒 curl は `-c` で受け取った Cookie をファイルに保存し、`-b` でそのファイルから送る。2回目は `visits=1` を送ったので 2 になる

```
curl -s -c cookie.txt -b cookie.txt -o /dev/null http://127.0.0.1:8000/
curl -s -c cookie.txt -b cookie.txt http://127.0.0.1:8000/ | grep visits
cat cookie.txt      # ドメイン・パス・期限（UNIX 時刻）・名前・値
```

🖊 Cookie を書き換える

📒 Cookie の値は利用者が自由に変えられる。だから「誰がログイン中か」を Cookie の値に入れてはいけない

```
curl -s -b 'visits=999' http://127.0.0.1:8000/ | grep visits     # 1000
```

🖊 curl でログインする

📚 Web技術の基本 6章 セキュリティと認証、MDN Set-Cookie

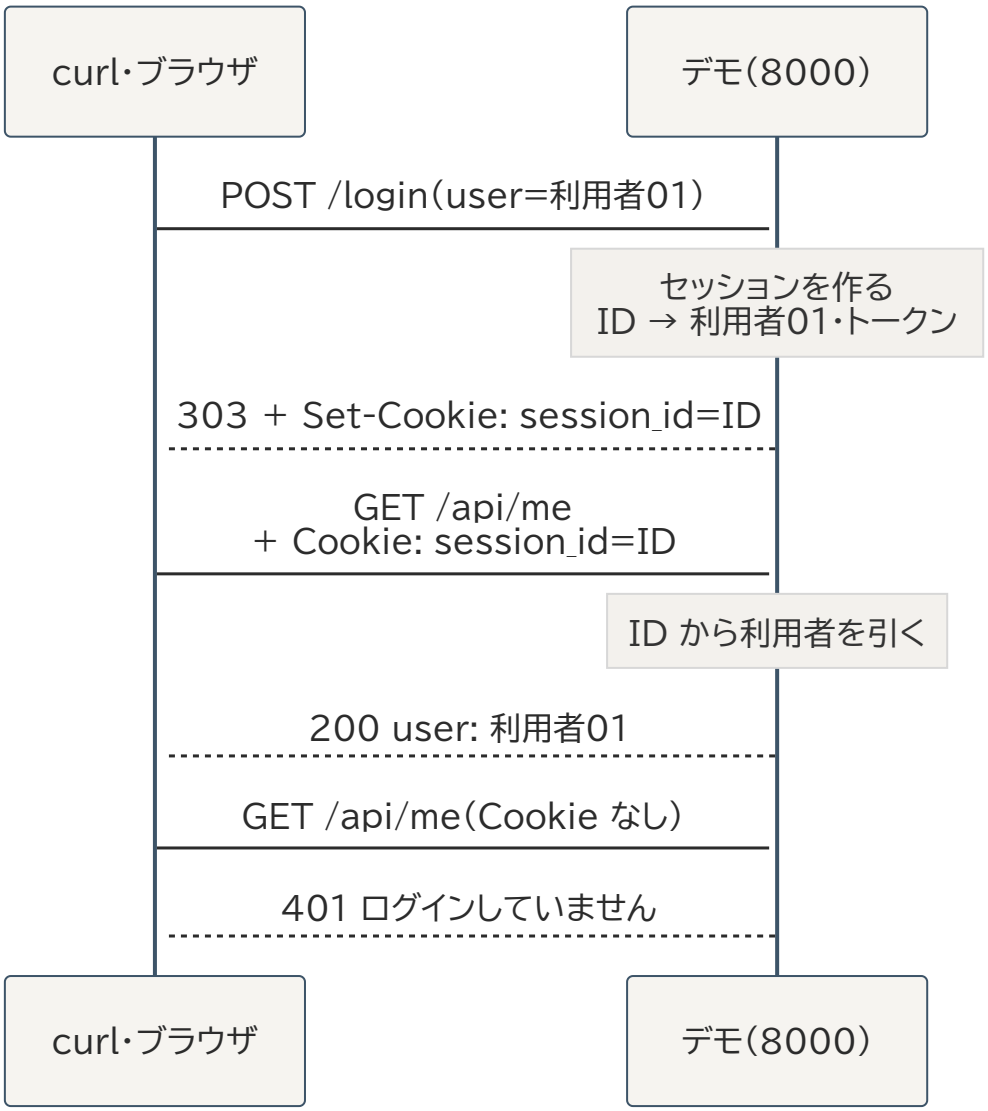
📒 ログインすると、サーバはセッション（利用者とトークン）を自分のメモリに作り、その ID だけを Cookie で渡す。パスワードの確認はデモでは省いている

```
curl -si -c cookie.txt -b cookie.txt --data-urlencode user=利用者01
http://127.0.0.1:8000/login
```

```
HTTP/1.1 303 See Other
Location: /                                   ← 次はトップを GET する
Set-Cookie: session_id=aRVPP-wnci17Yh3oXeChXg; HttpOnly; Path=/; SameSite=Lax; Max-
Age=1800
  session_id=…    ← セッション ID（推測できない乱数。人によって違う）
  HttpOnly        ← JavaScript から読めない
  Path=/          ← このサイトのどの URL にも付ける
  SameSite=Lax    ← ほかのサイトから送られた POST には付けない
  Max-Age=1800    ← 1800秒（30分）で消える
```

🖋 Cookie を付けて API を呼ぶ

📒 同じ URL でも、Cookie の有無で答えが変わる。サーバは ID から利用者を引いている



```
curl -s -b cookie.txt http://127.0.0.1:8000/api/me    # "user": "利用者01"
curl -s http://127.0.0.1:8000/api/me                  # 401 "ログインしていません"
```

🖊 ログアウトでセッションを消す

📒 ログアウトはブラウザの Cookie を消させる（`Max-Age=0`）だけでなく、サーバのセッションも消す。控えておいた古い ID を送っても、もう通らない

```
cp cookie.txt old-cookie.txt
curl -s -o /dev/null -b cookie.txt -c cookie.txt -X POST http://127.0.0.1:8000/logout
curl -s -b old-cookie.txt http://127.0.0.1:8000/api/me    # 401
```

🖊 ブラウザでログインする（ブラウザ）

📒 `http://127.0.0.1:8000/` を開き、「利用者03」でログインする。ネットワークの `login`（303）の応答ヘッダーに `Set-Cookie`、次の `127.0.0.1`（GET）の要求ヘッダーに `Cookie`。「アプリケーション」→「Cookie」で `session_id` の HttpOnly に✓

🖊 JavaScript から Cookie を読む

📒 「コンソール」で打つ。`visits` だけが見え、`HttpOnly` の `session_id` は見えない




```
document.cookie    // 'visits=3'（session_id は出ない）
```

4. CSRF: 自動で付く Cookie の罠（5分）

🖊 トークン付きのフォームでピン留めする（ブラウザ）

📚 Web技術の基本 6章、MDN Set-Cookie（SameSite）

📒 「牛乳を買う」をピン留めする。`Ctrl+U` でページのソースを開くと、フォームに `csrf_token` の hidden がある。値はセッションごとに違い、このページを開いた本人のフォームにだけ入っている

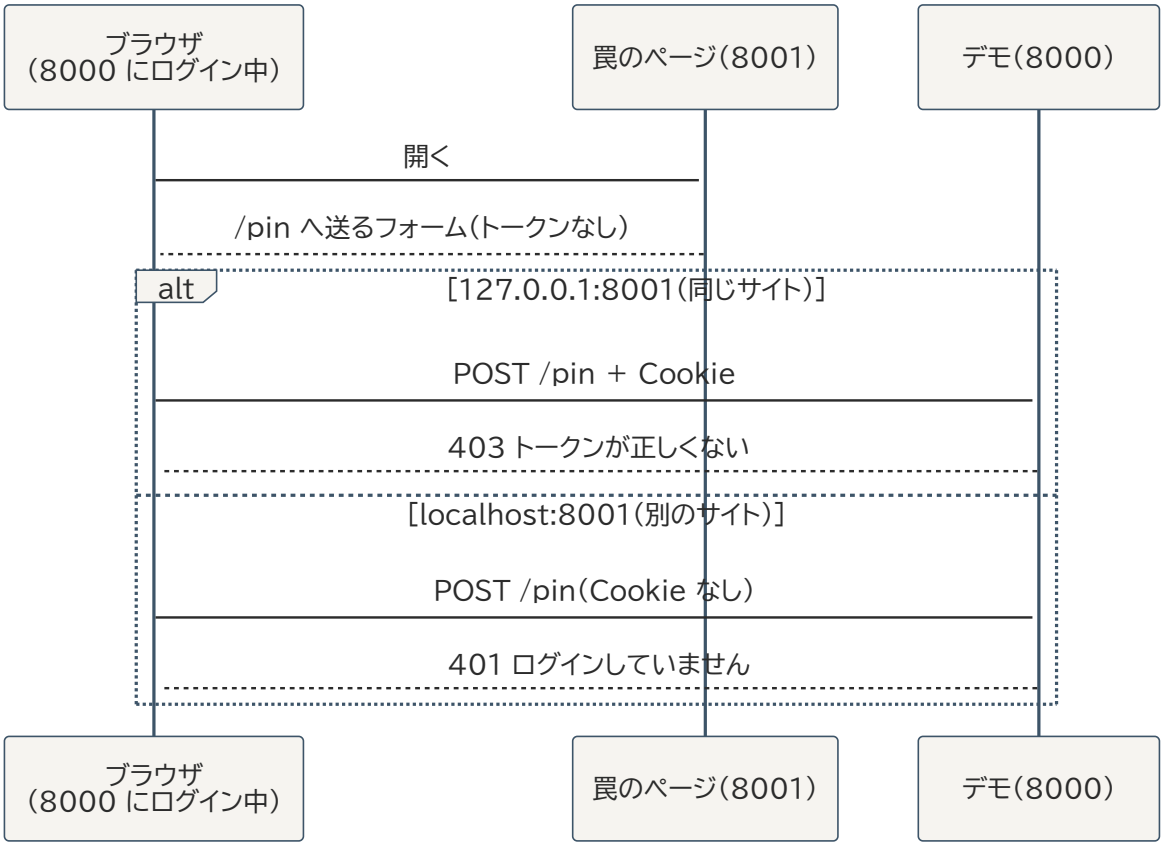
🖊 同じサイトの罠のページから送る

📒 `http://127.0.0.1:8001/trap` を開いて「受け取る」を押す。ポートが違うのでオリジンは別だが、サイトは同じなので Cookie が付く（ネットワークの `pin` の要求ヘッダー）。トークンがないので 403

開く URL	8000 から見たオリジン	8000 から見たサイト	`SameSite=Lax` の Cookie
`http://127.0.0.1:8000`	同じ	同じ	付く
`http://127.0.0.1:8001`	違う（ポート）	同じ（ポートを見ない）	付く
`http://localhost:8001`	違う	違う（ドメインが違う）	POST には付かない

🖊 別のサイトの罠のページから送る

📒 `http://localhost:8001/trap` で同じことをする。別のサイトからの POST なので `SameSite=Lax` で Cookie が付かず、401。トークンと `SameSite` の２つで守っている



5. Web API と JSON（11分）

🔗 一覧を取る（ターミナル②）

📚 Web技術の基本 5章 Webアプリケーションの基本（Web API）、4章 データ形式（JSON）

📒 本文は JSON。`Content-Type: application/json` で形式を伝える。`ETag` と `Cache-Control: no-cache` は手順2の CSS と同じ

```
curl -si http://127.0.0.1:8000/api/tasks
```

🔗 jq を入れる

📒 `jq` は JSON を整形し、一部を取り出す道具。入れられないときは `python3 -m json.tool --no-ensure-ascii` で整形だけする

```
sudo apt install -y jq     # 社内ではプロキシの設定が要る（第2回）
```

🔗 絞り込んで取り出す

📒 `?done=false` はクエリ文字列（URL の `?` の後ろ）で、未完了だけに絞る。`.[]` は配列の要素を1つずつ、`.title` はその中の項目

```
curl -s 'http://127.0.0.1:8000/api/tasks?done=false' | jq -r '.[].title'
```

🔗 追加する

📚 MDN HTTP レスポンスステータスコード

📒 JSON を送るときは `Content-Type: application/json` を付ける。作れたら 201 と、作ったものの URL（`Location`）が返る

```
curl -si -X POST -H 'Content-Type: application/json' -d '{"title": "傘を持っていく"}'
http://127.0.0.1:8000/api/tasks
```

```
HTTP/1.1 201 Created
Location: /api/tasks/3                    ← 作ったタスクの URL
Content-Type: application/json

{
  "id": 3,                                ← 数
  "title": "傘を持っていく",               ← 文字列
  "done": false                           ← 真偽値（true／false）
}
```

🖊 Location の URL を取る

```
curl -s http://127.0.0.1:8000/api/tasks/3
```

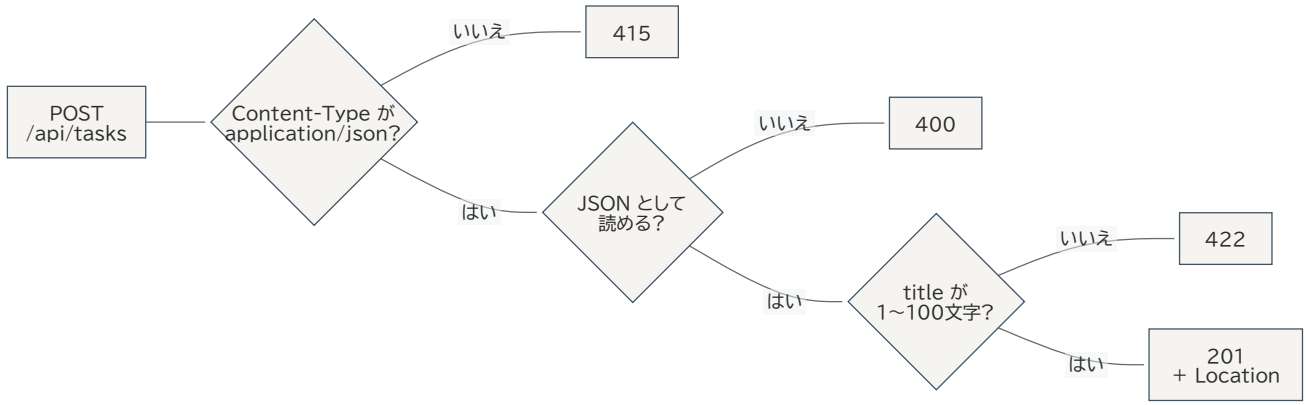
🖊 更新して消す

📒 PATCH は送った項目だけを変える。DELETE の成功は 204 で本文なし。消した後に取ると 404

```
curl -s -X PATCH -H 'Content-Type: application/json' -d '{"done": true}'
http://127.0.0.1:8000/api/tasks/3
curl -si -X DELETE http://127.0.0.1:8000/api/tasks/3 | head -1    # 204
curl -si http://127.0.0.1:8000/api/tasks/3 | head -1              # 404
```

🖊 わざと失敗させる

📒 失敗の理由ごとにコードが違う。サーバは上から順に確かめ、最初に引っかかったコードを返す。本文の `detail` に理由がある



```
A=http://127.0.0.1:8000/api/tasks
curl -s -X POST -d 'title=x' $A                                          # 415 フォームの形で送った
curl -s -X POST -H 'Content-Type: application/json' -d '{"title": "x"' $A    # 400 } がない
curl -s -X POST -H 'Content-Type: application/json' -d '{"title": ""}' $A    # 422 空のタイトル
curl -si -X DELETE $A | grep -E '^HTTP|^Allow'                           # 405 と Allow: GET, POST
curl -s $A/99                                                            # 404
```

🖊 変わっていなければ 304

📒 応答の `ETag` を `If-None-Match` で送ると、一覧が変わっていなければ 304 で本文なし。1件追加すると中身が変わるので 200 と新しい `ETag`

```
curl -si http://127.0.0.1:8000/api/tasks | grep -i etag
curl -si -H 'If-None-Match: "ここに ETag の値"' http://127.0.0.1:8000/api/tasks | head -1
# 304
```

🖊 公開 API を呼ぶ

📒 JSONPlaceholder は練習用の公開 API で、`/todos` に TODO の例がある。社内では第2回のプロキシの環境変数で外に出る。つながらないときは、同じことをデモの `/api/tasks` で行う

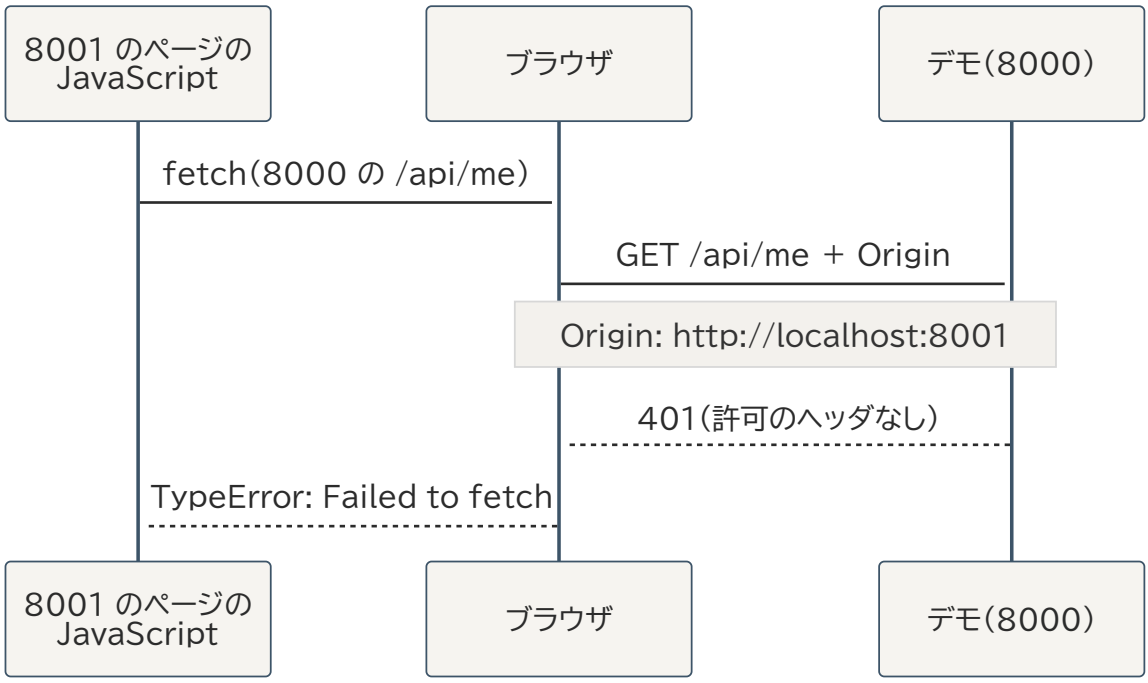
```
curl -si https://jsonplaceholder.typicode.com/todos/1                    # content-type・etag・cache-control を読む
curl -s 'https://jsonplaceholder.typicode.com/todos?userId=1&completed=false' | jq length
# 9
curl -si -X POST -H 'Content-Type: application/json' -d '{"title": "牛乳を買う"}' https://jsonplaceholder.typicode.com/todos | grep -iE '^HTTP|^location'
curl -s -o /dev/null -w '%{http_code}\n' https://jsonplaceholder.typicode.com/todos/201
# 404（練習用なので保存しない）
```

6. CORS とプロキシ（4 分）

🖊 別のオリジンから fetch する（ブラウザ）

📚 MDN オリジン間リソース共有（CORS）

📒 `http://localhost:8001/` を開き、コンソールで 8000 の `/api/me` を fetch する。コンソールに CORS のエラーが出るが、ターミナル①には `GET /api/me` と `Origin: http://localhost:8001` のログが出る。リクエストは届き、結果を JavaScript に渡すのをブラウザが止めた




```
await fetch("http://127.0.0.1:8000/api/me")
// Access to fetch at 'http://127.0.0.1:8000/api/me' from origin 'http://localhost:8001'
// has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present
...
```

🖊 許可された API を読む

📒 `/api/tasks` は `Access-Control-Allow-Origin: http://localhost:8001` を返すので読める。ネットワークで応答ヘッダーを確かめる

```
await (await fetch("http://127.0.0.1:8000/api/tasks")).json()
```

🔗 curl では読めることを見る（ターミナル②）

📒 CORS はブラウザの中の読み取りの制限で、サーバを守る仕組みではない。curl は `Origin` を名乗っても読める。サーバの変更を防ぐのは、手順４のトークン

```
curl -s -H 'Origin: http://localhost:8001' http://127.0.0.1:8000/api/me
```

🔗 プロキシを通っていないか確かめる

📒 `127.0.0.1` 宛ては、`no_proxy` によりプロキシを通さず直接つなぐ（第２回）。`Uses proxy` が出たら `no_proxy` を直す

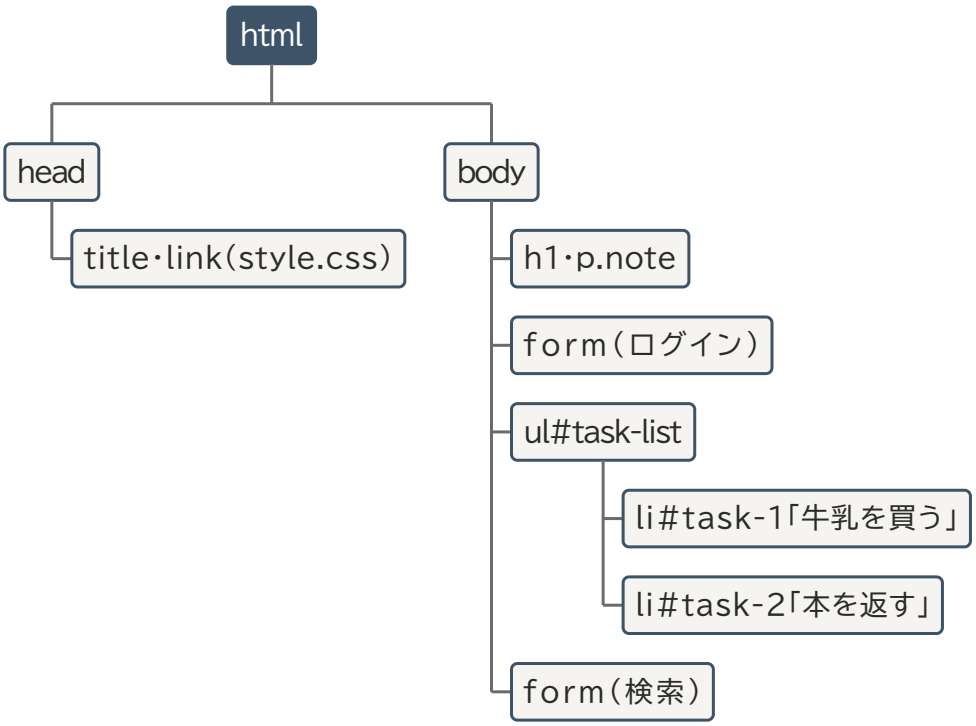
```
curl -sv -o /dev/null http://127.0.0.1:8000/ 2>&1 | grep -iE 'proxy|trying'   # Trying 127.0.0.1:8000 だけ
curl -sv -o /dev/null https://jsonplaceholder.typicode.com/ 2>&1 | grep -iE 'proxy|trying'
```

7. DOM・fetch・XSS をコンソールで（４分）

🔗 DOM の木を見る（ブラウザ）

📚 Web技術の基本 ４章（HTML・DOM）、MDN DOM の紹介

📒 `http://127.0.0.1:8000/` で「要素」を開く。ブラウザは HTML から DOM という木を作り、画面はこの木を描いたもの。JavaScript はこの木を書き換える



🖊 文字を書き換える

📒 画面は変わるが、変わったのはブラウザの中の DOM だけ。再読み込みでサーバの HTML に戻る

```
document.querySelector("#task-list li").textContent = "書き換えた"
```

🖊 fetch で追加して DOM に足す

📚 MDN フェッチ API の使用

📒 ページを移動せずに API に送り、返った JSON から `<li>` を作って足す。同じオリジンなので CORS は関係ない

```
const res = await fetch("/api/tasks", {method: "POST", headers: {"Content-Type":
"application/json"}, body: JSON.stringify({title: "コンソールから追加"})})
const task = await res.json()
const li = document.createElement("li"); li.textContent = task.title;
document.querySelector("#task-list").append(li)
```

🖊 textContent と innerHTML を比べる

📚 Web技術の基本 6章(XSS)

📒 `textContent` は文字として入れるので、タグがそのまま見える。`innerHTML` は HTML として解釈し、`onerror` のスクリプトが動いてダイアログが出る

```
document.querySelector("#task-list li").textContent = "<img src=x onerror=alert(1)>"
document.querySelector("#task-list li").innerHTML = "<img src=x onerror=alert(1)>"
```

🖊 エスケープしないページで XSS を起こす

📒 `/search-unsafe` は検索語をエスケープせずに HTML に入れる悪い例。ダイアログに出るのは `visits` だけで、`HttpOnly` の `session_id` は盗めない。`/search` はエスケープするので文字のまま出る

```
http://127.0.0.1:8000/search-unsafe?q=<img src=x onerror=alert(document.cookie)>
http://127.0.0.1:8000/search?q=<img src=x onerror=alert(document.cookie)>
```

- 終わったら、ターミナル①のデモのサーバを `Ctrl+C` で止める

読む章

区分	資料	範囲	読み方
参考書	イラスト図解式 この一冊で全部わかるWeb技術の基本 第2版(Web技術の基本)	3章 HTTPでやりとりする仕組み	通読(第10回でも使う)
参考書	同上	4章 データ形式(HTML・DOM・JSON)、5章 Webアプリケーションの基本(Web API)、6章 セキュリティと認証(Cookie・XSS・CSRF)	該当章(4章は第12回、5章は第10〜12回でも使う)
公式資料	MDN Web Docs	HTTP の概要、HTTP ヘッダー、HTTP レスポンスステータスコード、HTTP キャッシュ、HTTP Cookie の使用、同一オリジンポリシー、オリジン間リソース共有 (CORS)、フェッチ API の使用、DOM の紹介、JSON の操作	引く
紹介(副読本)	Webを支える技術	第2部 URI、第3部 HTTP	持っている人は該当章(第10回の API 設計でも使う)

第４回 実行環境の整備

tag: env-mise, env-python
前提タグ: linux-シェル, linux-アプリ, linux-ネットワーク
所要: 座学15分 / ハンズオン45分

ゴール

- システムの Python を直接使わない理由を、OS が使っていることとバージョンの違いから説明できる
- mise で Python と uv のバージョンをディレクトリごとに固定し、`cd` で切り替わる仕組みを説明できる
- uv で仮想環境を作り、`pyproject.toml` と `uv.lock` で依存パッケージを固定できる
- `mise.toml` に環境変数とタスクを書き、`mise run` で動かせる
- 渡すファイルと渡さないファイル(`.venv/`)を分け、隣の人が手順書(README)だけで再現できるように書ける

事前準備

- 第２回の環境設定手順書(`~/handsoff/workspace/session02/環境設定手順書.md`)の設定が済んでいること。社内ネットワークならプロキシの環境変数が `~/.bashrc` にある
- 講師から聞く: `mise.run`・`github.com`・`pypi.org` にプロキシ経由で届くか(open-questions.md の Q1・Q5)、社内 CA 証明書が要るか(Q3)

まとめ

冒頭の座学で使います。ハンズオンで打ったコマンドと確認は、その場で環境構築手順書(README.md)に書いていきます。

システムの Python を使わない理由

理由	中身	参照
OS の持ち物	`/usr/bin/python3` は apt で入り、OS のツールが使う。パッケージを入れると OS のツールが動かなくなりうるので、Ubuntu は pip で入れるのを止めている(`EXTERNALLY-MANAGED`)	モダンLinux 6.5
PC ごとに版が違う	版は Ubuntu の版で決まる。「自分の PC では動く」の原因になる	Python 公式 チュートリアル 12.1

理由	中身	参照
研修の方針	プロジェクトごとに Python 3.12 に固定する。システムの Python には何も入れない	mise 公式 Getting Started

3 つの層

層	道具	固定するファイル	固定するもの	参照
OS	apt	—	システムの Python・git・curl	モダンLinux 6.5
ツール	mise	`mise.toml`	Python・uv の版、環境変数、タスク	mise 公式 Configuration
パッケージ	uv	`pyproject.toml`	直接使うパッケージ（`rich ⩾ 15.0.0`）	uv 公式 Managing dependencies
		`uv.lock`	間接的に使うものも含めた全パッケージの正確な版	uv 公式 Locking and syncing
		`.venv/`	仮想環境（実物）。`uv.lock` から作り直せるので**渡さない**	Python 公式 venv

mise

コマンド・書き方	意味	参照（mise 公式）
`eval "$(~/.local/bin/mise activate bash)"`	`.bashrc` に書く。`cd` のたびとプロンプトの前に `mise.toml` を探して PATH を変える	Getting Started
`mise use --path . python@3.12`	このディレクトリの `mise.toml` に書いて入れる。`--path .` がないと親の `mise.toml` に書く	Getting Started
`mise install`	`mise.toml` に書いてある版を入れる（受け取った人が使う）	Dev Tools
`mise ls --current`・`mise config ls`	今使う版とその出どころ／読んでいる `mise.toml`	Configuration
`[env]`	環境変数。`_.python.venv` で `cd` したときに `.venv` を有効にする	Environments
`[tasks.名前]`・`mise tasks`・`mise run 名前`	よく打つコマンドに名前を付ける／一覧／実行	Tasks
`mise trust`	コピー・受け取った `mise.toml` を中身を見てから信頼する。未信頼の間は `[env]` を読まない	Configuration

uv

コマンド	意味	参照(uv 公式)
`uv init --app --vcs none --no-pin-python`	`pyproject.toml` と `main.py` を作る。Git は第５回、Python の版は `mise.toml` で決める	Creating projects
`uv add 名前`	`pyproject.toml` に書き、`uv.lock` を作り、`.venv` に入れるまでを一度に行う	Managing dependencies
`uv tree`	依存パッケージの木	Managing dependencies
`uv lock`	`pyproject.toml` から `uv.lock` を作り直す	Locking and syncing
`uv sync --locked`	`uv.lock` どおりに `.venv` を作る。`pyproject.toml` と食い違えば止まる	Locking and syncing
`uv run ファイル`	`.venv` の Python で動かす	Running commands

社内で必要な設定

通信先・設定	中身	参照
`mise.run`・`github.com`	mise のインストールと、mise による Python・uv の取得	open-questions Q5
`pypi.org`・`files.pythonhosted.org`	uv によるパッケージの取得	open-questions Q5
プロキシ	mise・uv は `https_proxy` などの環境変数を読む(第２回で設定済み)	uv 公式 Environment variables
社内 CA 証明書	uv は既定では OS に登録した証明書を読まない。`export UV_NATIVE_TLS=1` を `~/.bashrc` に書く	uv 公式 Environment variables

ハンズオン

1ステップ1コマンド。各ステップは「🖋 やること → 📚 参考書・公式資料の節 → 📒 説明 → コマンド」。作業場所は `~/handsoff/workspace/session04/`。見本は examples/session04/。

1. システムの Python を確かめる（4 分）

> 🖊 作業場所に移る

```
cd ~/handsoff/workspace/session04
```

> 🖊 システムの Python を見る

📚 Python 公式 チュートリアル 12.1 はじめに

📒 Ubuntu に最初から入っている Python は OS のもの。版は Ubuntu の版で決まり、隣の人と違うことがある

```
which -a python3; python3 --version   # 隣の人と比べる
```

> 🖊 誰が使っているかを見る

📚 モダンLinux 6.5 パッケージとパッケージマネージャ

📒 apt や OS のツールが、この Python に頼っている。研修では３つの層（まとめの表）に分け、プロジェクトで使う Python とパッケージを OS から切り離す

```
dpkg -S /usr/bin/python3; apt-cache rdepends --installed python3 | head
```

> 🖊 パッケージを入れさせない印を見る

📒 Ubuntu はこのファイルを置いて、システムの Python に pip でパッケージを入れるのを止めている

```
head -5 /usr/lib/python3*/EXTERNALLY-MANAGED
```

2. mise を入れる（7 分）

> 🖊 環境構築手順書を用意する

📒 第２回と同じく、打ったコマンドと確認をその場で書く。手順７で隣の人がこれだけを見て再現し、第５回でこのディレクトリをリポジトリにする

```
cp ~/handsoff/docs/session04/assets/環境構築手順書.md README.md; code README.md
```

- `code` が使えないときは `nano README.md`

🖊 mise を入れる

📚 mise 公式 Getting Started

📒 mise は1つのファイルとして `~/.local/bin/mise` に入る。社内で止まったら、プロキシの環境変数を確かめる（第2回）

```
curl https://mise.run | sh
```

🖊 入ったことを確かめる

```
~/.local/bin/mise --version
```

🖊 .bashrc のコピーを取る

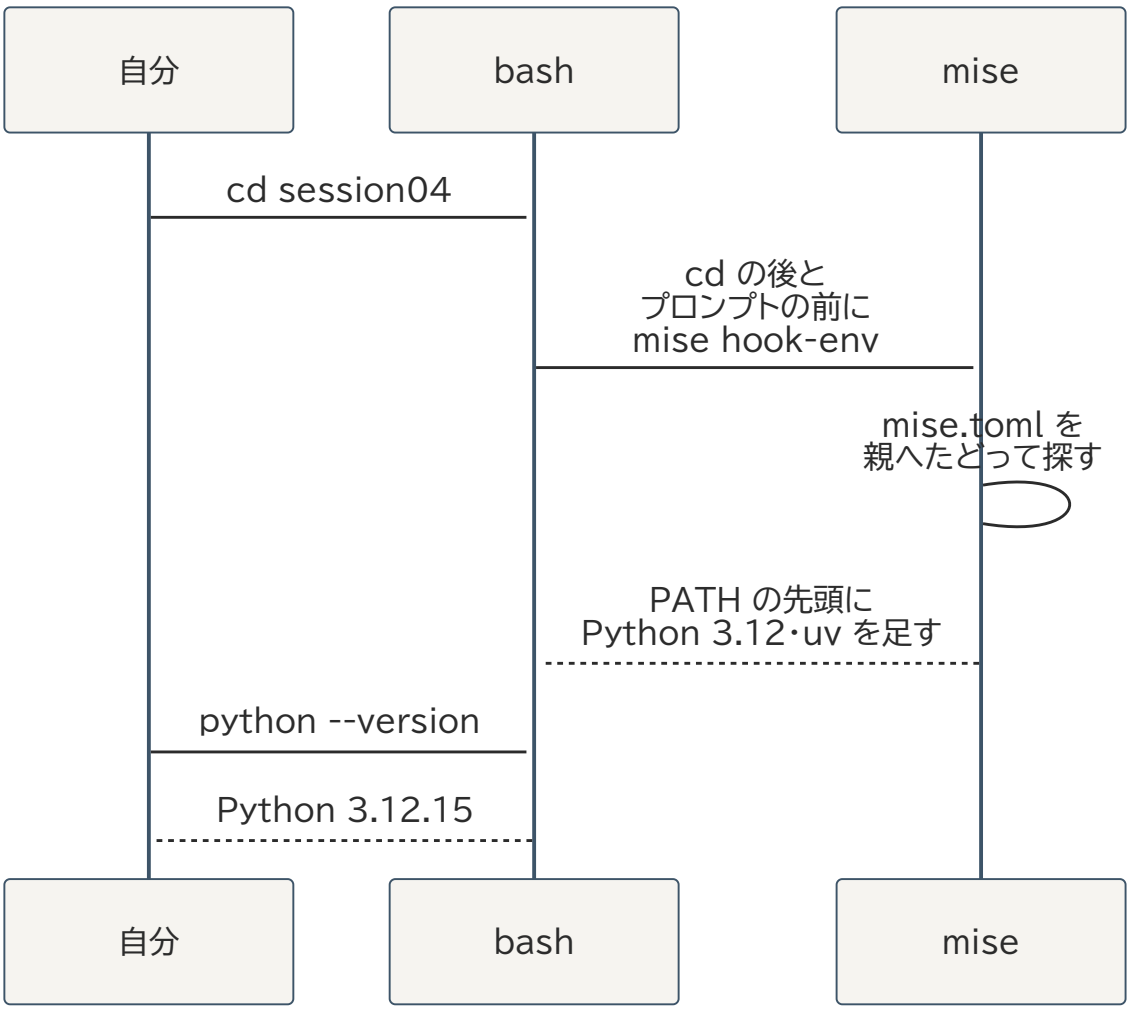
📒 第2回の `~/.bashrc.bak` は第2回の前の状態なので、上書きせずに別の名前で取る

```
cp ~/.bashrc ~/.bashrc.before-mise
```

🖊 .bashrc に activate を書く

📚 mise 公式 Getting Started（Activate mise）

📒 activate すると、`cd` のたびとプロンプトを出す前に mise が動く。mise は今のディレクトリから親へたどって `mise.toml` を探し、PATH を書き換える




```
echo 'eval "$(~/.local/bin/mise activate bash)"' >> ~/.bashrc; exec bash
```

> 🖊 **mise と cd が関数になったのを見る**

📒 activate は `mise` と `cd` を関数にする。第２回の探す順番では、関数は組み込みの `cd` より先に見つかる

```
mise --version; type -t mise cd   # function が2つ
```

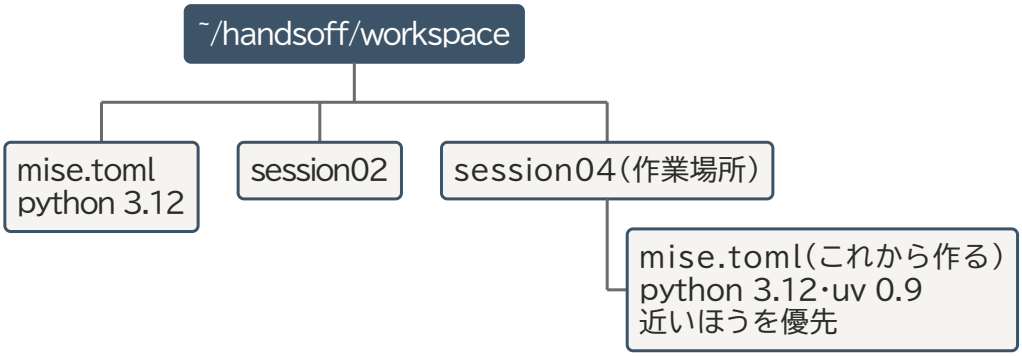
- 書く:「1. mise を入れる」(インストール → コピー → 追記 → `exec bash` → 確認は `mise --version`)

3. mise で Python と uv を固定する(7分)

> 🖊 **上の階層の mise.toml を見る**

📚 mise 公式 Configuration

📒 `workspace/` には研修用の `mise.toml` がある。mise は今のディレクトリから親へたどり、近いほうの設定を優先する




```
cat ../mise.toml; mise config ls
```

🖊 このディレクトリに固定する

📚 mise 公式 Getting Started（mise use）

📒 `mise use` は版を `mise.toml` に書き、まだなければ入れる。`--path .` を付けないと、いちばん近い `../mise.toml` に書き込む

```
mise use --path . python@3.12 uv@0.9
```

🖊 できた mise.toml を見る

```
cat mise.toml; mise config ls   # session04/mise.toml が増える
```

🖊 どの Python が動くかを見る

📒 mise は PATH の先頭に、入れた Python と uv のディレクトリを足す。`python` と打つと、`/usr/bin` より先にそこが見つかる

```
which python uv; python --version; mise ls --current
```

🖊 外に出て比べる

📒 `cd` するだけで切り替わる。ディレクトリの外ではシステムの Python に戻り、uv は見つからない

```
cd ~; python3 --version; type uv; cd -
```

- 書く:「2. Python と uv を入れる」。受け取った人は `mise use` ではなく、`mise.toml` に書いてある版を `mise install` で入れる。確認は `mise ls --current`

4. uv でパッケージを入れる(8分)

🖊 プロジェクトを作る

📚 uv 公式 Creating projects

📒 `pyproject.toml` はプロジェクトの名前と、直接使うパッケージを書くファイル。Git は第5回で自分で作り、Python の版は `mise.toml` だけで決める(`.python-version` を作らない)

```
uv init --app --vcs none --no-pin-python --name hello-todo
ls; cat pyproject.toml
```

🖊 パッケージを足す

📚 uv 公式 Managing dependencies

📒 `uv add` は、`pyproject.toml` への追記、`uv.lock` での版の固定、`.venv` へのインストールを一度に行う




```
uv add rich
```

🖊 何が入ったかを見る

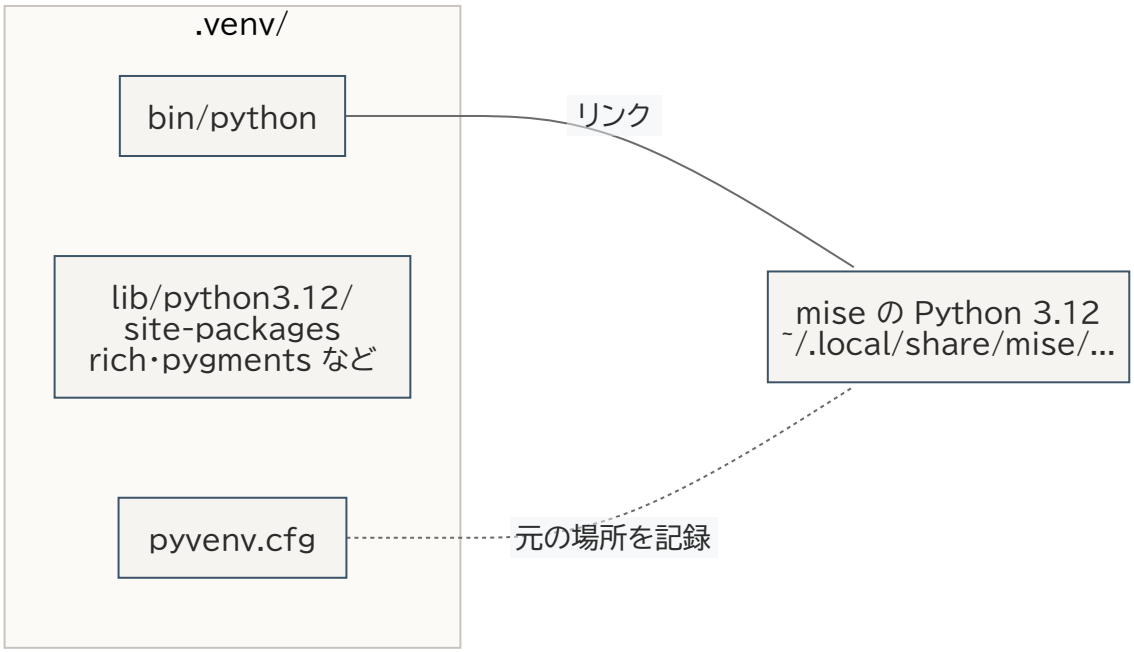
📒 rich が使うパッケージも一緒に入る。`uv.lock` は、この木にあるパッケージ全部の版を記録する

```
uv tree; grep -A1 '^name' uv.lock
```

🖊 仮想環境の中を見る

📚 Python 公式 チュートリアル 12.2 仮想環境の作成

📒 `.venv` の `python` は、mise が入れた Python へのリンク。パッケージは `.venv` の中にだけ入るので、システムの Python とほかのプロジェクトは変わらない




```
ls -l .venv/bin/python; cat .venv/pyvenv.cfg
```

🖊 動かす

📚 uv 公式 Running commands

📒 見本の `main.py` は rich でタスクの表を出し、最後の行に使った Python の場所を出す。`uv run` は `.venv` の Python で動かす

```
cp ~/handsoff/examples/session04/main.py .; uv run main.py
```

- 確認: 最後の行が `.../session04/.venv/bin/python`

5. mise.toml に環境変数とタスクを書く（8分）

🖊 環境変数を書く

📚 mise 公式 Environments

📒 `UV_PYTHON_DOWNLOADS` で、uv が自分で Python を落とさないようにする（mise の Python だけを使う）。`_.python.venv` で、`cd` したときに `.venv` が有効になる

```
cat >> mise.toml <<'EOF'

[env]
UV_PYTHON_DOWNLOADS = "never"
_.python.venv = { path = ".venv", create = true }
EOF
```

🖊 タスクを書く

📚 mise 公式 Tasks

📒 よく打つコマンドに名前を付けて `mise.toml` に残す。受け取った人は `mise tasks` で、何を打てばよいかがわかる

```
cat >> mise.toml <<'EOF'

[tasks.setup]
description = "依存パッケージを uv.lock どおりに入れる"
run = "uv sync --locked"

[tasks.hello]
description = "タスク一覧を表示する"
run = "python main.py"
EOF
```

🖊 タスクを動かす

📒 `mise run` は `mise.toml` の版と環境変数を設定してからコマンドを動かす。`python` は `.venv` の Python になる

```
mise tasks; mise run hello; which python   # .venv/bin/python
```

🖊 Python の版に上限を付ける

📚 uv 公式 Locking and syncing

📒 `⩾3.12` だけでは、mise を通さずに新しいシステムの Python で動かしても気づけない。上限を付けて 3.12 だけにする

```
sed -i 's/^requires-python = "⩾3.12"$/requires-python = "⩾3.12,<3.13"/' pyproject.toml;
grep requires-python pyproject.toml
```

🖊 食い違ったまま setup する

📒 `--locked` は、`pyproject.toml` と `uv.lock` が食い違うと、`uv.lock` を書き換えずに止まる。受け取った人の手元で版が勝手に変わるのを防ぐ

```
mise run setup   # The lockfile at `uv.lock` needs to be updated ...
```

🖊 uv.lock を作り直す

```
uv lock; mise run setup
```

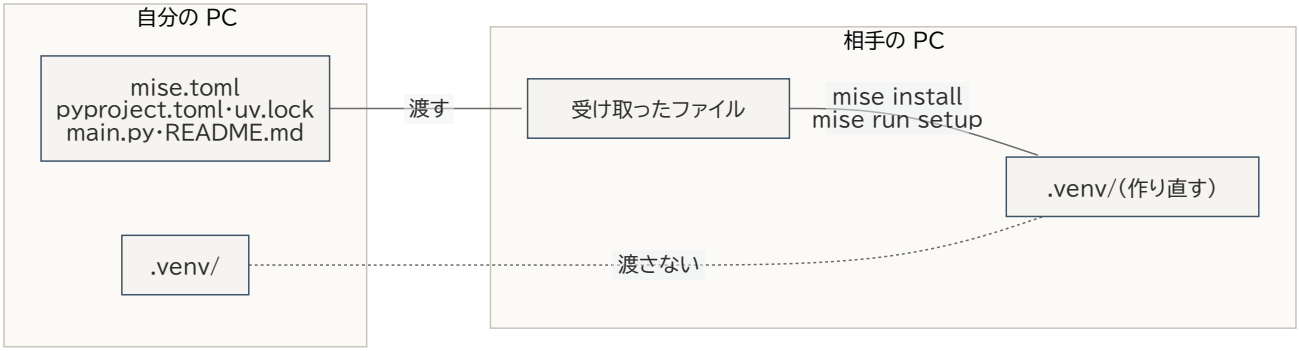
- 書く:「3. 依存パッケージを入れる」(`mise run setup`)と「4. 動かす」(`mise run hello`、確認は最後の行の Python の場所)

6. 渡すものを決め、別の場所で再現する(5 分)

🖊 渡すものと渡さないものを分ける

📚 Python 公式 チュートリアル 12.3 pip を使ったパッケージ管理

📒 `.venv` は `uv.lock` からいつでも作り直せる。中のリンクは自分の PC のパスを指すので、渡しても相手の PC では動かない




```
du -sh .venv; ls
```

- 書く:「渡すファイルと渡さないファイル」

🖊 別の場所にコピーする

📒 `workspace/` の外にコピーするので、`../mise.toml` には頼れない。`.venv` は渡さないのでコピーしない

```
mkdir ~/session04-check && cp mise.toml pyproject.toml uv.lock main.py README.md
~/session04-check/ && cd ~/session04-check
```

🖊 中身を見て信頼する

📚 mise 公式 Configuration(mise trust)

📒 `[env]` と `[tasks]` はコマンドを動かせる。コピーした `mise.toml` は信頼するまで `[env]` を読まない(`is not trusted` と出る)。`mise run` は自動で信頼するので、その前に中身を見る

```
cat mise.toml; mise trust
```

🖊 README の手順で動かす

```
mise install; mise run setup; mise run hello
```

- 確認: 最後の行が `~/session04-check/.venv/bin/python`。終わったら `cd ~/handsoff/workspace/session04; rm -rf ~/session04-check`

7. 隣の人の手順書で再現する（6 分）

🖊 渡すコピーを用意して席を替わる

📒 お互いに相手の PC の前に座り、相手の手順書だけを見て再現する。書いていないことは推測で補わず、止まった所と推測した所をメモして渡す。コピー先を手順 6 と別の名前にするので、信頼の手順も試せる

```
mkdir ~/session04-try && cp mise.toml pyproject.toml uv.lock main.py README.md
~/session04-try/
```

🖊 前提を確かめる（隣の人の PC で）

📒 第 2 回の設定は済んでいるので、隣の人の環境設定手順書の「確認」だけを打ち、前提がそろっているかを見る

```
type hello; env | grep -i _proxy   # 隣の人の手順書の「確認」の行を打つ
```

🖊 README だけで再現する（隣の人の PC で）

📒 新しいターミナルで、README の「2. Python と uv を入れる」から順に打つ。ディレクトリは `~/session04-try` に読み替える

```
cd ~/session04-try   # この後は README のとおりに
```

🖊 指摘を手順書に直す

📒 自分の席に戻り、メモを受け取って README を直す。止まった所は「うまくいかないとき」に書く

```
rm -rf ~/session04-try; grep -c '（書く）' README.md   # 0 になるまで埋める
```

- 書く:「うまくいかないとき」と「元に戻す方法」（`cp ~/.bashrc.before-mise ~/.bashrc`）

読む章

区分	資料	範囲	読み方
無料	mise 公式	Getting Started、Configuration、Environments、Tasks	主な資料（引く）

区分	資料	範囲	読み方
無料	uv 公式	Concepts の Projects（Creating projects、Managing dependencies、Locking and syncing、Running commands）、Environment variables	主な資料（引く）
無料	Python 公式ドキュメント	チュートリアル 12章 仮想環境とパッケージ、標準ライブラリ venv	該当章
参考書	入門 モダンLinux（モダンLinux）	6章 アプリケーション、パッケージ管理、コンテナ（6.4、6.5）	該当章（第2回の続き）
紹介（副読本）	Effective Python 第2版	10章 項目83 仮想環境	持っている人は該当章
紹介（副読本）	達人プログラマー 第2版	基本ツール（シェル、プレーンテキスト、バージョン管理）	持っている人は通読（第1〜5回で基本ツールの範囲）

本で補いたい人は、候補: 改訂新版 Python実践レシピ

環境構築手順

WSL の Ubuntu に mise と uv を入れ、`mise run hello` が動くところまでの手順です。

- 対象: (書く。例: WSL の Ubuntu 26.04)
- 所要時間: (書く)
- 前提: 第２回の環境設定手順書(PATH・プロキシ)が済んでいること。社内ネットワークでは `mise.run`・`github.com`・`pypi.org` にプロキシ経由で届くこと

1. mise を入れる(PC ごとに１回だけ)

> (書く: インストール → .bashrc のコピー → activate の行を追記 → 読み直す)

確認: (書く: 何が表示されれば次に進んでよいか)

2. Python と uv を入れる

> (書く: このディレクトリに移る → mise.toml の中身を見て信頼する → 入れる)

確認: (書く: `mise ls --current` の表示)

3. 依存パッケージを入れる

> (書く)

確認: (書く)

4. 動かす

> (書く)

確認: (書く: 最後の行に出る Python の場所)

渡すファイルと渡さないファイル

ファイル	役割	渡すか
`mise.toml`	（書く）	（書く）
`pyproject.toml`	（書く）	（書く）
`uv.lock`	（書く）	（書く）
`main.py`	動作確認用のプログラム	渡す
`.venv/`	（書く）	（書く）

うまくいかないとき

症状（表示）	原因	対処
（書く）		

元に戻す方法

> （書く：.bashrc を mise を入れる前に戻す）

確認：（書く）

第５回 Git と GitHub

tag: git-基礎, git-github, 標準, test-レビュー
前提タグ: linux-ファイル, linux-アプリ, env-mise, env-python
所要: 座学15分 / ハンズオン45分

ゴール

- 作業ツリー・ステージ・リポジトリ・リモートの４つの場所と、`add`・`commit`・`push` でファイルがどう移るかを説明できる
- `.gitignore` を先に書き、目的ごとの小さなコミットに「何を・なぜ」のメッセージを付けられる
- main を直接触らず、Issue → ブランチ → PR → セルフレビュー → マージの流れで変更を入れ、マージで Issue を閉じられる
- ２本のブランチが同じ行を変えたコンフリクトを、両方の目的が成り立つ形で解決できる

事前準備

- 第４回のハンズオンで作った一式が `~/handsoff/workspace/session04/` にある（`README.md`・`mise.toml`・`pyproject.toml`・`uv.lock`・`main.py`）
- `<Gitホスティング>` のアカウントと、push に使うトークン。講師が案内する（open-questions.md の Q4）
- 社内ネットワークでは、第２回の手順６で入れたプロキシの環境変数が `~/.bashrc` にある。git は環境変数を読むので、git の設定は要らない（Q1）

まとめ

冒頭の座学で使います。ハンズオンでは、第４回の環境構築手順書をリポジトリにし、Issue からマージまでを１人で２回通します。

４つの場所

場所	ひとことで	次の場所へ	中を見る	参照（Pro Git）
作業ツリー	編集しているファイル	`git add`	`git status`・`git diff`	1.3、2.2
ステージ	次のコミットに入れるもの。変更の一部だけを選べる	`git commit`	`git diff --staged`	2.2

場所	ひとことで	次の場所へ	中を見る	参照(Pro Git)
リポジトリ(`.git/`)	コミットの履歴	`git push`	`git log`	2.3
リモート(`origin`)	`<Gitホスティング>` 上のリポジトリ。Issue と PR もここ	`git fetch`・`git pull` で手元へ	`git branch -a`	2.5、3.5

コマンド

コマンド	意味	参照(Pro Git)
`git config --global 名前 値`	全リポジトリで使う設定(`~/.gitconfig`)	1.6
`git init`	**今いるディレクトリ**をリポジトリにする(`.git/` を作る)	2.1
`git status`・`git status --short`	どのファイルがどの場所にあるか。迷ったらこれ	2.2
`git add ファイル`・`git commit -m "何を" -m "なぜ"`	ステージに載せる／記録する。2つ目の `-m` が本文	2.2
`git diff`・`git diff --staged`	作業ツリーとステージ／ステージと最新のコミットの差分	2.2
`git restore ファイル`	作業ツリーの変更を捨て、最新のコミットに戻す	2.4
`git log --oneline --graph --all`	履歴を1行ずつ、分岐と合流を線で	2.3、3.2
`git remote add origin URL`・`git push -u origin 名前`	リモートを登録する／ブランチを初めて送る	2.5
`git switch -c 名前`・`git switch 名前`	ブランチを作って移る／移る	3.2
`git fetch origin`・`git merge origin/main`	リモートの最新を取る／今のブランチに取り込む	3.2、3.5
`git merge --abort`	マージを途中でやめ、マージの前に戻す	3.2
`git pull`・`git branch -d 名前`	fetch + merge／マージ済みのブランチを消す	2.5、3.2

Issue・ブランチ・PR

用語	ひとことで	参照
Issue	やること。目的と完了条件を書く	Pro Git 6.2
ブランチ	コミットを指す名前。main から分けて、main を壊さずに作業する	Pro Git 3.1
PR	ブランチの変更を main に入れる提案とレビューの場(GitLab では Merge Request)	Pro Git 6.2

用語	ひとことで	参照
`Closes #1`	PR の本文に書くと、マージで Issue #1 が閉じる（GitHub・GitLab 共通）	Pro Git 6.2
セルフレビュー	マージの前に、自分の PR の差分を他人のつもりで読む	SE基礎 2章・3章
コンフリクト	2本のブランチが同じ箇所（隣の行を含む）を変え、Git が自動でまとめられない状態	Pro Git 3.2

判断の基準（研修の規約）

場面	基準	参照
コミット	1コミット1目的。1行目＝何を（〜する）、本文＝なぜ（差分を見てもわからないこと）	Pro Git 5.2
入れないもの	`.venv/` と秘密情報（`.env`）。**add の前に** `.gitignore` に書く	Pro Git 2.2
ブランチ	main に直接コミットしない（作った直後の数コミットだけ例外）。1 Issue = 1 ブランチ `issue-<番号>-<内容>`	Pro Git 3.4
PR の本文	目的（`Closes #番号`）・変更点・確認方法（テンプレート）	Pro Git 6.2
セルフレビュー	目的・確認方法・読みやすさ・履歴・安全を見て、確かめたことをコメントに残す	SE基礎 2章・3章
コンフリクト	**どちらかを選ばず、両方の目的が成り立つ形**にする。記号の行（`<<<<<<<`・`=======`・`>>>>>>>`）はすべて消す	Pro Git 3.2
第6回から	アプリのリポジトリ（`workspace/todo-app/`）でも、変更は同じ流れ（Issue → ブランチ → PR → セルフレビュー → マージ）で入れる	—

ハンズオン

1ステップ1コマンド。各ステップは「🖋 やること → 📚 参考書・公式資料の節 → 📒 説明 → コマンド」。作業場所は `~/handsoff/workspace/session05/`（手順書のリポジトリ）。見本は examples/session05/。ブラウザで行うステップは、GitHub の画面の名前で書き、GitLab の名前を括弧に入れる。

1. Git の設定（3分）

> 🖋 **Git の版を見る**

📒 `git switch` は 2.23 以降、`init.defaultBranch` は 2.28 以降で使える

```
git --version
```

🖊 名前とメールアドレスを設定する

📚 Pro Git 1.6 最初のGitの構成

📒 名前とメールアドレスは、コミットごとに作者として記録される。`--global` はホームの `~/.gitconfig` に書き、全リポジトリで使う

```
git config --global user.name "自分の名前（ローマ字）"
git config --global user.email "<Gitホスティング> に登録したメールアドレス"
```

🖊 ブランチ名・改行・日本語・エディタを設定する

📒 最初のブランチを main にし、改行を LF にそろえ、日本語のファイル名を化けさせない。エディタは、マージコミットのメッセージを書くときなどに開く

```
git config --global init.defaultBranch main
git config --global core.autocrlf input
git config --global core.quotepath false
git config --global core.editor nano
```

🖊 設定を確かめる

```
git config --global --list   # 6行が出る
```

2. リポジトリを作る（4分）

🖊 作業場所に移る

📒 `git init` は今いるディレクトリをリポジトリにする。`~` や `~/handsoff` で打つと、ホームや研修ディレクトリ全体が対象になるので、必ず `pwd` で確かめる

```
cd ~/handsoff/workspace/session05; pwd
```

🖊 第４回の一式をコピーする

📒 `.venv` は渡さないもの（第４回の手順６）なのでコピーしない。作業場所の案内の `README.md` は、手順書で上書きしてよい

```
cp ../session04/{README.md,mise.toml,pyproject.toml,uv.lock,main.py} .
```

🖊 mise.toml を信頼して .venv を作る

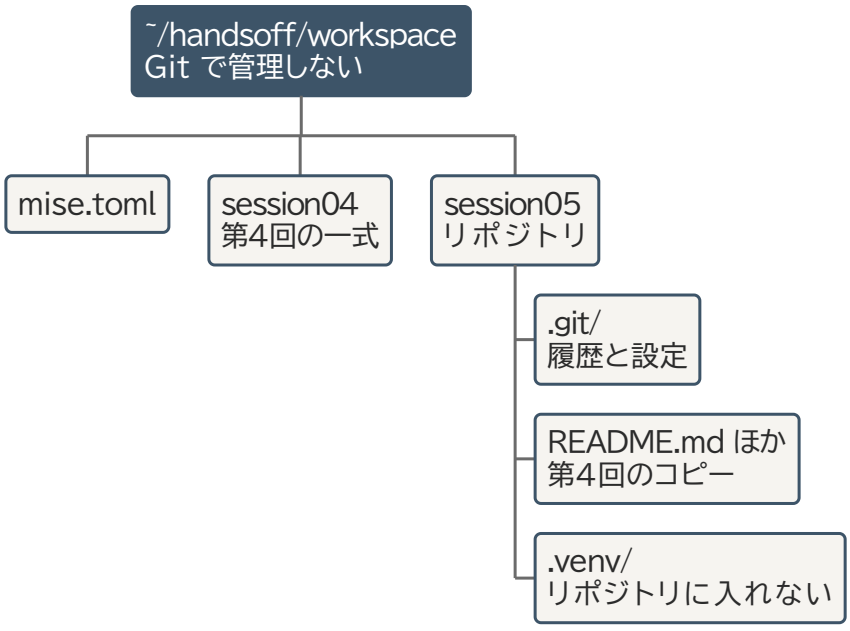
📒 コピーした `mise.toml` はまだ信頼されていない（第４回の手順６）。中身を見てから信頼し、手順書の「3. 依存パッケージを入れる」で `.venv` を作る

```
cat mise.toml; mise trust; mise run setup
```

🖊 リポジトリにする

📚 Pro Git 2.1 Gitリポジトリの取得

📒 `git init` は `.git/` を作る。履歴と設定はすべてここに入る。リポジトリにするのは `session05` だけで、`workspace/` と `~/handsoff` は Git で管理しない




```
git init; ls -a
```

🖊 状態を見る

📚 Pro Git 2.2 変更内容のリポジトリへの記録

📒 まだ何も記録していないので、ファイルはすべて「追跡されていない（Untracked）」。迷ったらまず `git status`

```
git status   # On branch main / No commits yet
```

3. 先に .gitignore を書く（5 分）

🖊 .venv が出てこない理由を見る

📒 uv が作る `.venv` には `*` だけを書いた `.gitignore` が入っており、`.venv` の中身はすべて無視される。`python -m venv`（3.12）で作ると入らないので、リポジトリの `.gitignore` にも書く

```
cat .venv/.gitignore   # *
```

🖊 秘密情報のファイルを置く

📒 `.env` は秘密情報を書くことが多いファイル名。ここではダミーを置き、`git status` に出ることを見る

```
echo 'HTTP_PROXY=http://user:password@proxy.example.com:8080' > .env; git status --short
# ?? .env
```

🖊 .gitignore を書く

📚 Pro Git 2.2 変更内容のリポジトリへの記録（ファイルの無視）

📒 一度コミットしたファイルは、後で `.gitignore` に書いても過去のコミットに残る。そのため add より先に書く

```
nano .gitignore
```

```
# 仮想環境（mise.toml と uv.lock から作り直せる）
.venv/
# Python が実行時に作るファイル
__pycache__/
# 秘密情報
.env
```

🖊 .env が消えたのを見る

📒 `git check-ignore -v` は、どのファイルの何行目で無視されたかを出す

```
git status --short; git check-ignore -v .env .venv
```

🖊 .gitignore だけを最初にコミットする

📚 Pro Git 5.2 プロジェクトへの貢献（コミットのガイドライン）

📒 1行目に「何を」、2つ目の `-m`（本文）に「なぜ」を書く

```
git add .gitignore; git commit -m ".venv と .env を Git の管理から外す" -m "仮想環境は作り直せ、.env には秘密情報が入るため。"
```

- 確かめたら `rm .env`

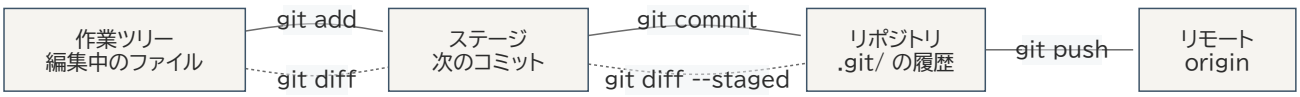
> 座学: add の前に `.gitignore` を書く理由（→ まとめ「判断の基準」）

4. 目的ごとにコミットする（7分）

🖊 手順書をステージに載せる

📚 Pro Git 1.3 Gitの基本、2.2 変更内容のリポジトリへの記録

📒 `add` で作業ツリーからステージへ、`commit` でステージからリポジトリへ移す。2段階なので、変更の一部だけをコミットできる



```
git add README.md; git status --short   # A  README.md
```

🖊 これからコミットする中身を見る

```
git diff --staged | head -20
```

🖊 手順書をコミットする

📒 1行目は「～する」の形で、そのコミットを入れると何が起きるかを書く（規約の2）

```
git commit -m "第4回の環境構築手順書を追加する"
```

🖊 版を固定するファイルをまとめてコミットする

📒 3つのファイルで1つの目的（版の固定）なので、1つのコミットにする

```
git add mise.toml pyproject.toml uv.lock
git commit -m "Python と依存パッケージのバージョンを固定する" -m "README の手順どおりに進めれば、誰の PC でも同じバージョンになるようにする。"
```

🖊 残りをコミットする

```
git add main.py; git commit -m "動作確認用の main.py を追加する"
```

🖊 履歴を見る

📚 Pro Git 2.3 コミット履歴の閲覧

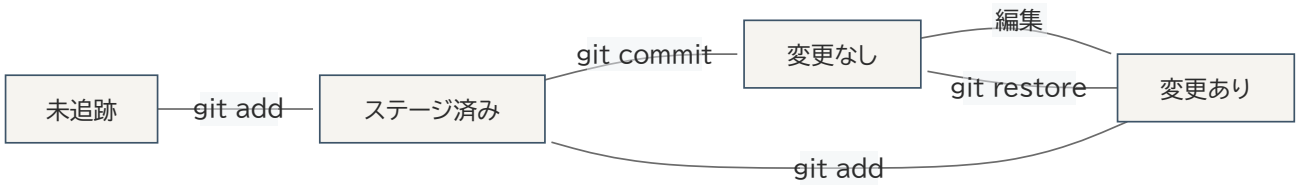
📒 コミットは1つ前のコミットを指してつながる。`git log --oneline` の1行目だけで、何をしたかがわかるようにする

```
git log --oneline; git status   # nothing to commit, working tree clean
```

🖊 間違えた変更を入れる

📚 Pro Git 2.4 作業のやり直し

📒 コミットした後のファイルは、編集すると「変更あり」になる。`add` する前なら、`git restore` で最新のコミットの状態に戻せる




```
echo 'まちがい' >> README.md; git diff
```

🖊 最新のコミットの状態に戻す

```
git restore README.md; git status
```

5. リモートに push する（4分）

🖊 空のリポジトリを作る（ブラウザ）

📒 `<Gitホスティング>` に非公開のリポジトリ `session05` を作る。README などの初期ファイルを付けると、手元にないコミットができて push が拒否されるので、何も付けない

🖊 リモートを登録する

📚 Pro Git 2.5 リモートでの作業

📒 `origin` はリモートの URL に付ける別名

```
git remote add origin <リポジトリのURL>; git remote -v
```

🖊 push する

📒 ユーザー名とパスワードを聞かれたら、パスワードにはトークンを入れる。`-u` で手元の main とリモートの main を結び付け、次からは `git push` だけで送れる

```
git push -u origin main
```

🖊 リモートのブランチを見る

📚 Pro Git 3.5 リモートブランチ

📒 `origin/main` は、最後にリモートとやり取りしたときのリモートの main を、手元に記録したもの

```
git branch -a; git status   # up to date with 'origin/main'
```

- 確認: ブラウザでファイルと４つのコミットが見える

6. Issue を立て、ブランチを２本作る（8分）

🖊 Issue を２つ立てる（ブラウザ）

📚 Pro Git 6.2 プロジェクトへの貢献

📒 Issue の本文には、目的と完了条件を１行ずつ書く。２つとも README.md の１行目の末尾を変えるので、後でコンフリクトが起きる

Issue	タイトル	目的	完了条件
#1	手順書のタイトルに対象の環境を書く	読んでよい手順書かを最初の行で判断できる	１行目の末尾に `（WSL の Ubuntu 向け）`
#2	手順書のタイトルに所要時間の目安を書く	始める前に作業の時間を確保できる	１行目の末尾に `（所要30分）`

🖊 １本目のブランチを作る

📚 Pro Git 3.1 ブランチとは

📒 ブランチはコミットを指す名前。`HEAD` は今いるブランチを指し、コミットすると今いるブランチだけが進む

```
git switch -c issue-1-target-env
```

🖊 1行目を変える

```
sed -i '1s/$/（WSL の Ubuntu 向け）/' README.md; git diff   # nano で直してもよい
```

🖊 コミットする

📒 `-a` は、追跡しているファイルの変更をまとめて add する

```
git commit -am "手順書のタイトルに対象の環境を書く" -m "読んでよい手順書かを最初の行で判断できるようにする。"
```

🖊 1本目を push する

📒 GitHub では、push の出力に PR を作る URL が出る

```
git push -u origin issue-1-target-env
```

🖊 main に戻る

📒 2本目は必ず main から作る。1本目の上に作ると、2本目に1本目の変更が入り、コンフリクトも起きない

```
git switch main; head -1 README.md   # # 環境構築手順 のまま
```

🖊 2本目のブランチで同じ行を変える

```
git switch -c issue-2-duration; sed -i '1s/$/（所要30分）/' README.md
```

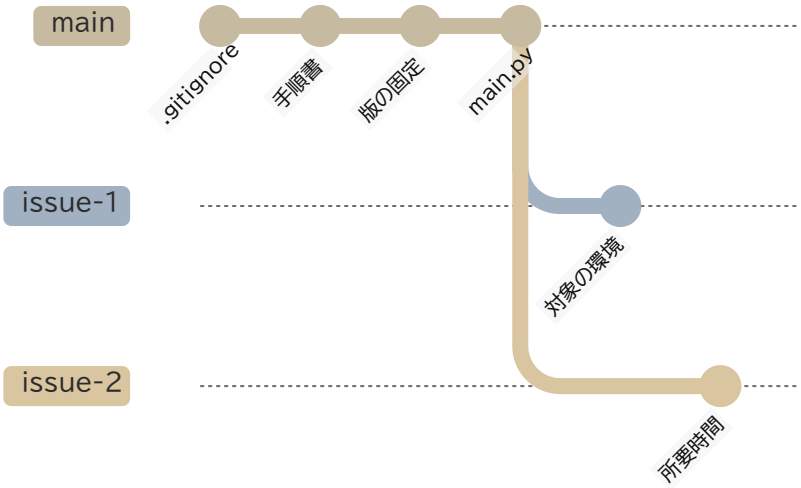
🖊 2本目をコミットして push する

```
git commit -am "手順書のタイトルに所要時間の目安を書く" -m "始める前に作業の時間を確保できるようにする。"
git push -u origin issue-2-duration
```

🖊 分かれ方を見る

📚 Pro Git 3.2 ブランチとマージの基本

📒 2本とも同じ main のコミットから分かれている。分かれ方が違ったら講師を呼ぶ。図ではブランチ名を `issue-1`・`issue-2` と略す




```
git log --oneline --graph --all
```

7. PR を出し、セルフレビューして1本目をマージする（6分）

🖊 PR を2つ作る（ブラウザ）

📚 Pro Git 6.2 プロジェクトへの貢献

📒 「Compare & pull request」（GitLab は push の出力の URL）から作る。マージ先は `main`、本文はテンプレートの目的・変更点・確認方法で書き、目的に `Closes #1`（2本目は `Closes #2`）を入れる

PR	ブランチ	目的に書く行
1本目	`issue-1-target-env` → `main`	`Closes #1`
2本目	`issue-2-duration` → `main`	`Closes #2`

🖊 手元でも PR の差分を見る

📒 `main...ブランチ` は、ブランチが main から分かれた後の変更だけを見せる。PR の「Files changed」（GitLab は「Changes」）と同じ差分になる

```
git diff main...issue-1-target-env
```

🖊 セルフレビューする（ブラウザ）

📚 SE基礎 2章 コードを読む、3章 コードを書く

📒 「Files changed」で差分を他人のつもりで読み、確かめたことを `[確認]` で始まる行コメントに1件以上残す。観点は規約の7（目的・確認方法・読みやすさ・履歴・安全）。自分の PR は承認（Approve）できない

🖊 1本目をマージする（ブラウザ）

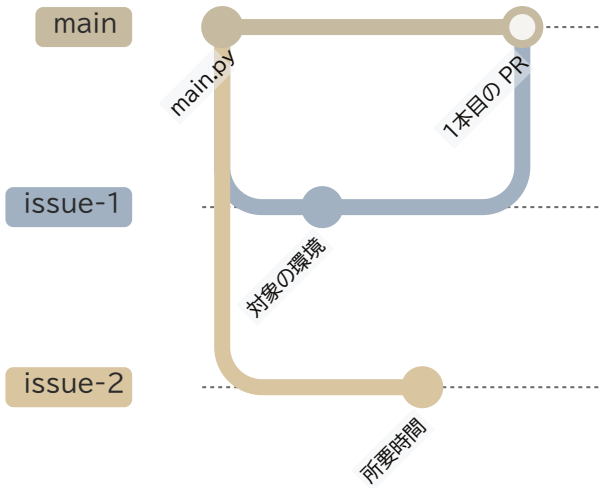
📒 「Merge pull request」（GitLab は「Merge」）で、既定のマージコミットでマージする。`Closes #1` で Issue #1 が閉じ、同じ行を変えた2本目の PR はコンフリクトの表示に変わる

- 確認: Issue #1 が Closed。2本目の PR に「This branch has conflicts」（GitLab は「Merge blocked」など）

🖊 リモートの main を取ってくる

📚 Pro Git 3.5 リモートブランチ

📒 マージはリモートで起きたので、手元の `origin/main` は古いまま。`git fetch` でリモートの最新を取ってくる（手元の main は変わらない）




```
git fetch origin; git log --oneline --graph --all
```

座学: PR の3項目とセルフレビューの観点（→ まとめ「判断の基準」／SE基礎 3章）

8. コンフリクトを解決して２本目をマージする（８分）

🖊 ２本目のブランチに main を取り込む

📚 Pro Git 3.2 ブランチとマージの基本（マージ時のコンフリクト）

📒 ２本のブランチが同じ行を変えたので、どちらを残すかを Git は決められず、マージが途中で止まる

```
git switch issue-2-duration; git merge origin/main   # CONFLICT (content): Merge conflict in README.md
```

🖊 状態を見る

📒 迷ったら `git merge --abort` で、マージの前に戻せる

```
git status   # both modified: README.md
```

🖊 記号を読む

📒 `HEAD` から `=======` までが今のブランチ（＃2）、`=======` から下が取り込んだ main（＃1）の内容

```
<<<<<<< HEAD
# 環境構築手順（所要30分）                ← 今のブランチ（#2）
=======
# 環境構築手順（WSL の Ubuntu 向け）      ← main に入った変更（#1）
>>>>>>> origin/main
```

```
head -5 README.md
```

🖊 両方を成り立たせて直す

📒 片方だけ残すと、もう一方の Issue は閉じたのに目的が失われる。記号の行はすべて消す

```
nano README.md   # 1行目を「# 環境構築手順（WSL の Ubuntu 向け・所要30分）」にする
```

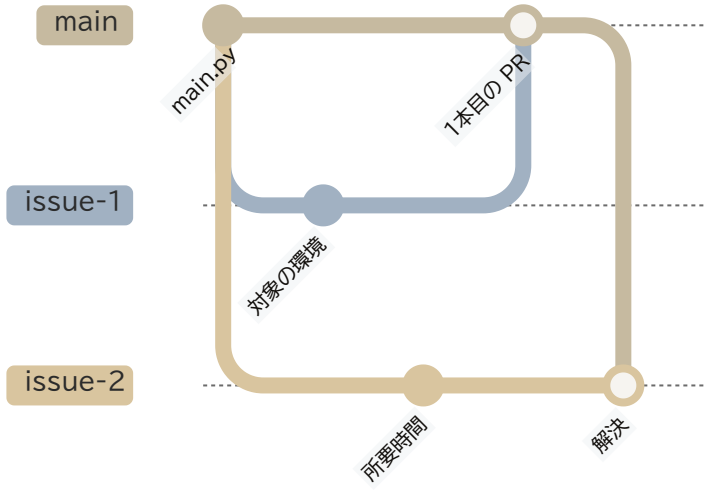
🖊 記号が残っていないか確かめる

📒 Git は記号が残っていても add·commit できる。add の前に自分で確かめる

```
grep -n '^<<<<<<<\|^=======\|^>>>>>>>' README.md   # 何も出なければよい
```

🖊 マージコミットを作って push する

📒 解決した結果は、2本目のブランチのマージコミットになる。push すると2本目の PR に加わり、コンフリクトの表示が消える




```
git add README.md; git commit --no-edit; git push
```

🖊 もう一度セルフレビューしてマージする（ブラウザ）

📒 2本目の PR の差分を読み直し、どう解決したかをコメントに残してからマージする。`Closes #2` で Issue #2 が閉じる

- 確認: Issue が2つとも Closed

🖊 手元の main をそろえる

📚 Pro Git 2.5 リモートでの作業

📒 `git pull` は `git fetch` と `git merge` をまとめて行う

```
git switch main; git pull; head -1 README.md   # 両方の変更が入った1行
```

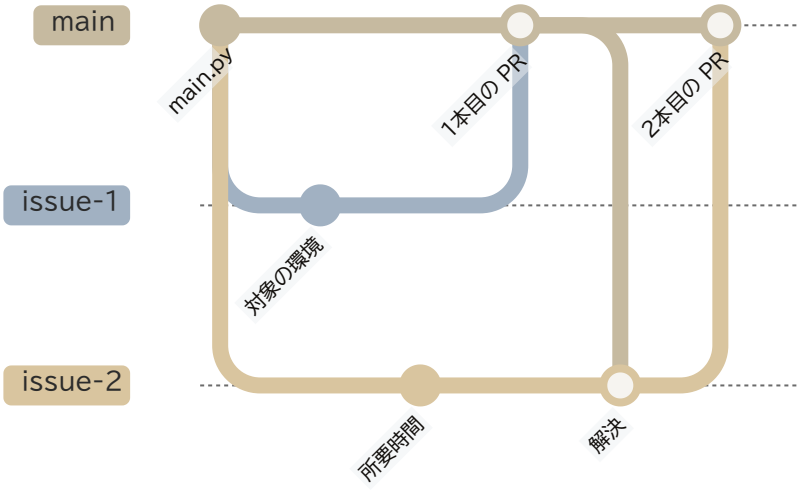
🖋 済んだブランチを消す

📒 `-d` はマージ済みのブランチだけを消す。マージしていないと消さずに止まる

```
git branch -d issue-1-target-env issue-2-duration
```

🖋 全体の履歴を見る

📒 最初の４つのコミットの後、main への変更はすべて PR のマージコミットを通っている。第６回からのアプリのリポジトリでも、この形を保つ




```
git log --oneline --graph
```

> 座学: コンフリクトは「両方を成り立たせる」。片方だけ残すと何が失われるか（→ まとめ「判断の基準」）

読む章

区分	資料	範囲	読み方
無料	Pro Git（日本語版）	1章 使い始める、2章 Git の基本、3.1〜3.2・3.5 ブランチ、5.2 プロジェクトへの貢献、6.2 GitHub のプロジェクトへの貢献	該当章（ハンズオンの「参照」の節を中心に）
無料	同上	2.4 作業のやり直し、3.6 リベース、7.7 リセットコマンド詳説	引く（取り消したいとき）
参考書	ソフトウェアエンジニアリングの基礎（SE基礎）	2章 コードを読む、3章 コードを書く	通読（セルフレビューの観点。第７回でも使う）

区分	資料	範囲	読み方
紹介(副読本)	達人プログラマー 第2版	基本ツール(バージョン管理)	持っている人は通読(第1〜5回で基本ツールの範囲)
紹介(副読本)	リーダブルコード	巻末解説の添削コミット	持っている人は(セルフレビューの参考)

研修の Git の規約（コミット・ブランチ・PR・セルフレビュー）

第５回から使う、研修共通の規約です。手順書のリポジトリ（`workspace/session05/`）と、第６回に作るアプリのリポジトリ（`workspace/todo-app/`）の両方で使います。第７回で自分のコーディング規約を決めるときに、見直してかまいません。

1. １つのコミットには１つの目的だけを入れる

- 「何のための変更か」を１文で言えない変更は、コミットを分ける
- 例: 「手順書の追加」と「誤字の修正」は別のコミットにする
- 迷ったら小さく分ける。分けたコミットはまとめられるが、混ざったコミットを後から分けるのは難しい

2. メッセージは「１行目＝何を」「本文＝なぜ」

```
README の mise のインストール手順に URL を追記する        ← 1行目: 何をしたか（日本語なら30文字程度まで）

隣の人に手順書を試してもらったとき、どこから入れるのかで   ← 本文: なぜそうしたか
迷ったため。                                              （1行目との間は1行空ける）
```

- １行目は「～する」の形で、そのコミットを入れると何が起きるかを書く
- 本文には、差分（`git diff`）を見てもわからないこと（理由、背景、試してやめたこと）を書く
- １行目だけで理由まで伝わる小さな変更なら、本文は省略してよい
- 日本語で書く

3. コミットしないもの

もの	理由	対処
`.venv/` などの仮想環境	`mise.toml` と `uv.lock` から作り直せる。ファイル数が多く、PC ごとに中身が違う	`.gitignore` に書く
`.env`、パスワード、トークン、プロキシの認証情報	履歴に一度入ると、後のコミットで消しても過去のコミットから読める	`.gitignore` に書く。入れてしまったら講師に相談し、パスワードを変える
`__pycache__/`、`tasks.json`・`var/` などの実行で自動でできるファイル	作り直せる。実行するたびに変わる	`.gitignore` に書く

4. コミットの前に確かめる

```
git status          # 意図しないファイルが入っていないか
git diff --staged   # これからコミットする差分
```

5. ブランチ運用

- main に直接コミットしない。main はいつでも動く・読める状態に保ち、変更はブランチ → PR → マージで入れる（例外は、リポジトリを作った直後の最初の数コミットだけ）
- 1 Issue = 1 ブランチ。名前は `issue-<番号>-<内容（英小文字とハイフン）>`（例: `issue-1-target-env`）
- ブランチは必ず最新の main から作る（`git switch main` → `git pull` → `git switch -c 名前`）
- ブランチは小さく短くする。放置するほど main との差が広がり、コンフリクトしやすい
- マージは既定のマージコミットで行う。マージしたブランチは消してよい

6. Issue と PR の書き方

項目	書くこと
Issue のタイトル	やることを「〜する」の形で
Issue の本文	目的（誰が何に困っているか）、完了条件（何ができていれば終わりか）
PR のタイトル	何を変えたか（Issue のタイトルと同じでよい）
PR の目的	なぜ必要か。`Closes #番号` で Issue と紐付ける（マージで Issue が閉じる）
PR の変更点	何を変えたか。変えていないことや、別の Issue で扱うことも
PR の確認方法	他人が同じ確認をできる手順と、期待する結果

テンプレート: docs/templates/pull_request.md。見本: examples/session05/pr-description.md

7. セルフレビューの観点

マージの前に、PR の差分（GitHub「Files changed」／GitLab「Changes」）を自分で上から読みます。他人のつもりで読み、気づいたことは行コメントに残します。

観点	見ること
目的	PR の目的と変更点が一致している。関係ない変更が混ざっていない
確認方法	書いた手順を自分で実行し、書いたとおりの結果になる
読みやすさ	初めて読む人に伝わる。誤字や手順の抜けがない
履歴	コミットが「何を・なぜ」の単位に分かれている

観点	見ること
安全	秘密情報、`.venv/`、コンフリクトの記号（`<<<<<<<` など）が入っていない

コメントは **事実**（どこで何が）・**理由**（なぜ気になるか）・**提案**（どうするか）をそろえ、先頭に `[必須]`（直してからマージ）／`[提案]`／`[質問]`／`[良い点]`／`[確認]`（確かめたこと）を付けます。見本: examples/session05/self-review.md

8. コンフリクトの解決

- どちらかを選ばず、両方の Issue の目的が成り立つ形にする。片方を捨てると、その Issue は閉じても目的が失われる
- どちらの目的とも違う新しい案を思いついたら、この PR で決めずに新しい Issue にする
- 記号の行（`<<<<<<<`・`=======`・`>>>>>>>`）はすべて消し、`grep` で残っていないことを確かめてから `git add` する
- 解決したら PR の差分をもう一度セルフレビューしてからマージする

悪い例と良い例

悪い例	何が困るか	良い例
`修正`	何を直したのか、履歴から探せない	`README の uv のバージョン指定の誤りを直す`
`いろいろ`	1つのコミットに複数の目的が混ざっている	目的ごとに分けてコミットする
`README.md を変更`	ファイル名は `git log --stat` でわかる。何のための変更かがわからない	`README に mise install の手順を追記する`
`第4回の分`	1つのコミットに複数の目的が混ざっている	`Python と依存パッケージのバージョンを固定する`
ブランチ名 `test`・`fix`	何の作業か、どの Issue かわからない	`issue-3-proxy-note`
PR の本文が空	目的も確認方法もわからず、レビューできない	目的（`Closes #3`）・変更点・確認方法を書く

第６回 Python の基本

tag: python-構文
前提タグ: env-python, git-基礎, git-github
所要: 座学15分 / ハンズオン45分

ゴール

- list と dict で TODO のデータを持ち、`d["k"]` と `d.get("k")`・内包表記・f-string を使い分けられる
- 型ヒント付きの関数を書き、データの操作と表示を別の関数に分けられる
- `pathlib`・`with`・`json` でタスクをファイルに保存し、読み直せる
- 起こりうる例外だけを `try` ／ `except` で扱い、利用者向けの文言と終了コードにできる
- `argparse` と `match` でサブコマンド(add / list / done)を振り分けられる
- アプリのリポジトリ `todo-app` を作って push し、機能の追加(`--due` と `list --overdue`)を Issue → ブランチ → PR → セルフレビュー → マージで入れられる

事前準備

- 第４回の mise が使え、`~/handsoff/workspace` で `python --version` が 3.12
- 第５回の Git の設定と push の認証(トークン)が済んでいる。`<Gitホスティング>` に新しい非公開リポジトリを作れる(open-questions.md の Q4)

まとめ

冒頭の座学で使います。ハンズオンでは雛形の関数の中身を順に書き、TODO の CLI を作ってアプリのリポジトリに入れます。

書き方

書き方	例	意味	参照(Python公式)
list	`[1, 2]`、`tasks[0]`、`tasks.append(x)`	順番のある並び。番号(0から)で取り出す	チュートリアル 3.1.3、5.1
dict	`{"id": 1, "done": False}`、`task["id"]`	キーで値を引く表	チュートリアル 5.5

書き方	例	意味	参照（Python 公式）
`d["k"]` ／ `d.get("k")`	`task.get("due")`	ないと `KeyError` ／ないと `None`（`d.get("k", 既定値)`）	ライブラリ `dict`
内包表記	`[t for t in tasks if not t["done"]]`	条件に合うものだけの新しいリスト	チュートリアル 5.1.3
f-string	`f"{task['id']:>3} [{status}]"`	文字列に値を埋め込む。`:>3` は３桁で右寄せ	チュートリアル 7.1.1
型ヒント	`def add_task(tasks: list[Task], title: str) → Task:`	引数と戻り値の型。実行には影響せず、読む人と mypy（第７回）が使う	チュートリアル 4.9.8、ライブラリ `typing`
型の別名	`type Task = dict[str, Any]`	型に名前を付ける（3.12 の書き方）	ライブラリ `typing`
`with`	`with path.open(encoding="utf-8") as f:`	ブロックを出るとき、例外が起きても必ず閉じる	チュートリアル 7.2、8.8
`pathlib`	`Path("tasks.json")`	パスを文字列ではなくオブジェクトで扱う	ライブラリ `pathlib`
`json`	`json.load(f)` ／ `json.dump(obj, f, ensure_ascii=False, indent=2)`	ファイルから読む／ファイルに書く	チュートリアル 7.2.2
例外	`try:` … `except FileNotFoundError:`、`raise ValueError("文言")`	起こりうる失敗を捕まえる／自分で知らせる	チュートリアル 8.3、8.4
`argparse`	`sub.add_parser("add")`、`type=int`	引数の解釈・`--help`・誤りの表示（終了コード 2）を自動で	argparse チュートリアル
`match`	`match args.command:` → `case "add":`	値で処理を振り分ける	チュートリアル 4.7
`if __name__ == "__main__":`	`sys.exit(main())`	ファイルを直接実行したときだけ動く（import では動かない）	ライブラリ `__main__`
`datetime.date`	`date.fromisoformat("2026-11-01")`、`d.isoformat()`	日付。文字列との変換と大小の比較ができる	ライブラリ `datetime`

JSON と Python の対応

JSON（`tasks.json`）	Python（`json.load` の結果）
オブジェクト `{"id": 1}`	dict
配列 `[...]`	list
文字列 `"牛乳を買う"`	str
数 `1`	int
`true` ／ `false`	`True` ／ `False`
`null`	`None`
（日付の型はない）	`date` は `"2026-11-01"` の文字列にして保存する

TODO の CLI（今日作るもの）

コマンド	動き
`python todo.py add "牛乳を買う"`	前後の空白を除き、1〜100 文字なら追加。id は最大＋1
`python todo.py add "レポートを出す" --due 2026-11-01`	期限付きで追加（手順 7）
`python todo.py list`	`1 [未着手] 牛乳を買う`。0 件なら `タスクはありません`
`python todo.py list --overdue`	未完了で期限が今日より前（`[期限切れ]`）のものだけ（手順 7）
`python todo.py done 1`	完了にする。無い id は `タスクが見つかりません: 1` で終了コード 1

判断の基準

場面	基準	参照
ファイルがない	初回は普通に起こる。`except FileNotFoundError:` で空のリストにする。`except:` だけは書かない（打ち間違いまで隠れる）	Python 公式 チュートリアル 8.3
後から足したキー	古いデータにはない。`task.get("due")` で読む	Python 公式 チュートリアル 5.5
操作と表示	データの操作（`add_task`）は値を返し、`print` は `main` だけで行う（第 8 回でテストしやすい）	SE基礎 3章
エラーの出し方	利用者向けの文言を `sys.stderr` に出し、終了コード 1。引数の誤りは argparse が終了コード 2 で止める	Python 公式 ライブラリ `sys`
今日の日付	`main` で 1 回だけ `date.today()` を取り、引数で渡す（第 8 回で日付を固定してテストできる）	SE基礎 3章

場面	基準	参照
リポジトリ	最初の数コミットだけ main に直接入れ、機能の追加からは Issue → ブランチ → PR → セルフレビュー → マージ（第５回）	研修の規約

ハンズオン

１ステップ１コマンド。各ステップは「🖊 やること → 📚 参考書・公式資料の節 → 📒 説明 → コマンドやコード」。作業場所は `~/handsoff/workspace/todo-app/`（手順２で作るアプリのリポジトリ）。見本は examples/session06/。コードは雛形の同じ名前の関数を、まるごと置き換える。

1. REPL で list と dict を試す（4分）

🖊 REPL を起動する

📚 Python 公式 チュートリアル 3 形式ばらない Python の紹介

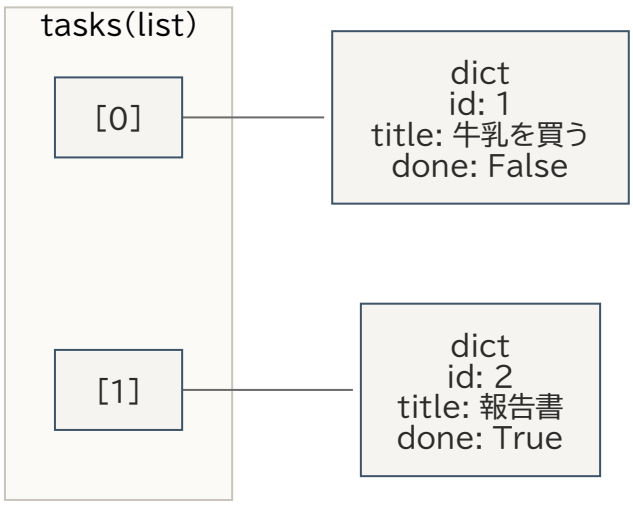
📒 REPL は1行ずつ打って結果を見る対話モード。`workspace` の中なら mise の Python 3.12 が動く

```
cd ~/handsoff/workspace; python
```

🖊 タスクのリストを作る

📚 Python 公式 チュートリアル 3.1.3 リスト型、5.5 辞書型

📒 タスク1件を dict、全体を dict の list で持つ。list は番号で、dict はキーで取り出す



```
tasks = [{"id": 1, "title": "牛乳を買う", "done": False}, {"id": 2, "title": "報告書",
"done": True}]
```

🖊 キーで値を取り出す

📚 Python 公式 チュートリアル 5.5 辞書型

📒 `d["k"]` はキーがないと `KeyError` で止まる。`d.get("k")` は `None` を返す。後から足すキー（手順７の `due`）は、古いデータにないので `get` で読む

```
tasks[0]["title"]; tasks[0].get("due"); tasks[0]["due"]   # 最後は KeyError: 'due'
```

🖊 内包表記で絞り込む

📚 Python 公式 チュートリアル 5.1.3 リストの内包表記

📒 `[式 for 要素 in リスト if 条件]` で、条件に合う要素だけの新しいリストを作る。元の `tasks` は変わらない

```
[t["title"] for t in tasks if not t["done"]]   # ['牛乳を買う']
```

🖊 次の番号を決める

📒 `max` に `default=0` を渡すと、空のリストでも 0 になり、最初の id が 1 になる

```
max((t["id"] for t in tasks), default=0) + 1; max((t["id"] for t in []), default=0) + 1
# 3 と 1
```

🖊 f-string で1行にする

📚 Python 公式 チュートリアル 7.1.1 フォーマット済み文字列リテラル

📒 `{}` の中に式を書く。`:>3` は３桁で右寄せ。外側を `"` にしたので、中のキーは `'` で囲む

```
f"{tasks[1]['id']:>3} [{'完了' if tasks[1]['done'] else '未着手'}] {tasks[1]['title']}"
exit()
```

座学: `d["k"]` と `d.get("k")` の使い分け（→ まとめ「判断の基準」）

2. アプリのリポジトリを作る（5分）

🖊 作業場所を作る

📒 第7〜12回もこのディレクトリで作業する。`git init` の前に `pwd` で確かめる（第5回の手順2）

```
mkdir ~/handsoff/workspace/todo-app; cd ~/handsoff/workspace/todo-app; pwd
```

🖊 設定ファイルを見本からコピーする

📚 uv 公式 Working on projects

📒 `mise.toml`・`uv.lock` は第4回と同じ形。`.gitignore` には、自分のタスクのデータ `tasks.json` も入れる

```
cp ~/handsoff/examples/todo-app-snapshots/session-
06/{.gitignore,mise.toml,pyproject.toml,uv.lock} .; ls -a
```

🖊 pyproject.toml を読む

📚 uv 公式 Configuring projects（Project packaging）

📒 `[tool.uv]` の `package = false` は、このプロジェクトをパッケージとして `.venv` に入れず、`todo.py` をそのまま実行するという意味。第4回の `uv init --app` の既定と同じで、第7回でパッケージにするときに消す

```
cat pyproject.toml
```

🖊 .venv を作る

📒 コピーした `mise.toml` は中身を見てから信頼する（第5回の手順2）

```
mise trust; mise run setup
```

🖊 リポジトリにして、設定をコミットする

📚 Pro Git 2.1 Gitリポジトリの取得、2.2 変更内容のリポジトリへの記録

📒 第5回と同じく、`.gitignore` を最初に1つでコミットする

```
git init
git add .gitignore; git commit -m ".venv と tasks.json を Git の管理から外す" -m "仮想環境は作り直せ、tasks.json は自分のデータでコードではないため。"
git add mise.toml pyproject.toml uv.lock; git commit -m "Python のバージョンと依存パッケージを固定する"
```

3. 雛形を置いて読む（4 分）

🖊 雛形をコピーする

📒 雛形には関数の名前・型ヒントと `main` だけが書いてあり、関数の中身は `raise NotImplementedError`

```
cp ~/handsoff/docs/session06/assets/todo_starter.py todo.py
```

🖊 ヘルプを出す

📚 Python 公式 argparse チュートリアル

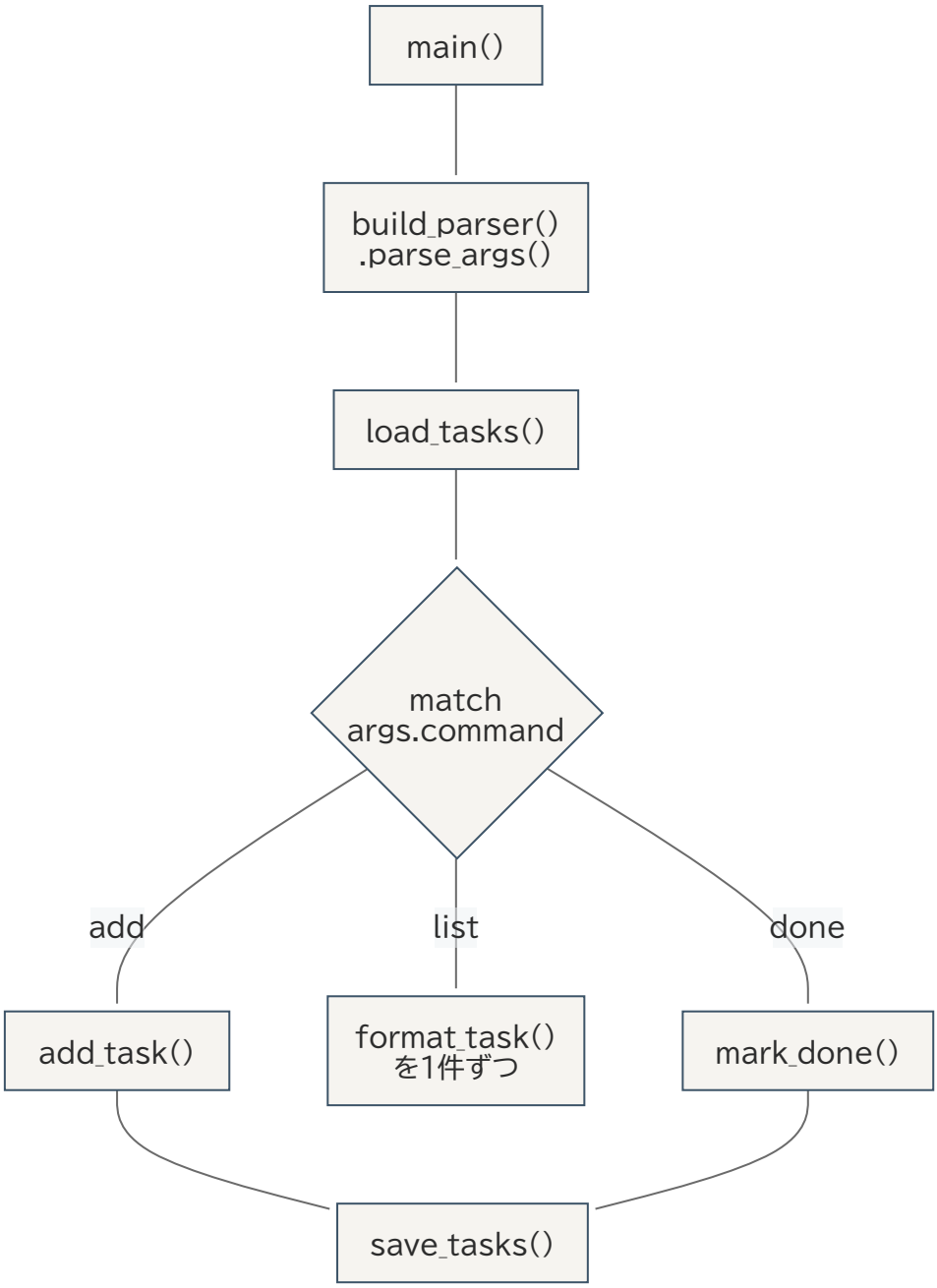
📒 `--help` と使い方の表示は argparse が作る。`build_parser` の `add_parser` の数だけサブコマンドが並ぶ

```
python todo.py --help; python todo.py add --help
```

🖊 main を読む

📚 Python 公式 チュートリアル 4.7 match 文、4.8 関数を定義する

📒 `main` は引数を解釈して `tasks.json` を読み、`match` でサブコマンドごとの関数を呼ぶ。最後の `if __name__ == "__main__":` で、直接実行したときだけ `main` が動く




```
less todo.py   # main と build_parser を読む。q で終わる
```

🖋 動かして止まる所を見る

📒 トレースバックは下から読む。最後の行がエラーの種類と文言、その上がどこで起きたか

```
python todo.py list   # NotImplementedError: 手順4で書く
```

4. 読み込みと保存: with・pathlib・JSON（7分）

🖊 load_tasks を書く

📚 Python 公式 チュートリアル 7.2 ファイルを読み書きする、8.3 例外を処理する

📒 ファイルがあるかを先に確かめず、開いてみて `FileNotFoundError` だけを捕まえる。`with` を抜けると、例外のときもファイルは閉じる

```
def load_tasks(path: Path) → list[Task]:
    """保存したタスクを読む。ファイルがまだなければ空のリスト。"""
    try:
        with path.open(encoding="utf-8") as f:
            tasks: list[Task] = json.load(f)
    except FileNotFoundError:
        return []
    return tasks
```

🖊 空の一覧を出す

📚 Python 公式 ライブラリ `pathlib`

📒 `Path("tasks.json")` は相対パスなので、コマンドを打ったディレクトリの `tasks.json` を見る

```
python todo.py list   # タスクはありません
```

🖊 save_tasks を書く

📚 Python 公式 チュートリアル 7.2.2 json による構造化されたデータの保存

📒 `json.dump` は Python の値を JSON にして書き、`json.load` はその逆を行う。`ensure_ascii=False` で日本語をそのまま、`indent=2` で読みやすく書く

```
def save_tasks(path: Path, tasks: list[Task]) → None:
    # ensure_ascii=False: 日本語を "\u3042" のようなエスケープ表記にせず、そのまま書く
    with path.open("w", encoding="utf-8") as f:
        json.dump(tasks, f, ensure_ascii=False, indent=2)
```

🖊 変換の結果を見る

📒 `json.dumps` は、ファイルではなく文字列にする。`False` が `false`、`None` が `null` になる

```
python -c 'import json; print(json.dumps({"title": "牛乳", "done": False, "due": None},
ensure_ascii=False))'
```

5. add・list・done を書く（8分）

🖊 format_task を書く

📚 Python 公式 チュートリアル 7.1.1 フォーマット済み文字列リテラル

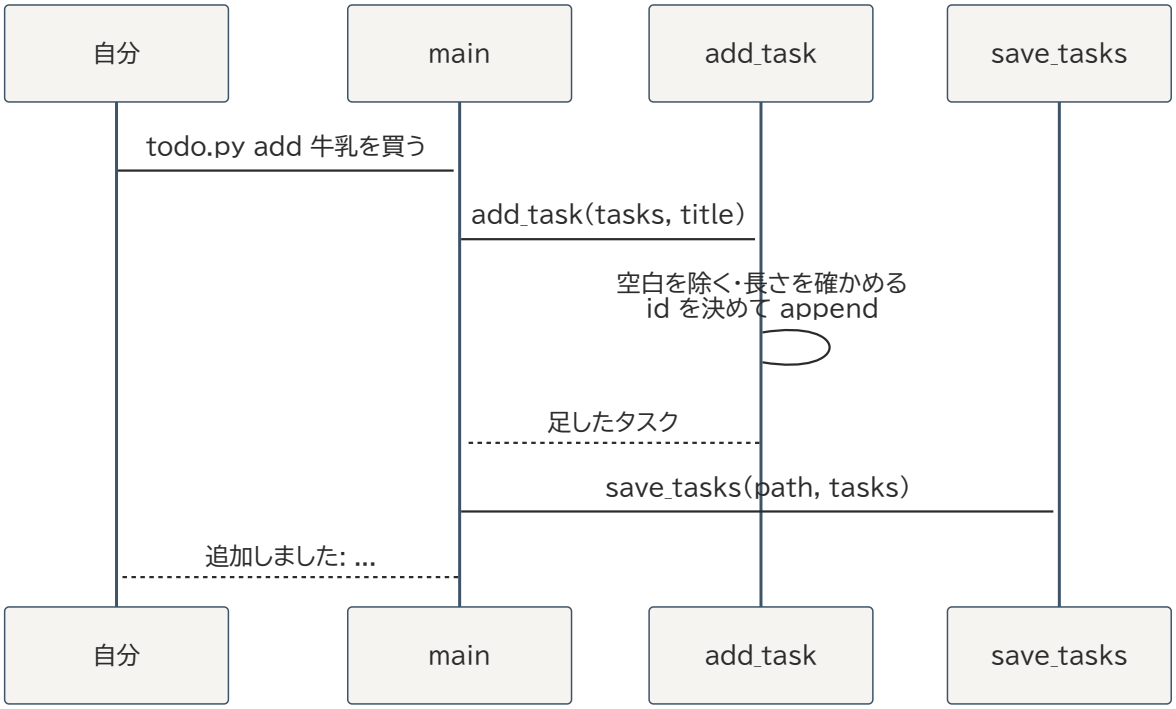
📒 表示の形を1か所にまとめる。add・list・done のどれもこの関数で1行にする

```
def format_task(task: Task) → str:
    status = "完了" if task["done"] else "未着手"
    return f"{task['id']:>3} [{status}] {task['title']}"
```

🖊 add_task を書く

📚 Python 公式 チュートリアル 4.8 関数を定義する、8.4 例外を送出する

📒 `add_task` は入力を確かめてリストに足し、足したタスクを返す。`print` も保存もしない。表示と保存は `main` が行う




```python
def add_task(tasks: list[Task], title: str) -> Task:
    title = title.strip()
    if not 1 <= len(title) <= TITLE_MAX:
        raise ValueError(f"タイトルは1〜{TITLE_MAX}文字で指定してください")
    new_id = max((t["id"] for t in tasks), default=0) + 1
    task: Task = {"id": new_id, "title": title, "done": False}
    tasks.append(task)
    return task
```

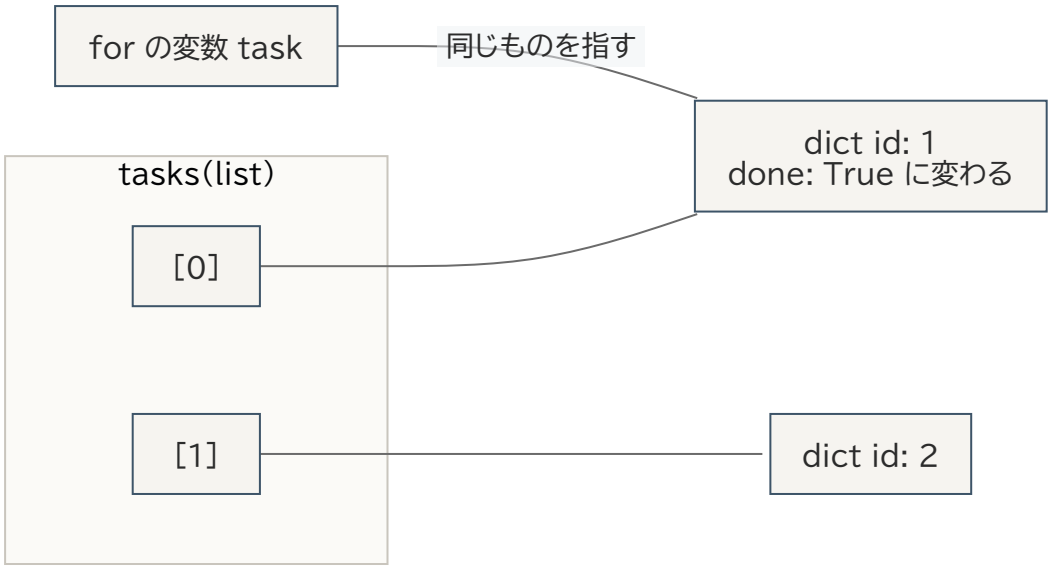
🖊 追加して一覧を見る

```
python todo.py add "牛乳を買う"; python todo.py add "報告書を書く"
python todo.py list; cat tasks.json
```

🖊 mark_done を書く

📚 Python 公式 チュートリアル 4.2 for 文

📒 `for` で取り出した `task` は、リストの中の dict そのもの(コピーではない)。`task["done"] = True` でリストの中身も変わり、それを保存する




```
def mark_done(tasks: list[Task], task_id: int) → Task:
    for task in tasks:
        if task["id"] == task_id:
            task["done"] = True
            return task
    raise KeyError(task_id)
```

🖋 完了にする

```
python todo.py done 1; python todo.py list   #   1 [完了] 牛乳を買う
```

6. エラーを確かめて push する（4 分）

🖋 空のタイトルを渡す

📚 Python 公式 チュートリアル 8.3 例外を処理する

📒 `add_task` が送出した `ValueError` を `main` の `except` が捕まえ、文言を標準エラー出力に出して 1 を返す。`sys.exit(main())` で、その値が終了コードになる

```
python todo.py add "   "; echo $?   # タイトルは1〜100文字で指定してください / 1
```

🖊 無い番号を渡す

```
python todo.py done 99; echo $?    # タスクが見つかりません: 99 / 1
```

🖊 数でない番号を渡す

📒 `type=int` で変換できない値は、`main` に届く前に argparse が止める

```
python todo.py done abc; echo $?   # invalid int value: 'abc' / 2
```

🖊 todo.py をコミットする

📒 `tasks.json` は `.gitignore` にあるので、`git status` に出ない

```
git status --short; git add todo.py; git commit -m "TODO の CLI（add・list・done）を作る"
```

🖊 空のリポジトリを作る（ブラウザ）

📒 `<Gitホスティング>` に非公開のリポジトリ `todo-app` を、初期ファイルを付けずに作る（第５回の手順５）

🖊 push する

📚 Pro Git 2.5 リモートでの作業

```
git remote add origin <リポジトリのURL>; git push -u origin main
```

- 確認: ブラウザで３つのコミットが見える

> 座学: 例外を捕まえる場所と、捕まえてよい種類（→ まとめ「判断の基準」）

7. 期限を足す: Issue とブランチ（８分）

🖊 Issue を立てる（ブラウザ）

📚 Pro Git 6.2 プロジェクトへの貢献

📒 ここからは第５回と同じく、main に直接コミットしない

Issue	タイトル	目的	完了条件
#1	期限を付け、期限切れだけを表示できるようにする	締め切りのあるタスクを見落とさない	`add --due` で期限を保存でき、`list --overdue` で期限切れだけが出る

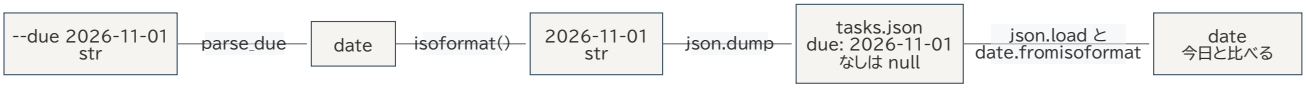
🖊 ブランチを作る

```
git switch -c issue-1-due
```

🖊 期限を日付に変える関数を書く

📚 Python 公式 ライブラリ `datetime`（`date.fromisoformat`）

📒 JSON に日付の型はないので、`"2026-11-01"` の文字列で保存し、比べるときに `date` に戻す。`parse_due` は argparse の `type=` に渡し、形が違えば `ArgumentTypeError` で知らせる。import の行に `from datetime import date` を足し、`save_tasks` の下に書く




```
def parse_due(text: str) → date:
    """--due の値を日付にする。argparse の type に渡す。"""
    try:
        return date.fromisoformat(text)
    except ValueError:
        raise argparse.ArgumentTypeError(
            f"期限は YYYY-MM-DD の形で指定してください: {text}"
        ) from None
```

🖊 期限切れを判定する関数を書く

📒 完了したもの・期限がないもの・期限が今日のものは期限切れにしない。`today` は引数で受け取る（→ まとめ「判断の基準」）

```
def is_overdue(task: Task, today: date) → bool:
    """期限切れか。完了したものと、期限がないものは期限切れにならない。"""
    due = task.get("due")  # 期限のないタスクは None
    return not task["done"] and due is not None and date.fromisoformat(due) < today
```

🔗 format_task と add_task を変える

📒 `date | None` は「日付か None」の型。手順５の `tasks.json` には `due` のキーがないので、`task.get` で読む

```
def format_task(task: Task, today: date) → str:
    if task["done"]:
        status = "完了"
    elif is_overdue(task, today):
        status = "期限切れ"
    else:
        status = "未着手"
    due = task.get("due")
    due_text = f"（期限 {due}）" if due else ""
    return f"{task['id']:>3} [{status}] {task['title']}{due_text}"


def add_task(tasks: list[Task], title: str, due: date | None) → Task:
    title = title.strip()
    if not 1 ⩽ len(title) ⩽ TITLE_MAX:
        raise ValueError(f"タイトルは1〜{TITLE_MAX}文字で指定してください")
    new_id = max((t["id"] for t in tasks), default=0) + 1
    task: Task = {
        "id": new_id,
        "title": title,
        "due": due.isoformat() if due else None,
        "done": False,
    }
    tasks.append(task)
    return task
```

🔗 引数を足す

📚 Python 公式 argparse チュートリアル

📒 `action="store_true"` の引数は、付けると `True`、付けないと `False` になる。`list` は組み込み関数の名前なので、変数は `list_` にする

```
    add.add_argument("--due", type=parse_due, help="期限（YYYY-MM-DD）")   #
add.add_argument("title", ...) の下

    list_ = sub.add_parser("list", help="タスクの一覧を表示する")              #
sub.add_parser("list", ...) の行を置き換える
    list_.add_argument("--overdue", action="store_true", help="期限切れだけを表示する")
```

🖊 main を変える

📒 `today` を１回だけ取り、`format_task` に渡す。`--overdue` のときは内包表記で絞り込む

```
def main(argv: list[str] | None = None) → int:
    args = build_parser().parse_args(argv)
    tasks = load_tasks(TASKS_FILE)
    today = date.today()

    # どのコマンドかで処理を振り分ける（パターンマッチ）
    match args.command:
        case "add":
            try:
                task = add_task(tasks, args.title, args.due)
            except ValueError as e:
                print(e, file=sys.stderr)
                return 1
            save_tasks(TASKS_FILE, tasks)
            print(f"追加しました: {format_task(task, today)}")
        case "list":
            shown = tasks
            if args.overdue:
                shown = [t for t in tasks if is_overdue(t, today)]
            if not shown:
                print("タスクはありません")
            for task in shown:
                print(format_task(task, today))
        case "done":
            try:
                task = mark_done(tasks, args.id)
            except KeyError:
                print(f"タスクが見つかりません: {args.id}", file=sys.stderr)
                return 1
            save_tasks(TASKS_FILE, tasks)
            print(f"完了にしました: {format_task(task, today)}")
    return 0
```

🖋 期限付きで追加する

```
python todo.py add "レポートを出す" --due 2026-04-01; python todo.py add "本を返す" --due
2030-03-31
```

🖊 一覧と期限切れを見る

📒 手順５で足した `due` のないタスクも、そのまま表示される

```
python todo.py list; python todo.py list --overdue   # 後のほうは［期限切れ］の1件だけ
```

🖊 形の違う期限を渡す

```
python todo.py add "テスト" --due 2026/11/1; echo $?   # 期限は YYYY-MM-DD の形で… / 2
```

8. PR を出してマージする（5 分）

🖊 コミットして push する

📚 Pro Git 3.5 リモートブランチ

```
git diff --stat; git commit -am "期限と期限切れだけの一覧を足す" -m "締め切りのあるタスクを見落とさないようにする。"
git push -u origin issue-1-due
```

🖊 PR を作り、セルフレビューしてマージする（ブラウザ）

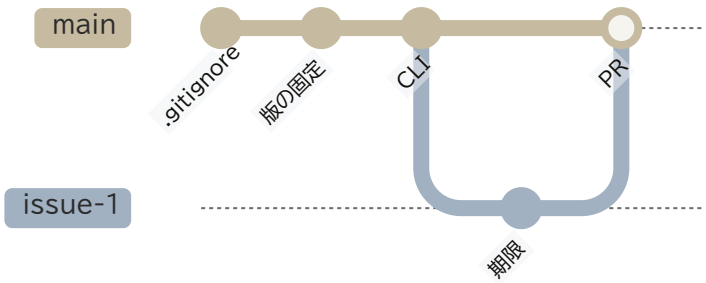
📚 SE基礎 2章 コードを読む、3章 コードを書く

📒 本文はテンプレートの目的（`Closes #1`）・変更点・確認方法で書く。確認方法には手順７で打ったコマンドと結果を書く。差分を他人のつもりで読み、`[確認]` の行コメントを１件以上残してからマージする

🖊 手元の main をそろえる

📚 Pro Git 2.5 リモートでの作業、3.2 ブランチとマージの基本

📒 main への変更は、最初の３つのコミットのあとは PR のマージコミットだけを通る。第７回からもこの形で足していく。図ではブランチ名を `issue-1` と略す




```
git switch main; git pull; git branch -d issue-1-due; git log --oneline --graph
```

- 確認: Issue #1 が Closed。`python todo.py list --overdue` が main でも動く

読む章

区分	資料	範囲	読み方
無料	Python 公式ドキュメント(チュートリアル)	3章 形式ばらない Python の紹介、4章 その他の制御フローツール(4.7 `match` 文、4.8〜4.9 関数)、5章 データ構造、7章 入力と出力、8章 エラーと例外	通読(第7回の前に)
無料	同上(ライブラリと HOWTO)	`pathlib`、`json`、`datetime.date`、`typing`、argparse チュートリアル、用語集(EAFP・LBYL)	引く(困ったとき)
無料	uv 公式ドキュメント	Configuring projects(Project packaging、`package = false`)	引く(第7回でパッケージにするとき)
参考書	ソフトウェアエンジニアリングの基礎(SE基礎)	3章 コードを書く	該当章(関数の分け方。第7回でも使う)
紹介(副読本)	Effective Python 第2版	1〜2章(項目16 `dict.get` など)、8章 項目65〜66(例外と `with`)	持っている人は該当章
紹介(候補)	改訂新版 Python実践レシピ	型ヒント・pathlib・json・argparse の章	本で補いたい人は

第７回 読みやすく壊れにくい Python

tag: python-oop, python-可読性, test-静的解析, 標準
前提タグ: python-構文, env-python, git-github
所要: 座学15分 / ハンズオン45分

ゴール

- 1ファイルの CLI を `src/todo/` パッケージ(models・store・cli)に分け、`uv run todo` で動かせる
- `dataclass(frozen=True)` と `Enum` でタスクと状態を表し、属性の打ち間違いを実行前に mypy で見つけられる
- `typing.Protocol` で保存先の約束を書き、継承しなくても形が合えば差し替えられること(ダックタイピング)を説明できる
- ruff・mypy の設定(機械が確かめる規約)と README(人が見る規約)をリポジトリに入れ、`mise run check` が通る
- 整理を目的ごとのコミットに分け、Issue → ブランチ → PR → セルフレビュー → マージで入れられる

事前準備

- 第６回のハンズオンの最後(手順８のマージ)まで済み、`~/handsoff/workspace/todo-app` の main が `<Gitホスティング>` と同じ
- 遅れた人: session-06/ を examples/README.md の手順でコピーし、push してから始める
- 社内ネットワークでは PyPI に届く(手順５の `uv add` 。open-questions.md の Q5)

まとめ

冒頭の座学で使います。ハンズオンでは、第６回の `todo.py` を動きを変えずに分け、ruff と mypy で確かめてから PR でマージします。

第６回 → 第７回

第６回	第７回	参照
`{"id": 1, "title": ..., "done": False}`	`Task(id=1, title=...)` (`dataclass` 。属性の打ち間違いは mypy が指摘)	Python 公式 `dataclasses`
`task["done"] = True` (中身を書き換える)	`task.mark_done()` が新しい `Task` を返す(`frozen=True` 、 `replace`)	Python 公式 `dataclasses`

文字列 `"期限切れ"`	`Status.OVERDUE`（`Enum`。取りうる値が3つに決まる）	Python 公式 `enum`
`load_tasks`／`save_tasks` を `main` が直接呼ぶ	`main` は `TaskStore` を満たすもの（`JsonStore`／`MemoryStore`）を受け取る	Python 公式 `typing.Protocol`
1ファイル `todo.py`	`src/todo/models.py`・`store.py`・`cli.py`	SE基礎 3章
`python todo.py list`	`uv run todo list`（`[project.scripts]` の `todo = "todo.cli:main"`）	uv 公式
エラーの文言は関数ごと、`--due` の誤りは終了コード 2	文言は `models.py` の1か所にまとめる。`--due` の誤りも終了コード 1	SE基礎 3章

用語と考え方

用語	ひとことで	参照
パッケージ	`__init__.py` のあるディレクトリ。`from todo.models import Task` で読める。`src/` の下に置くと、入れたものだけが import できる	uv 公式、Python 公式 チュートリアル 6.4
dataclass	属性を並べるだけで `__init__` と表示を作るクラス	Python 公式 `dataclasses`
イミュータブル	作った後に変えられない。どこで変わったかを追わなくてよい	Python 公式 `dataclasses`（frozen）
Enum	取りうる値を列挙した型。`Status.DONE.value` が `"完了"`	Python 公式 `enum`
ダックタイピング	型（クラス）ではなく、持っているメソッド（振る舞い）で扱う	Python 公式 用語集
`Protocol`	「このメソッドを持つこと」を型にする。継承しなくても、形が合えば mypy が認める	Python 公式 `typing`、mypy 公式
早期 return（ガード節）	例外的な場合を先に `return`／`raise` して、入れ子を浅くする	SE基礎 3章
`ruff check`／`ruff format`	リント（規約違反・バグになりやすい書き方）／整形	Ruff 公式
`mypy`（strict）	型ヒントの食い違いを実行前に見つける。型ヒントのない関数も指摘する	mypy 公式

パッケージの形

ファイル	持つもの	知らないこと
`models.py`	`Status`、`Task`、`validate_title`、`parse_due`、`InvalidTaskError`	保存先・画面

ファイル	持つもの	知らないこと
`store.py`	`TaskStore`（Protocol）、`JsonStore`、`MemoryStore`、`TaskNotFoundError`	画面・引数
`cli.py`	引数の解釈・表示・終了コード（`main(argv, store, today)`）	JSON の形

規約の置き場所

規約	置き場所	確かめ方
書き方の形（字下げ・行の長さ・import の順・未使用・命名）	`pyproject.toml` の `[tool.ruff]`	`mise run lint`（直すのは `mise run fmt`）
型（型ヒントを書く・食い違いがない）	`pyproject.toml` の `[tool.mypy]`	`mise run typecheck`
名前の意味・関数の分け方・コメント（なぜ）・データの持ち方	`README.md` の「コーディング規約」	セルフレビュー

判断の基準

場面	基準	参照
規約を足す	機械で確かめられるものは ruff・mypy の設定にする。README には人が見ることだけを書く	Ruff 公式、リーダブルコード
指摘を黙らせる	`# noqa: F401` や `# type: ignore[...]` は、理由をコメントに書けるときだけ	Ruff 公式、mypy 公式
状態の判定	`Task.status(today)` の1か所に置く。`cli.py` で `"期限切れ"` を組み立てない	SE基礎 3章
入力チェック	入口（`cli.py` の `run_add`）で `validate_title`・`parse_due` を呼ぶ。文言は `models.py` に1か所（第11回の Web でも使う）	SE基礎 3章
差し替え	`main` は保存先と「今日」を引数で受け取り、省略したときだけ `JsonStore` と `date.today()` にする（第8回のテスト）	SE基礎 3章
コミット	移す・作り直す・道具を入れる・文書を書くを別のコミットにする。移すだけのコミットは動きを変えない	研修の規約

ハンズオン

1ステップ1コマンド。各ステップは「🖋 やること → 📚 参考書・公式資料の節 → 📒 説明 → コマンドやコード」。作業場所は `~/handsoff/workspace/todo-app/`。見本は session-07/（examples/session07/）。

1. Issue を立て、todo.py をパッケージに移す（7分）

🖊 Issue を立てる（ブラウザ）

📚 Pro Git 6.2 プロジェクトへの貢献

📒 第6回の PR が #2 なので、この Issue は #3 になる（GitLab では番号の付き方が違うので、出た番号を使う）

Issue	タイトル	目的	完了条件
#3	todo.py を読みやすく壊れにくい形に整理する	打ち間違いを実行前に見つけ、保存先を差し替えられるようにする	`uv run todo list` が第6回と同じ表示で、`mise run check` が通る

🖊 ブランチを作る

```
cd ~/handsoff/workspace/todo-app
git switch main; git pull; git switch -c issue-3-package
```

🖊 パッケージのディレクトリを作る

📚 Python 公式 チュートリアル 6.4 パッケージ

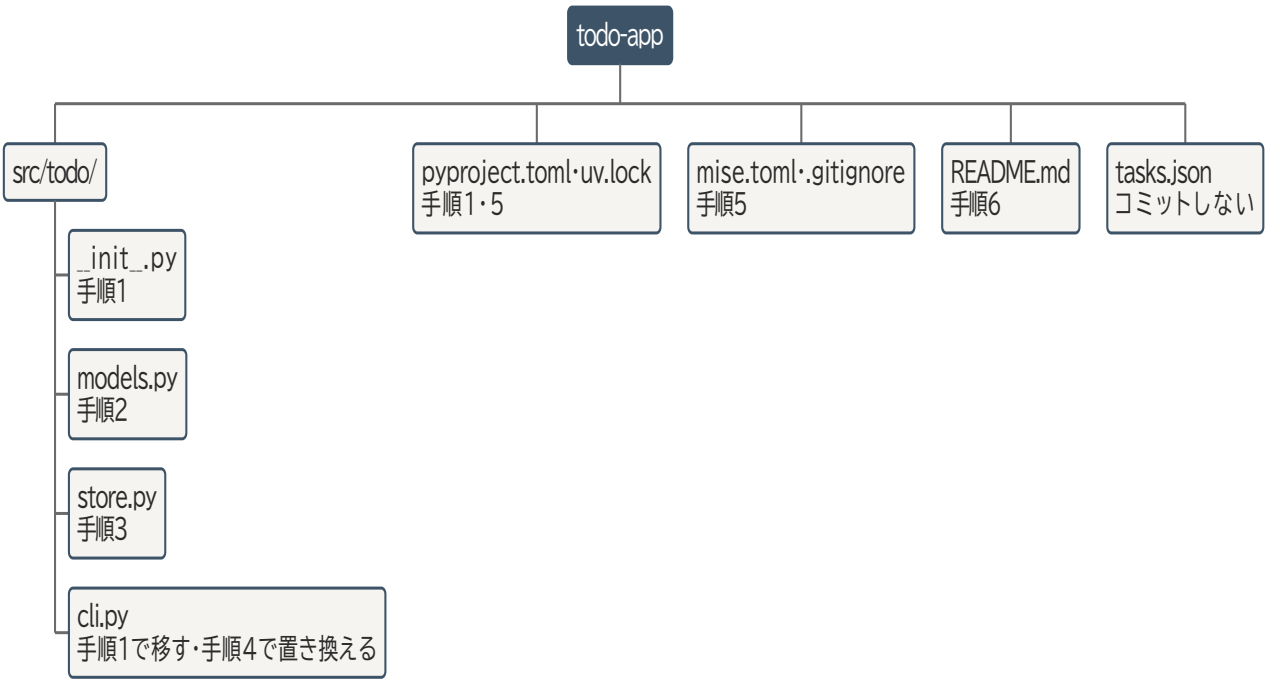
📒 1つのファイルを、役割ごとの3つのモジュール（`models.py`・`store.py`・`cli.py`）に分ける（→ まとめ「パッケージの形」）

```
mkdir -p src/todo; echo '"""TODO・タスク管理アプリ。"""' > src/todo/__init__.py
```

🖊 todo.py を移す

📚 Pro Git 2.2 変更内容のリポジトリへの記録（ファイルの移動）

📒 `git mv` で移すと、Git は「名前が変わった」と記録する。中身は手順4で `cli.py` に置き換える。それまでは第6回のコードのまま動く




```
git mv todo.py src/todo/cli.py; git status --short
```

🖊 pyproject.toml を書き換える

📚 uv 公式 Configuring projects(Entry points、Build systems)、The uv build backend (Modules)

📒 `package = false` を消し、パッケージとして `.venv` に入れる。`[project.scripts]` で `todo` というコマンドを作る。プロジェクト名は `todo-app` で、モジュール名 `todo` と違うので `module-name` で指定する

```
[project]
name = "todo-app"
version = "0.1.0"
description = "TODO・タスク管理アプリ"
# 上限も付けて 3.12 に固定する（mise を通さずにほかの Python で動かしてしまうのを防ぐ）
requires-python = "≥3.12,<3.13"
dependencies = []

# uv run todo ... で todo.cli の main() を呼ぶ
[project.scripts]
todo = "todo.cli:main"

[build-system]
requires = ["uv_build≥0.9,<0.10"]
build-backend = "uv_build"

[tool.uv.build-backend]
module-name = "todo"
```

🖋 .venv に入れ直す

📚 uv 公式 Locking and syncing

📒 `pyproject.toml` を変えたので、`mise run setup`（`--locked`）は止まる（第４回の手順５）。`uv sync` で `uv.lock` を直し、`todo-app` を `.venv` に入れる

```
uv sync
```

🖋 uv run todo で動かす

📒 `uv run todo` は `.venv/bin/todo` を動かし、それが `todo.cli` の `main()` を呼ぶ。`.venv` には `src/todo` への参照が入るので、編集はすぐ反映される

```
uv run todo list; uv run todo --help   # 第6回と同じ一覧。usage の名前が todo になる
```

🖋 移しただけでコミットする

📒 動きを変えないコミットを先に分けておくと、後のコミットの差分が読みやすい

```
git add -A; git status --short; git commit -m "todo.py をパッケージ src/todo に移す" -m "uv run todo で動かし、役割ごとのモジュールに分ける準備をする。"
```

2. models.py: Task と Status（8分）

🖊 Status と InvalidTaskError を書く

📚 Python 公式 ライブラリ `enum`、チュートリアル 8.6 ユーザー定義例外

📒 `Enum` で状態を３つに限る。`InvalidTaskError` は `ValueError` を継承し、その文言をそのまま利用者に見せる。エディタで `src/todo/models.py` を新しく作り、下を書く

```
"""タスクと、その状態の決まり。保存先や画面には依存しない。"""

from dataclasses import dataclass, replace
from datetime import date
from enum import Enum

TITLE_MAX = 100


class Status(Enum):
    TODO = "未着手"
    DONE = "完了"
    OVERDUE = "期限切れ"  # 完了していない・期限が今日より前


class InvalidTaskError(ValueError):
    """入力が正しくない。メッセージはそのまま利用者に見せる。"""
```

🖊 Task を書く

📚 Python 公式 ライブラリ `dataclasses`（frozen、`replace`）

📒 状態は保存せず、`today` と比べてその都度決める。完了したタスクは期限切れにならない

```python
@dataclass(frozen=True)
class Task:
    """タスク1件。frozen=True なので作った後は変えられない（変えるときは新しく作る）。"""

    id: int
    title: str
    due: date | None = None
    done: bool = False

    def status(self, today: date) -> Status:
        if self.done:
            return Status.DONE  # 完了したものは期限切れにならない
        if self.due is not None and self.due < today:
            return Status.OVERDUE
        return Status.TODO

    def mark_done(self) -> "Task":
        """完了にしたタスクを返す（自分自身は変えない）。"""
        return replace(self, done=True)
```

🖋 入力チェックを書く

📚 SE基礎 3章 コードを書く

📒 タイトルと期限の決まりと文言を、この２つの関数に集める。どちらも、先に例外の場合を `raise` する（ガード節）

```
def validate_title(raw: str) → str:
    """前後の空白を除いたタイトルを返す。1〜100文字でなければ InvalidTaskError。"""
    title = raw.strip()
    if not title:
        raise InvalidTaskError("タイトルを入力してください")
    if len(title) > TITLE_MAX:
        raise InvalidTaskError(f"タイトルは{TITLE_MAX}文字以内で入力してください")
    return title


def parse_due(raw: str) → date | None:
    """期限の文字列（YYYY-MM-DD）を日付にする。空なら期限なし（None）。"""
    raw = raw.strip()
    if not raw:
        return None
    try:
        return date.fromisoformat(raw)
    except ValueError:
        raise InvalidTaskError("期限は YYYY-MM-DD の形で入力してください") from None
```

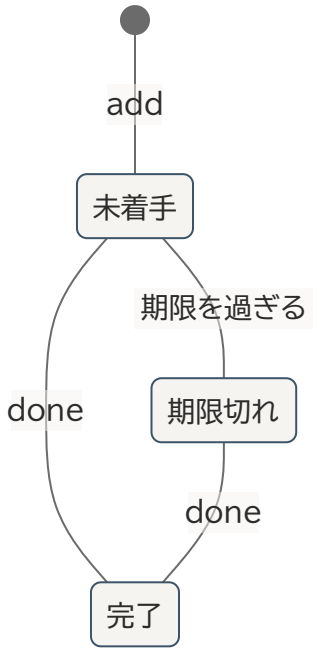
🖊 Task を作って変えてみる

📒 `frozen=True` なので、属性に代入すると `FrozenInstanceError` になる。変えるときは `replace` で新しい `Task` を作る

```
uv run python -c "from todo.models import Task; t = Task(id=1, title='牛乳を買う');
print(t); t.done = True"
```

🖊 状態を確かめる

📒 状態は保存しない。同じタスクでも、渡す日付で状態が変わる




```
uv run python -c "from datetime import date; from todo.models import Task; t = Task(1, 'レポート', due=date(2026, 4, 1)); print(t.status(date(2026, 3, 1)), t.status(date(2026, 10, 1)), t.mark_done().status(date(2026, 10, 1)))"
```

- 確認: `Status.TODO Status.OVERDUE Status.DONE`

> 🖊 入力チェックを確かめる

```
uv run python -c "from todo.models import validate_title; print(repr(validate_title('  牛乳を買う  ')))"; validate_title('   ')"
```

- 確認: `'牛乳を買う'` の後に `InvalidTaskError: タイトルを入力してください`

3. store.py: Protocol で保存先を約束する(8 分)

> 🖊 素材をコピーして JsonStore を読む

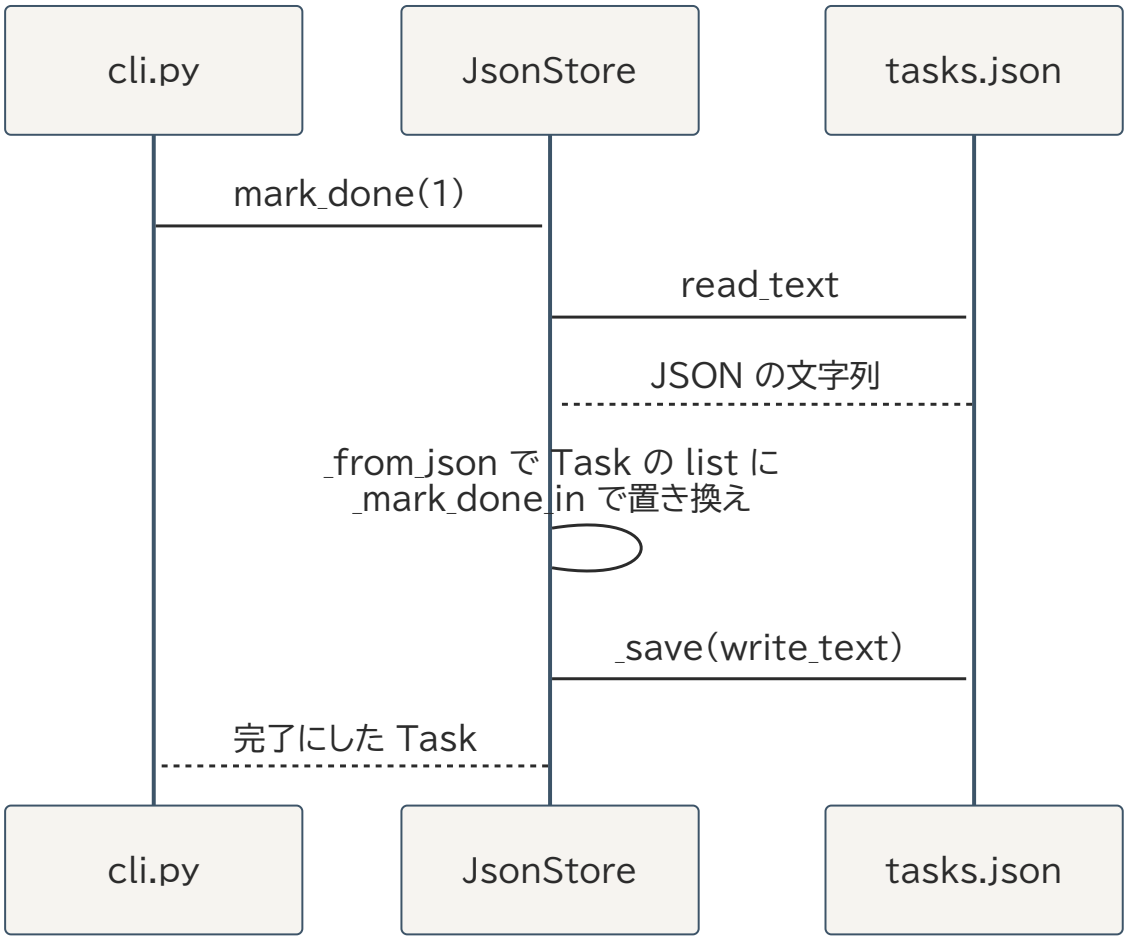
📚 Python 公式 ライブラリ `typing`(Protocol)、用語集(ダックタイピング)

📒 `JsonStore` は第 6 回の `load_tasks`・`save_tasks` の役割を引き継ぐ。`_from_json` は、古いタスクに `due` のキーがなくても読める

```
cp ~/handsoff/docs/session07/assets/store.py src/todo/store.py; less src/todo/store.py
```

🖋 JsonStore の流れを追う

📒 `JsonStore` は呼ばれるたびにファイルを読み、変えたリストを丸ごと書き直す。探して置き換える処理は `_mark_done_in` にまとめ、`MemoryStore` と共有する



🖋 TaskStore を書く

📚 mypy 公式 Protocols and structural subtyping

📒 `Protocol` には、保存先が持つべきメソッドの形だけを書く。中身は `...` でよい。「手順 3: ここに TaskStore」のコメントの行を置き換える

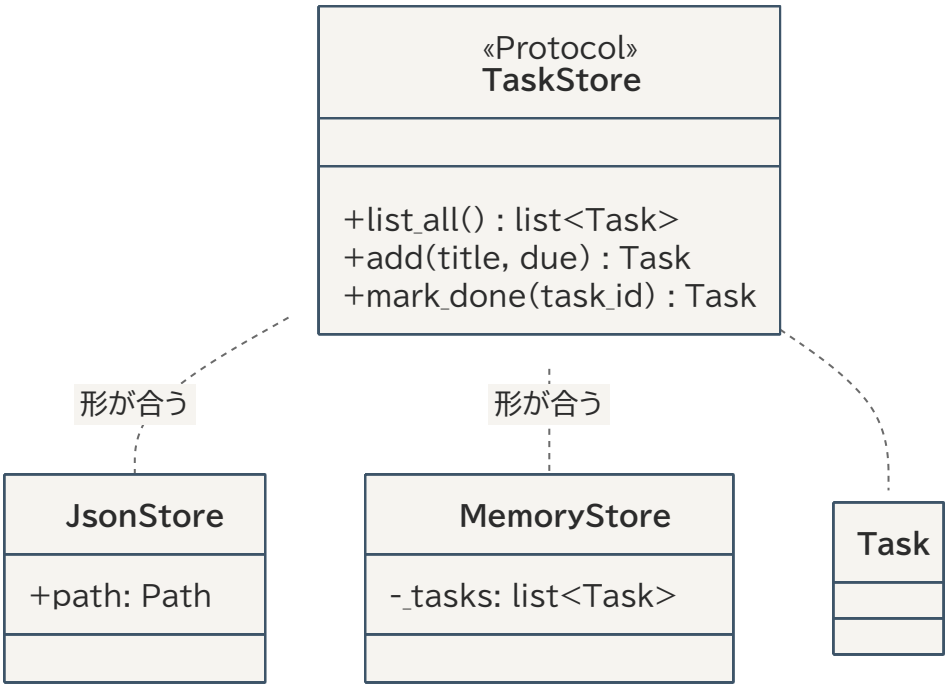
```python
class TaskStore(Protocol):
    def list_all(self) -> list[Task]:
        """全タスクを id の順に返す。"""
        ...

    def add(self, title: str, due: date | None) -> Task:
        """新しい id を付けて保存し、保存したタスクを返す。"""
        ...

    def mark_done(self, task_id: int) -> Task:
        """完了にして返す。なければ TaskNotFoundError。"""
        ...
```

🖊 MemoryStore を書く

📒 メモリに持つだけの保存先。`TaskStore` を継承しないが、同じ３つのメソッドを持つ。「手順3: ここにMemoryStore」のコメントの行を置き換える



```
class MemoryStore:
    """メモリに持つだけの保存先。プログラムが終わると消える（テスト用）。"""

    def __init__(self, tasks: list[Task] | None = None) -> None:
        self._tasks = list(tasks or [])

    def list_all(self) -> list[Task]:
        return list(self._tasks)

    def add(self, title: str, due: date | None) -> Task:
        task = Task(id=_next_id(self._tasks), title=title, due=due)
        self._tasks.append(task)
        return task

    def mark_done(self, task_id: int) -> Task:
        self._tasks, done = _mark_done_in(self._tasks, task_id)
        return done
```

> 🖋 **２つの保存先で同じ操作をする**

📒 `MemoryStore` はファイルを作らない。`JsonStore` は第６回の `tasks.json` をそのまま `Task` のリストにする

```
uv run python -c "from todo.store import MemoryStore; s = MemoryStore(); s.add('牛乳を買う', None); s.mark_done(1); print(s.list_all())"
uv run python -c "from pathlib import Path; from todo.store import JsonStore; print(JsonStore(Path('tasks.json')).list_all())"
```

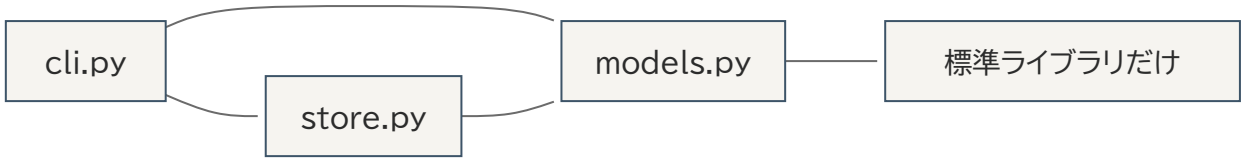
> 座学: `JsonStore` も `MemoryStore` も `TaskStore` を継承していない。それでも `TaskStore` として渡せる理由（→ まとめ「用語と考え方」）

4. cli.py を置き換える（６分）

> 🖋 **素材で cli.py を置き換える**

📚 SE基礎 ２章 コードを読む

📒 `cli.py` は `models` と `store` を使い、`models` はほかのモジュールを使わない。依存は一方向で、`cli.py` は JSON の形を知らない




```
cp ~/handsoff/docs/session07/assets/cli.py src/todo/cli.py; git diff --stat
```

🖊 main を読む

📒 `main` は保存先と「今日」を引数で受け取る。省略すると `JsonStore` と `date.today()` を使う。第８回のテストでは `MemoryStore` と固定の日付を渡す

```
less src/todo/cli.py   # main と run_list の早期 return を読む。q で終わる
```

🖊 前と同じ一覧が出るか確かめる

```
uv run todo list; uv run todo list --overdue
```

- 確認: 手順１の `uv run todo list` と同じ表示

🖊 エラーの出方を確かめる

📒 エラーの文言は `models.py` で決めたものになる。`--due` は `cli.py` で `parse_due` を呼ぶので、形の誤りも終了コード 1 になる

```
uv run todo add "   "; echo $?                # タイトルを入力してください / 1
uv run todo add "テスト" --due 2026/11/1; echo $?   # 期限は YYYY-MM-DD の形で入力してください
/ 1
uv run todo done 99; echo $?                  # タスクが見つかりません: 99 / 1
```

🖊 作り直しをコミットする

```
git add src; git commit -m "タスクを dataclass に、保存先を Protocol にする" -m "dict のキーの
打ち間違いを実行前に見つけ、保存先を差し替えられるようにする。"
```

5. ruff と mypy で確かめる（9分）

🖊 開発用の道具を入れる

📚 uv 公式 Managing dependencies（Development dependencies）

📒 `--dev` で入れたものは、アプリの実行には要らない開発用の依存（`[dependency-groups]` の `dev`）になる

```
uv add --dev ruff mypy
```

🖊 キャッシュを .gitignore に足す

📒 ruff と mypy は、結果のキャッシュをリポジトリの中に作る。見本の `.gitignore` は、第６回のものに２行を足したもの

```
cp ~/handsoff/examples/todo-app-snapshots/session-07/.gitignore .; git diff .gitignore
```

🖊 規約を pyproject.toml に書く

📚 Ruff 公式 Configuring Ruff・Rules、mypy 公式 The mypy command line（--strict）、PEP 8

📒 `select` のルールには、１行ずつ選んだ理由をコメントで書く。`pyproject.toml` の末尾に足す

```toml
# ---- コーディング規約（自動で確かめられるものは ruff と mypy に任せる） ----

[tool.ruff]
line-length = 100
target-version = "py312"

[tool.ruff.lint]
select = [
    "E",    # pycodestyle（PEP 8 の書き方）
    "W",    # pycodestyle の警告
    "F",    # pyflakes（未使用の import・変数など）
    "I",    # import の並び順
    "N",    # 命名規則（pep8-naming）
    "UP",   # 新しい書き方への置き換え（pyupgrade）
    "B",    # バグになりやすい書き方（flake8-bugbear）
    "SIM",  # もっと簡単に書ける箇所
    "E722", # 例外の種類を書かない except（握りつぶし）
]

[tool.mypy]
python_version = "3.12"
# 型ヒントのない関数や Any の混入も指摘する
strict = true
files = ["src"]
```

🖋 確かめるタスクを mise.toml に書く

📚 mise 公式 Tasks

📒 `check` は `lint` と `typecheck` の両方を動かす。PR を出す前に必ず通す。`mise.toml` の末尾に足す

```
[tasks.lint]
description = "ruff でリントとフォーマットを確かめる"
run = ["ruff check .", "ruff format --check ."]

[tasks.fmt]
description = "ruff でフォーマットし、直せるリントを直す"
run = ["ruff format .", "ruff check --fix ."]

[tasks.typecheck]
description = "mypy で型を確かめる"
run = "mypy"

[tasks.check]
description = "PR を出す前の確認（リント・型）"
depends = ["lint", "typecheck"]
```

🔗 check を通す

```
mise run check
```

- 確認: `All checks passed!` と `Success: no issues found`。指摘が出たら `mise run fmt` の後にもう一度

🔗 わざと属性を打ち間違える

📒 dict の `task["titel"]` は、その行を実行したときに初めて `KeyError` になる。dataclass の `task.titel` は、実行する前に mypy が見つける

```
sed -i 's/{task.title}{due}/{task.titel}{due}/' src/todo/cli.py; mise run typecheck   # "Task" has no attribute "titel"; maybe "title"?
git restore src/todo/cli.py
```

🔗 JsonStore のメソッドの名前を変えてみる

📒 `JsonStore` の `mark_done` だけを `finish` にすると、`TaskStore` の形に合わなくなる。`list` は動くので、mypy がなければ `done` を打つまで気づかない

```
sed -i '/^class JsonStore/,$ s/def mark_done(/def finish(/' src/todo/store.py; mise run
typecheck   # cli.py の store に JsonStore を渡せない
uv run todo done 1                                  # AttributeError: 'JsonStore' object
has no attribute 'mark_done'
git restore src/todo/store.py
```

🖊 使っていない import を足して ruff に直させる

📒 未使用の import(F401)と並び順(I001)は、`mise run fmt` が直す。直せないものは指摘だけが残る

```
sed -i 's/^from enum import Enum/from enum import Enum\nimport os/' src/todo/models.py;
mise run lint   # I001 と F401
mise run fmt; git diff src   # 差分なし（元に戻った）
```

🖊 道具と規約をコミットする

📒 `.mypy_cache/` と `.ruff_cache/` は `git status` に出ない

```
git status --short; git add -A; git commit -m "ruff と mypy でコーディング規約と型を確かめる"
-m "形と型は機械に任せ、レビューでは名前と分け方を見る。"
```

6. README に人が見る規約を書く（3 分）

🖊 見本の README をコピーして読む

📚 SE 基礎 3 章 コードを書く、PEP 8（命名規約）

📒 README には、開発環境・使い方・コードの構成と、ruff・mypy では確かめられない規約を書く。自分の決まりがあれば「コーディング規約」の表に足す

```
cp ~/handsoff/examples/todo-app-snapshots/session-07/README.md .; less README.md
```

🖊 README をコミットする

```
git add README.md; git commit -m "README に開発環境・使い方・コーディング規約を書く"
```

7. PR を出し、セルフレビューしてマージする（4 分）

🖊 push する

📚 Pro Git 3.5 リモートブランチ

```
git log --oneline main..; git push -u origin issue-3-package
```

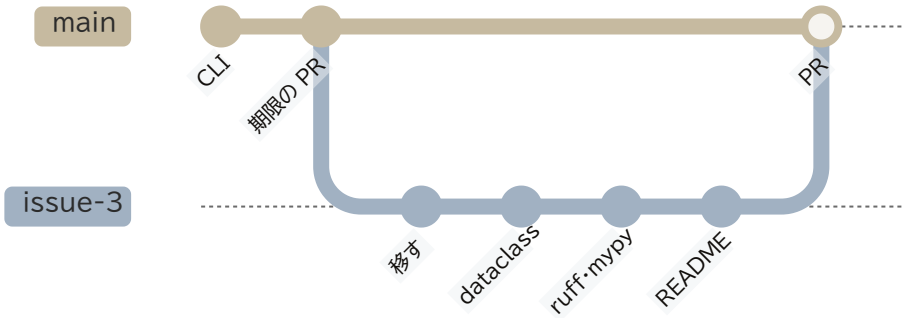
🖊 PR を作り、セルフレビューする（ブラウザ）

📚 SE 基礎 2 章 コードを読む、3 章 コードを書く

📒 本文はテンプレートで書き、変更点に「変えたこと（エラーの文言・終了コード）」と「変えていないこと（一覧の表示・`tasks.json` の形）」を分けて書く。差分を他人のつもりで読み、`[確認]` の行コメントを 1 件以上残す（見本: examples/session07/pr-description.md）

🖊 マージして手元の main をそろえる

📒 1 本の PR に、目的の違う 4 つのコミットが並ぶ。図ではブランチ名を `issue-3` と略す




```
git switch main; git pull; git branch -d issue-3-package; mise run check; uv run todo
list
```

- 確認: Issue #3 が Closed。main で `mise run check` が通る

読む章

区分	資料	範囲	読み方
参考書	ソフトウェアエンジニアリングの基礎(SE基礎)	2章 コードを読む、3章 コードを書く	通読
無料	Python 公式ドキュメント	チュートリアル 6.4 パッケージ、ライブラリ `dataclasses`・`enum`・`typing`(Protocol)、用語集(ダックタイピング)	該当章
無料	PEP 8・Ruff・mypy 公式	PEP 8 の命名規約／Ruff の Configuring Ruff とルール一覧／mypy の Protocols and structural subtyping、`--strict`	該当章(ルールは困ったときに引く)
無料	uv 公式ドキュメント	Configuring projects(Entry points、Build systems)、The uv build backend、Managing dependencies(Development dependencies)	引く
紹介(副読本)	Effective Python 第2版	5章 クラスと継承、10章 項目90 静的解析	持っている人は該当章
紹介(副読本)	リーダブルコード	2〜6章 名前とコメント、7〜9章 制御フローと変数	持っている人は通読

第８回 テストと TDD

tag: test-自動, test-CI
前提タグ: python-oop, test-静的解析, git-github
所要: 座学15分 / ハンズオン45分

ゴール

- pytest の `assert`・`parametrize`・fixture・`tmp_path`・`capsys` でテストを書き、失敗の表示を読める
- 機能を足す前に、今の動作をテストで固定できる
- 「期限が近い順に並べる」をレッド → グリーン → リファクタリングの順で足し、テストと実装のコミットを分けられる
- push のたびに ruff・mypy・pytest を GitHub Actions で動かし、CI が通ってから PR をマージできる

事前準備

- 第７回のハンズオンの最後（手順７のマージ）まで済み、`~/handsoff/workspace/todo-app` の main で `mise run check` が通る
- 遅れた人: session-07/ を examples/README.md の手順でコピーし、push してから始める
- アプリのリポジトリで GitHub Actions が使える（open-questions.md の Q11。GitLab は Q4）。社内ネットワークでは PyPI に届く（手順１の `uv add`。Q5）

まとめ

冒頭の座学で使います。ハンズオンでは、今の動作をテストで固定し、CI を置いてから、「期限が近い順に並べる」をテストを先に書いて足します。

テストの種類

種類	このアプリでは	速さ・数	参照
単体テスト	`test_models.py`・`test_sort.py`（関数１つ。ファイルも日付も使わない）	速い。一番多く書く	SE基礎 5章
結合テスト	`test_store.py`（`JsonStore` が本当にファイルに書く）、`test_cli.py`（`main` から保存先まで通す）	少し遅い	SE基礎 5章
E2E テスト	第11回の `test_web.py`（HTTP で画面まで）	遅い。少なく	SE基礎 5章

pytest の書き方

書き方	意味	参照
`tests/test_*.py` の `def test_*():`	テストとして集められる。`mise run test`（`uv run pytest -v` で1件ずつ、`-k 語` で名前を絞る）	pytest 公式 Get Started
`assert 実際 == 期待`	失敗すると、両辺の値と違う所を表示する	pytest 公式 assertions
`with pytest.raises(例外, match="文言"):`	例外が出ることと、その文言を確かめる	pytest 公式 assertions
`@pytest.mark.parametrize`	同じテストを、入力と期待の組ごとに別のテストとして動かす	pytest 公式 parametrize
`@pytest.fixture`	準備を関数にし、テストは引数名で受け取る。`params=` で準備ごとに繰り返す	pytest 公式 fixtures
`tmp_path`	テストごとに新しい空の一時ディレクトリ（`Path`）	pytest 公式 tmp_path
`capsys`	`print` の出力を受け取る（`capsys.readouterr().out`・`.err`）	pytest 公式 capture

テストしやすい形

形	このアプリでは	参照
判定を関数に分ける	`Task.status(today)`、`sort_by_due(tasks)`	SE基礎 5章
「今日」と保存先を外から渡す	`main(argv, store=MemoryStore(...), today=date(2026, 11, 1))`（第7回の手順4）	SE基礎 5章
同じテストを保存先ごとに	`MemoryStore` と `tmp_path` の `JsonStore`。両方で通れば、どちらも `TaskStore` の約束を守っている	SE基礎 5章
境目をテストにする	期限の当日・期限なし・同じ期限。間違いは境目に出やすい	SE基礎 5章
テスト名で仕様を書く	`test_期限なしは最後に並べる`（日本語でよい）	リーダブルコード 14章

TDD

決まり	理由・やり方	参照
レッド → グリーン → リファクタリング	テストを書いて失敗を見る → 通る最小の実装 → 通ったまま整える	SE基礎 5章
先に失敗を見る	最初から通るテストは、何も確かめていないことがある	SE基礎 5章
グリーンは一番簡単な実装でよい	次のテストで本物に近づける（三角測量）	SE基礎 5章
コミットを分け、1行目の頭に `テスト:` `レッド:` `グリーン:` `リファクタ:`	PR の履歴で順序を追える。テストと実装は別々に `git add` する	研修の規約

並び順の仕様: 期限が近い順、期限なしは最後、期限が同じなら id の順

CI

項目	内容	参照
CI	push・PR のたびに、GitHub の用意した PC で `mise run setup` と `mise run check` を動かす	SE基礎 10章
置き場所	`.github/workflows/check.yml`（GitLab は `.gitlab-ci.yml` に読み替え）	GitHub Actions 公式、mise 公式
判断	CI が赤の PR はマージしない。ログの失敗した行を読み、手元の `mise run check` で再現して直す	SE基礎 10章

ハンズオン

1ステップ1コマンド。各ステップは「🖋 やること → 📚 参考書・公式資料の節 → 📒 説明 → コマンドやコード」。作業場所は `~/handsoff/workspace/todo-app/`。見本は session-08/（examples/session08/）。

1. Issue とブランチを作り、pytest を入れる（5分）

> 🖋 **Issue を立てる（ブラウザ）**

📚 Pro Git 6.2 プロジェクトへの貢献

📒 第7回の PR が #4 なので、この Issue は #5 になる（GitLab では出た番号を使う）

Issue	タイトル	目的	完了条件
#5	一覧を期限が近い順に並べる	締め切りの近いタスクを先に見る。今の動作をテストで固定し、push のたびに自動で確かめる	`uv run todo list` が期限が近い順（期限なしは最後）。`mise run check` が手元と CI で通る

🔗 ブランチを作る

```
cd ~/handsoff/workspace/todo-app
git switch main; git pull; git switch -c issue-5-sort
```

🔗 pytest を入れる

📚 SE基礎 5章 自動テスト、uv 公式 Managing dependencies（Development dependencies）

📒 速い単体テストを多く書き、遅いテストは少なくする。今日は単体と結合を書く（第11回で E2E）

```
uv add --dev pytest
```

🔗 キャッシュを .gitignore に足す

📒 pytest は `.pytest_cache/` を作る。見本の `.gitignore` は、第7回のものに1行を足したもの

```
cp ~/handsoff/examples/todo-app-snapshots/session-08/.gitignore .; git diff .gitignore
```

🔗 pytest と ruff・mypy の設定を書く

📚 pytest 公式 Configuration、Ruff 公式 Configuring Ruff（per-file-ignores）

📒 テストは `tests/` に置く。テスト名の日本語を命名規則（N802）が指摘するので、`tests/` だけ外す。mypy はテストの型も確かめる。`[tool.mypy]` の `files` を `["src", "tests"]` に変え、`pyproject.toml` の末尾に下を足す

```
[tool.ruff.lint.per-file-ignores]
# テスト関数名には確かめたいことを日本語で書く。N802 は英小文字以外を指摘するため外す
"tests/**" = ["N802"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

🖊 test タスクを mise.toml に足す

📚 mise 公式 Tasks

📒 `check` に `test` を足す。PR を出す前の確認が、リント・型・テストの３つになる。`[tasks.check]` を書き換え、`[tasks.test]` を末尾に足す

```
[tasks.check]
description = "PR を出す前の確認（リント・型・テスト）"
depends = ["lint", "typecheck", "test"]

[tasks.test]
description = "自動テスト（pytest）"
run = "pytest"
```

🖊 テストがないときの結果を見る

📒 テストが１件もないと、pytest は `no tests ran` で失敗する（終了コード 5）。テストを書き忘れたまま「通った」ことにはならない

```
mkdir tests; mise run test
```

2. assert と parametrize（7分）

🖊 状態のテストを書く

📚 pytest 公式 How to parametrize fixtures and test functions、How to write and report assertions in tests

📒 １つのテスト関数が、組の数（６つ）だけ別のテストとして動く。エディタで `tests/test_models.py` を新しく作り、下を書く

```python
"""タスクの状態と入力チェック（models.py）。"""

from datetime import date

import pytest

from todo.models import InvalidTaskError, Status, Task, validate_title

TODAY = date(2026, 11, 1)


@pytest.mark.parametrize(
    ("due", "done", "expected"),
    [
        (None, False, Status.TODO),  # 期限なし
        (date(2026, 11, 2), False, Status.TODO),  # 期限は明日
        (TODAY, False, Status.TODO),  # 期限の当日はまだ期限切れではない
        (date(2026, 10, 31), False, Status.OVERDUE),  # 期限は昨日
        (date(2026, 10, 31), True, Status.DONE),  # 完了したものは期限切れにならない
        (None, True, Status.DONE),
    ],
)
def test_状態は完了と期限と今日で決まる(due: date | None, done: bool, expected: Status) →
None:
    task = Task(id=1, title="牛乳を買う", due=due, done=done)

    assert task.status(TODAY) is expected
```

🖋 例外のテストを書く

📒 `pytest.raises` の中で例外が出なければ、テストは失敗する。`match` で文言も確かめる。同じファイルの末尾に足す

```python
@pytest.mark.parametrize("title", ["", "   "])
def test_タイトルが空ならエラー(title: str) → None:
    with pytest.raises(InvalidTaskError, match="タイトルを入力してください"):
        validate_title(title)
```

🖊 テストを動かす

```
mise run test; uv run pytest -v   # 8 passed。-v では組ごとに [due1-False-Status.TODO] のような名前が付く
```

🖊 わざと壊して、失敗の表示を読む

📒 期限の当日（境目）を間違えると、当日の組だけが落ちる。`>` の行が落ちた `assert`、`E` の行が実際の値と期待の値。読んだら元に戻す

```
sed -i 's/self.due < today/self.due ⩽ today/' src/todo/models.py; mise run test
git restore src/todo/models.py
```

- 確認: `FAILED tests/test_models.py::test_状態は完了と期限と今日で決まる[due2-False-Status.TODO]` と `assert <Status.OVERDUE: '期限切れ'> is <Status.TODO: '未着手'>`

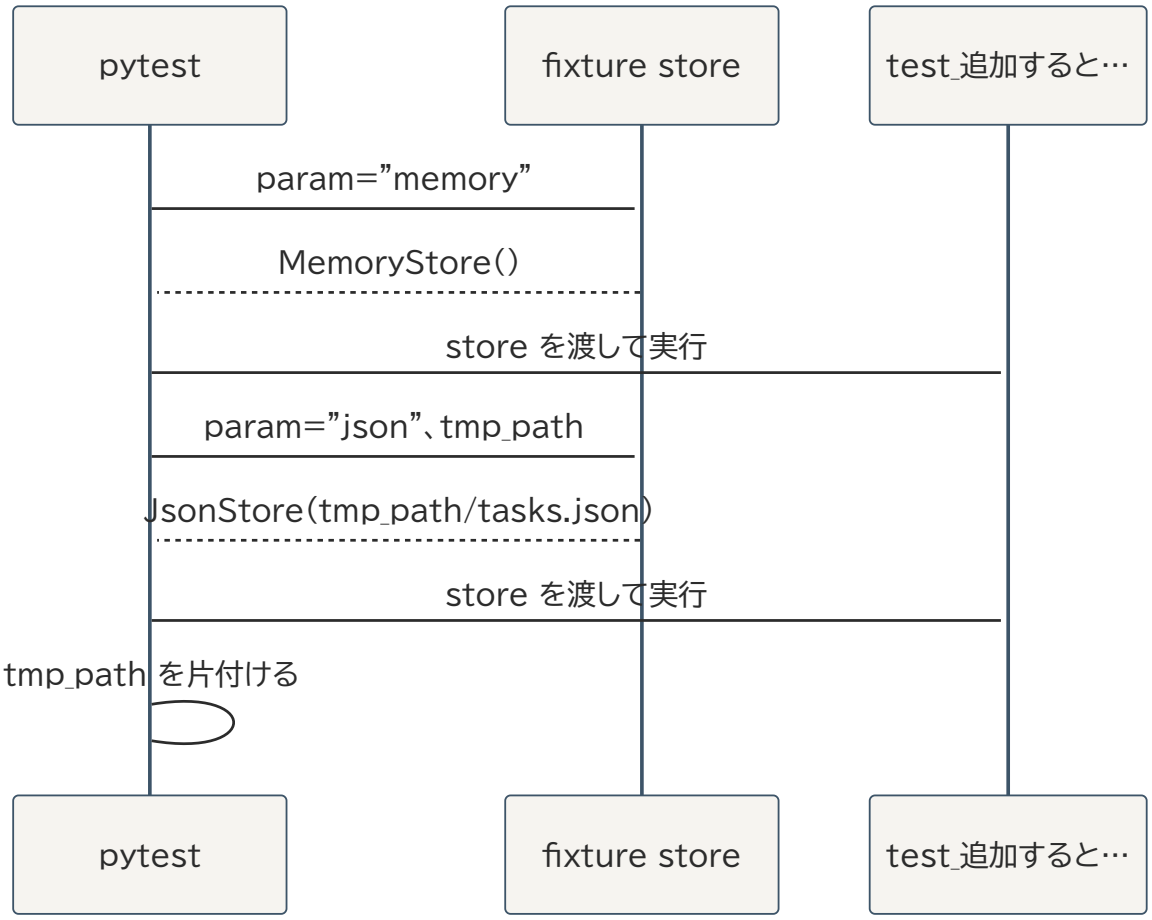
> 座学: 失敗の表示のどこを見るか。境目のテストがなければ、この間違いに気づけたか（→ まとめ「テストしやすい形」）

3. fixture と tmp_path（6分）

🖊 保存先のテストをコピーする

📚 pytest 公式 How to use fixtures（Parametrizing fixtures）、How to use temporary directories and files in tests

📒 テストは引数名 `store` で fixture を受け取る。`params` が２つなので、各テストが `MemoryStore` と `JsonStore` で１回ずつ動く。`tmp_path` はテストごとに新しい空のディレクトリで、手元の `tasks.json` には触れない




```
cp ~/handsoff/docs/session08/assets/test_store.py tests/; less tests/test_store.py
```

🖊 保存先ごとに動くのを見る

📒 同じテストが両方で通れば、どちらも `TaskStore` の約束(`list_all`・`add`・`mark_done`)を守っている

```
uv run pytest -v tests/test_store.py   # [memory] と [json] が並ぶ
```

🖊「最初は空」のテストを書く

📒「手順 3: ここに test_最初は空 を書く」のコメントの行を置き換える。fixture を使うので、これも 2 回動く

```
def test_最初は空(store: TaskStore) → None:
    assert store.list_all() == []
```

```
mise run test   # 18 passed
```

> 座学: `JsonStore` のテストが手元の `tasks.json` を書き換えない理由と、`MemoryStore` と同じテストが通る意味(→ まとめ「テストしやすい形」)

4. CLI の動作を固定する(5 分)

🖊 CLI のテストをコピーして読む

📚 pytest 公式 How to capture stdout/stderr output

📒 `run` は `main` に `MemoryStore` と固定の「今日」(2026-11-01)を渡す。表示は `capsys` で受け取って比べる

```
cp ~/handsoff/docs/session08/assets/test_cli.py tests/; less tests/test_cli.py
```

🖊 ない番号の done のテストを書く

📒 「手順 4: ここに test_done でない番号なら終了コード 1 を書く」の行を置き換える。エラーは標準エラー出力なので `.err` を見る

```
def test_doneでない番号なら終了コード1(
    store: MemoryStore, capsys: pytest.CaptureFixture[str]
) → None:
    assert run(store, "done", "9") == 1

    assert "タスクが見つかりません: 9" in capsys.readouterr().err
```

🖊 check を通す

```
mise run check   # 24 passed。指摘が出たら mise run fmt の後にもう一度
```

🖊 テストをコミットする

📒 動作は変えていないので `テスト:` を付ける。この後の変更で今の動作を壊すと、このテストが落ちる

```
git status --short; git add .gitignore pyproject.toml uv.lock mise.toml tests; git commit -m "テスト: 今の動作をテストで固定する" -m "期限の順に並べる機能を足す前に、add・list・done と保存先の今の動きを確かめておく。"
```

5. CI を置いて push する（5分）

🖊 ワークフローをコピーして読む

📚 GitHub Actions 公式 ワークフロー構文、mise 公式 Continuous Integration

📒 GitHub の用意した PC で、手元と同じ `mise run setup` と `mise run check` を動かす。`mise-action` が `mise.toml` の Python と uv を入れる

```
cp -r ~/handsoff/examples/todo-app-snapshots/session-08/.github .; less .github/workflows/check.yml
```

```
name: check
on:
  push:
  pull_request:
jobs:
  check:
    runs-on: ubuntu-latest
    env:
      MISE_TRUSTED_CONFIG_PATHS: ${{ github.workspace }}   # mise trust に当たる
    steps:
      - uses: actions/checkout@v5
      - uses: jdx/mise-action@v3   # mise.toml の Python・uv を入れる
        with:
          version: 2026.10.7
      - run: mise run setup        # uv.lock どおりに入れる
      - run: mise run check        # ruff・mypy・pytest
```

🖊 コミットして push する

📚 SE基礎 10章 本番環境へ

📒 push すると、GitHub がワークフローを動かす。結果は数分で出るので、待たずに手順6へ進む

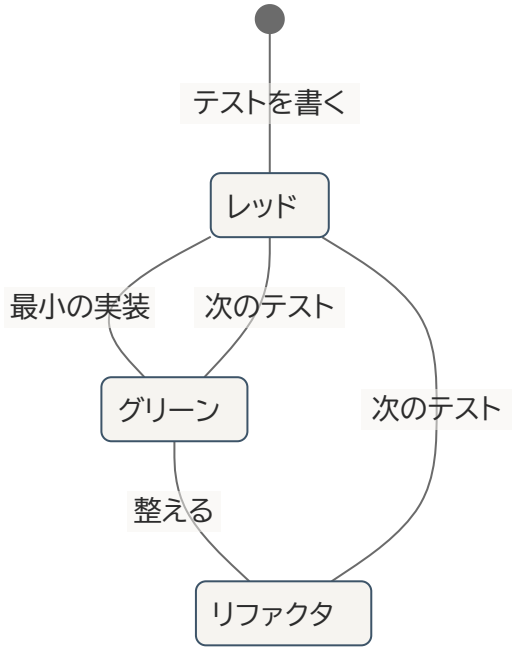
```
git add .github; git commit -m "PR ごとに mise run check を GitHub Actions で実行する"; git push -u origin issue-5-sort
```

6. TDD: 期限が近い順に並べる（10 分）

> 🖊 **レッド: 最初のテストを書く**

📚 SE 基礎 5 章 自動テスト

📒 `sort_by_due` はまだない。エディタで `tests/test_sort.py` を新しく作り、下を書く



```
"""期限が近い順に並べる（第8回に TDD で足した機能）。"""

from datetime import date

from todo.models import Task, sort_by_due


def test_期限が近い順に並べる() → None:
    tasks = [
        Task(id=1, title="a", due=date(2026, 11, 3)),
        Task(id=2, title="b", due=date(2026, 11, 1)),
        Task(id=3, title="c", due=date(2026, 11, 2)),
    ]

    assert [t.id for t in sort_by_due(tasks)] == [2, 3, 1]
```

🔗 失敗を見て、レッドを push する

📒 `ImportError: cannot import name 'sort_by_due'` で落ちれば、テストが本当に `sort_by_due` を確かめている。テストのファイルだけをコミットし、CI が赤になるのも見る

```
mise run test; git add tests/test_sort.py; git commit -m "レッド: 期限が近い順に並べるテスト";
git push
```

🔗 グリーン: 一番簡単な実装

📒 `models.py` の `Task` クラスの後に足す。期限なしのことはまだ考えない

```
def sort_by_due(tasks: list[Task]) → list[Task]:
    """期限が近い順に並べる。"""
    return sorted(tasks, key=lambda t: t.due)
```

```
mise run test; git add src; git commit -m "グリーン: 期限の順に並べる最小の実装"   # 25 passed
```

🔗 Actions の結果を見る（ブラウザ）

📚 GitHub Actions 公式 ワークフロー実行ログの使用

📒 リポジトリの Actions タブを開く。手順５の push は緑、レッドの push は赤で、ログの `mise run check` に手元と同じ `ImportError` が出ている

🖊 mypy の指摘を見る

📒 テストは通ったが、mypy は `t.due` が `None` のときに比べられないことを指摘する。次のテストで確かめる

```
mise run typecheck   # Argument "key" to "sorted" has incompatible type "Callable[[Task], date | None]" ...
```

🖊 レッド: 期限なしのテストを書く

📒 `test_sort.py` の末尾に足す。`TypeError: '<' not supported between instances of 'datetime.date' and 'NoneType'` で落ちる

```
def test_期限なしは最後に並べる() -> None:
    tasks = [
        Task(id=1, title="a"),
        Task(id=2, title="b", due=date(2026, 12, 31)),
    ]

    assert [t.id for t in sort_by_due(tasks)] == [2, 1]
```

```
mise run test; git add tests/test_sort.py; git commit -m "レッド: 期限なしは最後に並べるテスト"
```

🖊 グリーン: 期限なしを最後にする

📒 キーをタプル `(期限なしか, 期限)` にする。`False < True` なので期限ありが先。`t.due or date.min` は、期限なしでも比べられるようにするための値

```
    return sorted(tasks, key=lambda t: (t.due is None, t.due or date.min))
```

```
mise run check; git add src; git commit -m "グリーン: 期限なしを最後にする"   # mypy の指摘も消える
```

🖊 レッド: 同じ期限のテストを書く

📒 期限が同じものは元の並び(3 → 1)のまま残り、`assert [3, 1, 4, 2] == [1, 3, 2, 4]` で落ちる。テストを足すたびに、キーが本物に近づく(三角測量)

```
def test_期限が同じなら番号の順() -> None:
    tasks = [
        Task(id=3, title="c", due=date(2026, 11, 1)),
        Task(id=1, title="a", due=date(2026, 11, 1)),
        Task(id=4, title="d"),
        Task(id=2, title="b"),
    ]

    assert [t.id for t in sort_by_due(tasks)] == [1, 3, 2, 4]
```

```
mise run test; git add tests/test_sort.py; git commit -m "レッド: 期限が同じなら番号の順のテスト"
```

🖊 グリーン: キーに id を足す

📒 タプルは前の要素から順に比べ、同じなら次の要素で比べる。`(False, 11/1, 1)` → `(False, 11/1, 3)` → `(True, min, 2)` → `(True, min, 4)` の順になる。`sort_by_due` を下に置き換える

```
def sort_by_due(tasks: list[Task]) -> list[Task]:
    """期限が近い順に並べる。期限なしは最後、期限が同じなら id の順。"""
    # タプルは前の要素から順に比べる: (期限なしか, 期限, id)。False < True なので期限ありが先
    return sorted(tasks, key=lambda t: (t.due is None, t.due or date.min, t.id))
```

```
mise run test; git add src; git commit -m "グリーン: 期限が同じなら番号の順にする"   # 27 passed
```

🖊 リファクタ: テストの重複をまとめる

📒 3つのテストに同じ `[t.id for t in ...]` がある。`ids` にまとめ、テストが通ったままであることを確かめる。`import` の下に関数を足し、3か所を `ids(sort_by_due(tasks))` に置き換える

```
def ids(tasks: list[Task]) -> list[int]:
    return [t.id for t in tasks]
```

```
mise run check; git commit -am "リファクタ: テストの番号の取り出しを ids にまとめる"
```

座学: 最小の実装で最初のテストが通ってしまったこと。次のテストと mypy で気づいたこと(→ まとめ「TDD」)

7. TDD: 一覧を期限が近い順に表示する(4分)

🖊 レッド: list のテストを書く

📒 `tests/test_cli.py` の「手順7: ここに …」の行を置き換える。今は id の順(1 → 2 → 3)に出るので落ちる

```
def test_listは期限が近い順に表示する(
    store: MemoryStore, capsys: pytest.CaptureFixture[str]
) → None:
    assert run(store, "list") == 0

    assert capsys.readouterr().out.splitlines() == [
        "  2 [期限切れ] 牛乳を買う (期限 2026-10-30)",
        "  1 [未着手] レポートを出す (期限 2026-11-05)",
        "  3 [未着手] 本を返す",
    ]
```

```
mise run test; git add tests/test_cli.py; git commit -m "レッド: list を期限が近い順に表示するテスト"
```

🖊 グリーン: run_list で並べる

📒 `cli.py` の `from todo.models import ...` に `sort_by_due` を足し、`run_list` の1行目を置き換える。期限切れだけの表示も、並べた後に絞るので同じ順になる

```
    tasks = sort_by_due(store.list_all())
```

```
mise run check; git add src; git commit -m "グリーン: list を期限が近い順に表示する"   # 28 passed
uv run todo list
```

8. PR を出し、CI が通ってからマージする（4 分）

🖊 README を更新する

📒 見本の README は、`mise run check` にテストが入ったこと、CI、`tests/`、テストの規約を足したもの

```
cp ~/handsoff/examples/todo-app-snapshots/session-08/README.md .; git diff README.md; git add README.md; git commit -m "README にテストと CI を書く"
```

🖊 履歴を確かめて push する

📒 テスト → CI → レッドとグリーンの繰り返し → リファクタ → README の順に並ぶ

```
git log --oneline main..; git push
```

🖊 PR を作り、セルフレビューする（ブラウザ）

📚 SE 基礎 2 章 コードを読む、10 章 本番環境へ

📒 本文はテンプレートで書き、目的に `Closes #5`、確認方法に `mise run check` と `uv run todo list` を書く。Commits タブでレッドとグリーンの順を確かめ、`[確認]` の行コメントを 1 件以上残す

🖊 CI が緑になってからマージする

📒 PR の下のチェック `check` が緑になるのを待つ。赤なら、ログの失敗した行を読み、手元の `mise run check` で再現して直す

```
git switch main; git pull; git branch -d issue-5-sort; mise run check; uv run todo list
```

- 確認: Issue #5 が Closed。main で `mise run check` が通り、`uv run todo list` が期限が近い順

読む章

区分	資料	範囲	読み方
参考書	ソフトウェアエンジニアリングの基礎（SE 基礎）	5 章 自動テスト、10 章 本番環境へ（CI）	通読（5 章は第 11・12 回でも使う）
無料	pytest 公式	Get Started、How-to（assertions、fixtures、parametrize、tmp_path、capture）	該当章

区分	資料	範囲	読み方
無料	GitHub Actions 公式・mise 公式	ワークフロー構文、ワークフロー実行ログ／Continuous Integration（mise-action）	困ったときに引く
紹介（副読本）	リーダブルコード	14 章 テストと読みやすさ	持っている人は通読

第９回 要件と図

tag: 設計-要件, pm-見積, pm-進捗
前提タグ: python-構文, test-自動, git-github
所要: 座学15分 / ハンズオン45分

ゴール

- 伝えたいことに合わせて図の種類を選び、Mermaid で Markdown に書いてプレビューできる
- 曖昧な依頼から質問で「なぜ・誰が・何を・どのくらい」を引き出し、要件メモ（業務フロー図・状態遷移図・非機能要件・対象外）に書ける
- ユーザーストーリーと、確かめられる形の受け入れ条件を書ける
- ストーリーをタスクに分けて幅で見積もり、Issue・ラベル・マイルストーンで管理できる

事前準備

- 第８回のハンズオンの最後（手順８のマージ）まで済み、`~/handsoff/workspace/todo-app` の main で `mise run check` が通る
- 遅れた人: session-08/ を examples/README.md の手順でコピーし、push してから始める
- VS Code で Markdown の Mermaid をプレビューできる（拡張機能 Markdown Preview Mermaid Support）。入れられなければ GitHub に push して画面で見る

まとめ

冒頭の座学で使います。ハンズオンでは、Mermaid を CLI の図で練習してから、講師の依頼「**CLI の TODO を Web でも使いたい**」を要件メモとストーリーにし、Issue に登録します。

設計書の考え方

項目	内容	参照
設計書の役割	作る前に「何を・どう作るか」を決め、残す	SE基礎 ４章
合格基準	**実装者が質問せずに作れるか**	SE基礎 ４章
書くもの	ストーリー（Issue）・要件メモ（第９回）、API・DB設計書・システム構成図（第10回）	—
書かないもの	画面設計書・テスト仕様書。細かい振る舞いは TDD のテストで示す	SE基礎 ５章
置き場所	リポジトリの `docs/` に Markdown で置く。図も Mermaid の文字で書くので、PR で差分を読める	Mermaid 公式

項目	内容	参照
社外サービスに貼らない	mermaid.live などに社内の内容を貼らない。プレビューは手元の VS Code か社内の GitHub	—

図の種類と使い分け

伝えたいこと	図	Mermaid	書く設計書(回)	参照
誰が・どの順に・何を。分かれ道	業務フロー図・処理の流れ	`flowchart`	要件メモ(第９回)	SE基礎 ４章
状態と、変わるきっかけ	状態遷移図	`stateDiagram-v2`	要件メモ(第９回)	SE基礎 ４章
誰と誰が何をどの順にやり取りするか	シーケンス図	`sequenceDiagram`	API設計書(第10回)、第11〜12回	SE基礎 ４章
データの持ち方と関係	ER図	`erDiagram`	DB設計書(第10回)	SE基礎 ４章
部品・通信経路・ポート	構成図	`flowchart` (`subgraph`)	システム構成図(第10回)	SE基礎 ９章
クラスと依存の向き	クラス図	`classDiagram`	(第７回で見た)	SE基礎 ４章

Mermaid の書き方

書き方	意味	参照
```` ```mermaid ```` 〜 ```` ``` ````	Markdown の中に図を書く。1行目で図の種類を決める	Mermaid 公式 Syntax Reference
`flowchart TD` / `A[文字]` / `B{文字}`	上から下へ(`LR` は左から右)／四角(処理)／菱形(分かれ道)	Mermaid 公式 Flowcharts
`A ⟶ B` / `A -- はい ⟶ B` / `A -.→ B`	矢印／ラベル付き／点線	Mermaid 公式 Flowcharts
`A["記号(あり)"]`、` `	記号を含む文字は `"` で囲む。長い文字は ` ` で折る	Mermaid 公式 Flowcharts
`subgraph 利用者` 〜 `end`	枠(人ごとのレーン、部品のまとまり)	Mermaid 公式 Flowcharts
`participant C as cli` / `C⟹M: 呼ぶ` / `M-->>C: 返す`	シーケンス図の登場人物／呼び出し／戻り	Mermaid 公式 Sequence diagrams
`[*] ⟶ TODO : 追加する`	状態遷移図の開始と、きっかけ付きの遷移	Mermaid 公式 State diagrams

図のレビュー観点

- 1つの図に1つの目的。何を伝える図か、図の上の1文で言える
- 名前が設計書の中で一貫している(「完了」と「済み」が混ざらない)
- 分かれ道・矢印のきっかけが、質問せずに読める

ユーザーストーリーと受け入れ条件

> <利用者>として、<したいこと>したい。なぜなら<理由>から。
> 受け入れ条件:
> - [ ] (1) <前提>のとき、<操作>すると、<結果(数字・文言・境目)>

項目	内容・例	参照
ストーリー	利用者として、ブラウザでタスクを追加したい。なぜなら、ターミナルを開かずに書き留めたいから	—
受け入れ条件	全部満たせば完了、という条件。数字・文言・境目で書く(「101文字以上なら追加せず『…』と出る」)	—
番号	(1)〜(15) は全ストーリーの通し番号。1つの条件が第11回でテスト1本以上になり、テスト名の頭に番号を付ける	—
Issue	1ストーリー = 1 Issue。タイトルは `S-01 タスクを追加する`、ラベル `ストーリー`(テンプレート)	—
確かめる観点	誰が嬉しいか言える／作り方まで決めていない／確かめられる／半日〜1日で終わる大きさ	—

要件メモ(テンプレート)

項目	書くこと	例	参照
目的・背景	何に困り、何を解決するか。業務フロー図・状態遷移図	ターミナルを開かないとタスクを見られない	SE基礎4章
非機能要件	**どのくらい** よくあるべきか(性能・セキュリティ・運用)を数字や確かめられる形で	1,000件で一覧を1秒以内	SE基礎9章
制約	変えられない前提	Python 3.12・FastAPI・SQLite	—
対象外	**今回やらないこと** と理由	ログイン(自分の PC だけで使う)	—
ストーリー一覧	Issue の一覧へのリンクと控え(`docs/stories.md`)	—	—

見積（pm-見積）

考え方	ひとことで	参照
タスクに分ける	ストーリーを半日以内の作業に分ける（土台・保存・画面・入力チェックなど）	達人プログラマー 見積もり
幅で見積もる	見積は約束ではなく予測。「2〜4時間」のように幅で書く。幅が広いところが不確かなところ	達人プログラマー 見積もり
根拠を一言	「CLI の add と同じ」「初めてなので上を広く」など	—
記録しない	実績との比較はしない。見積の根拠を言葉にする練習	—

Issue での管理（pm-進捗）

道具	使い方	参照
Issue	ストーリー1件・タスク1件ずつ（ストーリー用／タスク用）	Pro Git 6.2
ラベル	種類で絞り込む: `ストーリー`・`タスク`（第12回で `不具合`）	GitHub Docs ラベルを管理する
マイルストーン	期日までに閉じる Issue の束。`Web 版`（期日は第12回）。進み具合が閉じた数で見える	GitHub Docs マイルストーンについて
閉じ方	PR の本文に `Closes #番号`。マージで閉じる（ストーリーは第11回の PR #16 で閉じる）	Pro Git 6.2

ハンズオン

1ステップ1コマンド。各ステップは「🖋 やること → 📚 参考書・公式資料の節 → 📒 説明 → コマンドや書くもの」。作業場所は `~/handsoff/workspace/todo-app/`。見本は session-09/（examples/session09/）。

1. Issue とブランチを作る（3分）

> **🖋 Issue を立てる（ブラウザ）**

📚 Pro Git 6.2 プロジェクトへの貢献

📒 第8回の PR が #6 なので、この Issue は #7 になる（GitLab では出た番号を使う）。本文はタスク用テンプレートで書き、見積に「1〜2時間（要件を書くのは初めて）」のように幅と根拠を入れる

Issue	タイトル	目的	完了条件
#7	Web 版の要件メモとストーリーを書く	作る前に、何を作るかを依頼人と合わせる	`docs/requirements.md`・`docs/stories.md` が main にあり、ストーリーが Issue に登録されている

> 🖊 ブランチを作る

📒 今日は文書を書くだけで、コードは変えない。今日の要件メモとストーリーをもとに、第10回で設計書を、第11回でテストを書く

```
cd ~/handsoff/workspace/todo-app
git switch main; git pull; git switch -c issue-7-requirements
```

2. Mermaid の基本: CLI を図にする（7分）

> 🖊 練習のファイルを開く

📚 Mermaid 公式 Flowcharts Syntax

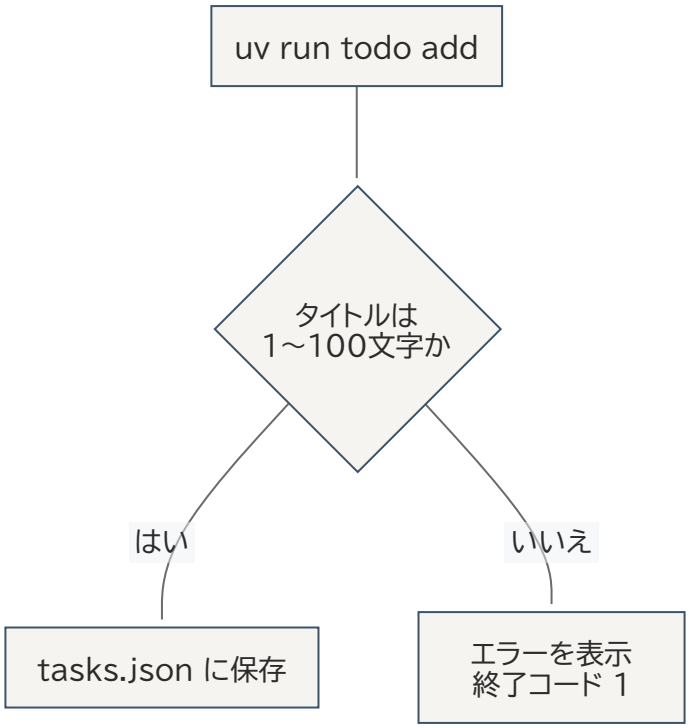
📒 練習はリポジトリの外のファイルで行う。図は ```` ```mermaid ```` のコードブロックに書き、`Ctrl+Shift+V`（横に並べるなら `Ctrl+K V`）でプレビューする

```
mkdir -p ~/handsoff/workspace/session09; code ~/handsoff/workspace/session09/practice.md
```

> 🖊 処理の流れを flowchart で書く

📒 `uv run todo add` の流れ。四角が処理、菱形が分かれ道、矢印のラベルが分かれる条件。下を書いてプレビューし、図のように描かれるのを確かめる

````
```mermaid
flowchart TD
  A[uv run todo add] ⟶ B{タイトルは1〜100文字か}
  B -- はい ⟶ C[tasks.json に保存]
  B -- いいえ ⟶ D[エラーを表示終了コード 1]
```
````



🖊 期限の分かれ道を足す

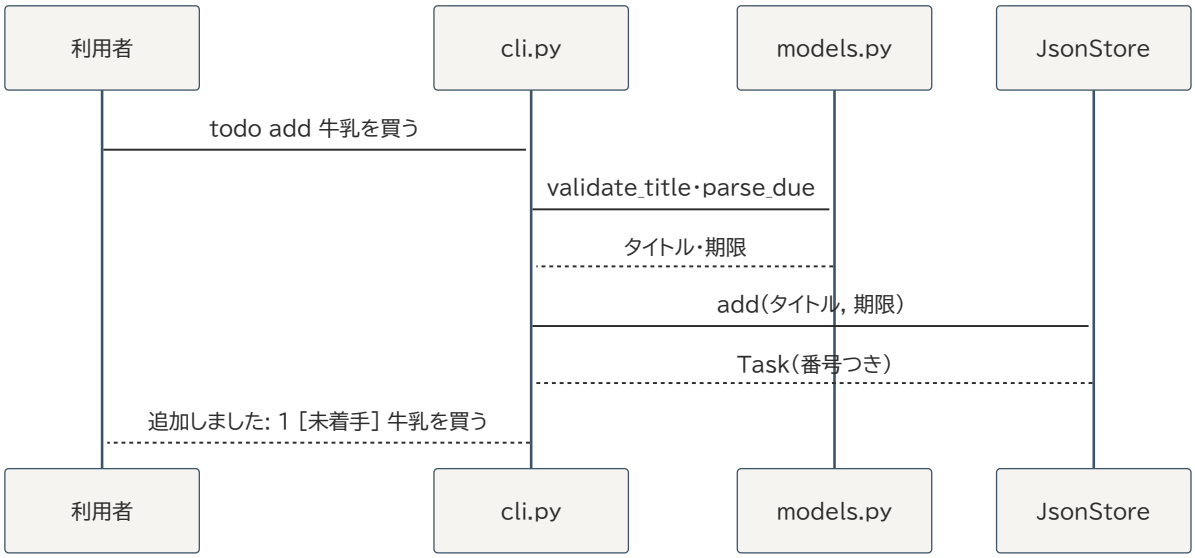
📒 `--due` が日付の形でないときもエラーになる（`parse_due`）。`B` と `C` の間に菱形を1つ足し、`D` に矢印をつなぐ。図が崩れたら、記号を含む文字を `"` で囲んだか確かめる

🖊 やり取りを sequenceDiagram で書く

📚 Mermaid 公式 Sequence diagrams

📒 同じ `add` を「誰が誰を呼ぶか」で見る。flowchart では見えなかった `cli.py`・`models.py`・`store.py` の分担が見える。同じファイルの下に書き足す

```mermaid
sequenceDiagram
  participant U as 利用者
  participant C as cli.py
  participant M as models.py
  participant S as JsonStore
  U->>C: todo add 牛乳を買う
  C->>M: validate_title・parse_due
  M-->>C: タイトル・期限
  C->>S: add(タイトル, 期限)
  S-->>C: Task（番号つき）
  C-->>U: 追加しました: 1 [未着手] 牛乳を買う
```



🖋 図を選ぶ

📚 SE基礎 4章 モデル

📒 図は「何を伝えたいか」で選ぶ。動き（順番・やり取り・状態。`flowchart`・`sequenceDiagram`・`stateDiagram-v2`）を伝える図と、形（データ・部品。`erDiagram`・構成図・`classDiagram`）を伝える図がある。今日は業務フロー図と状態遷移図を書き、残りは第10回で書く

> 座学: 同じ `add` を flowchart とシーケンス図で描くと、それぞれ何が見えて何が見えないか（→ まとめ「図の種類と使い分け」）

3. 依頼を聞き、質問する（7分）

🖊 要件メモのテンプレートを置く

📒 要件メモには、ストーリーに書けないこと（目的・非機能要件・制約・対象外）だけを書く。聞いた答えは、要件メモの各節に振り分けてメモする

```
mkdir docs; cp ~/handsoff/docs/templates/requirements.md docs/requirements.md
```

🖊 依頼を聞く

📒 講師が依頼人の役。最初の一言は「第６回から作っている CLI の TODO、ブラウザでも使いたいんです」だけ。依頼人は聞かれたことにだけ答える

🖊 質問する

📚 SE基礎 ４章 モデル、９章 ソフトウェアアーキテクチャ

📒 欲しい機能からではなく、困りごとから聞く。量や速さは「何件」「何秒」「何文字」のように数字で聞く。最後に「やらないこと」を確かめる

- 聞く順: なぜ（今の困りごと）→ 誰が・どこから使うか → 何をしたいか（CLI との違い）→ どのくらい（何件・何秒・何文字）→ やらないこと（編集・ログインなど）
- 聞き漏らしやすい観点: 期限は必須か／編集・削除は要るか／ログインは要るか／スマホから使うか／CLI のデータは移すか／いつまでに

🖊 答えを要件メモに書く

📒 決まっていないことは「6. 未定」に書く。答えが８件以上集まったら次へ進む

聞いたこと	書く節
困りごと・誰が使うか	1 目的・背景
何件・何秒・どこから見えてよいか・データの置き場所	3 非機能要件
言語・期日など変えられないこと	4 制約
「要らない」と言われたこと（理由も）	5 対象外

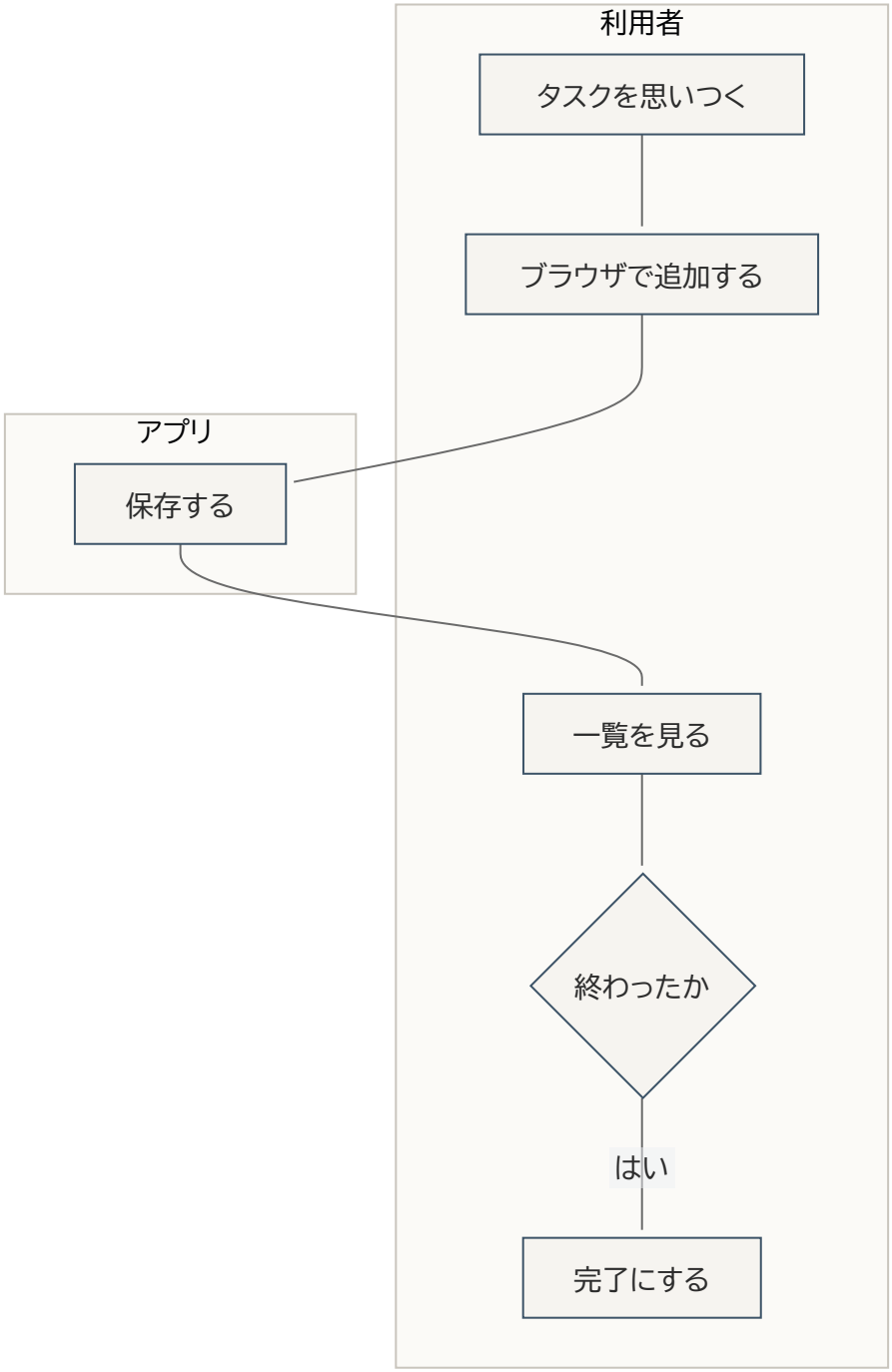
> 座学: 欲しい機能を直接聞くと、今の CLI をそのまま画面にしただけになる。対象外を書く理由（→ まとめ「要件メモ」）

4. 業務フロー図と状態遷移図を書く（6分）

🖋 業務フロー図を書く

📚 SE基礎 4章 モデル

📒 導入後の利用者の1日の流れ。`subgraph` で人とアプリのレーンに分け、アプリが受け持つところを分ける。テンプレートの図（下）に「朝に一覧を開く → 期限切れはあるか」の分かれ道を足す。図の上には、この図で何を伝えるかを1文で書く



🖊 状態遷移図を書く

📚 Mermaid 公式 State diagrams

📒 テンプレートの図(下)には `TODO` と `DONE` しかない。期限切れ(`OVERDUE`)と、そこへ移るきっかけ・そこから出るきっかけを足し、下の表の「条件」を不等号まで埋める



- 確認: 「完了したタスクは期限切れになるか」「期限の当日は期限切れか」「期限なしのタスクは」に図と表で答えられる

> 座学: 期限切れを DB に保存するか、表示するときに計算するか(第10回の DB設計書につながる。→ まとめ「図の種類と使い分け」)

5. ストーリーと受け入れ条件を書く(7分)

🖊 ストーリーの控えを作る

📒 ストーリーは Issue に書き、`docs/stories.md` に控えを残す。テンプレートは Issue 1件分の形

```
cp ~/handsoff/docs/templates/issue_story.md docs/stories.md
```

🖊 したいことを書き出す

📒 依頼人の「したいこと」を1つずつに分け、`S-01` から番号を付ける。1つが半日〜1日で終わる大きさにする。「要らない」と言われたことは要件メモの「対象外」へ移す

🖊 S-01 を1文で書く

📒 `<利用者>として、<したいこと>したい。なぜなら<理由>から。` の形で、ストーリーの節に書く。「なぜなら」には、手順3で聞いた困りごとを入れる

🖊 S-01 の受け入れ条件を書く

📒 受け入れ条件の節に3〜6行書く。依頼人が言った数字(100文字など)、うまくいかない場合(空のタイトル、日付でない期限)、そのときの文言を入れる。1行が第11回でテスト1本以上になる

- 確認: 受け入れ条件だけ読めば、来月の自分が依頼人に質問せずに作れるか

> 座学: 「追加できる」だけでは確かめられない。数字・文言・境目で書く(→ まとめ「ユーザーストーリーと受け入れ条件」)

6. タスクに分けて幅で見積もる(4分)

🖊 S-01 をタスクに分ける

📚 達人プログラマー 見積もり

📒 半日以内の作業に分け、それぞれ「1〜2時間」のように幅で見積もる。下は第6回の「期限を足す」を分けた例。初めての作業は幅を広く取る

タスク	幅	根拠
`parse_due`	0.5〜1時間	—
`--due` の引数	0.5〜1時間	—
`list --overdue`	1〜2.5時間	判定が初めて
合計	2〜4.5時間	—

🖊 見積の表を埋める

📒 `docs/stories.md` の「見積」の表に、タスク・幅・根拠を1行ずつ書き、最後に合計の行を足す。根拠は「似た作業は何か」「初めてのことは何か」で書く

- 確認: いちばん幅が広いタスクと、その理由を言える

> 座学: 幅が広いところが不確かなところ。見積は約束ではない(→ まとめ「見積」)

7. 見本と合わせてコミットする（3 分）

🖊 自分の版をコミットする

📒 自分で書いた版をまずコミットして残す。次のステップで見本に置き換えても、自分の版は履歴に残る

```
git add docs; git commit -m "Web 版の要件メモとストーリー S-01 を書く"
```

🖊 見本に置き換えて違いを見る

📒 見本は、講師（依頼人）と合わせた版。ストーリーは S-01〜S-04、受け入れ条件は通し番号（1）〜（15）。第 10 回・第 11 回はこの番号と文言を使うので、見本に置き換える。`git diff` で自分の版との違いを読む

```
cp ~/handsoff/examples/todo-app-snapshots/session-09/docs/*.md docs/; git diff --stat;
git diff docs/stories.md
```

🖊 README と一緒にコミットする

📒 見本の README は、ドキュメントの表に `docs/` の 2 つを足したもの

```
cp ~/handsoff/examples/todo-app-snapshots/session-09/README.md .; git add README.md docs;
git commit -m "依頼人と合わせた要件メモとストーリー S-01〜S-04 にする"
```

8. Issue・ラベル・マイルストーンで管理する（5 分）

🖊 ラベルを作る（ブラウザ）

📚 GitHub Docs ラベルを管理する

📒 Issues → Labels → New label で `ストーリー` と `タスク` を作り、Issue #7 に `タスク` を付ける（GitLab は Manage → Labels）

🖊 マイルストーンを作る（ブラウザ）

📚 GitHub Docs マイルストーンについて

📒 Issues → Milestones → New milestone で `Web 版` を作り、期日に第 12 回の日を入れる（GitLab は Plan → Milestones）

🔗 ストーリーを Issue に登録する(ブラウザ)

📒 `docs/stories.md` の S-01〜S-04 を1件ずつ Issue にする。タイトルは `S-01 タスクを追加する` の形、本文は各節の「ストーリー」から「関連」までをコピーし、ラベル `ストーリー` とマイルストーン `Web 版` を付ける。番号は #8〜#11 になる

🔗 絞り込んで見る(ブラウザ)

📒 Issue の一覧で `label:ストーリー` と絞り込むと4件が出る。マイルストーン `Web 版` を開くと、閉じた数(0/4)と期日が見える

🔗 Issue の番号を控えに書く

📒 `docs/stories.md` の一覧の表の「Issue」の列は見本では `#8` 〜 `#11`。番号が違ったら直してコミットする

```
git diff --stat; git commit -am "ストーリーの Issue の番号を控えに書く"    # 直したときだけ
```

9. PR を出してマージする(3分)

🔗 push して PR を作る

📚 Pro Git 6.2 プロジェクトへの貢献

📒 本文はテンプレートで書き、目的に `Closes #7`、確認方法に「GitHub の画面で図が描かれる」と書く。Files changed で `docs/requirements.md` を開き、表示を切り替えて図を見る。`[確認]` の行コメントを1件以上残す

```
git push -u origin issue-7-requirements
```

🔗 CI が緑になってからマージする

📒 コードは変えていないが、CI(`check`)が緑になるのを待ってからマージする。Issue #7 は閉じ、ストーリーの #8〜#11 は第11回の PR #16 で閉じる

```
git switch main; git pull; git branch -d issue-7-requirements; ls docs
```

- 確認: Issue #7 が Closed、#8〜#11 が Open でマイルストーン `Web 版` に入っている。main に `docs/requirements.md`・`docs/stories.md` がある

読む章

区分	資料	範囲	読み方
参考書	ソフトウェアエンジニアリングの基礎(SE基礎)	4章 モデル(どの図が必要か)	通読(第10回でも使う)
参考書	同上	9章 ソフトウェアアーキテクチャ(品質特性・非機能要件)	通読(第10回でも使う)
無料	Mermaid 公式	Syntax Reference(Flowcharts、Sequence diagrams、State diagrams)	図を書くときに引く
無料	Pro Git(日本語版)・GitHub Docs	6.2 プロジェクトへの貢献／ラベルを管理する、マイルストーンについて	Issue・ラベルで困ったときに引く
紹介(副読本)	達人プログラマー 第2版	見積もり	持っている人は通読
紹介(副読本)	ソフトウェアアーキテクチャの基礎 第2版	アーキテクチャ特性(非機能要件)の考え方	持っている人は

第10回 設計書作成

tag: 設計-詳細, 設計-構成, http-構成, db
前提タグ: 設計-要件, git-github
所要: 座学15分 / ハンズオン45分

ゴール

- ストーリーの受け入れ条件と要件メモから、API設計書・DB設計書・システム構成図をテンプレートに沿って書ける
- エンドポイントを「URL・メソッド・リクエストとレスポンスの例・ステータスコード・入力チェックとエラーの文言」で決められる
- シーケンス図・ER図・構成図を Mermaid で書ける
- 設計書を隣の人と交換して「質問せずに作れるか」を確かめ、出た質問を設計書に書き足せる

事前準備

- 第9回のハンズオンの最後(手順9のマージ)まで済み、`~/handsoff/workspace/todo-app` の main に `docs/requirements.md`・`docs/stories.md` がある。ストーリーの Issue #8〜#11 が Open
- 遅れた人: session-09/ を examples/README.md の手順でコピーし、push してから始める
- VS Code で Markdown の Mermaid をプレビューできる(第9回と同じ拡張機能)
- テンプレート: API・DB・システム構成図

まとめ

冒頭の座学で使います。ハンズオンでは、第9回の受け入れ条件 (1)〜(15) と要件メモをもとに3つの設計書の自分の版を書き、隣の人と交換して確かめてから、見本と合わせて PR でマージします。

設計書と図

合格基準: **実装者が質問せずに作れるか。**画面設計書は作らない(画面は受け入れ条件と API設計書から組み立てる)。

設計書(置き場所)	主な図	必ず書くこと	元にするもの	参照
API設計書(`docs/design/api.md`)	シーケンス図	エンドポイント、メソッド、リクエスト・レスポンス例、ステータスコード、入力チェックとエラーの文言	ストーリーと受け入れ条件	Web技術の基本 3章・5章
DB設計書(`docs/design/db.md`)	ER図	テーブル定義(カラム・型・NULL可否・キー)、制約、計算で求める値	状態遷移図、CLI の `Task`	SE基礎 8章
システム構成図(`docs/design/architecture.md`)	構成図	構成要素、通信経路とポート、ミドルウェアとバージョン、環境変数	非機能要件・制約	Web技術の基本 5章・7章、SE基礎 9章

エンドポイント(http-構成)

決まり	例	参照
URL は名詞(もの)、操作はメソッド(第3回の REST と同じ)	`GET /` 一覧、`POST /tasks` 追加	Web技術の基本 3章、MDN HTTP リクエストメソッド
フォーム送信の後は 303 で一覧へ	再読み込みで二重に追加しない(POST → リダイレクト → GET)	MDN 303 See Other
絞り込みはクエリ	`GET /?overdue=1` 期限切れだけ	Web技術の基本 3章
返すのは HTML	一覧ページ全体。第12回で HTML 断片を返す URL を足す	Web技術の基本 5章

エンドポイント	内容	成功	失敗
`GET /`	一覧ページ(`?overdue=1` で期限切れだけ)	200	—
`POST /tasks`	追加(フォーム: `title`, `due`)	303 → `/`	400(一覧ページにエラーの文言)
`POST /tasks/{id}/done`	完了にする	303 → `/`	404(無い id)

ステータスコードの意味は第3回の「ステータスコード」の表。このアプリで返す場面は次のとおり。

コード	このアプリで返す場面
200	一覧ページ
303	追加・完了のあと `/` へ
400	タイトルが空・101文字以上、期限が日付でない
404	`POST /tasks/999/done`

テーブル（db）

考え方	ひとことで	参照
1行1件	`tasks` テーブルの1行がタスク1件	SE基礎 8章
型と NULL	`due` は NULL 可（期限なし）。NULL の意味を書く	SE基礎 8章
計算できる値は持たない	期限切れ（`OVERDUE`）は `done` と `due` と今日から計算する	SE基礎 8章
SQLite の型	日付は ISO 8601 の文字列（`2026-11-01`）、真偽値は 0／1	SE基礎 8章
名前をそろえる	テーブル名・カラム名は英字。設計書・DB・コードで同じ名前	SE基礎 8章

カラム	型	NULL	キー・既定値
`id`	INTEGER	不可	PK、AUTOINCREMENT
`title`	TEXT	不可	—
`due`	TEXT（`YYYY-MM-DD`）	可（期限なし）	—
`done`	INTEGER（0／1）	不可	既定 0
`created_at`	TEXT	不可	—

構成（設計-構成）

項目	値	参照
経路	ブラウザ → uvicorn（FastAPI＋Jinja2）→ SQLite ファイル	Web技術の基本 5章
待ち受け	`127.0.0.1:8000`（自分の PC だけ）	Web技術の基本 7章
DB の場所	環境変数 `TODO_DB_PATH`（既定 `var/todo.db`）	SE基礎 9章
バージョン	Python 3.12（`mise.toml`）、ライブラリは `pyproject.toml` と `uv.lock`	—

Mermaid（図ごとの書き方）

図	1行目	例	参照
シーケンス図	`sequenceDiagram`	`ブラウザ-->>アプリ: POST /tasks`、`alt 入力が正しい` 〜 `else 正しくない` 〜 `end`	Mermaid 公式 Sequence diagrams
ER図	`erDiagram`	`tasks { INTEGER id PK "連番" }`。名前は英字、説明は `"期限"`。関係は `users ||--o{ tasks : "持つ"`	Mermaid 公式 Entity Relationship Diagrams
構成図	`flowchart LR`	`subgraph U["uvicorn 127.0.0.1:8000"]`、`B -- "HTTP" ⟶ A`、`D[(SQLite)]`	Mermaid 公式 Flowcharts

設計書の確かめ方

- 「この設計書だけで実装するとしたら」質問したい箇所を全部書く。質問が少ないほど良い設計書
- 指摘は「どこが・なぜ困るか・どうしたらよいか」
- 出た質問には口で答えず、設計書に書き足す（次に読む人は書き手に聞けない）

設計書	観点
共通	どのストーリー（Issue）のためかわかる。受け入れ条件がどれも、どこかの設計で満たせる。同じものを同じ名前で呼ぶ
API	全操作に URL・メソッド・リクエスト例・レスポンス例。状況ごとのステータス。エラーの文言（そのまま）と表示場所
DB	全カラムに型・NULL 可否・キー。NULL の意味。計算で求める値の条件（不等号まで）。同じ情報を２か所に持たない
構成図	構成要素・経路とポート・バージョン。設定を環境変数で切り替える方法

ハンズオン

1ステップ1コマンド。各ステップは「🖋 やること → 📚 参考書・公式資料の節 → 📒 説明 → コマンドや書くもの」。作業場所は `~/handsoff/workspace/todo-app/`。見本は session-10/（examples/session10/）。

1. Issue とブランチ、テンプレートを置く（3分）

> **🖋 Issue を立てる（ブラウザ）**

📚 Pro Git 6.2 プロジェクトへの貢献

📒 第９回の PR が #12 なので、この Issue は #13 になる（GitLab では出た番号を使う）。本文はタスク用テンプレートで書き、ラベル `タスク` を付ける

Issue	タイトル	目的	完了条件
#13	Web 版の API・DB 設計書とシステム構成図を書く	作る前に、どう作るかを決めて残す	`docs/design/` の３つが main にあり、隣の人の質問を反映している

> **🖋 ブランチを作る**

📒 ３つの設計書は、第９回の文書から作る。第11回は、この設計書の URL・ステータスコード・エラーの文言・テーブルでテストと実装を書く

```
cd ~/handsoff/workspace/todo-app
git switch main; git pull; git switch -c issue-13-design
```

🖊 テンプレートを置く

📒 設計書は `docs/design/` にまとめる。テンプレートの `<!-- -->` は書き方の注意で、書き終えたら消してよい

```
mkdir -p docs/design
cp ~/handsoff/docs/templates/api-design.md docs/design/api.md
cp ~/handsoff/docs/templates/db-design.md docs/design/db.md
cp ~/handsoff/docs/templates/architecture.md docs/design/architecture.md
ls docs/design
```

2. API設計書: URL とステータスコード（7分）

🖊 操作を URL とメソッドに当てはめる

📚 Web技術の基本 3章 HTTPでやりとりする仕組み、MDN HTTP リクエストメソッド

📒 URL は「何を」（もの）、メソッドは「どうする」。見るだけなら GET、追加・変更は POST（HTML のフォームは GET と POST しか送れない）。S-01〜S-04 を URL とメソッドに当てはめ、`docs/design/api.md` の「1. エンドポイント一覧」に A-01〜A-03 の3行を書く

🖊 POST の後は 303 で一覧へ戻す

📚 Web技術の基本 3章、MDN 303 See Other

📒 POST にそのまま一覧ページを返すと、再読み込みで同じ POST がもう一度送られ、タスクが二重に追加される。303 で `/` を GET させれば、再読み込みしても GET が繰り返されるだけになる

🖊 リクエストとレスポンスの例を書く

📚 Web技術の基本 3章

📒 「2. 各エンドポイント」の A-02 にパラメータの表と、成功・失敗の例を書く。失敗のレスポンスは HTML の要点（エラーの文言をどこに出すか、入力した値を残すか）だけでよい。ヘッダと本文の読み方は第3回

```
POST /tasks HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/x-www-form-urlencoded

title=%E7%89%9B%E4%B9%B3%E3%82%92%E8%B2%B7%E3%81%86&due=2026-11-01

HTTP/1.1 303 See Other                          ← 成功
Location: /

HTTP/1.1 400 Bad Request                        ← 失敗: 入力が正しくない
Content-Type: text/html; charset=utf-8

<input name="title" value="   ">                ← 入力した値を残す
<p class="error" role="alert">タイトルを入力してください</p>  ← エラー表示欄
<ul id="task-list">…</ul>                       ← 一覧もそのまま出す
```

🖊 ステータスコードを選ぶ

📚 MDN HTTP レスポンスステータスコード

📒 状況ごとに1つ選び、「3. ステータスコード」の表を埋める。入力が悪い(400)と対象が無い(404)を分ける

> 座学: 200 で一覧ページを返さずに 303 にする理由と、400 と 404 を分ける理由(→ まとめ「エンドポイント」)

3. API設計書: 入力チェックとシーケンス図(6分)

🖊 受け入れ条件から入力チェックの表を作る

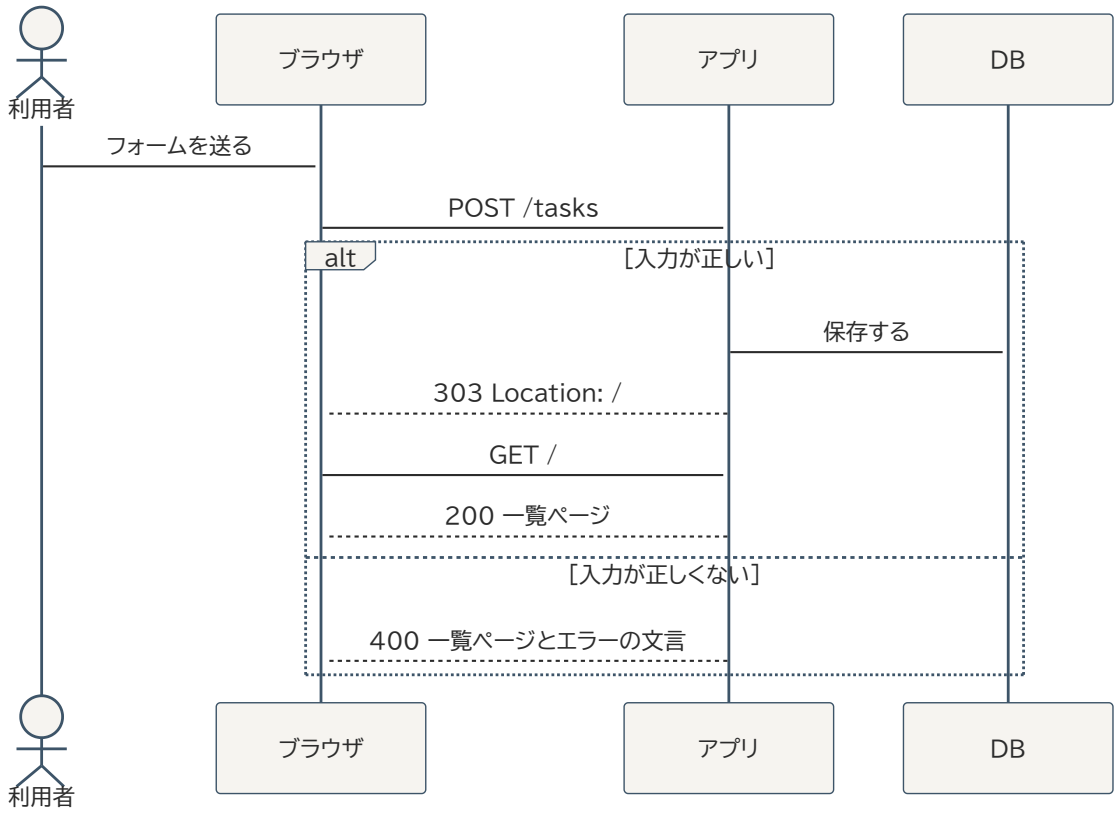
📚 SE基礎 4章 モデル

📒 `docs/stories.md` のうまくいかない場合を1行ずつ「4. 入力チェックとエラー表示」の表に写す。文言は一字も変えない(第11回のテストで同じ文言を確かめる)。表の下には、表示する場所と入力した値を残すかを書く

🖊 追加のシーケンス図を書く

📚 Mermaid 公式 Sequence diagrams

📒「5. シーケンス図」のテンプレート(下)に矢印を２本足す。アプリが入力をチェックする矢印(`アプリ→→アプリ: `)と、入力が正しくないときに一覧を読む `DB` への矢印(400 でも一覧を出すため)。図の上に、この図で何を伝えるかを１文で書く



- 確認: 受け入れ条件 (1)〜(6) と (14) の各行が、API設計書のどこかに対応する

> 座学: 画面設計書を作らない代わりに、API設計書に書くもの(返す HTML の要点・エラー表示欄・文言)(→ まとめ「設計書と図」)

4. DB設計書: テーブルと計算で求める値(7分)

🖊 Task をテーブルに写す

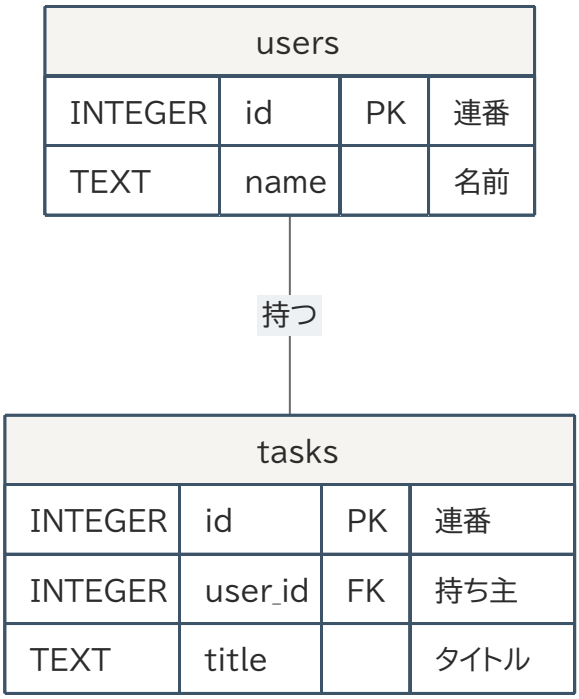
📚 SE基礎 8章 データの取り扱い

📒 CLI の `Task` (`models.py`)の項目を、`tasks` テーブルのカラムにする。SQLite には日付と真偽値の型がないので、日付は ISO 8601 の文字列、真偽値は 0／1 で持つ。ISO 8601 の日付は、文字列の大小と日付の前後が一致する

🖊 ER図とテーブル定義を書く

📚 Mermaid 公式 Entity Relationship Diagrams

📒 `docs/design/db.md` のER図に `done` と `created_at` を足し、「2. テーブル定義」と「3. 制約」の表を埋める。NULL を許すカラムには NULL の意味（`due` なら「期限なし」）を書く。テーブルが1つなら関係の線はない。線は、ログイン（対象外）を入れて `users` を足すときに下のように引く



🖊 状態は持たずに計算で求める

📚 SE基礎 8章

📒 要件メモの状態遷移図の3つの状態は、`done`・`due`・今日から決まる。期限切れを保存すると、日付が変わるたびに書き換えが要る。「4. DB に持たず計算で求める値」に、状態ごとの条件を不等号まで書く

- 確認: 「期限の当日は期限切れか」「期限なしのタスクは期限切れになるか」に、DB設計書だけで答えられる

🖊 並び順を書く

📒 「5. 並び順」に一覧の順を書く。期限なし（NULL）の位置と、期限が同じときの順まで決める（CLI の `sort_by_due` と同じ）

> 座学: 計算できる値を持つと、`done` や `due` と食い違う（→ まとめ「テーブル」）

5. システム構成図（5 分）

🖊 構成要素と通信経路を図にする

📚 Web技術の基本 5章 Webアプリケーションの基本、7章 構築と運用

📒 Web アプリは、画面（ブラウザ）・処理（アプリ）・データ（DB）の 3 層に分かれる。`docs/design/architecture.md` の構成図に、uvicorn の枠と待ち受けるアドレス・ポート、Jinja2 のテンプレート、SQLite のファイル名を足す。矢印には通信の種類（HTTP・ファイル）を書く

🖊 バージョンを書く

📚 SE基礎 9章 ソフトウェアアーキテクチャ

📒 「3. ミドルウェアとバージョン」に、何をどのバージョンで使い、どこで固定するかを書く。Python と uv は `mise.toml`、FastAPI などのライブラリは第11回で `pyproject.toml` と `uv.lock` に入る

```
grep -A3 '^\[tools\]' mise.toml
```

🖊 設定を環境変数で書く

📒 DB の場所は環境変数 `TODO_DB_PATH` で変える。開発では `mise.toml` の `[env]`、テストでは一時ディレクトリ、本番ではサーバの設定で渡す。「4. 設定（環境変数）」に名前・意味・既定値を書く

🖊 自分の版をコミットする

📒 交換の前に一度コミットして、自分の版を残す。埋めきれていない節があってもよい

```
git add docs/design; git commit -m "API・DB設計書とシステム構成図の自分の版を書く"
```

6. 交換して「質問せずに作れるか」を確かめる（10 分）

🖊 席を替わって読む

📚 SE基礎 4章

📒 隣の人の席に移り、その人の `docs/design/` を VS Code のプレビューで読む。書き手は説明しない。読み手は、この設計書だけで第11回の実装をすると考えて読む

🖊 設計書だけで答える

📒 下の問いに設計書だけで答える。答えが見つからない問いと、読んでいて迷った箇所が質問になる

#	実装者の問い	載っているはずの場所
1	空白だけのタイトルで追加したら、ステータスと文言は	API設計書 4章
2	入力エラーのとき、入れた値はフォームに残るか	API設計書 A-02
3	無い番号のタスクを完了にしたら	API設計書 3章・4章
4	期限の当日のタスクは期限切れか	DB設計書 4章
5	期限なしのタスクは一覧のどこに出るか	DB設計書 5章
6	DB のファイルはどこにでき、どう変えるか	システム構成図 4章

🖊 質問を書き込む

📒 相手の `docs/design/api.md` の「6. 設計レビューで出た質問」の表に、質問を1行ずつ書く。「答え」と「反映した場所」は空けておく。2件以上書いたら席に戻る

🖊 質問を設計書に反映する

📒 質問ごとに、本文に書き足すか、書かないかを決める。書き足したら「反映した場所」に節を、書かないなら「答え」に理由を書く

```
git diff --stat; git commit -am "設計レビューの質問を設計書に反映する"
```

> 座学: 伝わらなかった原因の多くは「書いたつもり」(→ まとめ「設計書の確かめ方」)

7. 見本と合わせてコミットする(4分)

🖊 見本に置き換えて違いを見る

📒 見本の設計書は、第9回の見本(依頼人と合わせた版)の受け入れ条件どおりに書いてある。第11回は見本のURL・エラーの文言・テーブルでテストを書くので、見本に置き換える。`git diff` で自分の版との違いを読む。自分の版と質問の反映は、履歴に残る

```
cp ~/handsoff/examples/todo-app-snapshots/session-10/docs/design/*.md docs/design/; git diff --stat; git diff docs/design/api.md
```

> 🖊 README とストーリーの控えも合わせてコミットする

📒 見本の README はドキュメントの表に設計書の３行を足したもの、`docs/stories.md` は S-01 の関連に設計書へのリンクを足したもの。第９回で Issue の番号を直した人は、コピーした後に `git diff docs/stories.md` で番号の列を戻す

```
cp ~/handsoff/examples/todo-app-snapshots/session-10/README.md .; cp
~/handsoff/examples/todo-app-snapshots/session-10/docs/stories.md docs/; git add
README.md docs; git commit -m "見本と合わせた設計書にする"
```

8. PR を出してマージする（３分）

> 🖊 push して PR を作る

📚 Pro Git 6.2 プロジェクトへの貢献

📒 本文はテンプレートで書き、目的に `Closes #13`、変更点に「交換で出た質問と、どう直したか」を書く。Files changed で `docs/design/api.md` を開き、表示を切り替えて図を見る。`[確認]` の行コメントを１件以上残す

```
git push -u origin issue-13-design
```

> 🖊 CI が緑になってからマージする

📒 コードは変えていないが、CI が緑になるのを待ってからマージする。ストーリーの Issue #8〜#11 は、第11回の PR #16 で閉じる

```
git switch main; git pull; git branch -d issue-13-design; ls docs/design
```

- 確認: Issue #13 が Closed、#8〜#11 は Open のまま。main に `docs/design/api.md`・`db.md`・`architecture.md` がある

読む章

区分	資料	範囲	読み方
参考書	ソフトウェアエンジニアリングの基礎（SE基礎）	４章 モデル、８章 データの取り扱い、９章 ソフトウェアアーキテクチャ	通読（８章は第11回でも使う）

区分	資料	範囲	読み方
参考書	イラスト図解式 この一冊で全部わかるWeb技術の基本 第2版(Web技術の基本)	3章 HTTPでやりとりする仕組み、5章 Webアプリケーションの基本、7章 構築と運用	該当章(3章は第3回で読んだ)
無料	MDN Web Docs	HTTP リクエストメソッド、HTTP レスポンスステータスコード(303 See Other)	困ったときに引く
無料	Mermaid 公式	Syntax Reference(Sequence diagrams、Entity Relationship Diagrams、Flowcharts)	図を書くときに引く
紹介(副読本)	Webを支える技術	第2部 URI、第3部 HTTP、第5部 Web サービスの設計	持っている人は(API 設計を深める)
紹介(副読本)	達人に学ぶDB設計徹底指南書 第2版	3章 正規化、4章 ER図	持っている人は(テーブルが増えたとき)

第11回 TDD による Web バックエンド

tag: web-サーバ, db, test-自動
前提タグ: 設計-詳細, python-oop, test-自動, test-CI
所要: 座学15分 / ハンズオン45分

ゴール

- 第7回の `TaskStore`（Protocol）に SQLite 版の `SqliteStore` を足し、同じテストで確かめられる
- 受け入れ条件と API設計書から TestClient のテストを先に書き、FastAPI＋Jinja2 の画面をレッド → グリーンの順に作れる
- 依存性の注入（`Depends` と `dependency_overrides`）で、テストのときだけ保存先と「今日」を差し替えられる
- テストと実装のコミットを分けた PR を CI が緑の状態でマージし、`Closes` でストーリーの Issue #8〜#11 を閉じられる

事前準備

- 第10回のハンズオンの最後（手順8のマージ）まで済み、`~/handsoff/workspace/todo-app` の main に `docs/design/` がある。ストーリーの Issue #8〜#11 が Open
- 遅れた人: session-10/ を examples/README.md の手順でコピーし、push してから始める
- 社内ネットワークでは PyPI に届く（手順1の `uv add`。open-questions.md の Q5）。`NO_PROXY`・`no_proxy` に `localhost,127.0.0.1`（第2回）
- 素材: assets/（`test_web.py`・`test_web_rest.py`・`web.py`）

まとめ

冒頭の座学で使います。ハンズオンでは、保存先 → 追加 → 入力エラー → 完了の順にテストを先に書いて Web 版を作り、残りの受け入れ条件のテストは見本からコピーして読みます。

層と道具（web-サーバ）

層	このアプリでは	道具	参照
サーバ	`mise run dev` で `127.0.0.1:8000` を待ち受ける	uvicorn	Web技術の基本 5章
URL と処理	`web.py`。`@app.post("/tasks")` で URL と関数を結ぶ。フォームの値は `Form()`、`task_id: int` は型ヒントで変換・検証（形が違えば 422）	FastAPI、python-multipart	FastAPI 公式 フォームデータ

層	このアプリでは	道具	参照
画面	`templates/index.html`。`{{ }}` は自動でエスケープ（`<` → `<`）	Jinja2	FastAPI 公式 Templates
入力チェック	第７回の `validate_title`・`parse_due` を Web でも呼ぶ。決まりと文言は１か所	—	API設計書 ４章
保存先	`sqlite_store.py` の `SqliteStore`。SQL はここだけ、値は `?` で渡す	sqlite3（標準ライブラリ）	Python 公式 sqlite3、SE基礎 ８章
テスト	`TestClient(app)` でサーバを立てずに HTTP を送る	TestClient、httpx2	FastAPI 公式 テスト

依存性の注入

依存	本番（`web.py`）	テスト（`tests/test_web.py`）	参照
保存先 `StoreDep`	`get_store`：`TODO_DB_PATH`（既定 `var/todo.db`）を開き、`yield` で渡し、返したら閉じる	`tmp_path / "test.db"`（テストごとに空）	FastAPI 公式 yield を持つ依存関係
今日 `TodayDep`	`get_today`：`date.today()`	`date(2026, 11, 1)` に固定	FastAPI 公式 依存関係
差し替え方	—	`app.dependency_overrides[get_store] = ...`、終わったら `.clear()`	FastAPI 公式 Testing Dependencies with Overrides

TDD とテストの元

決まり	やり方	参照
テストの元は受け入れ条件と API設計書	テスト名の頭に条件の番号（`test_3_…`）。ステータスコードと文言は設計書どおり	SE基礎 ５章
先に失敗を見る	失敗の理由（`ModuleNotFoundError`・404・422）が「まだ作っていないもの」を指しているか読む	SE基礎 ５章
グリーンは最小	次のテストで本物に近づける（第８回の三角測量）	SE基礎 ５章
コピーしたテストも確かめる	最初から通るテストは、わざと壊して落ちるのを見る	SE基礎 ５章
コミットを分ける	`レッド:` `グリーン:` `テスト:`。テストと実装は別々に `git add`	研修の規約

決まり	やり方	参照
CI が緑になってからマージ	PR に `Closes #番号` を1行ずつ書き、マージで Issue を閉じる	SE基礎 10章、GitHub Docs

アプリの形（API・DB 設計書と同じ）

エンドポイント	成功	失敗	受け入れ条件
`GET /`（`?overdue=1` で期限切れだけ）	200 一覧ページ	—	(7)〜(11)・(15)
`POST /tasks`（`title`・`due`）	303 → `/`	400 一覧ページに文言、入力した値を残す	(1)〜(6)
`POST /tasks/{id}/done`	303 → `/`	404 一覧ページに「タスクが見つかりません」	(12)〜(14)

- エラーの文言は `<p class="error" role="alert">`、一覧は `<ul id="task-list">`（第12回の JavaScript がこの id を使う）
- テーブル `tasks`（`id`・`title`・`due`・`done`・`created_at`）。日付は ISO 8601 の文字列、真偽値は 0／1（DB設計書）

ハンズオン

1ステップ1コマンド。各ステップは「🖋 やること → 📚 参考書・公式資料の節 → 📒 説明 → コマンドやコード」。作業場所は `~/handsoff/workspace/todo-app/`。見本は session-11/（examples/session11/）。

1. Issue とブランチ、依存パッケージ（4分）

> 🖋 **Issue を立てる（ブラウザ）**

📚 Pro Git 6.2 プロジェクトへの貢献

📒 第10回の PR が #14 なので、この Issue は #15 になる（GitLab では出た番号を使う）。本文はタスク用テンプレートで書き、ラベル `タスク` とマイルストーン `Web 版` を付ける。ストーリーの Issue #8〜#11 は、この作業の PR で閉じる

Issue	タイトル	目的	完了条件
#15	Web 版を TDD で作る（S-01〜S-04）	設計書どおりの Web 版を、受け入れ条件のテストを先に書いて作る	受け入れ条件 (1)〜(15) のテストが手元と CI で通り、#8〜#11 が閉じている

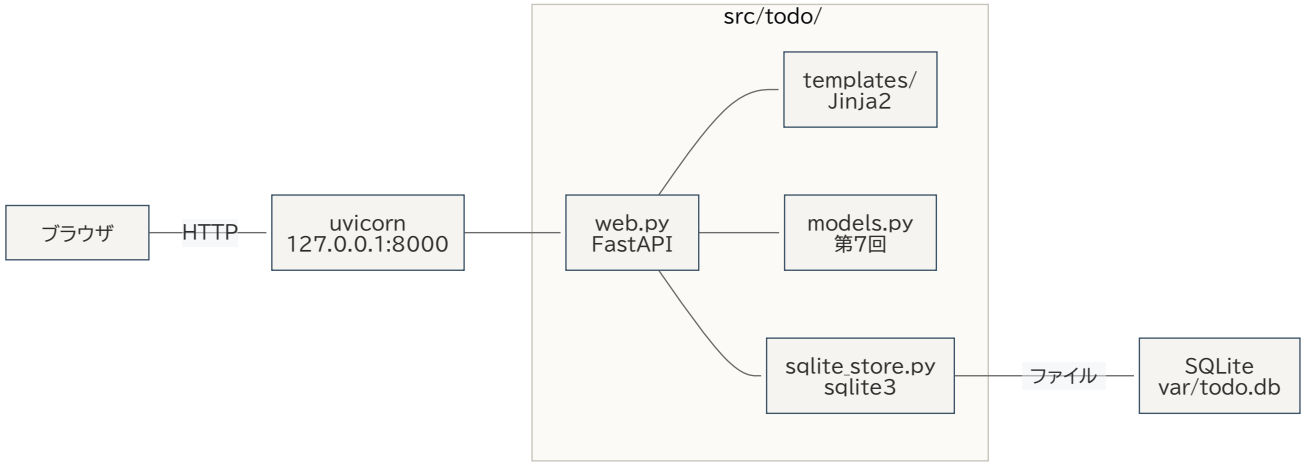
🖊 ブランチを作る

```
cd ~/handsoff/workspace/todo-app
git switch main; git pull; git switch -c issue-15-web
```

🖊 Web 版の依存パッケージを入れる

📚 Web技術の基本 5章 Webアプリケーションの基本、uv 公式 Managing dependencies

📒 第10回のシステム構成図の3層に、今日入れるパッケージを当てはめる。CLI(`cli.py`)はそのまま残り、Web版と同じ `models.py` を使う




```
uv add fastapi jinja2 uvicorn python-multipart
```

🖊 テスト用の HTTP クライアントを入れる

📚 FastAPI 公式 テスト

📒 TestClient は、サーバを立てずに `app` へ HTTP のリクエストを送る。中で使う HTTP クライアントが httpx2。テストでだけ使うので `--dev`

```
uv add --dev httpx2
```

🖊 .gitignore と mise.toml を見本にそろえる

📒 見本の `.gitignore` は開発用 DB の `var/` と `.env` を足したもの。`mise.toml` は `[env]` に `TODO_DB_PATH`、タスクに `dev`(開発用サーバ)と `db-reset`(開発用 DB を消す)を足したもの

```
cp ~/handsoff/examples/todo-app-snapshots/session-11/{.gitignore,mise.toml} .; git diff
.gitignore mise.toml
```

🖊 コミットする

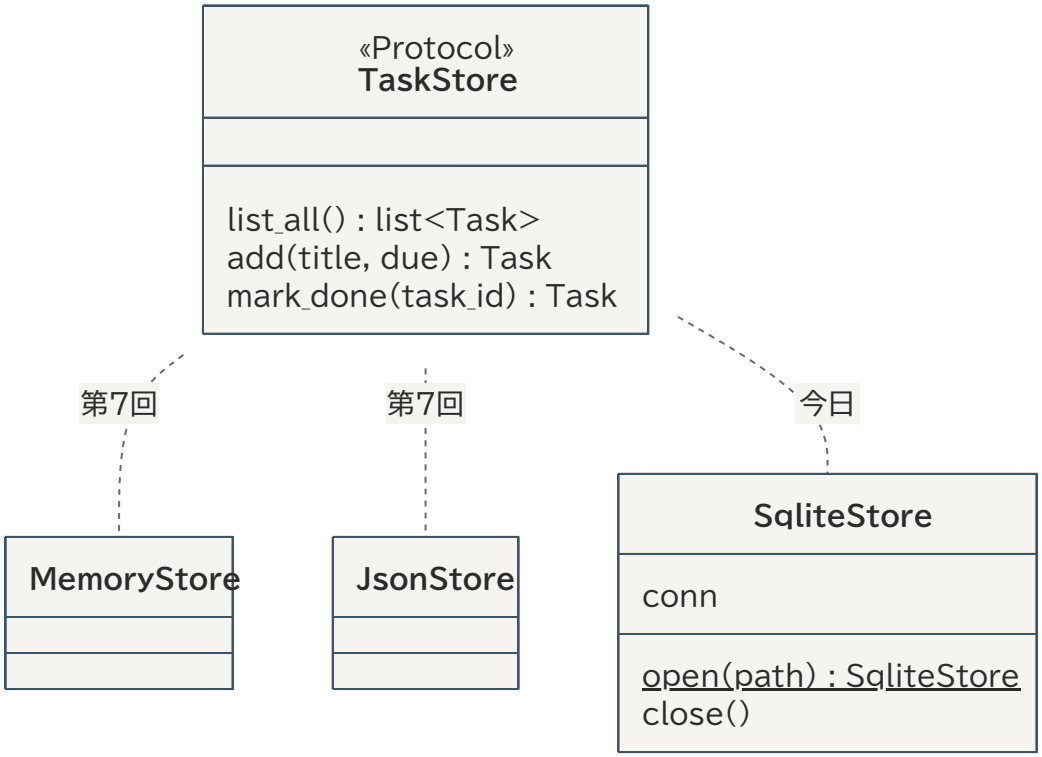
```
mise run check; git add .gitignore mise.toml pyproject.toml uv.lock; git commit -m "Web
版の依存パッケージと開発用のタスクを足す"
```

2. SqliteStore: Protocol に SQLite 版を足す（6 分）

🖊 レッド: 保存先のテストに sqlite を足す

📚 SE 基礎 5 章 自動テスト、pytest 公式 How to use fixtures

📒 見本の `test_store.py` は、第 8 回の fixture の `params` に `"sqlite"` を足し、開き直しても読めるテストを 1 つ足したもの。`SqliteStore` はまだないので `ModuleNotFoundError: No module named 'todo.sqlite_store'` で落ちる



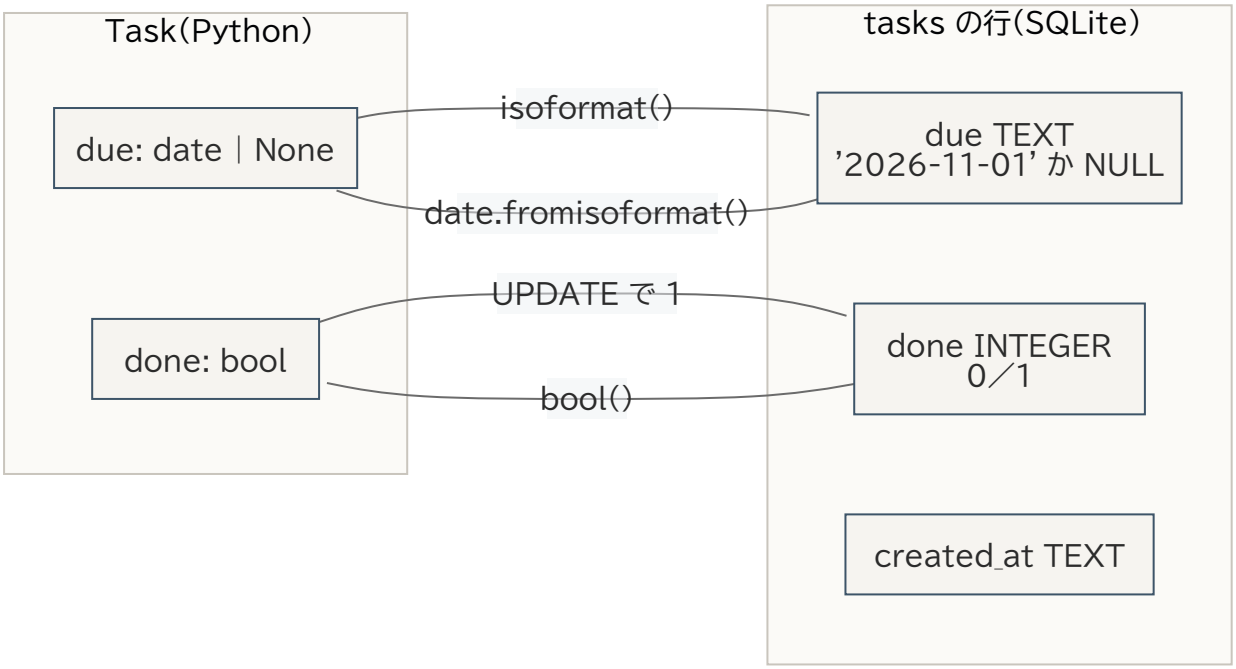

```
cp ~/handsoff/examples/todo-app-snapshots/session-11/tests/test_store.py tests/; git diff
tests/test_store.py; mise run test
```

```
git add tests/test_store.py; git commit -m "レッド: SqliteStore でも保存先のテストを動かす"
```

🖊 グリーン: SqliteStore をコピーして読む

📚 Python 公式 sqlite3、SE基礎 8章 データの取り扱い

📒 `open` はディレクトリとテーブルがなければ作る(`CREATE TABLE IF NOT EXISTS`)。`with self.conn:` はブロックを抜けるときにコミットし、例外ならロールバックする。値は SQL の文字列に埋め込まず `?` で渡す(SQL インジェクションを防ぐ)




```
cp ~/handsoff/examples/todo-app-snapshots/session-11/src/todo/sqlite_store.py src/todo/;
less src/todo/sqlite_store.py
```

🖊 3つの保存先で同じテストが通るのを見る

📒 同じテストが `[memory]`・`[json]`・`[sqlite]` で通れば、`SqliteStore` も `TaskStore` の約束を守っている

```
uv run pytest -v tests/test_store.py   # [sqlite] が並び、15 passed（全体では 33 件）
```

🖊 DB の中を見る

📒 `Task` は1行として保存される。期限は文字列、完了は 0、`created_at` はアプリが入れる。`var/` は `.gitignore` にあるのでコミットされない

```
uv run python -c 'from datetime import date; from pathlib import Path; from
todo.sqlite_store import SqliteStore; s = SqliteStore.open(Path("var/todo.db"));
print(s.add("牛乳を買う", date(2026, 11, 1))); print([tuple(r) for r in
s.conn.execute("SELECT * FROM tasks")])'
# Task(id=1, title='牛乳を買う', due=datetime.date(2026, 11, 1), done=False)
# [(1, '牛乳を買う', '2026-11-01', 0, '2026-11-01T10:00:00')]
```

> 🖊 グリーンをコミットする

```
mise run check; git add src/todo/sqlite_store.py; git commit -m "グリーン: SQLite に保存する
SqliteStore を足す"
```

> 座学: `web.py` は `TaskStore` だけを知っていれば、保存先が SQLite でもメモリでも同じに書ける（→ まとめ「層と道具」）

3. レッド: 追加のテストを書く（6 分）

> 🖊 テストの fixture と補助の関数をコピーして読む

📚 FastAPI 公式 テスト、Testing Dependencies with Overrides

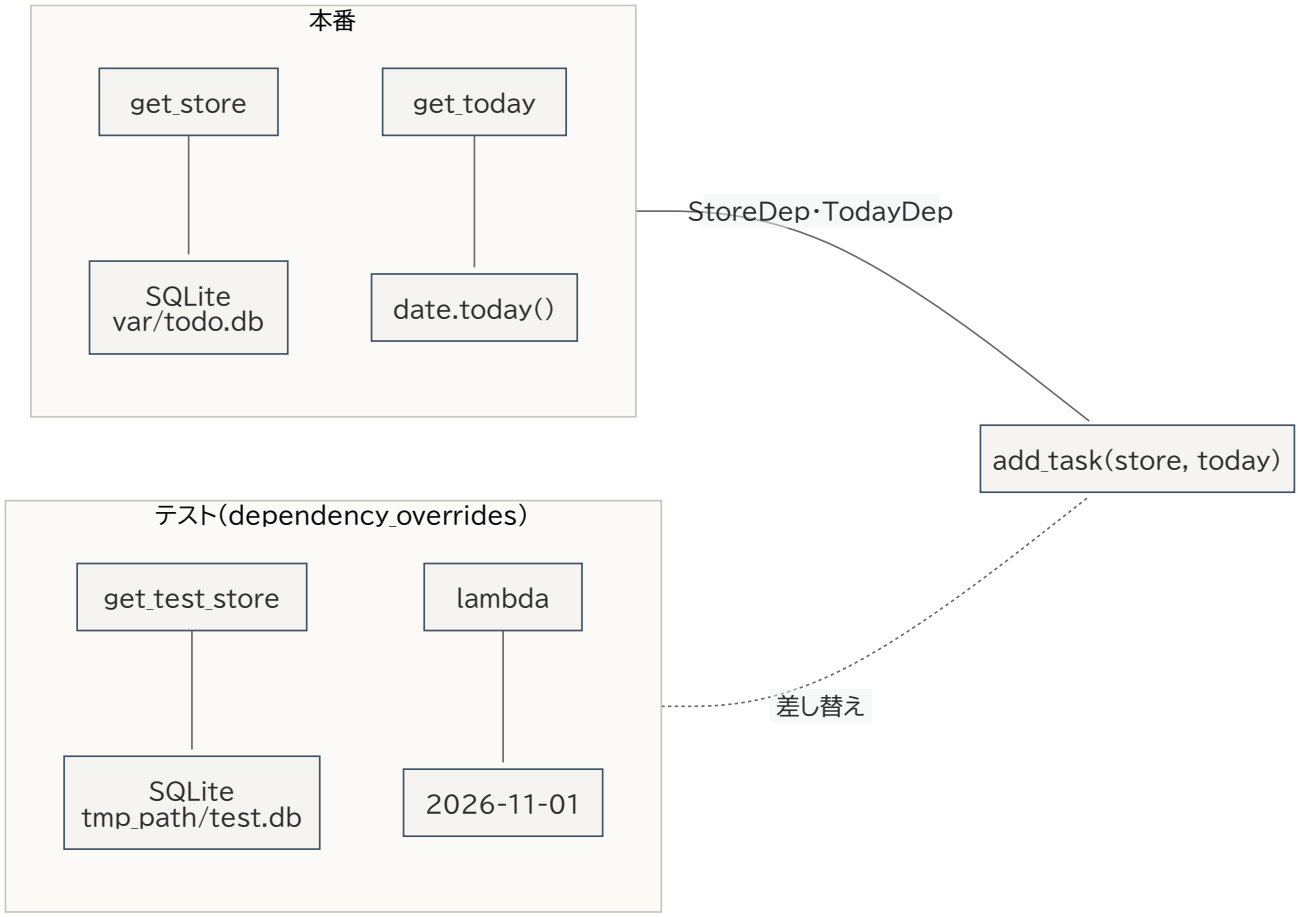
📒 fixture `client` は、`get_store` と `get_today` をテスト用に差し替えてから `TestClient` を渡す。`add` と `titles` はテストを短く書くための関数。今日は保存先 → 追加 → 入力エラー → 完了の順にテストを先に書く

```
cp ~/handsoff/docs/session11/assets/test_web.py tests/; less tests/test_web.py
```

> 🖊 依存性の注入で差し替える仕組みを知る

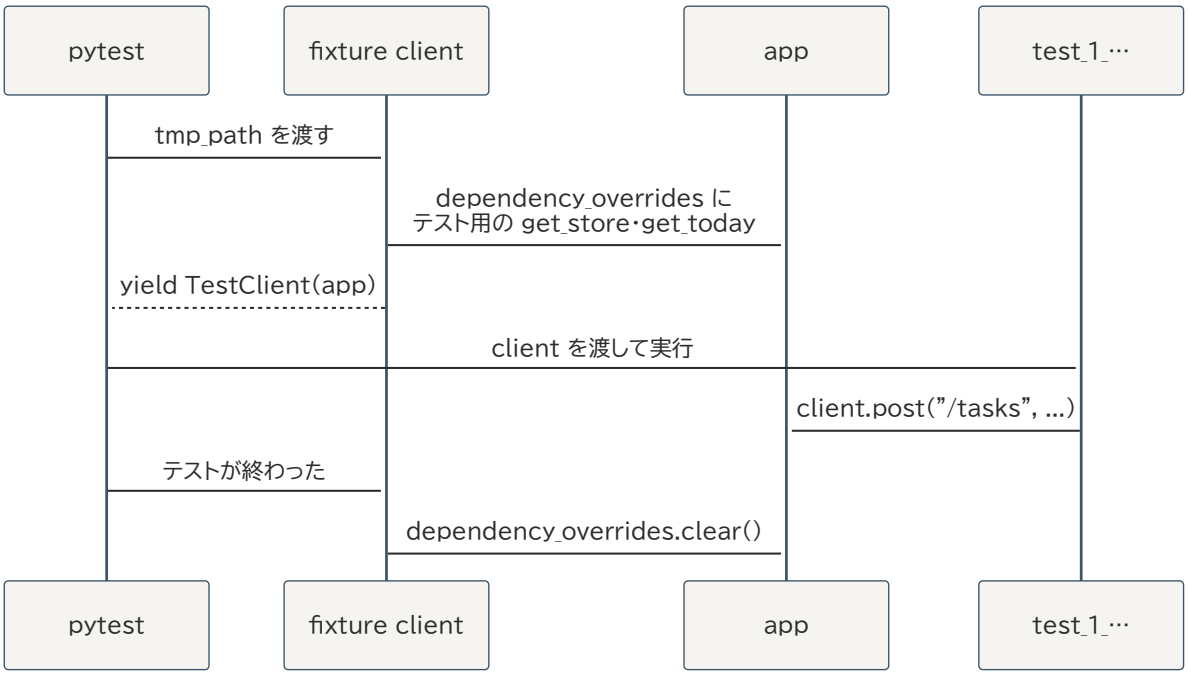
📚 FastAPI 公式 依存関係、yield を持つ依存関係

📒 ルートの関数は保存先と今日を自分で作らず、引数（`StoreDep`・`TodayDep`）で受け取る。本番では FastAPI が `get_store`・`get_today` を呼ぶ。テストでは `dependency_overrides` に書いた関数を代わりに呼ぶ



🖊 fixture の流れを読む

📒 `yield` の前が準備、後が片付け。テストごとに `tmp_path` が新しくなるので、DB は毎回空から始まり、開発用の `var/todo.db` には触れない



🖊 追加のテストを書く

📒 受け入れ条件（1）をテストにする。「手順 3: ここに …」の行を置き換える。TestClient は 303 のリダイレクトを既定で追うので、`response` は `GET /` の結果になる

```
def test_1_追加すると一覧へリダイレクトし未着手で出る(client: TestClient) → None:
    response = client.post("/tasks", data={"title": "牛乳を買う", "due": "2026-11-01"})

    assert response.status_code == 200  # リダイレクトの先（GET /）
    assert response.url.path == "/"
    assert titles(response.text) == ["牛乳を買う"]
    assert '<span class="status">未着手</span>' in response.text
```

🖊 失敗を見てコミットする

📒 `ModuleNotFoundError: No module named 'todo.web'` で落ちる。まだ `web.py` がないから落ちた、と言えればよい

```
mise run test; git add tests/test_web.py; git commit -m "レッド: タスクを追加すると一覧に出るテスト"
```

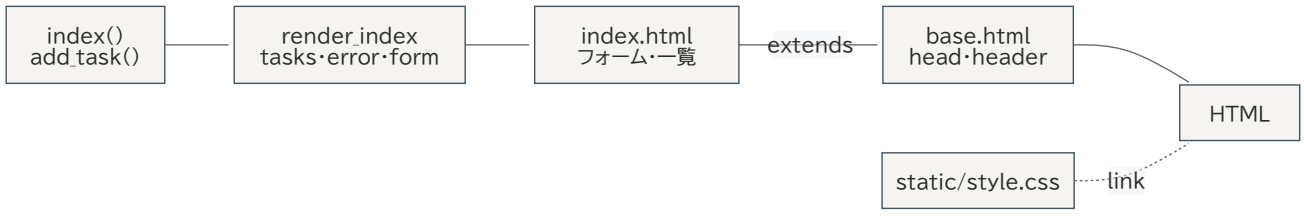
> 座学: 失敗の表示を全員で読む。何がないから落ちたか（→ まとめ「TDD とテストの元」）

4. グリーン: web.py とテンプレート（8 分）

🖊 素材の web.py とテンプレートをコピーする

📚 FastAPI 公式 Templates、Web 技術の基本 5 章

📒 素材の `web.py` には、依存関係（`get_store`・`get_today`・`StoreDep`・`TodayDep`）、一覧を描く `render_index`、`GET /` の `index` まである。テンプレートは見本をそのまま使う。`index.html` は `base.html` を継承し、`{% block content %}` だけを書く



```
cp ~/handsoff/docs/session11/assets/web.py src/todo/; cp -r ~/handsoff/examples/todo-app-snapshots/session-11/src/todo/{templates,static} src/todo/; ls src/todo
```

🖊 まだ落ちるのを見る

📒 `assert 404 == 200`。`POST /tasks` の URL がまだないので 404 になる

```
mise run test
```

🖊 add_task を書く

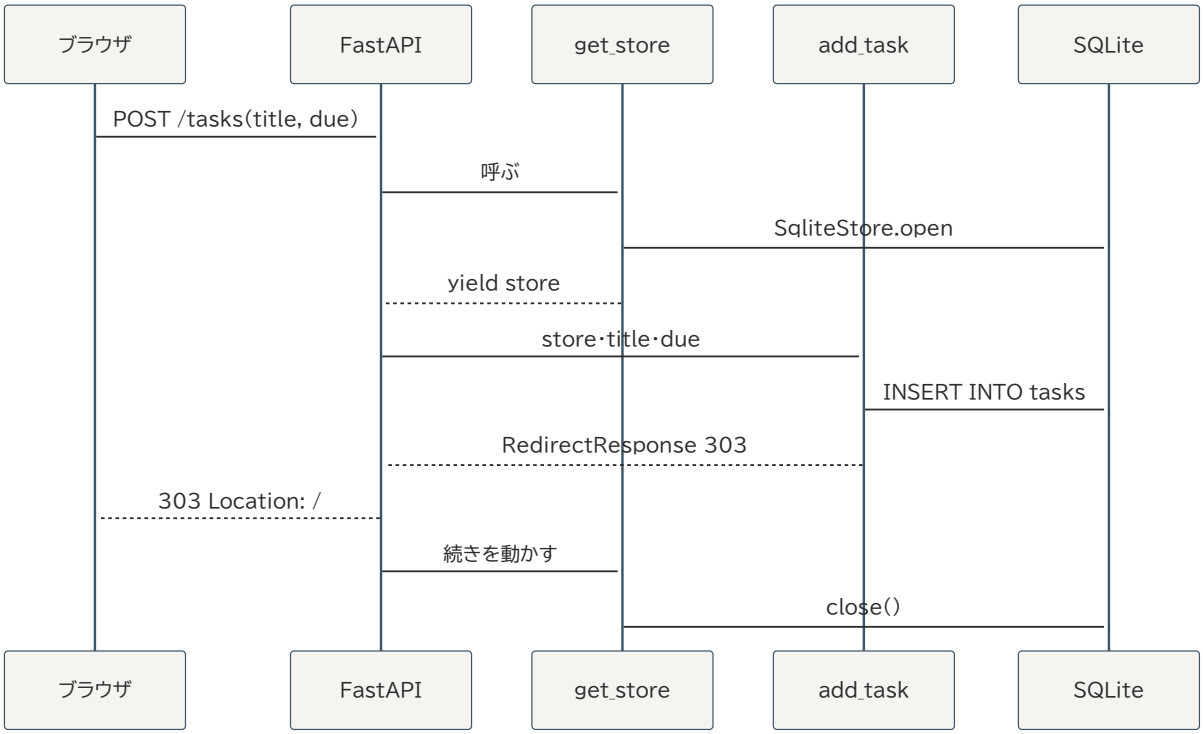
📚 FastAPI 公式 フォームデータ

📒「手順 4: ここに …」の行を置き換える。テストを通す最小の形にし、入力エラーのことはまだ考えない。成功したら 303 で `/` へ戻す（第10回の POST → 303 → GET）

```
@app.post("/tasks")
def add_task(
    store: StoreDep, title: Annotated[str, Form()], due: Annotated[str, Form()] = ""
) → RedirectResponse:
    """A-02 追加。入力のチェックは CLI と同じ関数（models.py）を使う。"""
    store.add(validate_title(title), parse_due(due))
    # POST → リダイレクト → GET にして、再読み込みで二重に追加されないようにする
    return RedirectResponse("/", status_code=303)
```

🖊 リクエストの流れを追う

📒 FastAPI は、リクエストごとに `get_store` を呼んで DB を開き、ルートの関数に渡す。レスポンスを返した後、`yield` の後ろ（`finally` の `close()`）が動いて閉じる



🖊 グリーンを確かめてコミットする

```
mise run check; git add src; git commit -m "グリーン: タスクの追加と一覧のページを作る"   # 34 passed
```

5. 入力エラーを 400 にする（5 分）

🖊 レッド: 空のタイトルのテストを書く

📚 SE基礎 5章、API設計書 4章

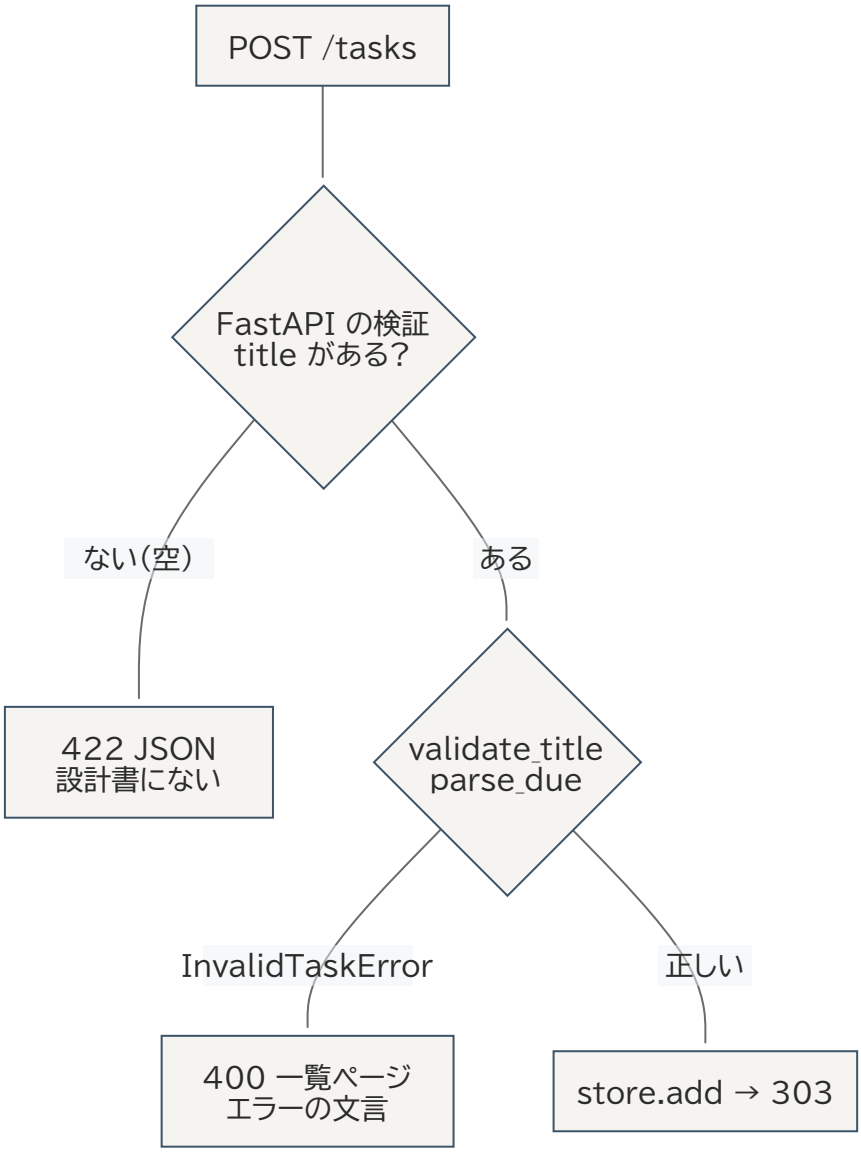
📒 受け入れ条件 (3) をテストにする。「手順5: ここに …」の行を置き換える。`""` と `" "`（空白だけ）の2組で動く

```
@pytest.mark.parametrize("title", ["", "   "])
def test_3_タイトルが空なら追加せずエラーを出す(client: TestClient, title: str) → None:
    response = client.post("/tasks", data={"title": title, "due": ""})

    assert response.status_code == 400
    assert '<p class="error" role="alert">タイトルを入力してください</p>' in response.text
    assert "タスクはありません" in response.text
```

🖊 2つの落ち方を読む

📒 `""` は `assert 422 == 400`。必須の `Form()` に空の値が来たので、FastAPI が関数を呼ぶ前に 422（JSON）を返した。`" "` は `InvalidTaskError: タイトルを入力してください`。`validate_title` の例外をどこも受け止めていない




```
mise run test; git add tests/test_web.py; git commit -m "レッド: タイトルが空なら 400 でエラーの文言を出すテスト"
```

🖊 グリーン: 例外を 400 の一覧ページにする

📒 `title` にも既定値 `""` を付け、空の値も関数まで届ける。`InvalidTaskError` を受け止めて、`render_index` で 400 の一覧ページを返す。文言は `str(e)`（CLI と同じ文言）、入力した値は `form` でフォームに残す。`from todo.models import` に `InvalidTaskError` を足し、`add_task` を下に置き換える

```
@app.post("/tasks", response_model=None)
def add_task(
    request: Request,
    store: StoreDep,
    today: TodayDep,
    title: Annotated[str, Form()] = "",
    due: Annotated[str, Form()] = "",
) → HTMLResponse | RedirectResponse:
    """A-02 追加。入力のチェックは CLI と同じ関数（models.py）を使う。"""
    try:
        store.add(validate_title(title), parse_due(due))
    except InvalidTaskError as e:
        form = {"title": title, "due": due}
        return render_index(request, store, today, error=str(e), form=form,
status_code=400)
    # POST → リダイレクト → GET にして、再読み込みで二重に追加されないようにする
    return RedirectResponse("/", status_code=303)
```

```
mise run check; git add src; git commit -m "グリーン: 入力エラーは 400 で一覧とエラーの文言を返す"   # 36 passed
```

> 座学: 型の変換は FastAPI に任せ、決まり（空・100文字・日付の形）は `models.py` の関数で確かめる。CLI と Web で文言が1か所にそろう（→ まとめ「層と道具」）

6. 完了と 404（5分）

> 🖋 **レッド: 完了と、ないタスクのテストを書く**

📚 FastAPI 公式 テスト、API設計書 A-03

📒 受け入れ条件 (13)・(14) をテストにする。「手順6: ここに …」の行を置き換える。303 そのものを確かめたいので `follow_redirects=False` を付ける

```
def test_13_完了にすると一覧へリダイレクトし期限切れだったものも完了と出る(
    client: TestClient,
) → None:
    add(client, "牛乳を買う", "2026-10-31")

    response = client.post("/tasks/1/done", follow_redirects=False)

    assert response.status_code == 303
    assert response.headers["location"] == "/"
    html = client.get("/").text
    assert '<span class="status">完了</span>' in html
    assert "期限切れ</span>" not in html


def test_14_ないタスクを完了にすると404(client: TestClient) → None:
    response = client.post("/tasks/99/done")

    assert response.status_code == 404
    assert '<p class="error" role="alert">タスクが見つかりません</p>' in response.text
```

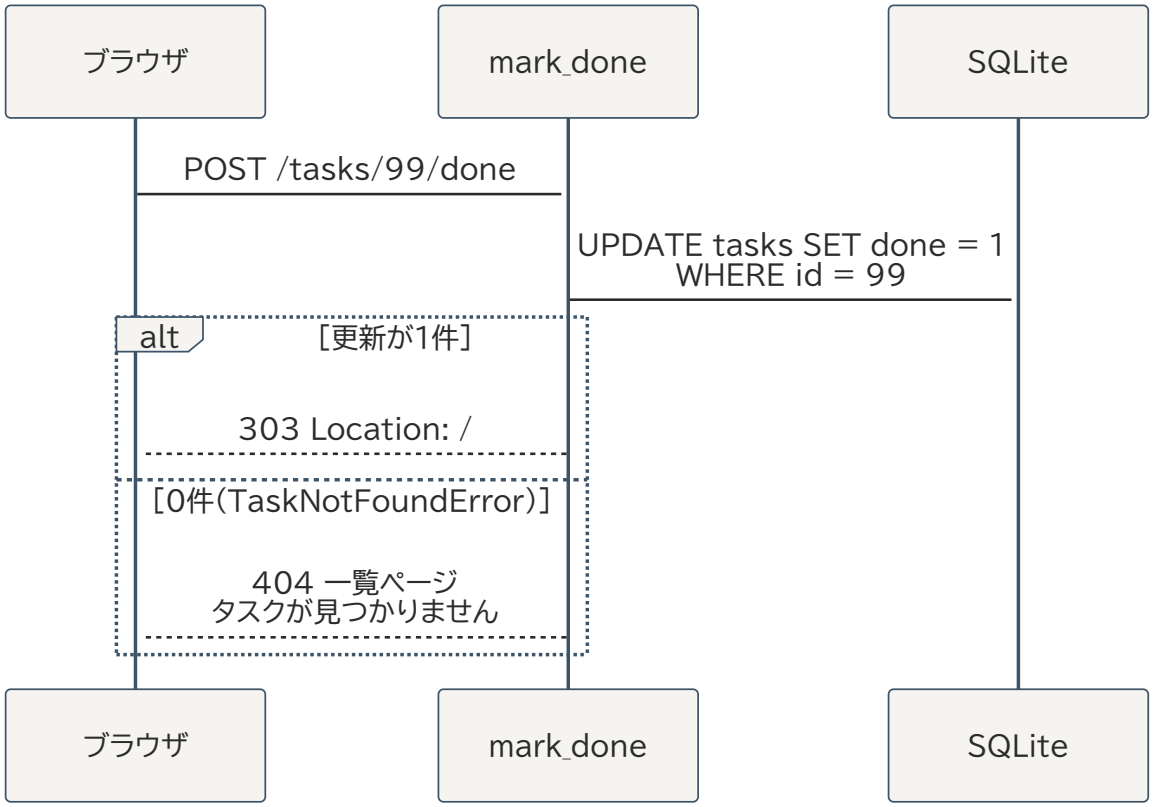
🖊 失敗を見てコミットする

📒 URL がまだないので、(13) は `assert 404 == 303` で落ちる。(14) はステータスの 404 は合っているが、本文が FastAPI の `{"detail":"Not Found"}` なので落ちる。文言まで確かめるから、URL がないだけの 404 と区別できる

```
mise run test; git add tests/test_web.py; git commit -m "レッド: 完了にすると 303、ないタスクは 404 のテスト"
```

🖊 グリーン: mark_done を書く

📒「手順 6: ここに …」の行を置き換え、`from todo.store import` に `TaskNotFoundError` を足す。`SqliteStore.mark_done` は更新した行が 0 件なら `TaskNotFoundError` を出す。ルートの `mark_done` はこの例外を受け止めて、404 の一覧ページを返す




```
@app.post("/tasks/{task_id}/done", response_model=None)
def mark_done(
    request: Request, task_id: int, store: StoreDep, today: TodayDep
) → HTMLResponse | RedirectResponse:
    """A-03 完了。"""
    try:
        store.mark_done(task_id)
    except TaskNotFoundError:
        return render_index(request, store, today, error="タスクが見つかりません",
status_code=404)
    return RedirectResponse("/", status_code=303)
```

```
mise run check; git add src; git commit -m "グリーン: タスクを完了にする"   # 38 passed
```

7. 残りの受け入れ条件のテストを足して読む(4分)

> 🖊 **見本のテストを末尾に足す**

📚 SE基礎 5章

📒 (2)・(4)〜(12)・(15) と、設計レビューで決めたこと・`TODO_DB_PATH` のテストを足す。テスト名の頭の番号で、受け入れ条件と対応を取る

```
cat ~/handsoff/docs/session11/assets/test_web_rest.py >> tests/test_web.py; mise run test
# 53 passed
```

🖊 わざと壊して、並び順のテストが落ちるのを見る

📒 コピーしたテストは最初から通った。本当に確かめているかは、壊して落ちるかで見る。一覧の並べ替えを外すと (7) だけが落ちる

```
sed -i 's/tasks = sort_by_due(store.list_all())/tasks = store.list_all()/'
src/todo/web.py; mise run test; git restore src/todo/web.py
```

🖊 わざと壊して、エスケープのテストが落ちるのを見る

📚 FastAPI 公式 Templates

📒 Jinja2 の `| safe` はエスケープを止める。タイトルの `<script>` がそのまま HTML になり、(10) が落ちる（第３回の XSS）

```
sed -i 's/{{ task.title }}/{{ task.title | safe }}/' src/todo/templates/index.html; mise
run test; git restore src/todo/templates/index.html
```

🖊 テストをコミットする

📒 実装を変えずに通ったテストなので `テスト:` を付ける

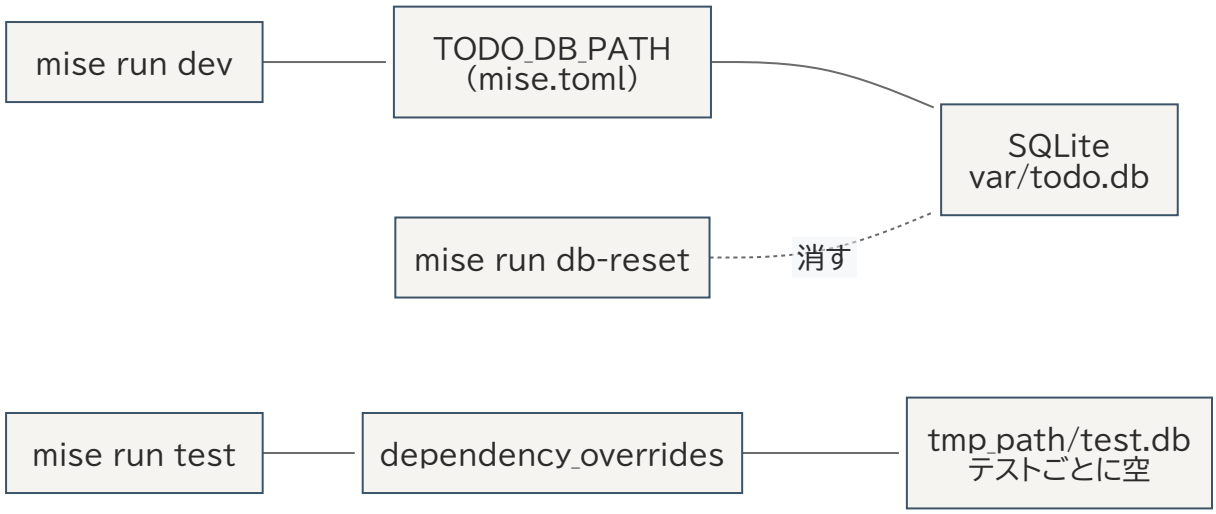
```
mise run check; git add tests/test_web.py; git commit -m "テスト: 残りの受け入れ条件のテストを
見本から足す"
```

8. ブラウザと curl で確かめる（３分）

🖊 開発用サーバを起動する

📚 Web技術の基本 ５章

📒 `--reload` はコードを保存すると読み直す。DB は `mise.toml` の `TODO_DB_PATH`（`var/todo.db`）で、テストの DB とは別




```
mise run dev     # 止めるときは Ctrl+C
```

- 確認: ブラウザで `http://127.0.0.1:8000/` を開くと、手順２で入れた「牛乳を買う」が出る。追加・完了・「期限切れだけ」を試す

🖊 303 のレスポンスを curl で見る

📒 別のターミナルで送る。303 と `Location`、日本語を送る `--data-urlencode` は第３回のログインと同じ

```
curl -i --data-urlencode "title=本を返す" -d "due=2026-11-05" http://127.0.0.1:8000/tasks
```

```
HTTP/1.1 303 See Other      ← 追加できた
server: uvicorn
content-length: 0
location: /                 ← 一覧ページ
```

🖊 400 と 404 を curl で見る

```
curl -si --data-urlencode "title=   " http://127.0.0.1:8000/tasks | grep -E "^HTTP|error"
curl -si -X POST http://127.0.0.1:8000/tasks/99/done | grep -E "^HTTP|error"
```

```
HTTP/1.1 400 Bad Request                                          ← 入力が正しくない
<p class="error" role="alert">タイトルを入力してください</p>        ← 一覧ページのエラー表示欄
HTTP/1.1 404 Not Found                                            ← 対象がない
<p class="error" role="alert">タスクが見つかりません</p>
```

9. PR を出してマージする（4 分）

🖊 設計書と見比べる

📚 SE基礎 4章 モデル

📒 API設計書 5章のシーケンス図（追加・完了）と、手順4・6の流れが同じかを見る。設計書どおりに作れたので設計書は変えない。変えるときは同じ PR で設計書も直す

```
git diff main --stat -- docs/design   # 何も出ない
```

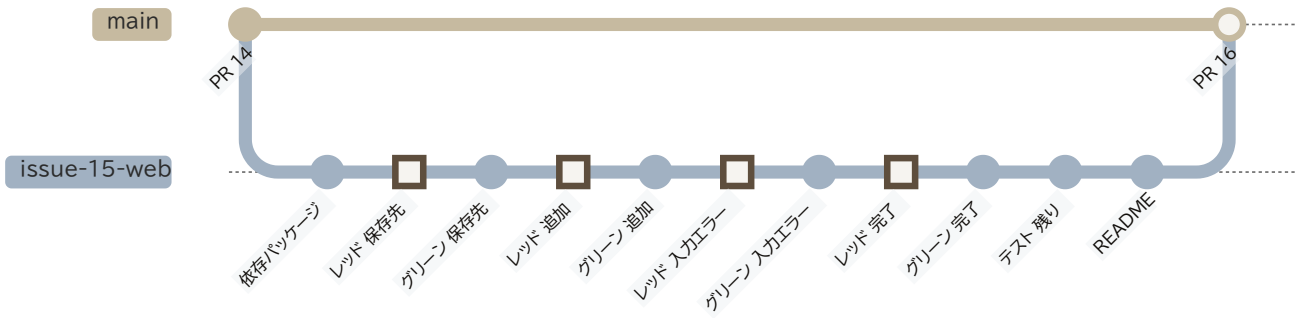
🖊 README と受け入れ条件の控えをそろえる

📒 見本の README は Web 版の使い方と `mise run dev`・`db-reset`・コードの構成を足したもの。`docs/stories.md` は受け入れ条件 (1)〜(15) にチェックを付けたもの

```
cp ~/handsoff/examples/todo-app-snapshots/session-11/README.md .; cp
~/handsoff/examples/todo-app-snapshots/session-11/docs/stories.md docs/; git diff --stat;
git add README.md docs/stories.md; git commit -m "README に Web 版の使い方を書き、受け入れ条件
にチェックを付ける"
```

🖊 履歴を確かめて push する

📒 レッドとグリーンが交互に並ぶ。図ではレッドのコミットを四角で示す



```
git log --oneline main..; git push -u origin issue-15-web
```

🖊 PR を作り、セルフレビューする（ブラウザ）

📚 Pro Git 6.2、GitHub Docs キーワードを使用して Pull Request を Issue にリンクする

📒 本文はテンプレートで書く。目的に `Closes #15` と、ストーリーの `Closes #8` ～ `Closes #11` を1行ずつ書く（`Closes #8, #9` では #9 が閉じない）。確認方法に `mise run check` と `mise run dev` での操作を書き、`[確認]` の行コメントを1件以上残す

🖊 CI が緑になってからマージする

📒 CI でも `mise run setup` が `uv.lock` どおりに FastAPI などを入れてから、53件のテストを動かす。赤なら、ログの失敗した行を読み、手元の `mise run check` で再現して直す

```
git switch main; git pull; git branch -d issue-15-web; mise run check
```

- 確認: Issue #8～#11・#15 が Closed、マイルストーン `Web 版` の進み具合が 100%。main で `mise run check` が通り、`mise run dev` で追加・一覧・完了ができる

読む章

区分	資料	範囲	読み方
参考書	ソフトウェアエンジニアリングの基礎（SE基礎）	5章 自動テスト、8章 データの取り扱い	通読（5章は第8回の続き、8章は第10回の続き）
参考書	イラスト図解式 この一冊で全部わかるWeb技術の基本 第2版（Web技術の基本）	5章 Webアプリケーションの基本	該当章
無料	FastAPI 公式ドキュメント	チュートリアル（フォームデータ、依存関係、yield を持つ依存関係、テスト）、高度なユーザーガイド（Templates、Testing Dependencies with Overrides）	該当章（書きながら引く）
無料	Python 公式ドキュメント	`sqlite3`（プレースホルダ、コンテキストマネージャとしての接続）	困ったときに引く
無料	GitHub Docs	キーワードを使用して Pull Request を Issue にリンクする	困ったときに引く

区分	資料	範囲	読み方
紹介(副読本)	達人に学ぶDB設計徹底指南書 第2版	6章 インデックス設計、7章 論理設計のアンチパターン	持っている人は(テーブルが増えたとき)

第12回 画面の部分更新と成果発表

tag: web-ブラウザ, test-レビュー
前提タグ: http, web-サーバ, test-自動
所要: 座学5分 / ハンズオン25分 / 発表準備5分 / 発表25分

ゴール

- HTML 断片を返す URL をテスト先行で足し、fetch で一覧だけを差し替えられる
- 差し替えた一覧のボタンでイベントが動かない理由を言い、イベントの委譲で直せる
- 完了ボタンの JSON 版と比べ、画面をどこで組み立てるかの違いを説明できる
- 見つけた不具合を Issue と不具合票にし、再現テスト(レッド)→ 修正(グリーン)→ PR の順で直せる
- 作った TODO アプリを、構成・デモ・根拠を添えた工夫1つで短く発表できる

事前準備

- 第11回のハンズオンの最後(PR #16 のマージ)まで済み、`~/handsoff/workspace/todo-app` の main で `mise run check` が通る(53 passed)。Issue・PR は #16 まで使用済み
- 第3回で、開発者ツールの Network タブとコンソール、`fetch` を使った
- 遅れた人: session-11/ を examples/README.md の手順でコピーし、push してから始める
- ブラウザは Chrome か Edge。社内ネットワークでは `NO_PROXY`・`no_proxy` に `localhost,127.0.0.1`(第2回)
- 発表はプロジェクターにつないだ自分の PC で行う(デモは `mise run dev`)
- 素材: assets/(`templates/`・`test_fragments.py`・`fragments.py`・`app.js`・`test_json.py`・`json_api.py`・`presentation.md`)

まとめ

冒頭の座学で使います。前半で第11回の Web 版の一覧を部分更新にして、そこで生まれた不具合を直し、後半で1人ずつ発表します。

部分更新の作り(web-ブラウザ)

部品	このアプリでは	参照
HTML 断片	ページ全体ではなく `<ul id="task-list">` だけを返す URL。ページと断片で同じテンプレート(`_task_list.html`)を使う	Web技術の基本 5章(Ajax)

部品	このアプリでは	参照
`data-fragment`	フォームの `action`（ふつうの送信先）は残し、fetch の送り先を属性で足す。JavaScript がなくても動く	MDN data-* 属性
`app.js`	submit を `preventDefault()` で止め、`fetch` で送り、返った断片で `outerHTML` を差し替える	MDN フェッチ API の使用
イベントの委譲	差し替えた新しいボタンには、読み込み時に付けたイベントがない。`document` に1つだけ付け、親へ伝わる submit を受け取る	MDN イベント入門

URL（API設計書 1.1）	返すもの
`GET /fragments/tasks`（A-04）	一覧の断片。`?overdue=1` で期限切れだけ
`POST /fragments/tasks`（A-05）・`POST /fragments/tasks/{id}/done`（A-06）	処理して一覧の断片。入力エラーは 400、ない id は 404 で `<p class="error" role="alert">` の断片
`POST /api/tasks/{id}/done`（A-07、比較用）	`{"id": 1, "done": true}`。ない id は 404 で `{"detail": "タスクが見つかりません"}`

HTML 断片と JSON

観点	HTML 断片（基本形）	JSON（比較用、完了ボタンだけ）
画面を組み立てる場所	サーバ（ページと同じテンプレート）	ブラウザ（`done-json.js`）
JavaScript の役割	送って差し替えるだけ。追加と完了で共通	JSON から DOM を書き換える。機能ごとに要る
表示の決まり	テンプレート1か所	テンプレートと JavaScript の2か所。並べ替え・絞り込みは直らない
テスト	返す HTML を pytest で確かめられる	画面の書き換えはブラウザ側のテストが別に要る
XSS	テンプレートが自動でエスケープ	受け取った値は `textContent` で入れる（第3回）

不具合票（test-レビュー）

項目	書くこと	参照
再現手順	誰がやっても同じになる番号付きの操作。日付は「昨日」「来週」のように今日から数える	不具合票のテンプレート
期待結果と根拠	本来どうなるか。受け入れ条件・設計書の該当箇所	SE基礎 6章

項目	書くこと	参照
実際の結果	見えたものをそのまま。Network タブの URL・ステータスも	SE基礎 6章
直す順	Issue(ラベル `不具合`)→ 票 → 再現テスト(レッド)→ 修正(グリーン)→ PR に `Closes`	研修の規約

発表

項目	基準	参照
聞き手	このアプリを初めて見る人。用語は構成図の箱の名前で言う	SE基礎 13章
順番	何を作ったか → 構成 → デモ → 工夫1つ(`docs/presentation.md` の4節)	素材 presentation.md
デモ	DB を空にし、見せるストーリーの順に3〜5手。部分更新は Network タブで見せる	—
工夫は1つ	主張を1文で言い、聞き手が後で開ける根拠を付ける(テスト名・PR の番号・設計書の節・不具合票)	SE基礎 13章
聞く側	良かった点を1つ+質問を1つ。質問は「なぜその設計か」「どう確かめたか」	SE基礎 13章

12回で通ったところ

フェーズ	回	身につけたこと	残ったもの
環境	1・2・4	Linux の全体像、シェル・PATH・プロキシ、mise と uv で再現できる環境	環境設定手順書、mise.toml
Web	3	HTTP・Cookie・CORS・Web API、DOM と fetch	—
構成管理	5	commit・ブランチ・PR・セルフレビュー・コンフリクトの解決	手順書のリポジトリ、PR 2本
Python	6〜8	CLI、dataclass と Protocol、ruff・mypy、pytest と TDD、CI	`todo-app` の PR #2・#4・#6
設計	9・10	ストーリーと受け入れ条件、幅のある見積、API・DB設計書、構成図	`docs/` 、PR #12・#14
実装	11・12	FastAPI+SQLite を TDD で、fetch による部分更新、不具合票	`src/` ・ `tests/` 、PR #16・#18・#20
総括	12	作ったものを根拠とともに伝える	発表資料、PR #22

地図: 研修で扱う知識の系統図、全体像(回ごとの流れ)

ハンズオン

1ステップ1コマンド。各ステップは「🖋 やること → 📚 参考書・公式資料の節 → 📒 説明 → コマンドやコード」。作業場所は `~/handsoff/workspace/todo-app/`。ターミナルを2つ使う(①開発用サーバ、②コマンド)。見本は session-12/(examples/session12/)。

1. 断片の URL と app.js をテスト先行で足す(6分)

> **🖋 開発用サーバを起動する(ターミナル①)**

📒 `mise run dev` は `--reload` 付きなので、ファイルを保存するたびにコードを読み直す。この回のあいだ起動したままにする

```
cd ~/handsoff/workspace/todo-app
git switch main; git pull; mise run dev
```

> **🖋 Issue を立てる(ブラウザ)**

📚 GitHub Docs Issue の作成

📒 第11回の PR が #16 なので、この Issue は #17 になる(GitLab では出た番号を使う)。本文はタスク用テンプレートで書き、ラベル `タスク` を付ける

Issue	タイトル	目的	完了条件
#17	一覧を fetch で部分更新する	追加・完了のたびにページ全体を読み直さず、一覧だけを差し替える	断片の URL(A-04〜A-06)のテストが通り、JavaScript がなくても動く。比較用の JSON 版がある

> **🖋 ブランチを作る(ターミナル②)**

```
cd ~/handsoff/workspace/todo-app
git switch -c issue-17-fetch
```

> **🖋 レッド: 素材のテストを足す**

📚 SE基礎 5章 自動テスト

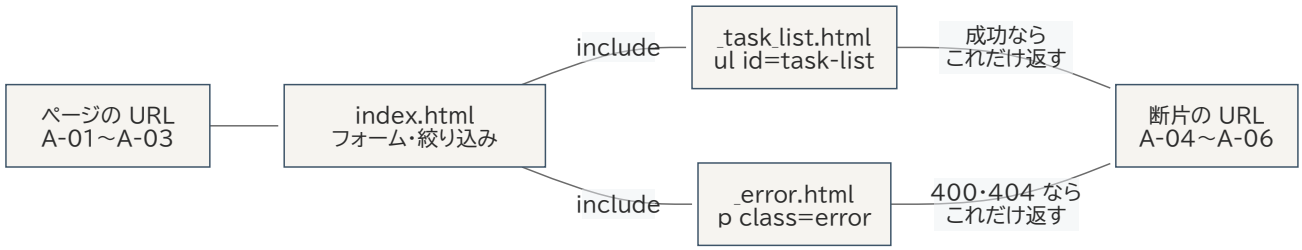
📒 素材のテスト7件を `tests/test_web.py` の末尾に足す。確かめるのは、一覧の断片(404)、断片での追加(405)と完了(404)、フォームの `data-fragment`、ページが `app.js` を読み込むこと

```
cat ~/handsoff/docs/session12/assets/test_fragments.py >> tests/test_web.py; mise run test   # 7 failed
git add tests/test_web.py; git commit -m "レッド: HTML 断片を返し、ページが app.js を読み込むテスト"
```

🖊 一覧を断片のテンプレートに切り出す

📚 FastAPI 公式 Templates

📒 素材の `_task_list.html` は `index.html` の `<ul id="task-list">` を切り出したもの。`index.html` は `{% include %}` で同じものを使い、フォームに `data-fragment` を足す。`base.html` は `app.js` を読み込む。落ちていたテストのうち、`data-fragment` のテストが通る

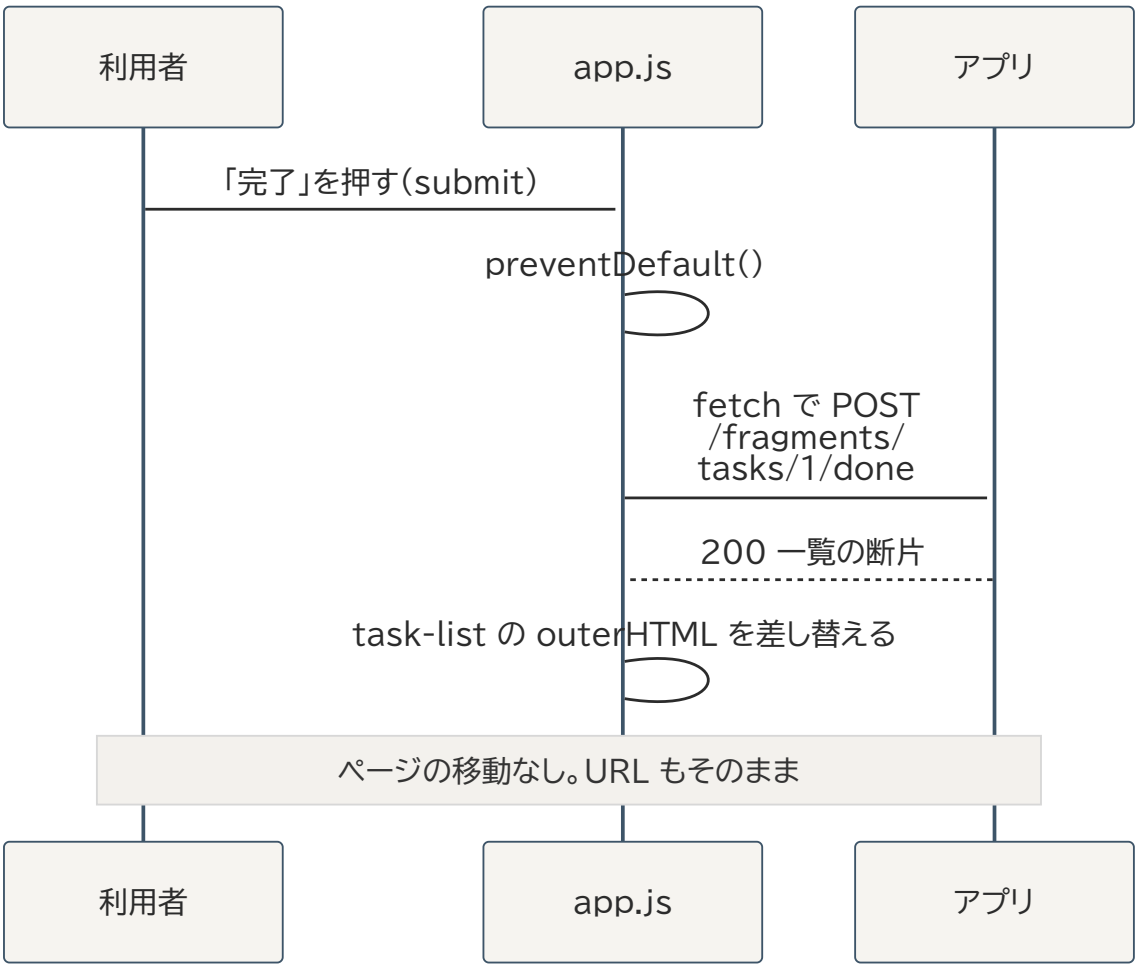



```
cp ~/handsoff/docs/session12/assets/templates/*.html src/todo/templates/; git diff --stat; mise run test   # 6 failed
```

🖊 グリーン: 素材のルートと app.js を足して読む

📚 MDN フェッチ API の使用

📒 `fragments.py` は第11回の `add_task`・`mark_done` と同じ処理で、成功なら 303 の代わりに一覧の断片、失敗なら一覧ページの代わりにエラーの断片を返す。`app.js` は送信を止めて fetch で送り、返った断片で一覧かエラー欄を差し替える




```
cp ~/handsoff/docs/session12/assets/app.js src/todo/static/; cat
~/handsoff/docs/session12/assets/fragments.py >> src/todo/web.py
less src/todo/static/app.js src/todo/web.py   # :n で次のファイル、q で終わる
mise run check; git add src; git commit -m "グリーン: 断片を返す URL と、送信を横取りする
app.js"   # 60 passed
```

2. イベントの委譲に直す(4 分)

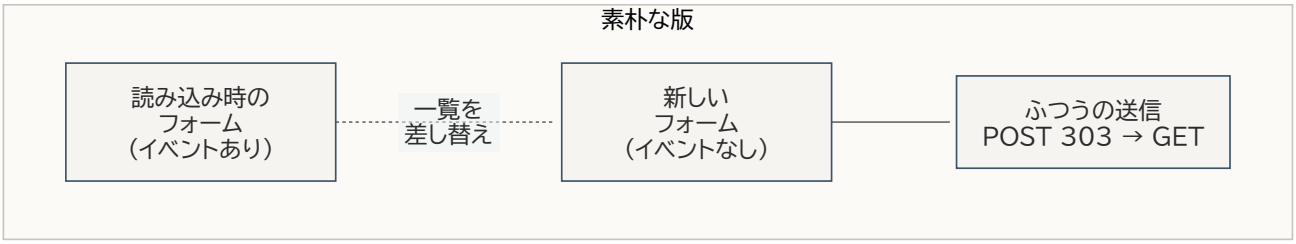
> 🖋 **ブラウザで試す**

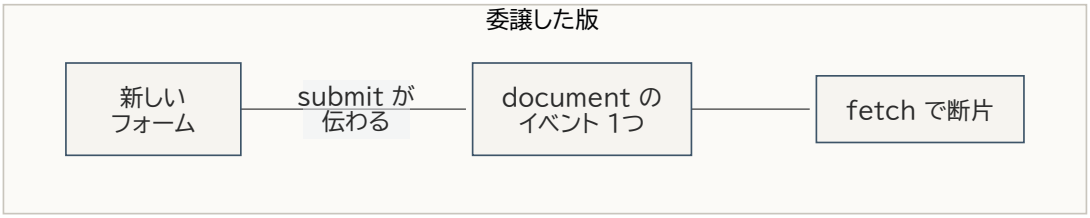
📒 `http://127.0.0.1:8000/` を再読み込みし、`F12` →「ネットワーク」→「ログを保持」に✓。2 件追加し、「完了」を押す。追加はタイプ `fetch` だが、完了は `document` の `POST /tasks/…/done`(303)→ `GET /` で、ページ全体を読み直す。なぜかを考える

🖊 見本の app.js に替えて違いを読む

📚 MDN イベント入門(イベントの委譲)

📒 追加で一覧を差し替えると、新しい「完了」ボタンには読み込み時に付けたイベントがない。見本の版は `document` に1つだけ付け、`event.target` で送られたフォームを見る。差し替えた後のフォームの submit も `document` まで伝わる






```
cp ~/handsoff/examples/todo-app-snapshots/session-12/src/todo/static/app.js
src/todo/static/; git diff
```

🖊 もう一度試してコミットする

📒 再読み込みして、追加 → 完了 → 完了がすべて `fetch` になることを見る。空白だけのタイトルで追加すると、400 のエラーの断片がエラー欄に入る。応答のサイズを `GET /`(ページ全体)と比べる

```
git commit -am "app.js のイベントを document にまとめて付ける（イベントの委譲）"
```

3. 完了ボタンだけ JSON 版にして比べる(5分)

🖊 レッド: JSON のテストを足す

📚 Web技術の基本 4章(JSON)

📒 完了にすると `{"id": 1, "done": true}`、ない id は 404 と `{"detail": "タスクが見つかりません"}` を返すテスト。URL がないので 404 と `{'detail': 'Not Found'}` で落ちる

```
cat ~/handsoff/docs/session12/assets/test_json.py >> tests/test_web.py; mise run test   # 2 failed
git add tests/test_web.py; git commit -m "レッド: JSON で完了にすると id と done を返すテスト"
```

🖊 グリーン: import を足し、素材のルートを足す

📚 FastAPI 公式 レスポンスモデル、エラーハンドリング

📒 `src/todo/web.py` の import を、下のように直す（`HTTPException` と `BaseModel` を足す）。素材の `DoneOut` は戻り値の型で、FastAPI が JSON にする

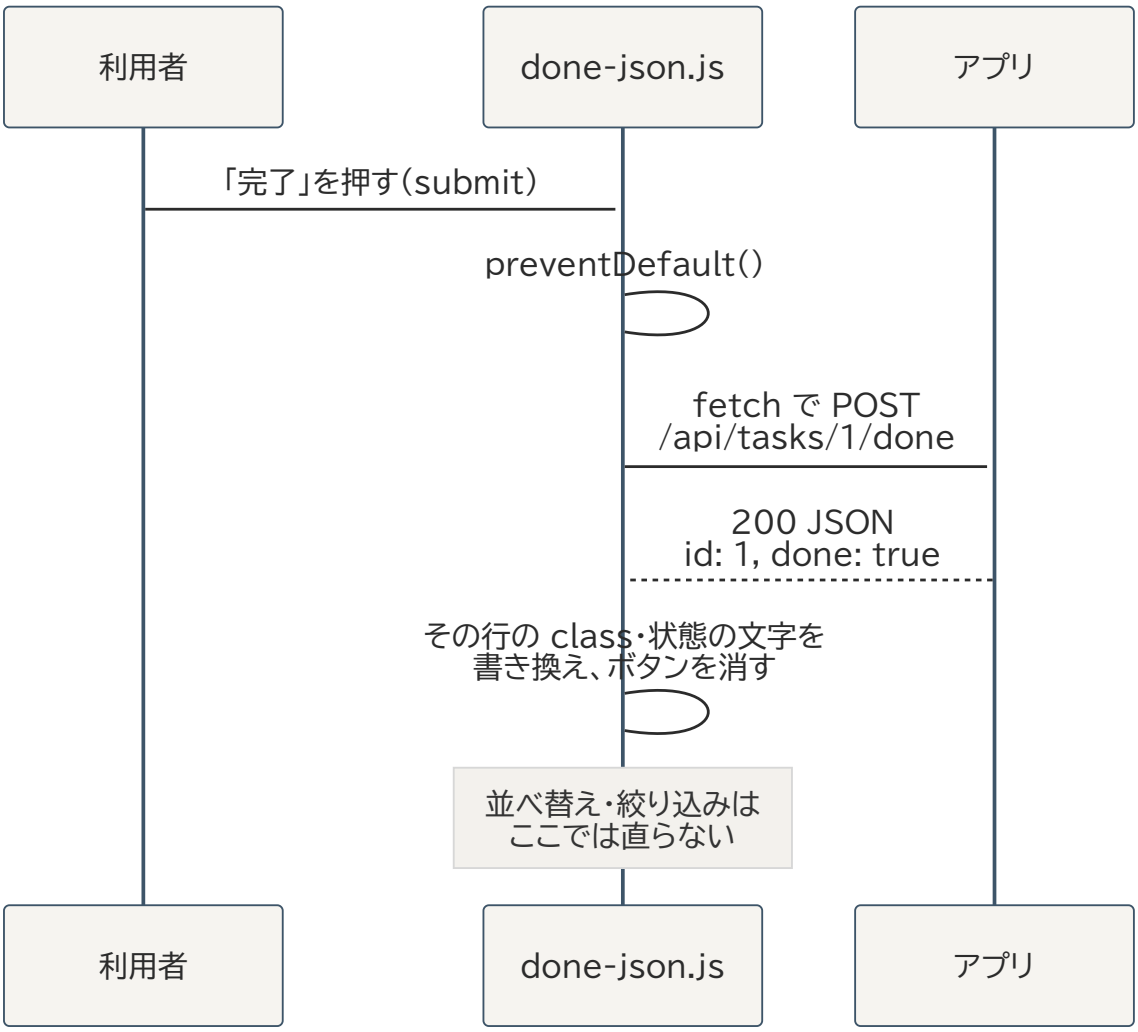
```
from fastapi import Depends, FastAPI, Form, HTTPException, Request
from fastapi.responses import HTMLResponse, RedirectResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel
```

```
cat ~/handsoff/docs/session12/assets/json_api.py >> src/todo/web.py; mise run check   # 62 passed
```

🖊 JSON から画面を組み立てる版をコピーして読む

📚 MDN Node.textContent

📒 見本の `done-json.js` は、受け取った JSON から、その行の class・状態の文字・ボタンを自分で書き換える。表示の決まりがテンプレートとここの２か所に分かれる




```
cp ~/handsoff/examples/todo-app-snapshots/session-12/src/todo/static/done-json.js
src/todo/static/; less src/todo/static/done-json.js
```

🖋 JSON 版に差し替えて比べる

📒 `base.html` の読み込みを替えて再読み込みし、期限が昨日のタスクを足す。「期限切れだけ」の表示で「完了」を押すと、JSON 版では行が「完了」のまま残る（断片の版では消える）。応答は約 20 バイト。比べたら戻す

```
sed -i 's/app.js" defer/done-json.js" defer/' src/todo/templates/base.html
git restore src/todo/templates/base.html   # 比べ終わったら
```

🖊 コミットする

```
mise run check; git add src; git commit -m "グリーン: JSON で完了にする URL と、比較用の done-json.js"
```

座学: 3つの作り(フォーム送信・HTML 断片・JSON)で、画面をどこで組み立て、どこでテストするか(→ まとめ「HTML 断片と JSON」)

4. 設計書をそろえて PR を出す(2分)

🖊 設計書を見本にそろえる

📚 SE基礎 4章 モデル

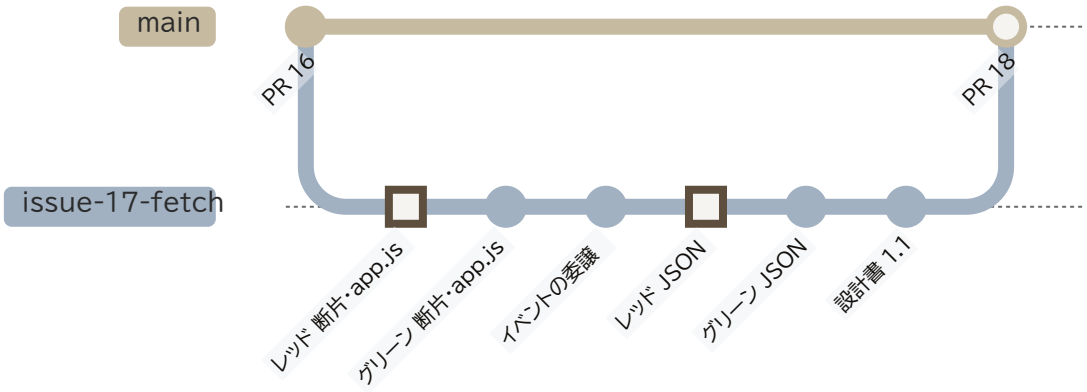
📒 見本の API設計書 1.1 は A-04〜A-07、`data-fragment`、断片と JSON のシーケンス図、7章の比較表を足したもの。システム構成図 1.1 はブラウザの JavaScript を足したもの。作った URL と見比べる

```
cp ~/handsoff/examples/todo-app-snapshots/session-12/docs/design/{api,architecture}.md docs/design/; git diff --stat
git commit -am "API設計書とシステム構成図に断片と JSON の URL を足す (1.1)"
```

🖊 push して PR を出し、マージする

📚 GitHub Docs キーワードを使用して Pull Request を Issue にリンクする

📒 PR は #18 で、目的に `Closes #17`、確認方法に `mise run check` と「Network で追加・完了が `fetch` になる」を書く。CI が緑になってからマージする



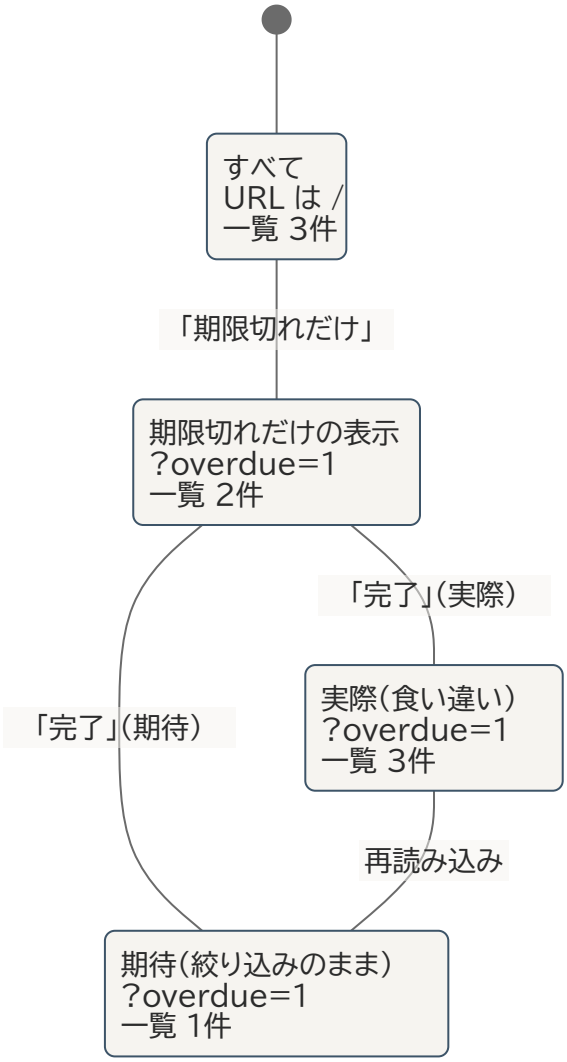
```
git push -u origin issue-17-fetch
git switch main; git pull; git branch -d issue-17-fetch   # マージした後
```

5. 不具合を見つけて票にし、テスト先行で直す（8 分）

🖋 受け入れ条件（15）を部分更新で試す（ブラウザ）

📚 SE 基礎 6 章 初めて触れるシステムの調査と変更

📒 `mise run db-reset` の後、「牛乳を買う」「本を返す」（期限 昨日）と「レポートを出す」（期限 来週）を追加し、「期限切れだけ」→「牛乳を買う」の「完了」を押す。一覧と、絞り込みのリンクと URL を見比べる




```
mise run db-reset   # ターミナル②で
```

🖊 Network で原因の見当を付ける

📒「完了」の行の URL を見る。`POST /fragments/tasks/1/done` に `?overdue=1` が付いていないので、サーバは全件の断片を返した。ページ全体を返す版は完了すると `/` に移ったので、食い違わなかった

🖊 不具合の Issue を立てる（ブラウザ）

📚 GitHub Docs ラベルを管理する

📒 Issue は #19。ラベル `不具合` を新しく作って付ける。本文は不具合票と同じ項目（概要・再現手順・期待結果・実際の結果）で書く

Issue	タイトル	ラベル
#19	期限切れだけの表示中に完了すると全件が表示される	`不具合`

🖊 ブランチを作り、不具合票を書く

📒 テンプレートをコピーして埋める。期待結果には根拠（受け入れ条件 (15)、API 設計書 2 章 A-01 の `data-fragment` の説明）を書く。書けたら見本の BUG-001 と見比べる

```
git switch -c issue-19-overdue-done; mkdir -p docs/bugs; cp
~/handsoff/docs/templates/bug-report.md docs/bugs/BUG-001.md
git add docs/bugs; git commit -m "不具合票 BUG-001 を書く"   # 埋め終わったら
```

🖊 レッド: 不具合を再現するテストを書く

📒 `tests/test_web.py` の末尾に、前と 2 行空けて足す（ruff の決まり）。「期限切れだけ」のページの完了ボタンの送り先に `?overdue=1` がないので落ちる。先に再現テストを書くと、直ったことと、二度と起きないことを確かめられる

```
# ---- 不具合票 docs/bugs/ の再現テスト ----


def test_BUG001_期限切れだけの表示中に完了しても期限切れだけを返す(client: TestClient) → None:
    add(client, "牛乳を買う", "2026-10-30")
    add(client, "本を返す", "2026-10-31")
    add(client, "レポートを出す", "2026-11-05")

    page = client.get("/?overdue=1").text
    # 期限切れだけの表示中は、完了ボタンの送り先にも ?overdue=1 が付く
    assert 'data-fragment="/fragments/tasks/1/done?overdue=1"' in page

    response = client.post("/fragments/tasks/1/done?overdue=1")

    assert titles(response.text) == ["本を返す"]
```

```
mise run test; git add tests/test_web.py; git commit -m "レッド: BUG-001 期限切れだけの表示中に完了すると全件が返る"
```

🖋 グリーン: 表示中の絞り込みを断片の URL に載せる

📒 `visible_tasks` の下に `overdue_query` を足し、`render_index` と `render_task_list` の `context` に `"query": overdue_query(overdue_only),` の行を足す。テンプレートは sed で `data-fragment` の末尾に `{{ query }}` を付ける

```
def overdue_query(overdue_only: bool) → str:
    """断片の URL に付けるクエリ。期限切れだけの表示中は、断片も期限切れだけにする（BUG-001）。"""
    return "?overdue=1" if overdue_only else ""
```

```
sed -i 's/data-fragment="\([^"]*\)"/data-fragment="\1{{ query }}"/'
src/todo/templates/{index,_task_list}.html; git diff
mise run check; git add src; git commit -m "グリーン: BUG-001 表示中の絞り込みを断片の URL に載せる"   # 63 passed
```

🖋 README をそろえ、PR を出してマージする

📚 GitHub Docs キーワードを使用して Pull Request を Issue にリンクする

📒 見本の README は、部分更新・比較用の JSON 版・`docs/bugs/` を足したもの。PR は #20 で、目的に `Closes #19`。ブラウザで最初の操作をもう一度試し、「本を返す」だけが残ることを確かめてからマージする




```
cp ~/handsoff/examples/todo-app-snapshots/session-12/README.md .; git commit -am "README に部分更新と不具合票の置き場所を書く"; git push -u origin issue-19-overdue-done
git switch main; git pull; git branch -d issue-19-overdue-done   # マージした後
```

6. 発表準備（5 分）

🖊 Issue を立ててブランチを作る

📒 Issue は #21「成果発表の資料を書く」（ラベル `タスク`）。完了条件は「`docs/presentation.md` が main にある」

```
git switch -c issue-21-presentation
```

🖊 発表資料の型をコピーする

📚 SE 基礎 13 章 テクノロジー業界でソフトスキルを磨く

📒 1〜3 節には全員に共通する下書きが入っている。自分のアプリと違う所だけ直し、発表者の名前を書く

```
cp ~/handsoff/docs/session12/assets/presentation.md docs/
```

🖊 作った記録の数字を集める

📒 1 節の表に写す。マージした PR の数、`レッド:` で始まるコミットの数（テストを先に書いた回数）、テストの件数

```
git log --oneline --merges | wc -l; git log --oneline --grep='^レッド' | wc -l; mise run test 2>&1 | tail -1
```

🖊 4節: 工夫を1つ選び、根拠を付ける

📒 下の候補から1つ選び、主張を1文にする。根拠は聞き手が後で開けるもの(ファイル・テスト名・PR の番号・設計書の節)にする

候補	根拠の例
サーバが HTML 断片を返し、JavaScript は差し替えるだけにした	`app.js`、API設計書 7章、PR #18
保存先と「今日」を差し替えてテストした(依存性の注入)	`tests/test_web.py` の fixture、PR #16
テストを先に書いた(レッド → グリーン)	`git log --grep='^レッド'` の数、PR の履歴
不具合を再現テストから直した	`docs/bugs/BUG-001.md`、`test_BUG001_…`、PR #20

🖊 デモを1回通す(ターミナル①)

📒 `Ctrl+C` で止めてから、DB を空にして起動し、3節の表の順に操作する

```
mise run db-reset && mise run dev
```

🖊 コミットして PR を出し、マージする(ターミナル②)

📒 PR は #22 で、目的に `Closes #21`。GitHub の PR の画面では Mermaid の図も描かれるので、発表で資料を見せるのに使える。CI が緑になってからマージする

```
git add docs; git commit -m "発表資料を書く"; git push -u origin issue-21-presentation
git switch main; git pull; git branch -d issue-21-presentation   # マージした後
```

7. 発表(25分)

🖊 順番と持ち時間を確かめる

📒 1人の持ち時間は人数から講師が決める。6人なら1人4分(発表とデモ2.5分・質疑1分・交代0.5分)

🖊 デモを起動しておく(発表者)

📒 前の人の質疑の間に、DB を空にして起動し、ブラウザと Network タブを開いておく

```
mise run db-reset && mise run dev
```

🖊 発表する（発表者）

📚 SE基礎 13章 テクノロジー業界でソフトスキルを磨く

📒 `docs/presentation.md` を画面に出して1節から話す。デモは3節の表の順に操作し、工夫は1つだけ話す。質問には根拠のファイルや PR を開いて答える

🖊 聞きながらメモする（聞き手）

📒 1人につき良かった点と質問を1つずつ書く。質問は設計の理由か確かめ方を聞く

発表者	良かった点（真似したいこと）	質問（なぜその設計か・どう確かめたか）

- 確認: Issue #17・#19・#21 が Closed。main で `mise run check` が通り（63 passed）、`docs/bugs/BUG-001.md` と `docs/presentation.md` がある

読む章

区分	資料	範囲	読み方
参考書	イラスト図解式 この一冊で全部わかるWeb技術の基本 第2版（Web技術の基本）	4章 データ形式（HTML・DOM・JSON）、5章 Webアプリケーションの基本（Ajax）	該当章
参考書	ソフトウェアエンジニアリングの基礎（SE基礎）	6章 初めて触れるシステムの調査と変更	通読（第1回に続いて2回目）
参考書	同上	13章 テクノロジー業界でソフトスキルを磨く	通読
無料	MDN Web Docs	フェッチ API の使用、イベント入門（イベントの委譲）、data-* 属性、Node.textContent	該当章（書きながら引く）
無料	FastAPI 公式ドキュメント	Templates、レスポンスモデル、エラーハンドリング（HTTPException）	困ったときに引く
無料	GitHub Docs（Issues）	Issue の作成、ラベルを管理する、Pull Request を Issue にリンクする	困ったときに引く

成果発表: TODO アプリ

項目	内容
発表者	
持ち時間	発表とデモ ○分 ＋ 質疑 ○分（講師が人数から決める）

1. 何を作ったか

コマンドラインの TODO（第６〜８回）を、ブラウザからも使えるようにした（第９〜12回）。

- タスクの追加（タイトル・期限）、一覧（期限が近い順）、完了
- 状態の表示（未着手・完了・期限切れ）と「期限切れだけ」の絞り込み
- 追加と完了は、ページを読み直さずに一覧だけを差し替える

作った記録	数	調べ方
マージした PR		`git log --oneline --merges | wc -l`
テストを先に書いたコミット		`git log --oneline --grep='^レッド' | wc -l`
テスト		`mise run test 2>&1 | tail -1`

2. システム構成



- CLI と Web は、タスクの決まり（`models.py`）と保存先の約束（`TaskStore`）を共有する。保存先は CLI が JSON、Web が SQLite
- 詳しくは システム構成図

3. デモ手順

順	操作	見せること	ストーリー
1	`mise run db-reset && mise run dev` で起動し、http://127.0.0.1:8000 を開く	空の一覧	S-02
2	期限が昨日・来週・なしのタスクを１件ずつ追加	期限が近い順に並び、昨日のものは「期限切れ」	S-01・S-02
3	「期限切れだけ」→「完了」（Network タブを開いておく）	一覧だけが差し替わる。返るのは `<ul id="task-list">` の断片	S-03・S-04

4. 設計とテストの工夫（1つ）

（主張を１文で）

- 根拠:

タグ一覧

知識体系マップ(curriculum.md)のタグを並べたものです。
同じタグ名を、資料の冒頭と Issue ラベルで共通に使います。
タグ名のリンク先は、そのタグを扱う回と副読本をまとめた索引です。

タグ	内容	扱う回
linux	OS(Linux)	1, 2
linux-ファイル	ファイルと権限:ファイル操作／パイプとリダイレクト／権限	1
linux-シェル	シェル:環境変数とPATH／.bashrc／シェルスクリプト	1, 2
linux-アプリ	パッケージとプロセス:apt／エディタ(nano／vim)／プロセス(ps・kill)／コンテナ(Docker)	1, 2
linux-ネットワーク	通信の基礎:ネットワーク層とアプリ層／TCP/IPとポート／DNS・hosts／curl・ss・ping	1, 2
http	Web通信	3, 10
http-基本	HTTPとは:アプリ層のプロトコル／リクエストとレスポンス／URI	3
http-ブラウザ	ブラウザの挙動:ページの読み込み／フォーム送信とリダイレクト／Cookie・セッション／CORS	3
http-構成	HTTPの構成:メソッド／ステータスコード／ヘッダ／キャッシュ	3, 10
http-リアルタイム	リアルタイム通信:ポーリング／SSE／WebSocket	3
db	データベース:ER図・テーブル設計／SQL(SQLite)	10, 11
sec	セキュリティ:XSSとエスケープ／CSRF／秘密情報の管理	3
python	プログラミング(Python)	6, 7
python-構文	言語機能:型ヒント／dict.get／内包表記／例外とwith／pathlib／パターンマッチ	6
python-oop	オブジェクト指向:ダックタイピング／Protocol／dataclass／ABC	7
python-可読性	読みやすいコード:命名／コメント／制御フロー／リファクタリング	7
web	Webアプリケーション	3, 11, 12
web-サーバ	サーバ側:FastAPI／Jinja2テンプレート／Pydantic／依存性の注入	11
web-ブラウザ	ブラウザ側:HTMLフォーム／fetch・DOM操作／DevTools／htmx	3, 12
設計	設計	9, 10
設計-要件	要件:ユーザーストーリー／受け入れ条件／非機能要件	9
設計-構成	システム構成:システム構成図	10
設計-詳細	詳細設計:API設計書／DB設計書	10

タグ	内容	扱う回
test	テストと品質	5, 7, 8, 11, 12
test-自動	自動テスト：pytest／TDD／テスト用DB	8, 11
test-静的解析	静的解析：ruff／mypy／pyright	7
test-CI	CI：GitHub Actions／PR ごとの ruff・pytest の自動実行	8
test-レビュー	レビューと不具合管理：コードレビュー／不具合票	5, 12
git	バージョン管理（Git）	5
git-基礎	Gitの基本：commit／diff・log／.gitignore／revert	5
git-github	GitHub での利用：ブランチ／PR・セルフレビュー／Issue／コンフリクト解決	5
env	実行環境	4
env-mise	ツールのバージョン管理：mise.toml／タスク	4
env-python	Pythonの環境：uv／仮想環境／ロックファイル	4
標準	開発標準：コーディング規約／ブランチ運用／コミット規約／レビュー基準	5, 7
pm	プロジェクトマネジメント	9
pm-見積	計画と見積：タスクの分解／幅のある見積	9
pm-進捗	タスク管理：Issue／ラベル・マイルストーン	9

db

データベース：ER図・テーブル設計／SQL（SQLite）

扱う回

- 第10回 設計書作成（db）
- 第11回 TDD による Web バックエンド（db）

副読本・参考書・公式資料

- 達人に学ぶDB設計徹底指南書 第2版（翔泳社）：3章 正規化、4章 ER図（第10回）／6章 インデックス設計、7章 論理設計のアンチパターン（第11回）

env

実行環境

下位タグ: env-mise, env-python

扱う回

- 第４回 実行環境の整備(env-mise, env-python)

前提にしている回

- 第５回 Git と GitHub
- 第６回 Python の基本
- 第７回 読みやすく壊れにくい Python

副読本・参考書・公式資料

- Effective Python 第２版:1〜２章(項目16 dict.get など)、８章 項目65〜66(例外と with)(第６回)／５章 クラスと継承(第７回)／10章 項目83 仮想環境(第４回)、項目90 静的解析(第７回)
- mise・uv 公式ドキュメント:まとまった書籍がないため、公式が主な資料

env-mise

ツールのバージョン管理：mise.toml／タスク

上位タグ: env

扱う回

- 第４回 実行環境の整備（env-mise）

前提にしている回

- 第５回 Git と GitHub

副読本・参考書・公式資料

- mise・uv 公式ドキュメント：まとまった書籍がないため、公式が主な資料

env-python

Pythonの環境：uv／仮想環境／ロックファイル

上位タグ: env

扱う回

- 第４回 実行環境の整備（env-python）

前提にしている回

- 第５回 Git と GitHub
- 第６回 Python の基本
- 第７回 読みやすく壊れにくい Python

副読本・参考書・公式資料

- Effective Python 第２版：1〜２章（項目16 dict.get など）、８章 項目65〜66（例外と with）（第６回）／５章 クラスと継承（第７回）／10章 項目83 仮想環境（第４回）、項目90 静的解析（第７回）
- mise・uv 公式ドキュメント：まとまった書籍がないため、公式が主な資料

git

バージョン管理(Git)

下位タグ: git-github, git-基礎

扱う回

- 第５回 Git と GitHub(git-基礎, git-github)

前提にしている回

- 第６回 Python の基本
- 第７回 読みやすく壊れにくい Python
- 第８回 テストと TDD
- 第９回 要件と図
- 第10回 設計書作成

副読本・参考書・公式資料

- 達人プログラマー 第２版(オーム社):基本ツール(シェル、プレーンテキスト、バージョン管理)(第１〜５回)／見積もり(第９回)
- Pro Git(日本語版):Git 公式の解説書。仕組みからブランチ運用まで網羅しており、Git のリファレンスはこれで足りる

git-github

GitHub での利用：ブランチ／PR・セルフレビュー／Issue／コンフリクト解決

上位タグ: git

扱う回

- 第５回 Git と GitHub(git-github)

前提にしている回

- 第６回 Python の基本
- 第７回 読みやすく壊れにくい Python
- 第８回 テストと TDD
- 第９回 要件と図
- 第10回 設計書作成

副読本・参考書・公式資料

- Pro Git(日本語版)：Git 公式の解説書。仕組みからブランチ運用まで網羅しており、Git のリファレンスはこれで足りる

git-基礎

Gitの基本：commit／diff・log／.gitignore／revert

上位タグ: git

扱う回

- 第５回 Git と GitHub(git-基礎)

前提にしている回

- 第６回 Python の基本

副読本・参考書・公式資料

- 達人プログラマー 第２版(オーム社)：基本ツール(シェル、プレーンテキスト、バージョン管理)(第１～５回)／見積もり(第９回)
- Pro Git(日本語版)：Git 公式の解説書。仕組みからブランチ運用まで網羅しており、Git のリファレンスはこれで足りる

http

Web通信

下位タグ: http-ブラウザ, http-リアルタイム, http-基本, http-構成

扱う回

- 第３回 Web 通信の基礎(HTTP・Cookie・Web API)(http)
- 第10回 設計書作成(http-構成)

前提にしている回

- 第12回 画面の部分更新と成果発表

副読本・参考書・公式資料

- Webを支える技術(技術評論社):第２部 URI、第３部 HTTP(第３・10回)／第５部 Web サービスの設計(第10回の API 設計)
- イラスト図解式 この一冊で全部わかるWeb技術の基本 第２版(SBクリエイティブ):見開き１テーマの図解で引く Web の入門書。２章 Webとネットワーク技術(第２回)／３章 HTTPでやりとりする仕組み(第３・10回)／４章 データ形式(HTML・DOM・JSON、第３・12回)／５章 Webアプリケーションの基本(Web API、第３回／３層構造、第10〜11回／Ajax、第12回)／６章 セキュリティと認証(プロキシは第２回、Cookie・セッション・CSRF・XSS は第３回)／７章 構築と運用(構成図、第10回)
- MDN Web Docs:HTTP、HTML、Cookie、WebSocket、CORS のリファレンス。Web の基礎はここを一次資料にする(第３回のヘッダ・キャッシュ・Cookie・CORS・フェッチ API・DOM、第10・12回でも引く)

http-ブラウザ

ブラウザの挙動：ページの読み込み／フォーム送信とリダイレクト／Cookie・セッション／CORS

上位タグ: http

扱う回

- 第３回 Web 通信の基礎（HTTP・Cookie・Web API）（http）

前提にしている回

- 第12回 画面の部分更新と成果発表

副読本・参考書・公式資料

- Webを支える技術（技術評論社）：第２部 URI、第３部 HTTP（第３・10回）／第５部 Web サービスの設計（第10回の API 設計）
- イラスト図解式 この一冊で全部わかるWeb技術の基本 第２版（SBクリエイティブ）：見開き１テーマの図解で引く Web の入門書。２章 Webとネットワーク技術（第２回）／３章 HTTPでやりとりする仕組み（第３・10回）／４章 データ形式（HTML・DOM・JSON、第３・12回）／５章 Webアプリケーションの基本（Web API、第３回／３層構造、第10〜11回／Ajax、第12回）／６章 セキュリティと認証（プロキシは第２回、Cookie・セッション・CSRF・XSS は第３回）／７章 構築と運用（構成図、第10回）
- MDN Web Docs：HTTP、HTML、Cookie、WebSocket、CORS のリファレンス。Web の基礎はここを一次資料にする（第３回のヘッダ・キャッシュ・Cookie・CORS・フェッチ API・DOM、第10・12回でも引く）

http-リアルタイム

リアルタイム通信：ポーリング／SSE／WebSocket

上位タグ: http

扱う回

- 第３回 Web 通信の基礎（HTTP・Cookie・Web API）（http）

前提にしている回

- 第12回 画面の部分更新と成果発表

副読本・参考書・公式資料

- Webを支える技術（技術評論社）：第２部 URI、第３部 HTTP（第３・10回）／第５部 Web サービスの設計（第10回の API 設計）
- イラスト図解式 この一冊で全部わかるWeb技術の基本 第２版（SBクリエイティブ）：見開き１テーマの図解で引く Web の入門書。２章 Webとネットワーク技術（第２回）／３章 HTTPでやりとりする仕組み（第３・10回）／４章 データ形式（HTML・DOM・JSON、第３・12回）／５章 Webアプリケーションの基本（Web API、第３回／３層構造、第10〜11回／Ajax、第12回）／６章 セキュリティと認証（プロキシは第２回、Cookie・セッション・CSRF・XSS は第３回）／７章 構築と運用（構成図、第10回）
- MDN Web Docs：HTTP、HTML、Cookie、WebSocket、CORS のリファレンス。Web の基礎はここを一次資料にする（第３回のヘッダ・キャッシュ・Cookie・CORS・フェッチ API・DOM、第10・12回でも引く）

http-基本

HTTPとは:アプリ層のプロトコル／リクエストとレスポンス／URI

上位タグ: http

扱う回

- 第３回 Web 通信の基礎(HTTP・Cookie・Web API)(http)

前提にしている回

- 第12回 画面の部分更新と成果発表

副読本・参考書・公式資料

- Webを支える技術(技術評論社):第２部 URI、第３部 HTTP(第３・10回)／第５部 Web サービスの設計(第10回の API 設計)
- イラスト図解式 この一冊で全部わかるWeb技術の基本 第２版(SBクリエイティブ):見開き１テーマの図解で引く Web の入門書。２章 Webとネットワーク技術(第２回)／３章 HTTPでやりとりする仕組み(第３・10回)／４章 データ形式(HTML・DOM・JSON、第３・12回)／５章 Webアプリケーションの基本(Web API、第３回／３層構造、第10〜11回／Ajax、第12回)／６章 セキュリティと認証(プロキシは第２回、Cookie・セッション・CSRF・XSS は第３回)／７章 構築と運用(構成図、第10回)
- MDN Web Docs:HTTP、HTML、Cookie、WebSocket、CORS のリファレンス。Web の基礎はここを一次資料にする(第３回のヘッダ・キャッシュ・Cookie・CORS・フェッチ API・DOM、第10・12回でも引く)

http-構成

HTTPの構成：メソッド／ステータスコード／ヘッダ／キャッシュ

上位タグ: http

扱う回

- 第３回 Web 通信の基礎（HTTP・Cookie・Web API）（http）
- 第10回 設計書作成（http-構成）

前提にしている回

- 第12回 画面の部分更新と成果発表

副読本・参考書・公式資料

- Webを支える技術（技術評論社）：第２部 URI、第３部 HTTP（第３・10回）／第５部 Web サービスの設計（第10回の API 設計）
- イラスト図解式 この一冊で全部わかるWeb技術の基本 第２版（SBクリエイティブ）：見開き１テーマの図解で引く Web の入門書。２章 Webとネットワーク技術（第２回）／３章 HTTPでやりとりする仕組み（第３・10回）／４章 データ形式（HTML・DOM・JSON、第３・12回）／５章 Webアプリケーションの基本（Web API、第３回／３層構造、第10〜11回／Ajax、第12回）／６章 セキュリティと認証（プロキシは第２回、Cookie・セッション・CSRF・XSS は第３回）／７章 構築と運用（構成図、第10回）
- MDN Web Docs：HTTP、HTML、Cookie、WebSocket、CORS のリファレンス。Web の基礎はここを一次資料にする（第３回のヘッダ・キャッシュ・Cookie・CORS・フェッチ API・DOM、第10・12回でも引く）

linux

OS（Linux）

下位タグ: linux-アプリ, linux-シェル, linux-ネットワーク, linux-ファイル

扱う回

- 第１回 知識体系の紹介＋Linux の全体像（linux）
- 第２回 シェル・環境変数・通信の切り分け（linux-シェル, linux-アプリ, linux-ネットワーク）

前提にしている回

- 第２回 シェル・環境変数・通信の切り分け
- 第３回 Web 通信の基礎（HTTP・Cookie・Web API）
- 第４回 実行環境の整備
- 第５回 Git と GitHub

副読本・参考書・公式資料

- 達人プログラマー 第２版（オーム社）：基本ツール（シェル、プレーンテキスト、バージョン管理）（第１〜５回）／見積もり（第９回）
- 入門 モダンLinux（オライリー・ジャパン）：Linux の仕組みを理解する本。第１回はこの本の流れで全体を通して見る。１章 Linuxの入門・２章 Linuxカーネル（第１回）／３章 シェルとスクリプト、４章 アクセス制御（第１〜２回）／５章 ファイルシステム（第１回）／６章 アプリケーション、パッケージ管理、コンテナ（6.6 コンテナは第１回、パッケージは第２・４回）／７章 ネットワーク（第１〜２回）／9.1 プロセス間通信（シグナル、第１回）／付録A 便利なコマンド集（第１回、以降は引く）・付録B モダンLinuxツール（第１回）
- イラスト図解式 この一冊で全部わかるWeb技術の基本 第２版（SBクリエイティブ）：見開き１テーマの図解で引く Web の入門書。２章 Webとネットワーク技術（第２回）／３章 HTTPでやりとりする仕組み（第３・10回）／４章 データ形式（HTML・DOM・JSON、第３・12回）／５章 Webアプリケーションの基本（Web API、第３回／３層構造、第10〜11回／Ajax、第12回）／６章 セキュリティと認証（プロキシは第２回、Cookie・セッション・CSRF・XSS は第３回）／７章 構築と運用（構成図、第10回）

linux-アプリ

パッケージとプロセス：apt／エディタ（nano／vim）／プロセス（ps・kill）／コンテナ（Docker）

上位タグ: linux

扱う回

- 第１回 知識体系の紹介＋Linux の全体像（linux）
- 第２回 シェル・環境変数・通信の切り分け（linux-アプリ）

前提にしている回

- 第２回 シェル・環境変数・通信の切り分け
- 第４回 実行環境の整備
- 第５回 Git と GitHub

副読本・参考書・公式資料

- 入門 モダンLinux（オライリー・ジャパン）：Linux の仕組みを理解する本。第１回はこの本の流れで全体を通して見る。1章 Linuxの入門・2章 Linuxカーネル（第１回）／3章 シェルとスクリプト、4章 アクセス制御（第1〜2回）／5章 ファイルシステム（第１回）／6章 アプリケーション、パッケージ管理、コンテナ（6.6 コンテナは第１回、パッケージは第2・4回）／7章 ネットワーク（第1〜2回）／9.1 プロセス間通信（シグナル、第１回）／付録A 便利なコマンド集（第１回、以降は引く）・付録B モダンLinuxツール（第１回）

linux-シェル

シェル：環境変数とPATH／.bashrc／シェルスクリプト

上位タグ: linux

扱う回

- 第１回 知識体系の紹介＋Linux の全体像（linux）
- 第２回 シェル・環境変数・通信の切り分け（linux-シェル）

前提にしている回

- 第２回 シェル・環境変数・通信の切り分け
- 第３回 Web 通信の基礎（HTTP・Cookie・Web API）
- 第４回 実行環境の整備

副読本・参考書・公式資料

- 達人プログラマー 第２版（オーム社）：基本ツール（シェル、プレーンテキスト、バージョン管理）（第１〜５回）／見積もり（第９回）
- 入門 モダンLinux（オライリー・ジャパン）：Linux の仕組みを理解する本。第１回はこの本の流れで全体を通して見る。1章 Linuxの入門・2章 Linuxカーネル（第１回）／3章 シェルとスクリプト、4章 アクセス制御（第１〜２回）／5章 ファイルシステム（第１回）／6章 アプリケーション、パッケージ管理、コンテナ（6.6 コンテナは第１回、パッケージは第２・４回）／7章 ネットワーク（第１〜２回）／9.1 プロセス間通信（シグナル、第１回）／付録A 便利なコマンド集（第１回、以降は引く）・付録B モダンLinuxツール（第１回）

linux-ネットワーク

通信の基礎：ネットワーク層とアプリ層／TCP/IPとポート／DNS・hosts／curl・ss・ping

上位タグ: linux

扱う回

- 第１回 知識体系の紹介＋Linux の全体像（linux）
- 第２回 シェル・環境変数・通信の切り分け（linux-ネットワーク）

前提にしている回

- 第２回 シェル・環境変数・通信の切り分け
- 第３回 Web 通信の基礎（HTTP・Cookie・Web API）
- 第４回 実行環境の整備

副読本・参考書・公式資料

- 入門 モダンLinux（オライリー・ジャパン）：Linux の仕組みを理解する本。第１回はこの本の流れで全体を通して見る。1章 Linuxの入門・2章 Linuxカーネル（第１回）／3章 シェルとスクリプト、4章 アクセス制御（第1〜2回）／5章 ファイルシステム（第１回）／6章 アプリケーション、パッケージ管理、コンテナ（6.6 コンテナは第１回、パッケージは第2・4回）／7章 ネットワーク（第1〜2回）／9.1 プロセス間通信（シグナル、第１回）／付録A 便利なコマンド集（第１回、以降は引く）・付録B モダンLinuxツール（第１回）
- イラスト図解式 この一冊で全部わかるWeb技術の基本 第２版（SBクリエイティブ）：見開き１テーマの図解で引く Web の入門書。2章 Webとネットワーク技術（第２回）／3章 HTTPでやりとりする仕組み（第3・10回）／4章 データ形式（HTML・DOM・JSON、第3・12回）／5章 Webアプリケーションの基本（Web API、第３回／３層構造、第10〜11回／Ajax、第12回）／6章 セキュリティと認証（プロキシは第２回、Cookie・セッション・CSRF・XSS は第３回）／7章 構築と運用（構成図、第10回）

linux-ファイル

ファイルと権限：ファイル操作／パイプとリダイレクト／権限

上位タグ: linux

扱う回

- 第１回 知識体系の紹介＋Linux の全体像（linux）

前提にしている回

- 第２回 シェル・環境変数・通信の切り分け
- 第５回 Git と GitHub

副読本・参考書・公式資料

- 入門 モダンLinux（オライリー・ジャパン）：Linux の仕組みを理解する本。第１回はこの本の流れで全体を通して見る。1章 Linuxの入門・2章 Linuxカーネル（第１回）／3章 シェルとスクリプト、4章 アクセス制御（第１〜２回）／5章 ファイルシステム（第１回）／6章 アプリケーション、パッケージ管理、コンテナ（6.6 コンテナは第１回、パッケージは第２・４回）／7章 ネットワーク（第１〜２回）／9.1 プロセス間通信（シグナル、第１回）／付録A 便利なコマンド集（第１回、以降は引く）・付録B モダンLinuxツール（第１回）

pm

プロジェクトマネジメント

下位タグ: pm-見積, pm-進捗

扱う回

- 第９回 要件と図（pm-見積, pm-進捗）

副読本・参考書・公式資料

- 達人プログラマー 第２版（オーム社）：基本ツール（シェル、プレーンテキスト、バージョン管理）（第１〜５回）／見積もり（第９回）
- ソフトウェアエンジニアリングの基礎（オライリー・ジャパン）：研修の軸として読み進める１冊。２章 コードを読む・３章 コードを書く（第５・７回）／４章 モデル＝どの図が必要か（第９〜10回）／５章 自動テスト（第８・11回）／６章 初めて触れるシステムの調査と変更（第1・12回）／８章 データの取り扱い（第10〜11回）／９章 ソフトウェアアーキテクチャ（第９〜10回）／10章 本番環境へ（第８回の CI）／12章 学び方を学ぶ（第１回）／13章 ソフトスキル（第12回）
- Mermaid 公式・GitHub Docs（Issues）：図の書き方（Syntax Reference）と、Issue・ラベル・マイルストーンの使い方の一次資料（第９回）

pm-見積

計画と見積:タスクの分解／幅のある見積

上位タグ: pm

扱う回

- 第９回 要件と図(pm-見積)

副読本・参考書・公式資料

- 達人プログラマー 第２版(オーム社):基本ツール(シェル、プレーンテキスト、バージョン管理)(第１〜５回)／見積もり(第９回)
- ソフトウェアエンジニアリングの基礎(オライリー・ジャパン):研修の軸として読み進める１冊。２章 コードを読む・３章 コードを書く(第５・７回)／４章 モデル＝どの図が必要か(第９〜10回)／５章 自動テスト(第８・11回)／６章 初めて触れるシステムの調査と変更(第1・12回)／８章 データの取り扱い(第10〜11回)／９章 ソフトウェアアーキテクチャ(第９〜10回)／10章 本番環境へ(第８回の CI)／12章 学び方を学ぶ(第１回)／13章 ソフトスキル(第12回)

pm-進捗

タスク管理：Issue／ラベル・マイルストーン

上位タグ: pm

扱う回

- 第９回 要件と図（pm-進捗）

副読本・参考書・公式資料

- ソフトウェアエンジニアリングの基礎（オライリー・ジャパン）：研修の軸として読み進める１冊。２章 コードを読む・３章 コードを書く（第５・７回）／４章 モデル＝どの図が必要か（第９〜10回）／５章 自動テスト（第８・11回）／６章 初めて触れるシステムの調査と変更（第１・12回）／８章 データの取り扱い（第10〜11回）／９章 ソフトウェアアーキテクチャ（第９〜10回）／10章 本番環境へ（第８回の CI）／12章 学び方を学ぶ（第１回）／13章 ソフトスキル（第12回）
- Mermaid 公式・GitHub Docs（Issues）：図の書き方（Syntax Reference）と、Issue・ラベル・マイルストーンの使い方の一次資料（第９回）

python

プログラミング（Python）

下位タグ: python-oop, python-可読性, python-構文

扱う回

- 第６回 Python の基本（python-構文）
- 第７回 読みやすく壊れにくい Python（python-oop, python-可読性）

前提にしている回

- 第７回 読みやすく壊れにくい Python
- 第８回 テストと TDD
- 第９回 要件と図
- 第11回 TDD による Web バックエンド

副読本・参考書・公式資料

- Effective Python 第２版：1〜２章（項目16 dict.get など）、８章 項目65〜66（例外と with）（第６回）／５章 クラスと継承（第７回）／10章 項目83 仮想環境（第４回）、項目90 静的解析（第７回）
- リーダブルコード：2〜６章 名前とコメント（第７回の規約づくり）／7〜９章 制御フローと変数（第７回のリファクタリング演習）／14章 テストと読みやすさ（第８回）／巻末解説の添削コミット（第５回のセルフレビュー）
- Python 公式ドキュメント：言語仕様と標準ライブラリの原典
- PEP 8・Ruff・mypy 公式：コーディング規約と、規約・型を自動で確かめる道具の一次資料（第７回）

python-oop

オブジェクト指向：ダックタイピング／Protocol／dataclass／ABC

上位タグ: python

扱う回

- 第７回 読みやすく壊れにくい Python（python-oop）

前提にしている回

- 第８回 テストと TDD
- 第11回 TDD による Web バックエンド

副読本・参考書・公式資料

- Effective Python 第２版：1〜２章（項目16 dict.get など）、８章 項目65〜66（例外と with）（第６回）／５章 クラスと継承（第７回）／10章 項目83 仮想環境（第４回）、項目90 静的解析（第７回）
- Python 公式ドキュメント：言語仕様と標準ライブラリの原典

python-可読性

読みやすいコード：命名／コメント／制御フロー／リファクタリング

上位タグ: python

扱う回

- 第７回 読みやすく壊れにくい Python（python-可読性）

副読本・参考書・公式資料

- リーダブルコード：2〜6章 名前とコメント（第７回の規約づくり）／7〜9章 制御フローと変数（第７回のリファクタリング演習）／14章 テストと読みやすさ（第８回）／巻末解説の添削コミット（第５回のセルフレビュー）
- Python 公式ドキュメント：言語仕様と標準ライブラリの原典
- PEP 8・Ruff・mypy 公式：コーディング規約と、規約・型を自動で確かめる道具の一次資料（第７回）

python-構文

言語機能：型ヒント／dict.get／内包表記／例外とwith／pathlib／パターンマッチ

上位タグ: python

扱う回

- 第６回 Python の基本（python-構文）

前提にしている回

- 第７回 読みやすく壊れにくい Python
- 第９回 要件と図

副読本・参考書・公式資料

- Effective Python 第２版：1〜2章（項目16 dict.get など）、8章 項目65〜66（例外と with）（第６回）／5章 クラスと継承（第７回）／10章 項目83 仮想環境（第４回）、項目90 静的解析（第７回）
- Python 公式ドキュメント：言語仕様と標準ライブラリの原典

sec

セキュリティ：XSSとエスケープ／CSRF／秘密情報の管理

扱う回

- 第３回 Web 通信の基礎（HTTP・Cookie・Web API）（sec）

副読本・参考書・公式資料

- イラスト図解式 この一冊で全部わかるWeb技術の基本 第２版（SBクリエイティブ）：見開き１テーマの図解で引く Web の入門書。２章 Webとネットワーク技術（第２回）／３章 HTTPでやりとりする仕組み（第３・10回）／４章 データ形式（HTML・DOM・JSON、第３・12回）／５章 Webアプリケーションの基本（Web API、第３回／３層構造、第10〜11回／Ajax、第12回）／６章 セキュリティと認証（プロキシは第２回、Cookie・セッション・CSRF・XSS は第３回）／７章 構築と運用（構成図、第10回）
- MDN Web Docs：HTTP、HTML、Cookie、WebSocket、CORS のリファレンス。Web の基礎はここを一次資料にする（第３回のヘッダ・キャッシュ・Cookie・CORS・フェッチ API・DOM、第10・12回でも引く）

test

テストと品質

下位タグ: test-CI, test-レビュー, test-自動, test-静的解析

扱う回

- 第５回 Git と GitHub(test-レビュー)
- 第７回 読みやすく壊れにくい Python(test-静的解析)
- 第８回 テストと TDD(test-自動, test-CI)
- 第11回 TDD による Web バックエンド(test-自動)
- 第12回 画面の部分更新と成果発表(test-レビュー)

前提にしている回

- 第８回 テストと TDD
- 第９回 要件と図
- 第11回 TDD による Web バックエンド
- 第12回 画面の部分更新と成果発表

副読本・参考書・公式資料

- Effective Python 第２版:1〜２章(項目16 dict.get など)、８章 項目65〜66(例外と with)(第６回)／５章 クラスと継承(第７回)／10章 項目83 仮想環境(第４回)、項目90 静的解析(第７回)
- リーダブルコード:2〜６章 名前とコメント(第７回の規約づくり)／7〜９章 制御フローと変数(第７回のリファクタリング演習)／14章 テストと読みやすさ(第８回)／巻末解説の添削コミット(第５回のセルフレビュー)
- ソフトウェアエンジニアリングの基礎(オライリー・ジャパン):研修の軸として読み進める１冊。２章 コードを読む・３章 コードを書く(第５・７回)／４章 モデル＝どの図が必要か(第９〜10回)／５章 自動テスト(第８・11回)／６章 初めて触れるシステムの調査と変更(第1・12回)／８章 データの取り扱い(第10〜11回)／９章 ソフトウェアアーキテクチャ(第９〜10回)／10章 本番環境へ(第８回の CI)／12章 学び方を学ぶ(第１回)／13章 ソフトスキル(第12回)
- PEP 8・Ruff・mypy 公式:コーディング規約と、規約・型を自動で確かめる道具の一次資料(第７回)
- pytest 公式・GitHub Actions 公式:テストの書き方(fixture・parametrize)と CI の設定の一次資料(第８回)

test-CI

CI：GitHub Actions／PR ごとの ruff・pytest の自動実行

上位タグ: test

扱う回

- 第８回 テストと TDD（test-CI）

前提にしている回

- 第11回 TDD による Web バックエンド

副読本・参考書・公式資料

- ソフトウェアエンジニアリングの基礎（オライリー・ジャパン）：研修の軸として読み進める１冊。２章 コードを読む・３章 コードを書く（第５・７回）／４章 モデル＝どの図が必要か（第９〜10回）／５章 自動テスト（第８・11回）／６章 初めて触れるシステムの調査と変更（第１・12回）／８章 データの取り扱い（第10〜11回）／９章 ソフトウェアアーキテクチャ（第９〜10回）／10章 本番環境へ（第８回の CI）／12章 学び方を学ぶ（第１回）／13章 ソフトスキル（第12回）
- pytest 公式・GitHub Actions 公式：テストの書き方（fixture・parametrize）と CI の設定の一次資料（第８回）

test-レビュー

レビューと不具合管理：コードレビュー／不具合票

上位タグ: test

扱う回

- 第５回 Git と GitHub(test-レビュー)
- 第１２回 画面の部分更新と成果発表(test-レビュー)

副読本・参考書・公式資料

- リーダブルコード：2〜6章 名前とコメント(第７回の規約づくり)／7〜9章 制御フローと変数(第７回のリファクタリング演習)／14章 テストと読みやすさ(第８回)／巻末解説の添削コミット(第５回のセルフレビュー)
- ソフトウェアエンジニアリングの基礎(オライリー・ジャパン)：研修の軸として読み進める１冊。2章 コードを読む・3章 コードを書く(第5・7回)／4章 モデル＝どの図が必要か(第９〜10回)／5章 自動テスト(第8・11回)／6章 初めて触れるシステムの調査と変更(第1・12回)／8章 データの取り扱い(第10〜11回)／9章 ソフトウェアアーキテクチャ(第９〜10回)／10章 本番環境へ(第８回の CI)／12章 学び方を学ぶ(第1回)／13章 ソフトスキル(第12回)

test-自動

自動テスト：pytest／TDD／テスト用DB

上位タグ: test

扱う回

- 第８回 テストと TDD（test-自動）
- 第11回 TDD による Web バックエンド（test-自動）

前提にしている回

- 第９回 要件と図
- 第11回 TDD による Web バックエンド
- 第12回 画面の部分更新と成果発表

副読本・参考書・公式資料

- ソフトウェアエンジニアリングの基礎（オライリー・ジャパン）：研修の軸として読み進める１冊。２章 コードを読む・３章 コードを書く（第５・７回）／４章 モデル＝どの図が必要か（第９〜10回）／５章 自動テスト（第８・11回）／６章 初めて触れるシステムの調査と変更（第１・12回）／８章 データの取り扱い（第10〜11回）／９章 ソフトウェアアーキテクチャ（第９〜10回）／10章 本番環境へ（第８回の CI）／12章 学び方を学ぶ（第１回）／13章 ソフトスキル（第12回）
- pytest 公式・GitHub Actions 公式：テストの書き方（fixture・parametrize）と CI の設定の一次資料（第８回）

test-静的解析

静的解析：ruff／mypy／pyright

上位タグ: test

扱う回

- 第７回 読みやすく壊れにくい Python（test-静的解析）

前提にしている回

- 第８回 テストと TDD

副読本・参考書・公式資料

- Effective Python 第２版：1〜２章（項目16 dict.get など）、８章 項目65〜66（例外と with）（第６回）／５章 クラスと継承（第７回）／10章 項目83 仮想環境（第４回）、項目90 静的解析（第７回）
- ソフトウェアエンジニアリングの基礎（オライリー・ジャパン）：研修の軸として読み進める１冊。２章 コードを読む・３章 コードを書く（第５・７回）／４章 モデル＝どの図が必要か（第９〜10回）／５章 自動テスト（第８・11回）／６章 初めて触れるシステムの調査と変更（第１・12回）／８章 データの取り扱い（第10〜11回）／９章 ソフトウェアアーキテクチャ（第９〜10回）／10章 本番環境へ（第８回の CI）／12章 学び方を学ぶ（第１回）／13章 ソフトスキル（第12回）
- PEP 8・Ruff・mypy 公式：コーディング規約と、規約・型を自動で確かめる道具の一次資料（第７回）

web

Webアプリケーション

下位タグ: web-サーバ, web-ブラウザ

扱う回

- 第３回 Web 通信の基礎(HTTP・Cookie・Web API)(web-ブラウザ)
- 第11回 TDD による Web バックエンド(web-サーバ)
- 第12回 画面の部分更新と成果発表(web-ブラウザ)

前提にしている回

- 第12回 画面の部分更新と成果発表

副読本・参考書・公式資料

- MDN Web Docs:HTTP、HTML、Cookie、WebSocket、CORS のリファレンス。Web の基礎はここを一次資料にする(第３回のヘッダ・キャッシュ・Cookie・CORS・フェッチ API・DOM、第10・12回でも引く)
- htmx 公式・Hypermedia Systems:htmx の属性リファレンスと例集。Hypermedia Systems は htmx 作者らによる無料の書籍で、サーバが HTML を返す設計の考え方がわかる(第12回の講師の補足で紹介)
- FastAPI 公式ドキュメント:チュートリアルがそのまま第11回の教材になる

web-サーバ

サーバ側：FastAPI／Jinja2テンプレート／Pydantic／依存性の注入

上位タグ: web

扱う回

- 第11回 TDD による Web バックエンド（web-サーバ）

前提にしている回

- 第12回 画面の部分更新と成果発表

副読本・参考書・公式資料

- FastAPI 公式ドキュメント：チュートリアルがそのまま第11回の教材になる

web-ブラウザ

ブラウザ側：HTMLフォーム／fetch・DOM操作／DevTools／htmx

上位タグ: web

扱う回

- 第３回 Web 通信の基礎（HTTP・Cookie・Web API）（web-ブラウザ）
- 第12回 画面の部分更新と成果発表（web-ブラウザ）

副読本・参考書・公式資料

- MDN Web Docs：HTTP、HTML、Cookie、WebSocket、CORS のリファレンス。Web の基礎はここを一次資料にする（第３回のヘッダ・キャッシュ・Cookie・CORS・フェッチ API・DOM、第10・12回でも引く）
- htmx 公式・Hypermedia Systems：htmx の属性リファレンスと例集。Hypermedia Systems は htmx 作者らによる無料の書籍で、サーバが HTML を返す設計の考え方がわかる（第12回の講師の補足で紹介）

標準

開発標準：コーディング規約／ブランチ運用／コミット規約／レビュー基準

扱う回

- 第５回 Git と GitHub（標準）
- 第７回 読みやすく壊れにくい Python（標準）

副読本・参考書・公式資料

- リーダブルコード：2〜6章 名前とコメント（第７回の規約づくり）／7〜9章 制御フローと変数（第７回のリファクタリング演習）／14章 テストと読みやすさ（第８回）／巻末解説の添削コミット（第５回のセルフレビュー）
- ソフトウェアエンジニアリングの基礎（オライリー・ジャパン）：研修の軸として読み進める１冊。2章 コードを読む・3章 コードを書く（第5・7回）／4章 モデル＝どの図が必要か（第9〜10回）／5章 自動テスト（第8・11回）／6章 初めて触れるシステムの調査と変更（第1・12回）／8章 データの取り扱い（第10〜11回）／9章 ソフトウェアアーキテクチャ（第9〜10回）／10章 本番環境へ（第８回の CI）／12章 学び方を学ぶ（第1回）／13章 ソフトスキル（第12回）

設計

設計

下位タグ: 設計-構成, 設計-要件, 設計-詳細

扱う回

- 第９回 要件と図（設計-要件）
- 第10回 設計書作成（設計-詳細, 設計-構成）

前提にしている回

- 第10回 設計書作成
- 第11回 TDD による Web バックエンド

副読本・参考書・公式資料

- Webを支える技術（技術評論社）：第２部 URI、第３部 HTTP（第３・10回）／第５部 Web サービスの設計（第10回の API 設計）
- 達人に学ぶDB設計徹底指南書 第２版（翔泳社）：３章 正規化、４章 ER図（第10回）／６章 インデックス設計、７章 論理設計のアンチパターン（第11回）
- ソフトウェアアーキテクチャの基礎 第２版（オライリー・ジャパン）：アーキテクチャ特性（非機能要件）の考え方（第９回）／アーキテクチャスタイルの比較は配属後の発展課題
- ソフトウェアエンジニアリングの基礎（オライリー・ジャパン）：研修の軸として読み進める１冊。２章 コードを読む・３章 コードを書く（第５・７回）／４章 モデル＝どの図が必要か（第９〜10回）／５章 自動テスト（第８・11回）／６章 初めて触れるシステムの調査と変更（第１・12回）／８章 データの取り扱い（第10〜11回）／９章 ソフトウェアアーキテクチャ（第９〜10回）／10章 本番環境へ（第８回の CI）／12章 学び方を学ぶ（第１回）／13章 ソフトスキル（第12回）
- イラスト図解式 この一冊で全部わかるWeb技術の基本 第２版（SBクリエイティブ）：見開き１テーマの図解で引く Web の入門書。２章 Webとネットワーク技術（第２回）／３章 HTTPでやりとりする仕組み（第３・10回）／４章 データ形式（HTML・DOM・JSON、第３・12回）／５章 Webアプリケーションの基本（Web API、第３回／３層構造、第10〜11回／Ajax、第12回）／６章 セキュリティと認証（プロキシは第２回、Cookie・セッション・CSRF・XSS は第３回）／７章 構築と運用（構成図、第10回）
- Mermaid 公式・GitHub Docs（Issues）：図の書き方（Syntax Reference）と、Issue・ラベル・マイルストーンの使い方の一次資料（第９回）

設計-構成

システム構成:システム構成図

上位タグ: 設計

扱う回

- 第10回 設計書作成(設計-構成)

副読本・参考書・公式資料

- ソフトウェアアーキテクチャの基礎 第2版(オライリー・ジャパン):アーキテクチャ特性(非機能要件)の考え方(第9回)／アーキテクチャスタイルの比較は配属後の発展課題
- ソフトウェアエンジニアリングの基礎(オライリー・ジャパン):研修の軸として読み進める1冊。2章 コードを読む・3章 コードを書く(第5・7回)／4章 モデル＝どの図が必要か(第9〜10回)／5章 自動テスト(第8・11回)／6章 初めて触れるシステムの調査と変更(第1・12回)／8章 データの取り扱い(第10〜11回)／9章 ソフトウェアアーキテクチャ(第9〜10回)／10章 本番環境へ(第8回の CI)／12章 学び方を学ぶ(第1回)／13章 ソフトスキル(第12回)
- イラスト図解式 この一冊で全部わかるWeb技術の基本 第2版(SBクリエイティブ):見開き1テーマの図解で引く Web の入門書。2章 Webとネットワーク技術(第2回)／3章 HTTPでやりとりする仕組み(第3・10回)／4章 データ形式(HTML・DOM・JSON、第3・12回)／5章 Webアプリケーションの基本(Web API、第3回／3層構造、第10〜11回／Ajax、第12回)／6章 セキュリティと認証(プロキシは第2回、Cookie・セッション・CSRF・XSS は第3回)／7章 構築と運用(構成図、第10回)

設計-要件

要件：ユーザーストーリー／受け入れ条件／非機能要件

上位タグ: 設計

扱う回

- 第９回 要件と図（設計-要件）

前提にしている回

- 第10回 設計書作成

副読本・参考書・公式資料

- ソフトウェアエンジニアリングの基礎（オライリー・ジャパン）：研修の軸として読み進める１冊。２章 コードを読む・３章 コードを書く（第５・７回）／４章 モデル＝どの図が必要か（第９〜10回）／５章 自動テスト（第８・11回）／６章 初めて触れるシステムの調査と変更（第１・12回）／８章 データの取り扱い（第10〜11回）／９章 ソフトウェアアーキテクチャ（第９〜10回）／10章 本番環境へ（第８回の CI）／12章 学び方を学ぶ（第１回）／13章 ソフトスキル（第12回）
- Mermaid 公式・GitHub Docs（Issues）：図の書き方（Syntax Reference）と、Issue・ラベル・マイルストーンの使い方の一次資料（第９回）

設計-詳細

詳細設計：API設計書／DB設計書

上位タグ: 設計

扱う回

- 第10回 設計書作成（設計-詳細）

前提にしている回

- 第11回 TDD による Web バックエンド

副読本・参考書・公式資料

- Webを支える技術（技術評論社）：第２部 URI、第３部 HTTP（第３・10回）／第５部 Web サービスの設計（第10回の API 設計）
- 達人に学ぶDB設計徹底指南書 第２版（翔泳社）：３章 正規化、４章 ER図（第10回）／６章 インデックス設計、７章 論理設計のアンチパターン（第11回）
- ソフトウェアエンジニアリングの基礎（オライリー・ジャパン）：研修の軸として読み進める１冊。２章 コードを読む・３章 コードを書く（第５・７回）／４章 モデル＝どの図が必要か（第９〜10回）／５章 自動テスト（第８・11回）／６章 初めて触れるシステムの調査と変更（第１・12回）／８章 データの取り扱い（第10〜11回）／９章 ソフトウェアアーキテクチャ（第９〜10回）／10章 本番環境へ（第８回の CI）／12章 学び方を学ぶ（第１回）／13章 ソフトスキル（第12回）

テンプレート

研修で使うひな形です。自分のアプリのリポジトリ（`~/handsoff/workspace/todo-app/`）にコピーして使います。見本は `examples/todo-app-snapshots/` の各回のフォルダにあります。

アジャイルで設計書を少なくする前提です。何を作るかはユーザーストーリー（Issue）と受け入れ条件で決め、細かい振る舞いは TDD のテストで示します（画面設計書とテスト仕様書は作りません）。設計書は「実装者が質問せずに作れるか」を合格基準にします。図は Mermaid で Markdown に書き、Git で差分を追えるようにします。

各ファイルの先頭の `#` の見出しは、資料の目次に出すためのものです。Issue・PR の本文に使うときは、見出しの行を消してかまいません。

要件と設計書

ファイル	使う回	内容	見本
issue_story.md	9	ユーザーストーリーの Issue（「〜として、〜したい。なぜなら〜」、受け入れ条件、見積の幅と根拠、関連）	`session-09/docs/stories.md`（Issue の控え）
requirements.md	9	要件メモ（目的・背景、業務フロー図、状態遷移図、非機能要件、制約、対象外、ストーリー一覧へのリンク）	`session-09/docs/requirements.md`
api-design.md	10	API 設計書（エンドポイント、リクエスト・レスポンス例、ステータスコード、入力チェックとエラー表示の文言、シーケンス図、設計レビューで出た質問）	`session-10/docs/design/api.md`
db-design.md	10	DB 設計書（ER 図、テーブル定義、制約、計算で求める値）	`session-10/docs/design/db.md`
architecture.md	10	システム構成図（構成要素、通信経路とポート、ミドルウェアとバージョン、環境変数）	`session-10/docs/design/architecture.md`

「見本」は `examples/todo-app-snapshots/` の下のパスです。

Issue と PR

ファイル	使う回	内容	見本
pull_request.md	5 以降	PR の説明（目的・変更点・確認方法・セルフチェック）	—
issue_task.md	5 以降	タスクの Issue（目的・やること・完了条件、第９回以降は見積の幅）	—
bug-report.md	12	不具合票（再現手順・期待結果・実際の結果）	`session-12/docs/bugs/BUG-001.md`

GitHub・GitLab では、次の場所に置いたテンプレートが自動で使われます。

ホスティング	PR のテンプレート	Issue のテンプレート
GitHub	`.github/pull_request_template.md`	`.github/ISSUE_TEMPLATE/story.md`・`task.md`・`bug.md`
GitLab	`.gitlab/merge_request_templates/Default.md`	`.gitlab/issue_templates/story.md`・`task.md`・`bug.md`

API設計書

合格基準: 実装者が質問せずに作れるか

項目	内容
版	1.0(第10回)
関連	要件メモ、DB設計書、ストーリーの Issue

1. エンドポイント一覧

ID	メソッド	URL	内容	成功時	ストーリー
A-01	GET	`/`	一覧ページ	200 ページ全体の HTML	#
A-02	POST			303 で `/` へ	

2. 各エンドポイント

A-02 POST

パラメータ	場所	型	必須	説明
	パス／クエリ／フォーム			

リクエスト例:

```
POST /tasks HTTP/1.1
Content-Type: application/x-www-form-urlencoded

title=...&due=...
```

レスポンス例(成功):

```
HTTP/1.1 303 See Other
Location: /
```

レスポンス例(失敗):

```
←!— 返す HTML の要点（エラーの文言をどこに出すか、入力した値を残すか） —→
```

処理:

3. ステータスコード

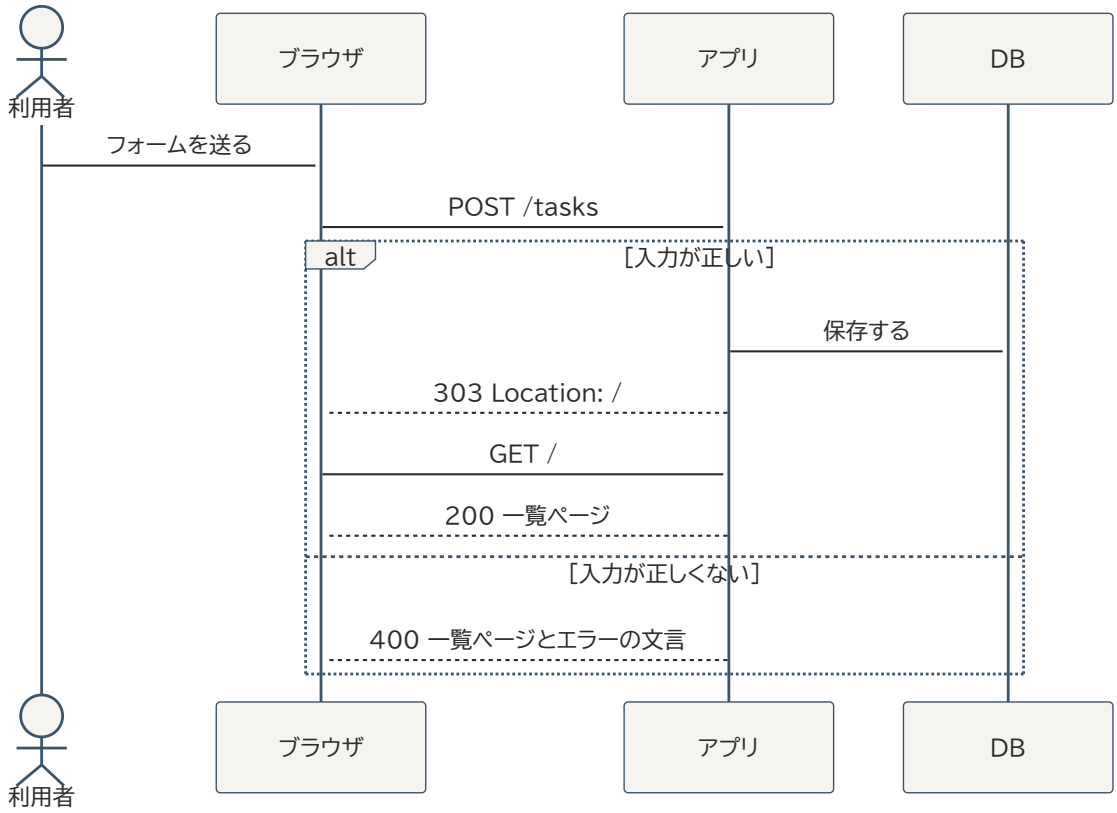
状況	ステータス	返すもの
成功(表示)	200	
成功(追加・更新)	303	`Location: /`
入力が正しくない	400	
対象が見つからない	404	

4. 入力チェックとエラー表示

操作	チェック	ステータス	表示する文言(そのまま)
		400	
		400	

- 表示する場所:
- 入力した値をフォームに残すか:
- 値は HTML として解釈せず文字として表示する(テンプレートの自動エスケープ)

5. シーケンス図



6. 設計レビューで出た質問

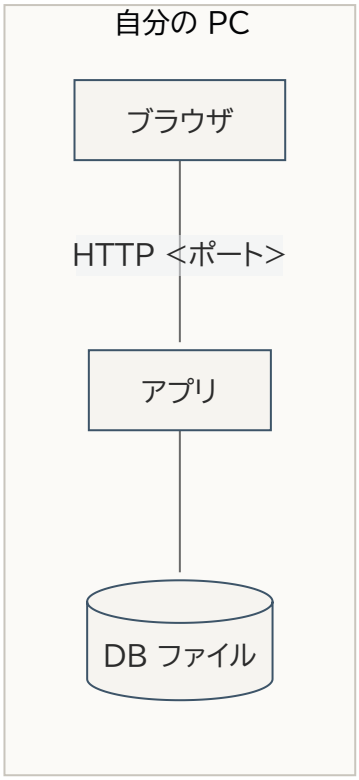
質問	答え	反映した場所

システム構成図

合格基準: 実装者が質問せずに作れるか

項目	内容
版	1.0（第10回）
関連	要件メモ の非機能要件

1. 構成図



構成要素	役割	備考

2. 通信経路とポート

送信元	宛先	プロトコル・アドレス・ポート

3. ミドルウェアとバージョン

ソフトウェア	バージョン	固定する場所
Python	3.12	`mise.toml`
		`pyproject.toml` と `uv.lock`

4. 設定（環境変数）

環境変数	意味	既定値

5. 起動のしかた

不具合票

概要

再現手順

3.

期待結果

実際の結果

環境

- ブランチ／コミット:
- ブラウザ／OS:
- その他:

重要度

補足

DB設計書

合格基準: 実装者が質問せずに作れるか

項目	内容
版	1.0（第10回）
DBMS	SQLite 3
DB の場所	
関連	要件メモ、API設計書

1. ER図

tasks			
INTEGER	id	PK	連番
TEXT	title		タイトル
TEXT	due		期限。NULL可

2. テーブル定義

<テーブル名>（<日本語の名前>）

カラム	型	NULL	キー・既定値	説明
id	INTEGER	不可	PK、AUTOINCREMENT	連番
		可／不可		

3. 制約

制約	内容	理由
NOT NULL		
DEFAULT		
CHECK		

4. DB に持たず計算で求める値

値	条件

5. 並び順

-

ユーザーストーリー

ストーリー

<利用者>として、<したいこと>したい。なぜなら<理由>から。

受け入れ条件

- [ ] (1)
- [ ] (2)

見積

タスク	見積	根拠
	～時間	

関連

-

タスク

目的

やること

- [ ]

完了条件

-

見積

- 見積: ～時間
- 根拠:

関連

-

PR の説明

目的

Closes #

変更点

-

確認方法

1.

セルフチェック

- [ ] 目的と変更点が一致している（関係のない変更が入っていない）
- [ ] コミットが「何を・なぜ」の単位に分かれていて、メッセージで説明できている
- [ ] 確認方法の手順を自分で一度実行した
- [ ] 秘密情報や `.venv`・`tasks.json`・`var/` などの不要なファイルが入っていない

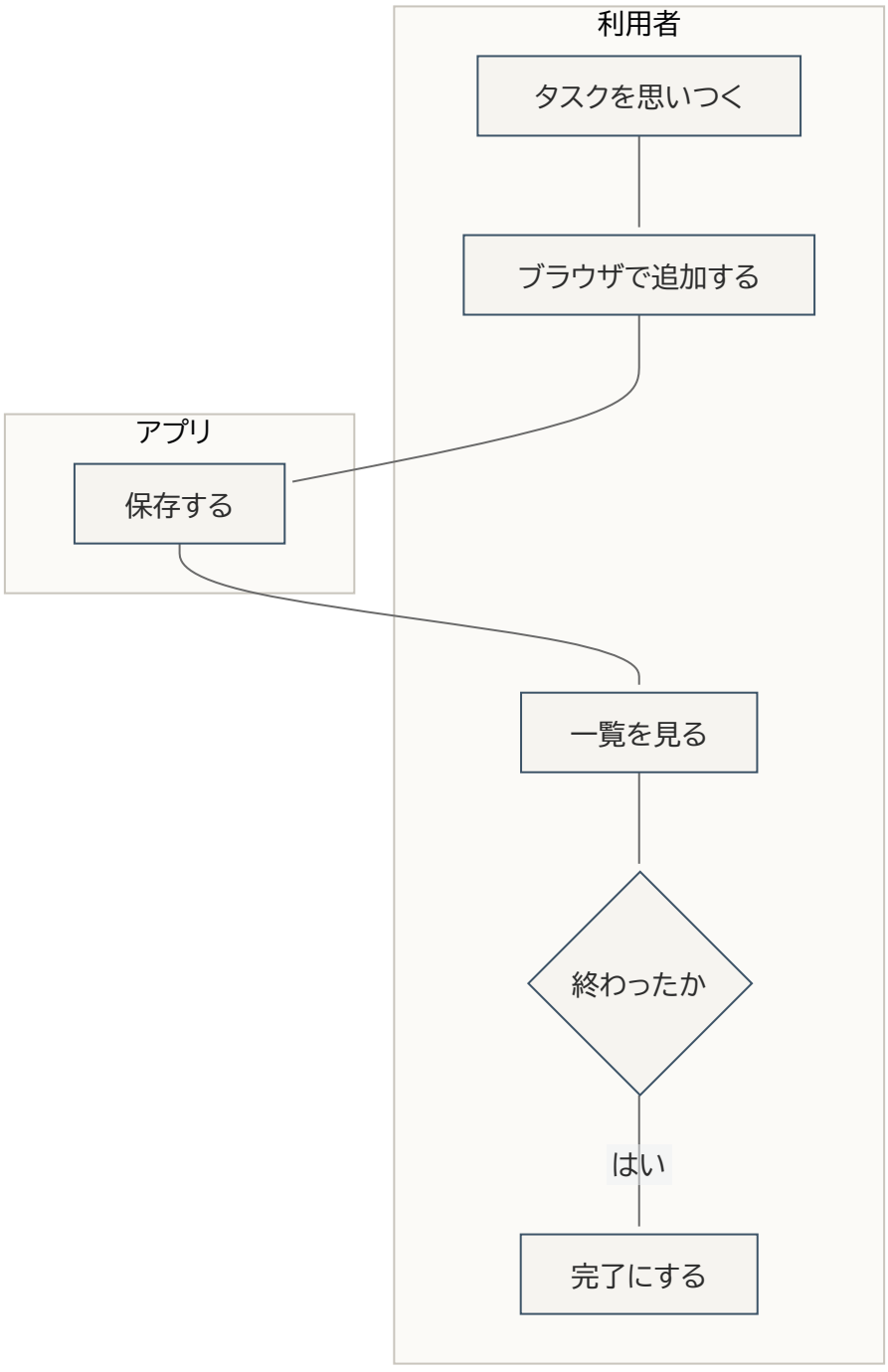
要件メモ <アプリ名>

合格基準: 実装者が質問せずに作れるか

項目	内容
版	1.0（第９回）
書いた人	
依頼人	

1. 目的・背景

業務フロー図



状態遷移図



状態	意味	条件
`TODO`	未着手	
`DONE`	完了	
`OVERDUE`	期限切れ	

2. ストーリー一覧

- Issue の一覧:
- 控え: stories.md

3. 非機能要件

区分	要件
性能	
セキュリティ	
運用	

4. 制約

-

5. 対象外

項目	理由

6. 未定・確かめたいこと

-

未確定事項

社内での確認が終わっていない事項の一覧です。資料では値を推測で埋めず、下の「プレースホルダー」の形で書いています。決まったら、この表の「決定内容」を埋め、プレースホルダーを置き換えてください。

置き換え漏れは次のコマンドで探せます（研修ディレクトリのルートで実行）。

```
grep -rn --include='*.md' --include='*.toml' --include='*.py' -e '<社内' -e '<プロキシ' -e '<Git' -e '<リポジトリ' -e '<研修' docs examples workspace tools | grep -v open-questions.md
```

ID	事項	プレースホルダー	影響する回	決定内容
Q1	社内プロキシのアドレスと認証の有無	`<社内プロキシのアドレス>`（`http://<ホスト>:<ポート>` の形を想定）、`<社内プロキシのホスト:ポート>`（認証ありの場合に `ユーザー名:パスワード@` を挟むため）、`<プロキシ認証の有無>`	2〜4	
Q2	受講者 PC で WSL を有効化できるか（管理者権限・情報システム部門の許可）	—	1〜4（全回の前提）	
Q3	社内 CA 証明書の入手方法	`<社内CA証明書の入手方法>`、`<社内CA証明書のファイル名>`	2〜4	
Q4	Git ホスティング（GitHub か GitLab か）、アカウントの発行方法、認証方式	`<Gitホスティング>`、`<リポジトリのURL>`	5〜12	
Q5	mise・uv・PyPI へプロキシ越しにアクセスできるか	—	4 以降	
Q6	受講者の人数（グループ分けは不要。2026-10-12 に全回を個人で進めることにした）	—	12（発表の持ち時間）	

ID	事項	プレースホルダー	影響する回	決定内容
Q7	週2回(全24回)に拡張するか	—	—	拡張しない。2026-10-12 の見直しで拡張案をなくした(週1回×全12回)。拡張案のテーマの教材は作らない
Q8	研修ディレクトリー式の配布方法と、受講者 PC での置き場所	`<研修ディレクトリの配布方法>`。置き場所は資料上 `~/handsoff` としている	1 以降	
Q9	(不要になった)模範リポジトリのコミット作者名	—	—	2026-10-12 に見本のアプリ(`examples/todo-app/`)を git を使わないフォルダにしたので不要
Q10	受講者 PC の Ubuntu のバージョンと、WSL が Windows のプロキシ設定を引き継ぐか	—	2〜4	
Q11	GitHub Actions (GitLab なら Runner)が受講者のリポジトリで使えるか	—	8 以降	
Q12	受講者 PC(WSL)で Docker が使えるか(Docker Desktop のライセンス、または WSL 内の Docker Engine の導入可否)	—	1	

補足

Q1・Q3 社内プロキシと CA 証明書

- 認証付きのプロキシの場合、`HTTP_PROXY` に ID とパスワードを書くことになる。第2回で「秘密情報をファイルや履歴に残さない」扱い(`read -s` で入力するなど)を教えている
- 第3回では、Windows のブラウザから WSL のデモのサーバ(`127.0.0.1:8000`・`localhost:8001`)を開くので、Windows のプロキシ設定の例外に `127.0.0.1;localhost` が要る
- 社内 CA 証明書の配布形式(PEM か DER か)によって、Ubuntu への登録手順(`update-ca-certificates`)が変わる

- uv は既定では OS の証明書ストアを使わない。社内 CA を読ませるには `UV_NATIVE_TLS=1`（または `SSL_CERT_FILE`）が要る（第2・4回の資料に記載）

Q4 Git ホスティング

- 決まるまで、PR・Issue・ブランチ保護の説明はどちらでも読める書き方にし、固有の操作は「GitHub の場合」「GitLab の場合」に分けて書く
- 用語の違い: Pull Request（GitHub）／Merge Request（GitLab）。資料では「PR」と書き、初出で両方を併記する
- カリキュラムには「GitHub の Issue ラベル」とあるが、GitLab でもラベル機能で同じ運用ができる
- 認証方式（HTTPS とトークンか、SSH 鍵か）とトークンの保存方法を決める必要がある。`git config credential.helper store` はトークンを平文で保存するため、社内規定の確認が要る
- 受講者に自分のリポジトリを作る権限があるか（第5回で手順書のリポジトリ、第6回で TODO アプリのリポジトリを作る）
- 受講者に Issue ラベルを作る権限があるか（第9回以降、ストーリーの Issue にタグ名のラベルを付ける）
- ブランチ保護（main への直接 push の禁止）は、第5回では講師が画面で見せるだけにした（講師用資料の「講師が見せる補足」）。GitHub の無料プランの非公開リポジトリでは使えず、GitLab ではプランによって設定できる範囲が違う

Q5 mise・uv・PyPI へのアクセス

研修の環境構築では、次の通信が発生します。プロキシで止められていないかを、事前に講師が確認してください。

通信先	用途
`mise.run`、`mise.jdx.dev`	mise のインストール
`github.com`、`objects.githubusercontent.com`	mise が Python（python-build-standalone）と uv を取得する
`pypi.org`、`files.pythonhosted.org`	uv がパッケージを取得する
`archive.ubuntu.com`、`security.ubuntu.com`、`esm.ubuntu.com`	apt（第2回）。`esm.ubuntu.com` は https で接続する
`untrusted-root.badssl.com`	第2回の演習で、証明書エラーの見え方を確かめる
`jsonplaceholder.typicode.com`	第3回の演習で、公開 API をプロキシ越しに呼ぶ（通らなければ講師の画面で見せる）
`registry.npmjs.org`、`nodejs.org`	第4回で講師が見せる補足（mise で Node.js を入れる）と、資料の HTML 版・PDF 版を作る tools の docs タスクで使う

htmx（第12回で講師が見せる補足）を CDN から読み込む場合は、その CDN（`unpkg.com` など）への通信も要ります。

Q8 研修ディレクトリの配布

- 研修ディレクトリのルートは git 管理しない方針なので、配布は zip などのファイル共有になる想定
- 配る一式は `tools/` の `mise run dist -- <出力先>` で作る。`.git` はすべて含めず、各回の `instructor.md`（講師用）、`handoff.md`、`docs/_template/` も除く。HTML 版・PDF 版（`docs/_build/`）は含める
- 見本の TODO アプリ（`examples/todo-app/` と各回終了時点の `examples/todo-app-snapshots/session-NN/`）は git を使わない普通のフォルダなので、そのままコピーする。遅れた受講者はそのフォルダを `workspace/` にコピーして追いつく（examples/README.md）
- 見本にはコミット履歴がない。TDD の順序やコミットの分け方は、講師が画面で見せる

Q10 Ubuntu のバージョンと WSL

- Ubuntu のバージョンによって sudo の実装が違い、`sudo -E` で環境変数を引き継げないことがある（第２回の資料を作るときに Ubuntu 26.04 の sudo-rs で確認）。第２回では `sudo --preserve-env=変数名` を使う形で書いている
- WSL が Windows のプロキシ設定を自動で引き継ぐ設定になっているかで、第２回の手順の一部が不要になる